408 考纲知识点手册

计算机组成原理

计算机学科专业基础综合　分册

按 2027 年全国硕士研究生招生考试大纲编排

对应大纲　第二部分计算机组成原理
本册章数　6 章
知识点条目　覆盖大纲本部分全部条目
主要素材　语雀「计算机考研 408 笔记」知识库
王道 2027 系列教材与课件
工作区历年真题与强化笔记

本分册为依据 2027 年考试大纲整理的「计算机组成原理」知识点汇编，供复习检索使用。

使用说明

本手册按《2027 年全国硕士研究生招生考试计算机学科专业基础考试大纲》的条目顺序编排，覆盖大纲「考查内容」中数据结构、计算机组成原理、操作系统、计算机网络四科的全部条目。

本册范围：本册是《408 考纲知识点手册》的「计算机组成原理」分册，对应 2027 年考试大纲「计算机组成原理」部分的全部考查内容，共 6 章。其余三科另见对应分册；四科合订本见《408 考纲知识点手册》。

一、编排方式

全书分四部分，共 24 章。四部分与大纲的四科一一对应，章与该科下的一级条目一一对应，章内小节按大纲的二级、三级条目顺序展开。每一章的开头用引用块照抄本章对应的大纲原文条目，便于对照检查覆盖情况；每一章末尾固定给出「本章小结表」与「易错清单」两张速查内容。

二、阅读约定

本手册用三种提示框把不同性质的内容区分开：

- 核心结论——必须记住的定义、公式、判定条件，考前反复看的就是这些。
- 易错点——统考中反复出现的陷阱，以及容易与相邻概念混淆的地方。
- 真题考法——该知识点在统考中以什么形式出现；能确认年份与题号的，如实标注。无法确认的不标，不编造。

正文中的公式一律用数学排版；算法与代码用等宽框给出；对比性内容尽量做成表格。凡涉及计算的知识点，都给出公式、符号含义、适用范围，并配一组可以自己动手复核的数字算例。

三、素材来源

本手册的素材来自三处：

- 语雀知识库「计算机考研 408 笔记」（月暗）导出的 182 篇笔记 Markdown 全文；
- 工作区中的王道 2027 系列教材、课件、强化笔记与历年真题汇编；
- 工作区 408 计算机考研目录下的各类复习笔记与答题本。

素材在整理过程中做了三件事：一是按大纲重新组织顺序；二是把原笔记中的口语化叙述、错别字与图片占位改写成规范的知识点条目，公式按上下文重新推导书写；三是逐条核对，凡素材之间口径冲突的，以大纲用词与统考真题为准，并在正文中说明差异。

四、使用建议

第一轮复习时，按章通读，重点看「核心结论」与例题，把「易错点」留到做题后再回看；第二轮复习时只看每章的「本章小结表」，遇到想不起来的再翻回正文；考前冲刺时只看「核心结论」与「易错清单」。

说明：本手册为复习资料汇编，用于知识点检索与自查，不替代教材与真题训练。

目录

使用说明 **i**

第一章　计算机系统概述 **1**

本章考纲要求 . . . 1
一、 计算机系统的基本组成 . . . 2
（一） 计算机系统的构成 . . . 2
（二） 计算机硬件系统的五大部件 . . . 2
（三） 各部件的工作关系（数据通路初貌） . . . 3
二、 计算机硬件的基本结构 . . . 3
（一） 冯 · 诺依曼结构 . . . 3
（二） 现代计算机以存储器为中心 . . . 4
（三） 计算机的层次结构 . . . 4
（四） 指令集体系结构（ISA）与微架构的界限 . . . 5
三、 计算机软件和硬件的关系 . . . 5
（一） 软件与硬件的逻辑等价性 . . . 5
（二） 固化的概念 . . . 6
四、 计算机系统的工作原理 . . . 6
（一） 「存储程序」工作方式 . . . 6
（二） 高级语言程序与机器语言程序之间的转换 . . . 6
（三） 程序和指令的执行过程 . . . 8
五、 性能评价的两个基本维度 . . . 9
（一） 响应时间 . . . 9
（二） 吞吐量 . . . 10
六、 CPU 时钟周期、主频、CPI 与 CPU 执行时间 . . . 10
（一） 时钟周期与主频 . . . 10
（二） CPI . . . 10
（三） CPU 执行时间的三个等价公式 . . . 11
（四） 用比例法快速解题 . . . 12
（五） 两台机器执行时间之比 . . . 13
七、 MIPS 与 FLOPS 类指标 . . . 13
（一） MIPS . . . 13
（二） GIPS 与指令执行速度 . . . 14
（三） MIPS 的三条重要局限 . . . 15
（四） FLOPS 系列指标 . . . 15

（五） 速率进制 vs 容量进制（必考辨析） . . . 16
八、 各种字长总结 . . . 17
（一） 机器字长 . . . 17
（二） 指令字长 . . . 17
（三） 存储字长与存储容量 . . . 18
（四） 编址单位与地址字长 . . . 18
（五） MAR、MDR 与地址线、数据线 . . . 19
（六） 数据字长 . . . 20
（七） 地址字长 . . . 20
（八） 字长之间的关系梳理 . . . 20
本章小结表 . . . 21
易错清单 . . . 23

第二章 数据的表示和运算 **25**
本章考纲要求 . . . 25
一、 数制与编码 . . . 25
（一） 进位计数制 . . . 25
（二） 进位计数制之间的相互转换 . . . 26
（三） 真值与机器数 . . . 27
（四） 定点数的编码表示 . . . 27
二、 运算方法和运算电路 . . . 31
（一） 基本运算部件 . . . 31
（二） 补码加/减运算器与标志位的生成 . . . 32
（三） 乘/除法运算的基本原理 . . . 35
（四） 乘法电路和除法电路的基本结构 . . . 39
三、 整数的表示和运算 . . . 41
（一） 无符号整数 . . . 41
（二） 带符号整数 . . . 41
（三） 零扩展与符号扩展 . . . 42
（四） 无符号与有符号混合运算的陷阱 . . . 42
（五） 有符号整数加减运算的溢出判断 . . . 43
四、 浮点数的表示和运算 . . . 44
（一） 浮点数的一般格式 . . . 44
（二） 浮点数的规格化 . . . 44
（三） IEEE 754 标准 . . . 44
（四） 浮点数的加减运算 . . . 47
（五） 浮点数加减运算的步骤汇总 . . . 51
本章小结表 . . . 51

（六） 表一：各种编码对照（$n+1$ 位编码，1 位符号位 + n 位数值位） 51
（七） 表二：四个标志位 51
（八） 表三：乘法与除法运算方法 52
（九） 表四：IEEE 754 关键数值（单精度） 52
（十） 表五：浮点数加减运算六步 52
易错清单 53

第三章 存储器层次结构 **55**
本章考纲要求 55
一、 存储器的分类 56
（一） 按存储介质分类 56
（二） 按存取方式分类 56
（三） 按信息可更改性与易失性分类 56
（四） 按作用与存取速度分类 57
（五） 存储器的性能指标 57
（六） 存储器的层次化结构 58
（七） 程序访问的局部性原理 59
二、 半导体随机存取存储器 60
（一） 半导体存储芯片的内部结构与译码方式 60
（二） 基本存储元件 60
（三） DRAM 的刷新 61
（四） DRAM 的地址复用 62
（五） 只读存储器 ROM 与 Flash 存储器 63
三、 主存储器 65
（一） DRAM 芯片与内存条 65
（二） 存储器容量的扩展 65
（三） 主存与 CPU 之间的连接 67
（四） 多模块存储器 69
四、 外部存储器 71
（一） 磁盘存储器 71
（二） 固态硬盘 SSD 74
五、 高速缓冲存储器（Cache） 75
（一） Cache 的基本原理 75
（二） Cache 与主存之间的映射方式 76
（三） Cache 中主存块的替换算法 80
（四） Cache 写策略 81
六、 虚拟存储器 83
（一） 虚拟存储器的基本概念 83

（二） 页式虚拟存储器 . . . 84
（三） 段式虚拟存储器 . . . 88
（四） 段页式虚拟存储器 . . . 88
本章小结表 . . . 89
易错清单 . . . 91
七、 本章公式速查 . . . 93

第四章 指令系统 **95**
本章考纲要求 . . . 95
一、 指令系统的基本概念 . . . 95
（一） 指令与指令系统 . . . 95
（二） 指令集体系结构（ISA） . . . 96
（三） 指令字长与机器字长 . . . 96
（四） 定长指令字与变长指令字 . . . 97
二、 指令格式 . . . 97
（一） 操作码与地址码 . . . 97
（二） 零地址 / 一地址 / 二地址 / 三地址指令 . . . 97
（三） 定长操作码 . . . 98
（四） 扩展操作码（可变长操作码） . . . 98
三、 寻址方式 . . . 100
（一） 指令寻址与数据寻址 . . . 100
（二） 各寻址方式详解 . . . 101
（三） 寻址方式完整对比表 . . . 104
四、 数据的对齐和大/小端存放方式 . . . 105
（一） 大端方式与小端方式 . . . 105
（二） 边界对齐与不对齐存储 . . . 106
（三） 结构体大小的计算规则 . . . 106
五、 CISC 和 RISC 的基本概念 . . . 109
（一） CISC 的基本概念与特点 . . . 109
（二） RISC 的基本概念与特点 . . . 110
（三） CISC 与 RISC 的完整对比 . . . 110
（四） 走向融合 . . . 110
六、 高级语言程序与机器级代码之间的对应 . . . 111
（一） 编译器、汇编器和链接器的基本概念 . . . 111
（二） 选择结构语句的机器级表示 . . . 112
（三） 循环结构语句的机器级表示 . . . 113
（四） 过程（函数）调用对应的机器级表示 . . . 116
（五） MIPS 汇编基础 . . . 119

本章小结表 . . . 123
易错清单 . . . 124

第五章 中央处理器 **126**
本章考纲要求 . . . 126
一、 CPU 的功能和基本结构 . . . 126
（一） CPU 的功能 . . . 126
（二） 运算器的组成与各部件的作用 . . . 127
（三） 控制器的组成与各部件的作用 . . . 128
（四） 各寄存器的位数与地址、字长的关系 . . . 128
二、 指令执行过程 . . . 129
（一） 指令周期的层级：指令周期、机器周期、时钟周期 . . . 129
（二） 取指周期的流程与数据通路 . . . 130
（三） 间址周期的流程与数据通路 . . . 130
（四） 执行周期的流程与数据通路 . . . 131
（五） 中断周期的流程与数据通路 . . . 131
（六） 各周期微操作的通用分析方法 . . . 132
三、 数据通路的功能和基本结构 . . . 132
（一） 数据通路的定义与组成元件 . . . 132
（二） 单总线结构的数据通路 . . . 133
（三） 多总线结构与专用数据通路 . . . 133
（四） 数据通路的数据传送过程与时钟周期数计算 . . . 133
四、 控制器的功能和工作原理 . . . 134
（一） 时序与控制 . . . 134
（二） 硬布线控制器 . . . 134
（三） 微程序控制器 . . . 135
（四） 微指令的格式与编码方式 . . . 136
（五） 微地址的形成方式 . . . 137
（六） 微指令条数与控制存储器容量的计算 . . . 138
五、 异常和中断机制 . . . 139
（一） 异常和中断的基本概念与区别 . . . 139
（二） 异常和中断的分类 . . . 139
（三） 异常和中断的检测与响应 . . . 140
（四） 多重中断与中断屏蔽 . . . 141
六、 指令流水线 . . . 143
（一） 指令流水线的基本概念 . . . 143
（二） 指令流水线的性能计算 . . . 144
（三） 结构冒险、数据冒险和控制冒险的处理 . . . 145

（四） 超标量和动态流水线的基本概念 . . . 147
七、 多处理器基本概念 . . . 148
（一） Flynn 分类法与向量处理器 . . . 148
（二） 硬件多线程 . . . 149
（三） 多核处理器 . . . 150
（四） 共享内存多处理器（SMP）与 UMA/NUMA、Cache 一致性问题 . . . 150
本章小结表 . . . 151
易错清单 . . . 152

第六章 总线和输入/输出系统 **154**
一、 总线和输入/输出系统 . . . 154
二、 总线 . . . 154
（一） 总线的基本概念 . . . 154
（二） 总线的组成及性能指标 . . . 156
（三） 总线事务和定时 . . . 159
三、 I/O 接口（I/O 控制器） . . . 162
（一） I/O 接口的功能和基本结构 . . . 162
（二） I/O 端口及其编址 . . . 163
四、 I/O 方式 . . . 164
（一） 程序查询方式 . . . 164
（二） 程序中断方式 . . . 166
（三） DMA 方式 . . . 172
（四） 三种 I/O 方式的完整对比 . . . 176
本章小结表 . . . 177
易错清单 . . . 178

第一章计算机系统概述

本章考纲要求

第二部分计算机组成原理［考查目标］

1. 掌握单处理器计算机系统中主要部件的工作原理、组成结构以及相互连接方式。
2. 掌握指令集体系结构的基本知识和基本实现方法，对计算机硬件相关问题进行分析，并能够对相关部件进行设计。
3. 理解计算机系统的整机概念，能够综合运用计算机组成的基本原理和基本方法，对高级编程语言（C 语言）程序中的相关问题进行分析，具备软硬件协同分析和设计能力。

一、计算机系统概述（一）计算机系统层次结构

1. 计算机系统的基本组成
2. 计算机硬件的基本结构
3. 计算机软件和硬件的关系
4. 计算机系统的工作原理

” 存储程序” 工作方式，高级语言程序与机器语言程序之间的转换，程序和指令的执行过程。（二）计算机性能指标吞吐量、响应时间；CPU 时钟周期、主频、CPI、CPU 执行时间；MIPS、MFLOPS、GFLOPS、TFLOPS、PFLOPS、EFLOPS、ZFLOPS。

本章是《计算机组成原理》的开篇，也是全书的「总纲」。它只包含两个二级条目，但作用很关键：（一）层次结构负责建立整机概念，告诉你一台机器由什么组成、程序如何在它上面跑起来；（二）性能指标负责建立定量语言，统考中所有关于「快慢」的计算题，最终都要落回 $T_{CPU} = IC \times CPI \times T_c$ 这一个式子上。

从历年统考来看，本章的命题非常有规律：层次结构部分几乎固定出一道文字辨析选择题（冯 · 诺依曼结构、ISA 与微架构的界限、语言与翻译程序）；性能指标部分是计算选择题的必考区，还可能出现在综合应用题第一小问（要求算出主频、CPI、MIPS、执行时间）。字长部分则是近年的新热点，2020、2021 年连续以选择题和大题小问的形式考查。

本章的写法是：先把概念的定义边界说清楚（辨析题的命门），再把每个公式推导一遍并代入数字验证（计算题的命门），最后用小结表和易错清单收口。

从分值分布上看，本章单独成题一般占 2 分选择题 1 ~ 2 道，但它的公式是后面《存储器层次结构》《中央处理器》《指令系统》三章的公共工具：算 Cache 缺失对 CPI 的影响要用 $T_{CPU} = IC \times CPI \times T_c$；算流水线加速比要用「周期数 × 周期长」；算总线带宽要用「一次传输位数 ÷ 传输周期」。所以本章不能只当「背概念」的一章来读，公式必须练到能闭卷推导、闭卷验算。以下所有公式都会给出推导链条和一组带数字的验证，读者应当自己动手把每个验证算一遍。

一、计算机系统的基本组成

（一）计算机系统的构成

计算机系统（**Computer System**）由硬件（**Hardware**）和软件（**Software**）两部分组成，两者缺一不可，只有硬件的机器叫「裸机」，什么都干不了。

- 硬件：有形的物理实体，是计算机系统的物理基础。包括主机箱内的 CPU、主存、主板、总线，以及外部的显示器、键盘、鼠标、硬盘等。
- 软件：由具有各类特殊功能的信息（程序）组成，分为系统软件和应用软件。

分类	定义	典型例子
系统软件	管理整个计算机系统，使系统资源得到合理调度、高效运行	操作系统（OS）、数据库管理系统（DBMS）、语言处理程序（编译/汇编/解释）、系统服务程序、标准库
应用软件	为完成用户特定任务而编写的软件	微信、WPS、Chrome、各类仿真软件、用户自己写的 C 程序

硬件与软件的分工

硬件是物理基础，软件是灵魂。一台计算机的性能既取决于硬件的物理极限（主频、存储带宽），也取决于软件对硬件的利用效率（编译器优化、算法复杂度）。统考反复出现的一道题就是「下列哪项能缩短程序执行时间」，答案往往三项全能：提高主频（改 T_c）、优化数据通路（改 CPI）、编译优化（改 IC）——因为执行时间公式里三个因子各占一个。

（二）计算机硬件系统的五大部件

按冯 · 诺依曼结构，计算机硬件由五大部件组成，这是必须背下来的一条：

1. 运算器（**Arithmetic Unit / ALU** 主体）：对数据进行加工处理，完成算术运算和逻辑运算。
2. 控制器（**Control Unit, CU**）：指挥各部件按指令要求协调工作，是整机的指挥中心。
3. 存储器（**Memory**）：存放程序和数据。分主存（内存）和辅存（外存）。
4. 输入设备（**Input Device**）：把人们熟悉的信息形式（文字、图像）转换为计算机能识别的二进制形式，如键盘、鼠标、扫描仪。
5. 输出设备（**Output Device**）：把计算机内部的二进制结果转换成人能识别的形式，如显示器、打印机。

在这五大部件之上，统考又习惯再加两个组合概念：

组合概念	构成	说明
CPU（中央处理器）	运算器 + 控制器	有时把 CPU 内部的寄存器（通用寄存器、PC、IR、MAR、MDR、PSW）也归入 CPU
主机	CPU + 主存（内存）	注意：不含外存。硬盘是辅存，属于外设，不属于主机

「主机」的范围

「主机 = CPU + 主存」是统考的固定口径。硬盘、光驱、键盘、显示器都不属于主机，它们属于 I/O 设备。见到「主机包括 CPU、主存和硬盘」这种说法，一律判错。另外，「CPU = 运算器 + 控制器」也要连同寄存器一起理解：现代计组把寄存器组看成控制器和运算器共同使用的部件，所以「CPU = 运算器 + 控制器 + 寄存器」也不算错，但严格按考纲原文写「CPU 由运算器和控制器组成」最保险。

（三）各部件的工作关系（数据通路初貌）

把五大部件连起来看，一次程序运行就是一条不断循环的信息流：

- 取指令：PC（程序计数器）给出指令地址 → MAR → 主存 → 取出指令 → MDR → IR（指令寄存器）。
- 分析指令：IR 中的操作码送控制器译码，地址码送地址形成部件。
- 执行指令：控制器发出控制信号，运算器从寄存器或主存取数运算，结果写回寄存器或主存；若为 I/O 指令，则由 I/O 接口驱动外设。
- **PC** 更新：顺序执行时 PC+1（或 + 指令字长对应的字节数），转移时由转移地址改写 PC。

这条链路的具体时序会在《中央处理器》一章展开，这里只要记住一个结论：任何一条指令的执行，都可以拆成「取指—译码—执行」三段，其中取指阶段永远第一个动作。

在这条链路上，有四个寄存器是必须在第一章就记住位置的，后面的所有大题都默认你熟悉它们：

寄存器	全称	作用	谁给它赋值
PC	程序计数器 Program Counter	存放下一条要执行指令的地址	取指后自动增量；转移指令在执行阶段改写
IR	指令寄存器 Instruction Register	存放当前正在执行的那条指令	由 MDR 送来的指令装入
MAR	主存地址寄存器	存放将要访问的主存单元地址	由 PC 或指令的地址码装入
MDR	主存数据寄存器	存放刚从主存读出或将要写入主存的数据	由主存或 CPU 装入

有一个极易混淆的点：**PC** 里装的是「下一条」指令的地址，不是「当前」指令的地址。当前正在执行的指令已经送进 IR 了，PC 在取指阶段结束时就已自增。这正是「顺序执行」与「转移」能够统一起来的原因——顺序执行时 PC 自增即指向下一条；转移指令在执行阶段把目标地址写进 PC，下一条就跳到别处去了。

二、计算机硬件的基本结构

（一）冯 · 诺依曼结构

冯 · 诺依曼结构（**von Neumann Architecture**）由美籍匈牙利数学家冯 · 诺依曼于 1945 年提出，核心思想是把程序当作数据一样存进存储器，让机器自动连续地执行。它的要点有四条：

1. 计算机由运算器、控制器、存储器、输入设备、输出设备五大部件组成。
2. 指令和数据都用二进制数表示，形式上无差别，存放在同一个存储器中（因此存储器既存指令又存数据，称为「指令和数据同等地位」）。
3. 程序执行前，指令和数据需预先存放在存储器中，这就是「存储程序（**Stored-Program**）」原理。
4. 指令按地址顺序访问，由 PC 自动指向下一条指令，程序可自动连续执行。

除此之外，原始的冯 · 诺依曼结构以运算器为中心：输入设备、输出设备、存储器三者之间的数据传送都要经过运算器转运，运算器既是计算部件又是数据中转站，这就形成了性能瓶颈。

冯 · 诺依曼结构的四句话口诀

五大部件、二进制表示、存储程序、按地址顺序执行。加上「早期以运算器为中心」和「指令与数据形式上无差别」两条限定，就覆盖了全部辨析题的选项来源。2009-11、2019-12 都直接考的这里。

冯 · 诺依曼结构的三个高频错项

- 「**CPU** 区分指令和数据的依据是它们的存储单元不同」——错。两者在同一个存储器里，存储形式上无差别。
- 「数据都在指令中直接给出」——错。只有立即寻址的立即数才在指令中直接给出，绝大多数操作数要按地址去存储器或寄存器里取。
- 「**CPU** 区分指令和数据的依据是操作码的译码结果」——错。译码发生在取指之后，而「取出来的是指令还是数据」这件事在取指阶段就已经由指令周期的不同阶段决定了。正确答案是「指令周期的不同阶段」。

关于「CPU 如何区分指令和数据」，正确的理解是按阶段而非按内容：

阶段	从存储器取出的内容	依据
取指周期（Fetch）	一定是指令	PC 提供的地址；此刻取出的内容被当作指令处理
执行周期（Execute）	可能是数据（操作数）	由指令的地址码给出的地址；可能是读操作数，也可能是写结果

（二）现代计算机以存储器为中心

原始的冯 · 诺依曼结构以运算器为中心，I/O 与主存之间的数据传送要绕道运算器，效率低。现代计算机改为以存储器为中心：

- 主存直接与 CPU 通过存储器总线交换信息；
- I/O 设备通过 I/O 接口（I/O 总线）直接与主存交换数据，不再经过运算器；
- 运算器专心负责运算，只与主存和寄存器打交道。

这个改动的直接好处是数据传送不再占用运算器，运算器和主存可以并行工作，整机效率显著提高。所以在「以谁为中心」的选择题里：

- 问「冯 · 诺依曼原始结构以什么为中心」→ 运算器；
- 问「现代计算机以什么为中心」→ 存储器。

（三）计算机的层次结构

计算机系统是一个多层次的层级系统，从最底层的硬件到最顶层的高级语言用户，可以这样分层：

层级	名称	由谁实现	使用者看到的内容
第 0 级	数字逻辑层（硬联逻辑）	门电路	高低电平
第 1 级	微程序机器层	微程序（固化在 ROM 中）	微指令
第 2 级	传统机器层（机器语言层）	硬件直接执行	机器指令
第 3 级	操作系统层	操作系统程序	系统调用、进程、文件
第 4 级	汇编语言层	汇编程序翻译	汇编助记符
第 5 级	高级语言层	编译程序翻译	C、Java 等

层级	名称	由谁实现	使用者看到的内容
第 6 级	应用语言层	应用软件	应用问题本身的描述

请注意一个重要事实：第 **0**～**2** 级是硬件实现的，第 **3**～**6** 级是软件实现的，而第 1 级（微程序机器层）是一个特殊的「软硬交界」层：它由硬件解释执行，但解释的对象是软件写出来的微程序。

（四）指令集体系结构（**ISA**）与微架构的界限

指令集体系结构（**Instruction Set Architecture, ISA**）是软件和硬件之间的界面，是程序员（或编译器编写者）所能看到的计算机属性，规定「做什么」；微架构（**Microarchitecture**）是 ISA 的具体实现，规定「怎么做」。

属于 **ISA**（程序员可见）	属于微架构（实现细节，程序员不可见）
指令字格式、指令类型、寻址方式	CPU 的时钟周期、主频
数据类型、数据表示（如 IEEE 754）	加法器的进位方式（串行/先行进位）
通用寄存器个数和位数	Cache 的容量、组相联路数、替换算法
程序可访问的寄存器 PC、PSR 等	数据通路的具体结构、流水线级数
是否采用虚拟存储、存储编址方式	微指令编码方式、控制器的具体实现

ISA 辨析题的固定套路

2022-18 的题干给四项，要求选出属于 ISA 的：Ⅰ指令字格式和指令类型（属于）、Ⅱ CPU 的时钟周期（不属于）、Ⅲ通用寄存器个数和位数（属于）、Ⅳ加法器的进位方式（不属于），答案「仅Ⅰ、Ⅲ」。判别口诀：凡是「程序员写程序时必须知道、不知道就写不出正确程序」的，属于 **ISA**；凡是「换了它程序照样正确跑，只是快慢不同」的，属于微架构。时钟周期、进位方式、Cache 容量都只影响快慢，不影响正确性，所以都不属于 ISA。

三、计算机软件和硬件的关系

（一）软件与硬件的逻辑等价性

软件和硬件在逻辑功能上是等价的。这句话的含义是：同一个功能，既可以用硬件实现，也可以用软件实现。

- 用硬件实现：速度快、成本高、不灵活（功能固化后难以更改）。
- 用软件实现：速度慢、成本低、灵活（改代码就能改功能）。

举例：乘法运算可以用硬件乘法器一个周期算完，也可以用程序「移位—相加」循环实现；浮点运算可以用专用的浮点运算部件，也可以用软件模拟；甚至一条复杂指令的功能，既可以用一条硬布线指令实现，也可以编译成若干条简单指令序列实现。

由此推出两条重要延伸：

1. 软硬件之间的界限不是固定不变的，随着器件成本和技术发展，两者可以互相转化。多数功能早期由软件实现，后来移到硬件；也有功能为了灵活性从硬件移回软件。
2. 软件的功能可以用硬件来实现，硬件的功能可以用软件来模拟，所以一台机器只要 ISA 兼容，用不同软硬件划分方式实现的机器在逻辑上是等价的。

（二）固化的概念

固化（**Firmware**）是指把软件写入只读存储器（**ROM**、**PROM**、**EPROM**、**EEPROM**、**Flash**）中，使其成为硬件的一部分，这种固化了的软件称为固件（**Firmware**）。

- 固化之后，这段程序不需要再被装入主存（或只需映射到地址空间），上电即可运行，速度更快、掉电不丢失。
- 但它的本质仍然是软件：它是程序，可以被读出来分析、可以通过重新烧写来修改（对可改写 ROM 而言）。
- 微程序就属于典型的固件：它「用硬件的方式解释执行」，但内容是软件写的微指令序列。

一句话区分

「固化」只是改变了软件的存放介质和调用方式，没有改变它是软件这一事实。因此「固化后的程序就变成了硬件」是错的；「固件是软件和硬件的结合体」是可以接受的说法。

软硬件关系的三个常见错项

- 「硬件实现的性能一定优于软件实现」——错，要看具体功能和实现成本，且这句话的绝对化表述本身就有问题。
- 「软硬件在物理上是等价的」——错，等价的是逻辑功能，不是物理实体。
- 「软件可以脱离硬件独立运行」——错。

四、计算机系统的工作原理

（一）「存储程序」工作方式

存储程序（**Stored Program**）工作方式是冯 · 诺依曼结构的核心，包含两层含义：

1. 事先编制程序：把要解决的问题编成由指令构成的程序。
2. 事先存储程序：把程序（指令序列）和原始数据一起送入主存，然后启动计算机，机器就能自动、连续地逐条取指执行，直到程序结束，中间不需要人工干预。

「自动连续」之所以能实现，靠的是三件事：

- **PC** 自动增量：每取完一条指令，PC 自动指向下一条；
- 指令按地址顺序存放：程序在存储器中连续排列；
- 转移指令可改写 **PC**：实现分支和循环，使「顺序执行」变成「可控制流的执行」。

（二）高级语言程序与机器语言程序之间的转换

这是本章文字题的另一个高频区，要能准确说出每一步由谁完成、产物是什么。

*1.*三种语言与三种翻译程序

语言层次	说明	硬件能否直接执行	翻译程序
机器语言	二进制编码的指令和数据	能，唯一能直接执行的	不需要翻译

语言层次	说明	硬件能否直接执行	翻译程序
汇编语言	用助记符（如 ADD、MOV）代替二进制操作码	不能	汇编程序（**Assembler**）
高级语言	C、C++、Java 等，与机器无关	不能	编译程序或解释程序

两条容易混的边界：

- 汇编语言与机器指令几乎一一对应，一条汇编指令翻译成一条机器指令，所以汇编语言是「机器相关的」，换一种 ISA 就要重写。
- 高级语言与机器无关，同一份 C 源程序经过不同编译器可以生成 x86、ARM、MIPS 等不同 ISA 的目标代码。这也是「可移植性」的来源。
- 硬件描述语言（如 **VHDL**、**Verilog**）不是「程序」，它描述的是电路的结构与行为，用于设计和综合硬件，CPU 不能直接执行它。2015-12 专门用这个选项做干扰。
- 汇编程序（**Assembler**）：把汇编语言源程序翻译成机器语言目标程序。
- 编译程序（**Compiler**）：把高级语言源程序整体翻译成汇编语言或机器语言目标程序，先翻译后执行，生成可独立运行的目标文件。
- 解释程序（**Interpreter**）：逐句读入高级语言源程序，边翻译边执行，不生成独立的目标代码文件，执行效率通常低于编译。

汇编程序 vs 编译程序

「将高级语言源程序转换为机器目标代码文件的程序」→ 编译程序（2016-12）。「汇编程序」的输入是汇编语言源程序，不是高级语言。这两个选项几乎每年都在同一道题里出现，是最经典的干扰项。另外注意「链接程序」只做目标文件的合并与重定位，不做语言翻译。

2.C 语言程序的完整转换过程

现代统考按 GCC 的流程来考（2022-20 就是直接考这四步的顺序），完整过程是：

预处理 → 编译 → 汇编 → 链接

步骤	由谁完成	输入	输出	做了什么事
预处理	预处理器 `cpp`	`.c`	`.i`	展开 `#include` 头文件、替换 `#define` 宏、处理条件编译
编译	编译器 `cc1`	`.i`	`.s`（汇编）	词法/语法/语义分析、优化、生成汇编代码
汇编	汇编器 `as`	`.s`	`.o`（可重定位目标文件）	把汇编指令翻译成机器指令，生成符号表与重定位信息
链接	链接器 `ld`	多个`.o`＋库	可执行文件	符号解析、地址重定位、把库函数合并进来

从语言层次看则是：高级语言源程序 →（编译）→ 汇编语言程序 →（汇编）→ 机器语言目标程序 →（链接）→ 可执行目标程序。只有最后一步产物才能被硬件直接执行，且必须装入主存后才能运行。

语言转换的三类考法

- **2015-12**：硬件能直接执行的语言 → 仅机器语言程序（汇编语言、硬件描述语言都不能）。

- **2016-12**：高级语言源程序 → 机器目标代码文件的程序 → 编译程序。
- **2022-20**：高级语言源程序 → 可执行目标文件的主要过程 → 预处理 → 编译 → 汇编 → 链接。

（三）程序和指令的执行过程

*1.*程序与指令的关系

- 指令（**Instruction**）：计算机执行某种操作的命令，是硬件能理解的最小功能单位。一条指令由操作码（做什么）和地址码（对谁做）两部分组成。
- 程序（**Program**）：完成特定任务的一条指令序列。
- 因此，程序的执行过程 = 指令的逐条执行过程。

*2.*指令执行的三个基本阶段

每条指令的执行都可以划分为：

1. 取指令（**Fetch**）：按 PC 的地址从主存取指令，送入 IR，同时 PC 自动增量。
2. 分析指令（**Decode**）：对 IR 中的操作码译码，判断要做什么操作；地址码送地址形成部件算出操作数地址。
3. 执行指令（**Execute**）：按操作码发出控制信号，完成运算或数据传送，必要时写回结果并更新 PC。

用寄存器级语言写出来，就是教科书上最经典的一段。为了贴近统考习惯，这里用通用的寄存器传送记号（`(X)` 表示 X 的内容，`M(addr)` 表示主存 addr 单元的内容），用汇编语言风格标注：

```
; 取指阶段（所有指令共有）
    (PC) -> MAR
    M(MAR) -> MDR           ; 从主存取指令
    (MDR) -> IR             ; 指令送指令寄存器
    (PC) + 1 -> PC          ; 顺序执行时 PC 增量

; 分析阶段
    OP(IR) -> CU            ; 操作码送控制器译码
    Ad(IR) -> MAR           ; 地址码送 MAR

; 执行阶段（以取数指令 LOAD 为例）
    M(MAR) -> MDR           ; 从主存取操作数
    (MDR) -> ACC            ; 送入累加器
```

这段伪代码的每一步都对应一条微操作，而微操作的执行就是数据在部件之间的一次传送。要注意两点：一是「`(PC) + 1 -> PC`」中的 1 在按字节编址且指令字长多字节时，应当是「指令所占的字节数」，本章按单字长指令简化；二是不同指令的取指和分析阶段完全相同，只有执行阶段不同，这为后面学习「指令周期的划分」和「流水线」埋下伏笔。

一次程序的执行，就是「取指—分析—执行」这个循环不断重复，直到遇到停机指令或程序结束。注意取指阶段是所有指令共有的，这是「CPU 靠指令周期的不同阶段区分指令和数据」这一结论的硬件基础。

三个必须记住的执行顺序结论

1. 取指永远是第一步，任何指令都必须先取出自己。
2. **PC** 的增量发生在取指阶段（取出指令后立即 +1 或 + 指令字节数），所以转移指令在执行阶段改写 PC 才是「有效」的。

3. 操作码译码发生在分析阶段，因此「靠译码结果区分指令和数据」在时序上是说不通的。

*3.*一个完整的执行流程实例

用一个具体的小程序把整条链路走通。设主存按字节编址，指令字长为 2 字节，程序从地址 1000 开始，内容是：

```
; 地址   指令              含义
; 1000   LOAD  R1, [2000]  把主存 2000 单元的数取到 R1
; 1002   ADD   R1, [2004]  R1 加上主存 2004 单元的数
; 1004   STORE [2008], R1  把 R1 写回主存 2008 单元
; 1006   HALT              停机
```

执行的第一轮循环（对应 1000 处的 LOAD）依次发生：

步骤	**PC** 值	发生的事
取指开始	1000	MAR ← 1000，从主存读出 1000、1001 两个字节
取指结束	**1002**	指令装入 IR，**PC** 自动 **+2**
分析	1002	操作码送控制器译码为「取数」；地址码 2000 送 MAR
执行	1002	从主存 2000 读数，经 MDR 送入 R1
下一轮	1002	取第 2 条指令（ADD）

关键观察：在取指结束时，**PC** 已经从 **1000** 变成 **1002**，此时 PC 指向的是下一条指令而不是当前指令。第二轮结束后 PC 变为 1004，第三轮后变为 1006，第四轮 HALT 被取出后程序停止。

如果把 ADD 换成 JMP 1000（无条件转移），则在执行阶段会把 1000 写入 PC，于是程序进入死循环——这就是「转移指令通过改写 PC 改变执行顺序」的直观体现。

程序装入主存后才可执行

「程序存放在磁盘上」和「程序能被 CPU 执行」是两件事。可执行文件必须由操作系统装入主存（或映射到进程地址空间）之后，CPU 才能按 PC 的地址去取指令。这也是「存储程序」与「程序装入」的分工：前者是机器的工作方式，后者是操作系统的职责。

五、性能评价的两个基本维度

（一）响应时间

响应时间（**Response Time**），又称执行时间（**Execution Time**），指从作业提交开始到作业完成所经过的总时间。

它的构成是：

$$T_{总} = T_{CPU} + T_{等待}$$

其中 T_{CPU} 是 CPU 真正花在运行该程序上的时间，$T_{等待}$ 包括磁盘访问、主存访问、I/O 操作、操作系统调度开销等。

性能与响应时间成反比：

$$\text{性能} = \frac{1}{\text{执行时间}}$$

适用场景：个人电脑、银行事务处理、实时系统——关心「这件事多久办完」。考试中的「程序执行时间」「**CPU** 时间」都属于这一维度。

（二）吞吐量

吞吐量（**Throughput**），又称带宽（**Bandwidth**），指单位时间内系统能处理的请求数或任务数。

- 适用场景：数据中心、Web 服务器、多媒体流——关心「单位时间能处理多少事」。
- 注意：吞吐量与响应时间不是同向变化的。例如把 CPU 换成更快的处理器，单个任务响应时间缩短，吞吐量通常上升；但如果在原系统上增加 I/O 缓冲和多道程序度，吞吐量可能显著上升，而单个任务的响应时间反而变长（因为任务之间要竞争资源）。

响应时间与吞吐量的关系

缩短响应时间的措施不一定提高吞吐量，提高吞吐量的措施也可能拉长响应时间。典型例子：用「排队后集中批处理」能提高吞吐量，但单个请求的响应时间变长。两者是不同维度的指标，不能互相替代。

六、CPU 时钟周期、主频、CPI 与 CPU 执行时间

这是本章的核心计算区，所有公式都要能独立推导。

（一）时钟周期与主频

- 时钟周期（**Clock Cycle / Clock Tick**）T_c：CPU 中最基本、最小的时间单位。在一个时钟周期内，CPU 只能完成一个最基本的动作（如一次寄存器传送）。
- 主频（**Clock Rate / Frequency**）f：时钟周期的倒数，表示每秒钟有多少个时钟周期。

$$f = \frac{1}{T_c}, \qquad T_c = \frac{1}{f}$$

单位换算（速率类，按 10^3 进制）：

$$1\ \text{Hz} = 1\ \text{次/秒}, \quad 1\ \text{kHz} = 10^3\ \text{Hz}, \quad 1\ \text{MHz} = 10^6\ \text{Hz}, \quad 1\ \text{GHz} = 10^9\ \text{Hz}, \quad 1\ \text{THz} = 10^{12}\ \text{Hz}$$

数字验证：$f = 1\ \text{GHz} = 10^9\ \text{Hz}$，则

$$T_c = \frac{1}{10^9\ \text{Hz}} = 10^{-9}\ \text{s} = 1\ \text{ns}$$

即 **1 GHz ↔ 1 ns**。同理 $f = 2\ \text{GHz} \Rightarrow T_c = 0.5\ \text{ns}$；$f = 1.5\ \text{GHz} \Rightarrow T_c = \frac{1}{1.5\times10^9} \approx 0.667\ \text{ns}$。

主频与时钟周期的速算

记牢这一组对应关系，考场上能省下大量时间：**1 GHz ↔ 1 ns**，**2 GHz ↔ 0.5 ns**，**4 GHz ↔ 0.25 ns**，**1 MHz ↔ 1 μs**，**1 kHz ↔ 1 ms**。反过来，时钟周期 $T_c = 2\ \text{ns}$ 时，$f = 1/(2\times10^{-9}) = 500\ \text{MHz}$。

（二）CPI

CPI（**Cycles Per Instruction**）：执行一条指令平均所需的时钟周期数。

$$\text{CPI} = \frac{\text{程序执行所需的总时钟周期数}}{\text{程序包含的指令条数}IC}$$

为什么是「平均」？不同类型的指令所需周期数不同：一条寄存器间加法可能只要 1 个周期，一次访存可能要几十个周期。因此 CPI 通常是加权平均值：

$$\text{CPI} = \sum_{i=1}^{n} \left(\text{CPI}_i \times \frac{IC_i}{IC} \right)$$

其中 CPI_i 是第 i 类指令的周期数，$\frac{IC_i}{IC}$ 是第 i 类指令在程序中所占的比例，且 $\sum \frac{IC_i}{IC} = 1$。

IPC（Instructions Per Cycle）是 CPI 的倒数：$IPC = 1/CPI$。现代处理器用流水线和多发射技术，IPC 可以大于 1（例如 4 发射的理想流水线 IPC 最高为 4）。

加权平均 CPI 的两种写法

设各类指令的周期数为 C_i、条数为 IC_i，则

$$\text{CPI} = \frac{\sum_i IC_i\, C_i}{\sum_i IC_i} = \sum_i C_i \cdot \frac{IC_i}{IC}$$

分子是总周期数，分母是总指令数。两种写法等价，做题时哪种数据给全就用哪种。给「比例」用右边，给「条数」用左边。

CPI 的两个常见陷阱

- **CPI** 是平均值，不能把各类指令的 **CPI** 直接相加。必须先按比例加权。
- **CPI** 与主频无关。换一块更高主频的同架构 CPU，CPI 基本不变，只是每个周期变短，执行时间随之变短。所以「提高主频能降低 CPI」是错的。

（三）CPU 执行时间的三个等价公式

从「执行时间 = 总周期数 × 周期长」出发，可以推出统考必备的三个等价形式。

推导过程：

程序执行所需的总时钟周期数 = 指令条数 × 平均 CPI：

$$\text{总时钟周期数} = IC \times CPI$$

而执行时间 = 总时钟周期数 × 单个时钟周期长：

$$T_{CPU} = \text{总时钟周期数} \times T_c = IC \times CPI \times T_c \qquad (1)$$

把 $T_c = 1/f$ 代入，得到只含主频的形式：

$$T_{CPU} = \frac{IC \times CPI}{f} \qquad (2)$$

再把「总时钟周期数」记作 N（即 $N = IC \times CPI$），得到最简洁的形式：

$$T_{CPU} = \frac{N}{f} \qquad (3)$$

CPU 性能公式（必背）

$$T_{CPU} = IC \times CPI \times T_c = \frac{IC \times CPI}{f} = \frac{N}{f}$$

- IC：指令条数（Instruction Count），由指令集和编译器决定；
- CPI：平均每条指令的时钟周期数，由体系结构和指令集决定；
- T_c / f：时钟周期 / 主频，由硬件工艺决定。

> 三个因子分别由「编译器」「体系结构」「工艺」控制，这正是统考爱考的「下列措施能否缩短执行时间」的解题依据。

数字验证（三式互推）：$f = 1$ GHz，程序共执行 $IC = 10000$ 条指令，其中 80% 的指令平均需 1 个周期，20% 的指令平均需 10 个周期。

先算混合 CPI：

$$CPI = 0.8 \times 1 + 0.2 \times 10 = 0.8 + 2.0 = 2.8$$

再算执行时间。用式（1）：$T_c = 1$ ns，

$$T_{CPU} = 10000 \times 2.8 \times 1 \text{ ns} = 28000 \text{ ns} = 28\ \mu\text{s}$$

用式（2）验证：

$$T_{CPU} = \frac{10000 \times 2.8}{1 \times 10^9 \text{ Hz}} = \frac{28000}{10^9} \text{ s} = 2.8 \times 10^{-5} \text{ s} = 28\ \mu\text{s}$$

用式（3）验证：$N = IC \times CPI = 10000 \times 2.8 = 28000$ 个周期，

$$T_{CPU} = \frac{28000}{10^9 \text{ Hz}} = 28\ \mu\text{s}$$

三式结果一致。这就是 **2022-12** 的原题（答案：平均 CPI 为 2.8，执行时间 28 μs）。

（四）用比例法快速解题

当题目只给「变化比例」而不给绝对值时，用比例法最快。由于 $T_{CPU} = IC \times CPI \times T_c$ 是乘积关系，各因子的变化倍数直接相乘：

$$\frac{T_{新}}{T_{旧}} = \frac{IC_{新}}{IC_{旧}} \times \frac{CPI_{新}}{CPI_{旧}} \times \frac{T_{c,新}}{T_{c,旧}}$$

数字验证（**2014-12**）：程序 P 在机器 M 上执行时间 20 s，编译优化后指令数减少到原来的 70%，CPI 增加到原来的 1.2 倍，机器未变（T_c 不变）。

$$\frac{T_{新}}{T_{旧}} = 0.7 \times 1.2 \times 1 = 0.84$$

$$T_{新} = 20 \times 0.84 = 16.8 \text{ s}$$

答案 16.8 s。

（五）两台机器执行时间之比

当两台机器 **ISA** 相同、运行同一个程序时，指令条数 IC 相同，比值式可以大幅简化：

$$\frac{T_{CPU,A}}{T_{CPU,B}} = \frac{IC \times CPI_A / f_A}{IC \times CPI_B / f_B} = \frac{CPI_A}{CPI_B} \times \frac{f_B}{f_A}$$

数字验证（**2017-12**）：M1 主频 1.5 GHz、平均 CPI 为 2；M2 主频 1.2 GHz、平均 CPI 为 1。两者 ISA 相同，运行同一基准程序。

$$\frac{T_{M1}}{T_{M2}} = \frac{2}{1} \times \frac{1.2}{1.5} = 2 \times 0.8 = 1.6$$

答案 1.6。交叉验证：设 $IC = 10^7$，

$$T_{M1} = \frac{10^7 \times 2}{1.5 \times 10^9} \approx 13.33 \text{ ms}, \qquad T_{M2} = \frac{10^7 \times 1}{1.2 \times 10^9} \approx 8.33 \text{ ms}$$

$$\frac{13.33}{8.33} = 1.6 \quad \checkmark$$

比值题的分子分母

「A 比 B」与「B 比 A」互为倒数，看题时一定要圈出「谁比谁」。另外主频高的机器不一定更快：M1 主频 1.5 GHz 比 M2 的 1.2 GHz 高，但 M1 的 CPI 是 2，M2 只有 1，结果 M1 反而慢 1.6 倍。这就是「主频不是性能的唯一决定因素」的标准范例。

CPU 时间的真题分布

2010-12（缩短执行时间的措施）、2012-12（CPU 提速与 I/O 时间）、2013-12（求 MIPS）、2014-12（比例法）、2017-12（两机时间比）、2022-12（平均 CPI 与执行时间）、2023-12（指令执行速度与用户 CPU 时间）。这条线的题目几乎每年一道，且全部围绕同一个公式。

七、MIPS 与 FLOPS 类指标

（一）MIPS

MIPS（Million Instructions Per Second）：每秒执行多少百万条指令。

$$\text{MIPS} = \frac{IC}{T_{CPU} \times 10^6} = \frac{IC}{\dfrac{IC \times CPI}{f} \times 10^6} = \frac{f}{CPI \times 10^6}$$

其中 f 的单位是 Hz。若把 f 直接以 MHz 为单位，则式子简化为：

$$\text{MIPS} = \frac{f(\text{MHz})}{CPI}$$

数字验证（**2013-12**）：主频 1.2 GHz，四类指令比例与 CPI 如下表。

指令类型	所占比例	**CPI**
A	50%	2
B	20%	3

指令类型	所占比例	**CPI**
C	10%	4
D	20%	5

第一步，加权平均 CPI：

$$CPI = 0.5 \times 2 + 0.2 \times 3 + 0.1 \times 4 + 0.2 \times 5 = 1.0 + 0.6 + 0.4 + 1.0 = 3$$

第二步，求 MIPS（把 1.2 GHz 化为 1200 MHz）：

$$\text{MIPS} = \frac{1200}{3} = 400$$

答案 400。

反向验证：$f = 1.2 \times 10^9$ Hz，$CPI = 3$，则每秒执行指令数 $= \frac{1.2\times10^9}{3} = 4 \times 10^8$ 条/秒 $= 400 \times 10^6$ 条/秒 $= 400$ MIPS。✓

再来一组对照验证，把「主频相同、MIPS 不同」这件事坐实：

机器	主频	平均 **CPI**	**MIPS**	每秒指令数
甲	2 GHz	1	$2000/1 = 2000$	2×10^9
乙	2 GHz	4	$2000/4 = 500$	5×10^8
丙	4 GHz	4	$4000/4 = 1000$	1×10^9

三行数据说明了三件事：主频相同而 **CPI** 不同，**MIPS** 差 **4** 倍（甲与乙）；主频翻倍可把 **MIPS** 翻倍（乙与丙）；**MIPS** 与 **CPI** 严格成反比（乙与丙 CPI 相同，MIPS 比 = 主频比）。记住这张表，MIPS 相关的四个选项就都能秒判。

（二）**GIPS** 与指令执行速度

与 MIPS 同类，**GIPS**（**Giga Instructions Per Second**）是每秒执行的十亿条指令数：

$$\text{GIPS} = \frac{f}{CPI \times 10^9} \qquad \text{(f 以 Hz 计)}$$

数字验证（**2023-12**）：主频 1.5 GHz、$CPI = 1.2$、程序指令条数 $IC = 5 \times 10^5$。

指令执行速度：

$$\text{GIPS} = \frac{1.5 \times 10^9}{1.2 \times 10^9} = 1.25$$

用户 CPU 时间：

$$T_{CPU} = \frac{IC \times CPI}{f} = \frac{5 \times 10^5 \times 1.2}{1.5 \times 10^9} = \frac{6 \times 10^5}{1.5 \times 10^9} = 4 \times 10^{-4}\ \text{s} = 0.4\ \text{ms}$$

答案 1.25 GIPS、0.4 ms。

（三）MIPS 的三条重要局限

MIPS 不能作为跨机器的绝对性能指标

1. **MIPS** 与 **CPI** 成反比：MIPS $= f/(CPI \times 10^6)$，CPI 越大 MIPS 越小。所以「MIPS 高」只说明「每秒执行了很多条指令」，不代表「算得快」。
2. **MIPS** 不能跨指令集比较。不同 ISA 的指令功能强弱差异极大：RISC 的一条简单指令可能只做一次加法，而 CISC 的一条复杂指令可能等价于 RISC 的 5 条指令。一台 **MIPS** 为 **100** 的 **CISC** 机器可能比 **MIPS** 为 **400** 的 **RISC** 机器实际更快，因为前者一条指令顶后者多条。
3. **MIPS** 对同一机器上的不同程序也不同。程序里访存指令比例高，CPI 就大，MIPS 就低。因此 MIPS 只能用于同一机器（或同一 **ISA** 系列）上的粗略对比。

MIPS 与 CPI 的关系式

$$\text{MIPS} = \frac{f}{CPI \times 10^6}$$

MIPS 与 **CPI** 成反比，与主频成正比。这是「MIPS 和 CPI 的反比关系」这一考点的全部内容。另外注意：未经特别说明时，MIPS 用的是混合 **CPI**（该程序在该机器上的平均 CPI），而不是某类指令的 CPI。

（四）FLOPS 系列指标

FLOPS（**Floating-point Operations Per Second**）衡量的是浮点运算速度，用于科学计算、超级计算机排名，比 MIPS 更能反映数值计算能力。

指标	全称	含义	数量级
MFLOPS	Million FLOPS	每秒百万（10^6）次浮点运算	10^6
GFLOPS	Giga FLOPS	每秒十亿（10^9）次浮点运算	10^9
TFLOPS	Tera FLOPS	每秒万亿（10^{12}）次浮点运算	10^{12}
PFLOPS	Peta FLOPS	每秒千万亿（10^{15}）次浮点运算	10^{15}
EFLOPS	Exa FLOPS	每秒百亿亿（10^{18}）次浮点运算	10^{18}
ZFLOPS	Zetta FLOPS	每秒十万亿亿（10^{21}）次浮点运算	10^{21}

注意这里全部按 10^3 进制（M→G→T→P→E→Z 每级 $\times 10^3$），这是与存储容量（按 2^{10}）的根本区别。

数字验证（**2021-12**）：神威 · 太湖之光浮点运算速度为 93.0146 PFLOPS，要求每秒完成的浮点操作次数。

$$1\ \text{PFLOPS} = 10^{15}\ \text{次/秒}$$

$$93.0146 \times 10^{15}\ \text{次/秒} \approx 9.3 \times 10^{16}\ \text{次/秒}$$

四个选项换算过来看：A 是 9.3×10^{13}（太小）、B 是 9.3×10^{15}（太小，差 10 倍）、C「9.3 千万亿次」$= 9.3 \times 10^7 \times 10^8$…按中文数量级，「亿亿」$= 10^8 \times 10^8 = 10^{16}$，所以 9.3 亿亿次 $= 9.3 \times 10^{16}$，与计算结果一致。答案 D。

中文数量级对照（考「亿亿次」这种说法时用）

- 万 $=10^4$，亿 $=10^8$，万亿 $=10^{12}$，千万亿 $=10^7\times10^8=10^{15}$（即 P），亿亿 $=10^8\times10^8=10^{16}$，百亿亿 $=10^2\times10^{16}=10^{18}$（即 E）。
- 判断技巧：把选项换算成 10 的幂，与 PFLOPS $=10^{15}$ 的数量级对齐即可，不必纠结中文措辞。

（五）速率进制 **vs** 容量进制（必考辨析）

这是本章最容易被「差 2.4%」坑到的点，必须在读题时第一秒分辨清楚。

场合	进制	换算
存储容量、编址范围、Cache 容量	按 2 的幂	1 KB $=2^{10}$ B $=1024$ B， 1 MB $=2^{20}$ B， 1 GB $=2^{30}$ B， 1 TB $=2^{40}$ B
速率、频率（主频、MIPS、FLOPS、总线带宽、传输速率）	按 10 的幂	1 kHz $=10^3$ Hz， 1 MHz $=10^6$ Hz， 1 GHz $=10^9$ Hz； 1 MB/s $=10^6$ B/s

量化对比：$2^{10}=1024$，而 $10^3=1000$，两者相差

$$\frac{1024-1000}{1000}=2.4\%$$

$2^{20}=1048576$ 与 $10^6=1000000$ 相差约 4.86%。单个单位差别不大，但跨多级相乘就会放大，命题人正是利用这一点设置「差不多但不相等」的干扰项。

主频换算绝不能用 **1024**

主频 1 GHz 就是 10^9 Hz，不是 $1024^3=2^{30}$ Hz。求时钟周期时用 $T_c=1/f$，1 GHz 对应 1 ns 是精确值，不能算成 0.931 ns（那是误按 2^{30} 的结果）。反过来，说「4 GB 内存」时是 4×2^{30} B $=2^{32}$ B，不是 4×10^9 B。

具体差异有多大？把同一串数字用两种进制算一遍，差距就很直观了：

单位	按 10 的幂	按 2 的幂	相差
K / k	$10^3=1000$	$2^{10}=1024$	+2.4%
M	$10^6=1000000$	$2^{20}=1048576$	+4.86%
G	10^9	$2^{30}=1073741824$	+7.37%
T	10^{12}	$2^{40}\approx1.0995\times10^{12}$	+9.95%

差别的规律是：每升一级，误差大约再加 **2.4～2.6** 个百分点。所以「512 GB 的固态硬盘」按厂商进制只有 512×10^9 B，而操作系统按 2^{30} 显示时会变成约 476.8 GB——这不是硬盘缩水，而是进制口径不同。

代入验证一遍：512×10^9 B $\div\,2^{30}=512\times10^9\div1073741824\approx476.84$。可见差距约 6.9%，与上表中 G 级的 7.37% 同量级。这也解释了为什么 8 GB 的 U 盘在电脑上只显示约 7.45 GB。

这个差异在带宽题里最致命。举一个必须分清的对照：

> 某系统以 1 MB/s 的速率把数据写入主存，同时主存中有 1 MB 的数据待读取。问各是多少字节?

- 「1 MB/s」是速率，按 10^3 进制：1 MB/s $= 10^6$ B/s，即每秒写入 1000000 字节。
- 「1 MB 数据」是容量，按 2^{10} 进制：1 MB $= 2^{20}$ B $= 1048576$ 字节。

两者写成同一个数字都是「1 MB」，实际相差 48576 字节（4.86%）。如果题目要求算「传完这 1 **MB** 数据需要多少秒」，正确做法是：

$$t = \frac{2^{20}\ \text{B}}{10^6\ \text{B/s}} = 1.048576\ \text{s}$$

若误把分子也按 10^6 算，就会得到整整齐齐的 1 s——那个「太整齐」的答案恰恰是错项的标志。同理，凡在一条式子里同时出现容量和速率，就要分别用各自进制换算后再相除，绝不能统一按一个进制处理。

判题时的操作准则可以归纳为一句话：

见到单位先问一句「这是容量还是速率」

容量、地址空间、**Cache** 大小、存储器芯片规格 → 用 2^{10} 系列（**1K=1024**，**1M=2^20**）；频率、主频、**MIPS**、**FLOPS**、带宽、传输速率 → 用 10^3 系列（**1k=1000**，**1M=10^6**）。例外说明：现代存储厂商标注硬盘容量时常用 10^3 进制，而统考里的「主存容量」一律按 2^{10} 进制理解；若题目明确写了换算关系，则以题目为准。

八、各种字长总结

字长是「硬件规格与性能的接口」，涉及机器字长、指令字长、存储字长、数据字长、地址字长等多个层次，2020、2021 年连续考查，属于必须彻底理清的一组概念。

（一）机器字长

机器字长（**Machine Word Length**）：**CPU** 在一次整数（定点整数）运算中能够处理的二进制数据的位数。

- 通常等于 **CPU** 中定点运算数据通路的宽度，也就是 **ALU** 的宽度和通用寄存器的位数。
- 32 位机器 / 64 位机器，说的就是机器字长。
- 机器字长直接影响数的表示范围和运算精度：字长越长，表示范围越大、精度越高。
- 机器字长通常取字节的整数倍（8、16、32、64 位）。

机器字长 ⇔ ALU 宽度 ⇔ 通用寄存器位数

机器字长 = **ALU** 宽度 = 通用寄存器位数，这三者在统考口径下是同一个数。但要注意它不等于指令寄存器（IR）的位数（那是指令字长），也不等于浮点寄存器的位数（浮点寄存器通常更宽，以保证精度）。

关于「字」这个概念的一个提醒：「字」的宽度不一定等于机器字长。字是某个系列机中固定的信息宽度计量单位。例如在 80x86 系列中，一个字的宽度是 16 位，所以 32 位是双字、64 位是四字；而在 IBM 303X 系列中，一个字是 32 位，16 位为半字、64 位为双字。统考默认按「字 = 机器字长」理解，除非题目另有说明。

（二）指令字长

指令字长（**Instruction Word Length**）：一条指令中所包含的二进制代码的位数。

- 取决于操作码的长度和操作数地址的个数及各地址的长度。
- 定长指令字机器：所有指令位数相同（目前定长指令字大多是 32 位）。
- 不定长指令字机器：指令有长有短，但每条指令的长度一般都是 **8** 的整数倍（因为存储器按字节编址，便于按字节存取）。

指令字长决定 **IR**（指令寄存器）的位数：若机器中指令字长最大为 32 位，则 IR 至少需要 32 位。

按指令长度与存储字长的关系，可把指令分为：

叫法	指令字长与存储字长的关系	取指需要访存次数
半字长指令	指令字长 = 存储字长 ÷ 2	1 次（取出后有半个单元冗余）
单字长指令	指令字长 = 存储字长	1 次
双字长指令	指令字长 = 存储字长 × 2	2 次

（三）存储字长与存储容量

存储字长（**Storage Word Length**）：一个存储单元所存储的二进制代码的位数。

- 一个存储单元对应一个地址。要读写某个存储单元的内容，必须给出它的地址。
- 主存容量 = 存储单元个数 × 存储字长，例如 1K × 8 位表示 1024 个单元，每个单元 8 位。

（四）编址单位与地址字长

编址单位（**Addressing Unit**）：一个地址所对应的存储单元的宽度。现代计算机绝大多数按字节编址，即编址单位为 8 位，每一个字节有一个地址。

这里有一个必须分清的区别，它决定了后面几章的一大类计算题：

概念	定义	由谁决定	典型值
编址单位	一个地址对应的存储单元宽度	指令集体系结构（ISA）规定	8 位（按字节编址）
存储字长	一个存储单元存放的二进制位数	存储器设计	8/16/32/64 位
存取宽度	一次从主存模块同时读写的位数	存储模块与总线匹配设计	64 位（8 片 ×8 位并行）
传输宽度	总线一次能传送的位数	总线设计（数据线条数）	= MDR 位数

按字节编址时，编址单位 = **8** 位，小于等于存储字长。这就带来两个后果：

1. 一个数据可能占多个存储单元。例如一个 32 位整数占 4 个字节、有 4 个地址；一个 64 位 `double` 占 8 个字节、有 8 个地址。
2. 一次访存取到的信息量取决于数据线宽度，而不是数据本身的长度。这正是一道经典统考题的考点：某机主存按字节编址，由 4 个 64M × 8 位 DRAM 芯片交叉编址构成，存储器总线宽 32 位，主存每次最多读写 32 位。`double` 型变量 x 的地址为 `804001AH`，则读取 x 需要几个存储周期？推导：`double` 占 8 B = 64 位，因此占 8 个连续地址单元，即 `804001AH` ~ `8040021H`。总线一次传 32 位 = 4 B，所以要传 $\lceil 8/4 \rceil = 2$ 次。但地址 `804001AH` 的低两位是 $10_2 = 2$，说明 x 不

是从 4 字节对齐的边界开始的，8 个字节跨在 3 个 4 字节行上（第 1 行只用到 2 个字节，第 2 行 4 个，第 3 行 2 个），因此需要 **3** 个存储周期。这个例子说明的是：编址单位决定「有多少个地址」，存取宽度（数据线宽度）决定「一次能搬多少」，两者不是同一个数。

地址字长 = **MAR** 的位数 = 地址线的根数，它决定了可寻址的空间大小：

$$\text{可寻址单元数} = 2^n \quad (n = \text{MAR 位数})$$

若按字节编址，则

$$\text{可寻址空间} = 2^n \text{ 字节}$$

寻址范围的三个连等式

$$\text{寻址范围} = 2^{\text{MAR 位数}} \text{ 个存储单元} = 2^{\text{地址线条数}} \text{ 个单元}$$

若按字节编址，则每个单元 = 1 B，于是

$$2^{\text{地址线条数}} \text{ B}$$

三个量（**MAR** 位数、地址线根数、地址字长）在统考里永远是同一个数，题目给哪个都能推出寻址范围。反之，给出寻址范围也能反推地址线根数：主存 4 GB 按字节编址，地址线需 $\log_2(4 \times 2^{30}) = 32$ 根。

（五）MAR、MDR 与地址线、数据线

部件	全称	位数由谁决定	说明
MAR	存储器地址寄存器	地址线条数	位数 = 地址线宽度，决定可寻址单元数 2^n
MDR	存储器数据寄存器	数据线条数	位数 = 数据线宽度，也就是一次访存能读写的数据位数

MDR 位数 ≠ 存储字长（这是最常错的等式）

MDR 的位数等于数据总线的宽度，与存储字长不一定相等。当存储字长大于数据线宽度时，一次访存需要分多个存储周期完成，MDR 只装得下一个「传输单位」。例如存储字长 32 位、数据线 16 位时，读一个存储单元要分 2 次传送。反过来，MAR 的位数一定等于地址线位数，这一点没有例外。

数字验证（**2021** 年综合题第一问）：某计算机 M 字长 16 位，按字节编址，CPU 与主存之间的地址线 20 位、数据线 8 位，采用 16 位定长指令字。求 ALU 宽度、可寻址主存空间大小、IR / MAR / MDR 的位数。

- **ALU** 宽度 = 机器字长 = **16** 位（ALU 一次处理一个机器字长）。
- 可寻址主存空间：地址线 20 位 ⇒ 单元数 2^{20}；按字节编址 ⇒ 每个单元 1 B，所以容量 $= 2^{20}$ B = 1 MB。
- **IR** 位数 = 指令字长 = **16** 位（指令字长决定 IR 位数）。
- **MAR** 位数 = 地址线位数 = **20** 位。
- **MDR** 位数 = 数据线位数 = **8** 位（不是 16 位，因为数据线只有 8 位）。

（六）数据字长

数据字长（**Data Word Length**）：存储、传输或处理一个数据时所用的二进制位数。

它和前面几个字长的关系是「具体值与一般值」的关系：

- 数据字长是具体的：说的是某一个数据占多少位，由数据类型决定。C 语言中 `char` 是 1 B（8 位），`short` 是 2 B（16 位），`int` 通常 4 B（32 位），`double` 是 8 B（64 位）。同一个程序里，不同数据的数据字长可以不同。
- 机器字长是一般的：说的是 CPU 一次定点整数运算能处理的位数，同一台机器上是一个固定值。
- 当数据类型宽度不超过机器字长时，一次运算即可处理完；当数据类型宽度大于机器字长时，需要分多次运算（例如 32 位机上做 64 位整数加法，要分低位、高位两次并处理进位）。

浮点数据的存储宽度可以超过机器字长，这正是「浮点寄存器宽度不等于机器字长」的原因（2020-12 的Ⅳ项）。所以数据字长这一概念的价值在于：它提醒你「数据的宽度」和「机器的宽度」是两件事，一个 64 位浮点数在 32 位机上照样能存、能算，只是要拆开来处理。

数据类型	C 语言典型宽度	数据字数（按字节编址占几个单元）
`char`	1 B	1 个字节单元
`short`	2 B	2 个字节单元
`int`	4 B	4 个字节单元
`double`	8 B	8 个字节单元

数据字长与机器字长的关系

数据字长可以小于、等于或大于机器字长。小于或等于时一次处理完；大于时需要拆成多段处理。统考里凡是问「某类型变量占几个存储单元」「读它需要几次访存」，考的都是数据字长与编址单位、存取宽度的配合关系。

（七）地址字长

地址字长（**Address Word Length**）：一个存储地址所占的二进制位数，也就是 MAR 的位数和地址线的根数。

- 它决定可寻址空间：n 位地址字长对应 2^n 个可寻址单元。
- 它与机器字长不一定相等。例如 32 位机器（机器字长 32 位）可以只配 20 位地址线，寻址 1 MB；也可以配 32 位地址线，寻址 4 GB。
- 统考口径：题中说「地址线 20 位」「MAR 20 位」「地址字长 20 位」都是同一件事，直接取 2^{20}。

机器字长 $\neq$ 地址字长

「32 位计算机」说的是机器字长 **32** 位（数据通路宽度），不能由此推断地址线也是 32 位。反过来，题目给了地址线根数就要用它算寻址范围，不要拿机器字长去算。2021 年的那道综合题正是「机器字长 16 位、地址线 20 位」的组合，两者不等，必须各用各的。

（八）字长之间的关系梳理

把这几个字长的约束关系理清，可以覆盖绝大多数选择题：

关系对	约束
存储字长 ↔ 机器字长	机器字长 ≥ 存储字长，通常是存储字长的整数倍（1、2、4 倍）。最常见是 1 倍（相等），这样一次访存即可取到能一次处理的数据
存储字长 ↔ 指令字长	指令字长可以是存储字长的 **0.5** 倍、**1** 倍、**2** 倍等（半字长/单字长/双字长指令）
指令字长 ↔ 机器字长	互不影响，没有任何约束关系。两者只是恰巧各自与存储字长成倍数关系
编址单位 ↔ 存储字长	按字节编址时编址单位为 8 位；存储字长可以等于或大于编址单位
MAR ↔ 地址线	位数相等
MDR ↔ 数据线	位数相等，与存储字长不一定相等
IR ↔ 指令字长	IR 位数 ≥ 最大指令字长

为什么「指令字长与机器字长无关」

机器字长强调「参与 **ALU** 运算的操作数位数」，而指令字长是「指令本身的位数」。有的指令没有参与 ALU 运算的操作数，甚至没有显式操作数（如空操作、停机），所以两者无法互相约束。记住一句话：机器字长看运算，指令字长看编码，两者只是各自与存储字长有倍数关系。

2020-12 的核心判据

题目问「位数一定与机器字长相同的部件」：Ⅰ ALU（是）、Ⅱ指令寄存器（不是，看指令字长）、Ⅲ通用寄存器（是）、Ⅳ浮点寄存器（不是，通常更宽），答案「仅Ⅰ、Ⅲ」。这个判据就是「谁直接参与定点整数运算，谁就等于机器字长」。

字长辨析的三个高频错项

1. 「**MDR** 的位数一定等于存储字长」——错，等于数据线宽度。
2. 「指令字长一定等于机器字长」——错，两者无约束关系；在变长指令系统（如 x86）中更是明显不等。
3. 「机器字长一定等于存储字长」——错，机器字长 ≥ 存储字长，只在最常见情况下相等。

本章小结表

考纲条目	核心内容	必须记住的结论
计算机系统的基本组成	硬件 + 软件；软件分系统软件与应用软件	只有硬件的机器是「裸机」，什么都干不了
硬件五大部件	运算器、控制器、存储器、输入设备、输出设备	**CPU** = 运算器 + 控制器；主机 = **CPU** + 主存（不含外存）
冯 · 诺依曼结构要点	五大部件；指令与数据同为二进制、同等地位、同存一个存储器；存储程序；按地址顺序执行	原始结构以运算器为中心；现代计算机以存储器为中心
CPU 区分指令与数据的依据	取指周期取出的是指令，执行周期取出的是数据	依据是指令周期的不同阶段，不是译码结果、不是存储单元
ISA 与微架构	ISA 规定「做什么」（程序员可见）；微架构规定「怎么做」	指令格式、通用寄存器属 ISA；时钟周期、进位方式、Cache 容量属微架构
软件与硬件的关系	逻辑功能等价；可互相转化；固化	固化只是换了存放介质，固件本质仍是软件

考纲条目	核心内容	必须记住的结论
「存储程序」工作方式	程序与数据预先存入主存，自动连续执行	靠 PC 自动增量 + 按地址顺序存放 + 转移指令改写 PC
语言与翻译程序	机器语言（硬件唯一直接执行）；汇编程序翻译汇编语言；编译程序整体翻译高级语言；解释程序逐句边翻译边执行	高级语言 → 机器目标代码的是编译程序
C 程序转换流程	预处理 → 编译 → 汇编 → 链接	输出依次为`.i` → `.s` → `.o` → 可执行文件
指令执行过程	取指 → 分析（译码）→ 执行	取指永远是第一步；PC 增量发生在取指阶段
响应时间	作业提交到完成的总时间 $T_{总} = T_{CPU} + T_{等待}$；性能 $\propto 1/$ 执行时间	关心「多久办完」；与吞吐量不同向变化
吞吐量	单位时间处理的请求或任务数	关心「单位时间办多少事」；提高吞吐量可能拉长响应时间
时钟周期与主频	$f = 1/T_c$；速率按 10^3 进制	1 GHz ↔ 1 ns，绝不能用 **1024** 换算主频
CPI	$CPI = \frac{总时钟周期数}{IC} = \sum_i CPI_i \times \frac{IC_i}{IC}$	CPI 是加权平均值；与主频无关
CPU 执行时间	$T_{CPU} = IC \times CPI \times T_c = \frac{IC \times CPI}{f} = \frac{N}{f}$	三式等价；IC、CPI、f 分别由编译器、体系结构、工艺决定
比例法	$\frac{T_{新}}{T_{旧}} = \frac{IC_{新}}{IC_{旧}} \times \frac{CPI_{新}}{CPI_{旧}} \times \frac{T_{c,新}}{T_{c,旧}}$	乘积关系，各因子倍数直接相乘
两机时间比	IC 相同时 $\frac{T_A}{T_B} = \frac{CPI_A}{CPI_B} \times \frac{f_B}{f_A}$	主频高不一定更快（2017-12 的 M1 反而慢 1.6 倍）
MIPS	$\text{MIPS} = \frac{f}{CPI \times 10^6} = \frac{f(\text{MHz})}{CPI}$	与 CPI 成反比；不能跨指令集比较
MIPS 局限	指令功能强弱不同；同一机器不同程序 MIPS 不同	只适合同 ISA 系列粗略对比
FLOPS 系列	M/G/T/P/E/Z 每级 $\times 10^3$	按 10 的幂，与存储容量的 2^{10} 进制不同
进制辨析	容量按 2^{10}；速率按 10^3	2^{10} 与 10^3 相差 2.4%，跨级会放大
机器字长	CPU 一次整数运算能处理的位数 = ALU 宽度 = 通用寄存器位数	决定数的表示范围与运算精度；不等于 IR 位数
指令字长	一条指令的位数，由操作码与地址码决定	决定 IR 位数；与机器字长无必然联系
存储字长	一个存储单元的位数	主存容量 = 单元数 × 存储字长
编址单位与地址字长	一个地址对应的单元宽度；地址字长 = MAR 位数	按字节编址时 1 地址 = 1 B；可寻址单元数 $= 2^{\text{地址字长}}$
数据字长	一个数据所占的位数，由数据类型决定（`char` 8 位、`int` 32 位、`double` 64 位）	可小于、等于或大于机器字长；大于时需分段处理
地址字长与机器字长	地址字长 = 地址线根数 = MAR 位数	两者不必相等；32 位机器可以只配 20 位地址线
MAR / MDR	MAR 位数 = 地址线数；MDR 位数 = 数据线数	**MDR** $\neq$ 存储字长（易错点第一名）
字长间关系	机器字长 $\geq$ 存储字长（整数倍）；指令字长是存储字长的 0.5/1/2 倍；指令字长与机器字长无关	记住三句：机器字长看运算，指令字长看编码，MDR 看数据线

易错清单

1. **CPU** 区分指令和数据的依据是「指令周期的不同阶段」，不是「操作码的译码结果」，也不是「所在的存储单元」。2009-11 的正确答案是 C。
2. 「主机 = **CPU** + 主存」，硬盘等外存属于 I/O 设备，不属于主机。见到「主机包括硬盘」直接判错。
3. 「**CPU** = 运算器 + 控制器」，这是考纲原文口径。寄存器可以理解成两者共用，但不要漏掉控制器。
4. 原始冯 · 诺依曼结构以运算器为中心，现代计算机以存储器为中心。两个「中心」不要互换。
5. 「数据都在指令中直接给出」是错的，只有立即寻址才是；数据可以来自寄存器、主存、堆栈等。2019-12 的错项正是这一句。
6. **ISA** 与微架构的分界看「换掉它程序是否仍然正确」：影响正确性的属 ISA（指令格式、寄存器个数与位数），只影响快慢的属微架构（时钟周期、进位方式、Cache 容量）。2022-18 答案「仅 Ⅰ、Ⅲ」。
7. 固化不等于变成硬件。固件写在 ROM 里，本质仍是软件，只是存放介质和调用方式变了。
8. 软硬件等价的是「逻辑功能」，不是物理实体。
9. 高级语言 $\rightarrow$ 机器目标代码文件用编译程序；汇编程序的输入是汇编语言，不是高级语言；解释程序不生成独立目标文件。
10. **C** 程序完整流程的顺序是「预处理 $\rightarrow$ 编译 $\rightarrow$ 汇编 $\rightarrow$ 链接」，2022-20 就是直接考这个顺序，不要把编译与汇编颠倒。
11. 取指阶段是所有指令共有的，且 **PC** 增量发生在取指阶段。因此「靠译码结果区分指令与数据」在时序上说不通。
12. 主频换算一律按 10^3 进制：1 GHz $= 10^9$ Hz，对应 $T_c = 1$ ns 是精确值。不要用 **1024** 去换算频率。
13. T_c 与 f 互为倒数，求执行时间时两式可互换验证；算完用另一种形式代回检查，能避免位数错误（如把 μs 写成 ms）。
14. **CPI** 必须按比例加权，不能把各类指令的 CPI 相加或取算术平均。$0.8 \times 1 + 0.2 \times 10 = 2.8$ 而不是 $(1 + 10)/2 = 5.5$。
15. **CPI** 与主频无关，同架构下提高主频不会降低 CPI，只会缩短每个周期。
16. **MIPS** 与 **CPI** 成反比：MIPS $= f/(CPI \times 10^6)$。CPI 越大 MIPS 越小，两者不可能同增。
17. **MIPS** 不能跨指令集比较。一条 CISC 复杂指令可能顶多条 RISC 指令，所以 MIPS 高的机器不一定更快。
18. **FLOPS** 系列按 10 的幂（M=10^6、G=10^9、T=10^{12}、P=10^{15}、E=10^{18}、Z=10^{21}），与存储容量的 2^{10} 进制完全不同。2^{10} 与 10^3 相差 2.4%，选「差不多」的选项时要留神。
19. 中文数量级要会换算：「亿亿」$= 10^{16}$，「千万亿」$= 10^{15}$（即 P），「百亿亿」$= 10^{18}$（即 E）。2021-12 的四个选项就是靠数量级互相干扰的。
20. **MAR** 位数 = 地址线位数；**MDR** 位数 = 数据线位数。**MDR** 的位数不等于存储字长——这是 2021 年大题反复挖的坑，存储字长 32 位而数据线 16 位时，MDR 只有 16 位，一次访存要传两次。
21. 指令字长决定 **IR** 位数，机器字长决定 **ALU** 宽度和通用寄存器位数，两者毫无约束关系。2020-12 答案「仅 Ⅰ、Ⅲ」的判据就是这一句。
22. 机器字长 $\geq$ 存储字长，通常是 1、2、4 倍关系；指令字长与机器字长无关，只是各自与存储字长成倍数。把「指令字长 = 机器字长」当结论去用，一定会错。

23. 按字节编址时可寻址空间 = $2^{\textbf{MAR}\text{ 位数}}$ 字节。地址线 20 位、按字节编址即 1 MB；同一题目里「字长 16 位」不能拿去做 $2^{20}\times 16$ 的乘法——那是把编址单位和字长搞混了。
24. 执行时间比值题的分子分母要对准。$\frac{T_A}{T_B}=\frac{CPI_A}{CPI_B}\cdot\frac{f_B}{f_A}$，主频在分母上，别写反。
25. 「速度提高 **50%**」不等于「时间减少 **50%**」。速度变为原来的 1.5 倍，时间变为原来的 $1/1.5\approx 0.667$ 倍。2012-12 中 CPU 时间 90 s 变为 $90/1.5=60$ s，加上不变的 I/O 10 s 得 70 s，选 D——若误算成 $90\times 0.5=45$ s 再加 10 s 就会误选 55 s。
26. **PC** 里装的是「下一条」指令的地址，不是当前指令的地址。当前指令已装入 IR，PC 在取指阶段就已自增；转移指令靠执行阶段改写 PC 生效。
27. 机器字长与地址字长不是一回事。「32 位计算机」只说明数据通路宽度是 32 位，地址线可能是 20 位、32 位或更多。算寻址范围必须用地址线根数。
28. 数据字长与机器字长可以不等。64 位 `double` 在 32 位机上照样能存能算，只是要拆成两段处理；不要因为「机器字长 32 位」就断定「数据不可能超过 32 位」。
29. 在一条式子里同时出现容量与速率时，必须分别用两种进制换算。例如「1 MB 数据以 1 MB/s 传输需要 $2^{20}/10^6=1.048576$ s」，而不是 1 s。答案「整整好好是 1」往往就是进制用错的红旗。
30. $\lceil\log_2 C\rceil$ 这类题不能靠「差不多」蒙。容量按 2 的幂还是 10 的幂，结果在很多数据下恰好相同，错误会被掩盖；判分依据是你取数是否按定义，而不是结果碰巧对。

第二章数据的表示和运算

本章考纲要求

第二部分计算机组成原理二、数据的表示和运算（一）数制与编码

1. 进位计数制及其数据之间的相互转换
2. 定点数的编码表示

（二）运算方法和运算电路

1. 基本运算部件：加法器，算术逻辑部件（ALU）。
2. 加/减运算：补码加/减运算器，标志位的生成。
3. 乘/除运算：乘/除法运算的基本原理，乘法电路和除法电路的基本结构。

（三）整数的表示和运算

1. 无符号整数的表示和运算
2. 带符号整数的表示和运算

（四）浮点数的表示和运算

1. 浮点数的表示：IEEE 754 标准。
2. 浮点数的加/减运算

本章是计组计算题最密集的一章。从历年统考看，它的出题方式只有三种：选择题考「边界值 + 编码转换 + 标志位判断」，大题考「**C** 语言程序段的机器级执行结果」，大题考「运算电路与标志位的逻辑表达式」。这三种题型都不要求背结论，要求的是能在纸上把位运算过程一步步推出来。所以本章的写法是：每个知识点给出定义、公式、可执行步骤，再用表格把一次完整的位运算过程摊开，让读者能照着复算一遍。

一、数制与编码

(一) 进位计数制

进位计数制用一组固定的数字符号和一个固定的基数（**radix**）来表示数。基数为 r 的 r 进制数中，使用的数字符号是 $0 \sim r-1$；每个数位上的数字所代表的实际值是「该数字 $\times$ 该位的位权」，第 i 位的位权是 r^i。任意一个 r 进制数可以写成按位权展开式：

$$N = \sum_{i=-m}^{n-1} a_i r^i, \qquad a_i \in \{0, 1, \ldots, r-1\}$$

其中整数部分有 n 位，小数部分有 m 位。计算机内部使用二进制，原因是四方面的：制造只有两个稳定状态的物理器件容易（高/低电位、有/无脉冲、正/负极）；二进制的计数与运算规则最简单（加法和乘法都只有四条）；二进制恰好与逻辑命题的「真/假」对应，便于逻辑运算；用逻辑门电路实现算术运算方便。这是统考选择题考过的结论。

进制	基数	数字符号	计算机中的用途
二进制	2	0 1	机器内部的物理表示
八进制	8	0 ~ 7	二进制的一种缩写写法
十六进制	16	0 ~ 9 A ~ F	书写机器数、地址、机器码的通用写法
十进制	10	0 ~ 9	人机交互的输入输出形式

四者之间是「同一个数在不同位权系统下的写法」，本质是同一个数值，不是四种不同的数。

（二）进位计数制之间的相互转换

（1）任意进制 → 十进制：按位权展开求和

直接代入位权公式即可，是唯一不需要技巧的方向。

例：$1101.101B = 1\times 2^3 + 1\times 2^2 + 0\times 2^1 + 1\times 2^0 + 1\times 2^{-1} + 0\times 2^{-2} + 1\times 2^{-3} = 13.625$

例：$2AF.8H = 2\times 16^2 + 10\times 16^1 + 15\times 16^0 + 8\times 16^{-1} = 687.5$

（2）十进制 → r 进制：整数部分除基取余，小数部分乘基取整

这是统考要考的「通用方法」，必须能说出为什么。

整数部分用「除基取余，逆序排列」。设 $N = (a_{n-1}\cdots a_1 a_0)_r$，两边对 r 取余，有 $N \bmod r = a_0$——因为 r 的更高次幂都能被 r 整除，只有最低位的 a_0 留在余数里。于是商为 $(a_{n-1}\cdots a_1)_r$，再对 r 取余就得到 a_1。反复执行直到商为 0，把每次得到的余数从后往前读出来，就是结果。

小数部分用「乘基取整，顺序排列」。设小数 $F = (0.b_{-1}b_{-2}\cdots b_{-m})_r$，两边乘 r，有 $rF = b_{-1} + (0.b_{-2}\cdots b_{-m})_r$——乘基数相当于把小数点右移一位，移出来的整数部分正好是最高小数位 b_{-1}。反复执行直到小数部分为 0 或达到所需精度，把每次得到的整数部分从前往后读出来。

转换方向	循环体	循环终止	读数方向
十进制整数 → r 进制整数	余数 $= N \bmod r$，记下余数，$N = \lfloor N/r \rfloor$	$N = 0$	余数逆序读出
十进制小数 → r 进制小数	$N = N \times r$，记下整数部分，N 取 N 的小数部分	$N = 0$ 或达到所需精度	整数部分顺序读出

例：$43.6875D = 101011.1011B$。整数部分 $43 = 32 + 8 + 2 + 1 = 101011B$（也可以用除 2 取余：$43 \to 21\cdots 1$、$21 \to 10\cdots 1$、$10 \to 5\cdots 0$、$5 \to 2\cdots 1$、$2 \to 1\cdots 0$、$1 \to 0\cdots 1$，逆序得到 $101011B$）；小数部分 $0.6875\times 2 = 1.375$ 记 1，$0.375\times 2 = 0.75$ 记 0，$0.75\times 2 = 1.5$ 记 1，$0.5\times 2 = 1.0$ 记 1，顺序得到 $.1011B$。

小数部分可能永远取不完

只有形如 $k/2^m$ 的十进制小数才能在有限位内被二进制精确表示。$0.3D$、$0.7D$、$1.2D$ 转二进制都是无限循环小数，必须按题目要求截取到指定位数，这就产生了表示误差。统考 2021 年就考过「下列数值中不能用 IEEE 754 浮点格式精确表示的是」，正确答案是 1.2，而 1.25、2.0、2.5 都能精确表示（$1.25 = 1.01B$、$2.5 = 10.1B$）。判断标准很简单：把十进制小数写成既约分数 p/q，若 q 是 2 的幂就能精确表示。

（3）二进制与八进制、十六进制的快速转换

八进制与十六进制的基数是 2^3 和 2^4，因此可以一位对三（四）位直接映射，不需要经过十进制。

- 二进制 → 八进制：以小数点为界，向两侧每 3 位一组，不足补 0，每组直接写成一位八进制数字。
- 二进制 → 十六进制：以小数点为界，向两侧每 4 位一组，不足补 0，每组直接写成一位十六进制数字。
- 反向转换：每位八（十六）进制数字展开成 3（4）位二进制，去掉两端多余的 0。

例：$1011010.1101B$ → 分组 001 011 010 . 110 100 → $132.64O$；分组 0101 1010 . 1101 → $5A.DH$。

本知识点的考法

统考对「进位计数制」的直接考查不多，通常嵌在其他题目里：给出十六进制机器数要求先转成二进制再判断符号位与溢出；或者判断某个十进制小数能否被 IEEE 754 精确表示。真正的必考项是「定点数的编码表示」，转换速度快慢直接决定后面所有计算题的速度。

（三）真值与机器数

- 真值（**true value**）：带正负号、符合人类书写习惯的数值，如 -3、$+5$、-0.75。真值可以用十进制写，也可以用二进制写（如 $-1011B$）。
- 机器数（**machine number**）：数值在计算机内部的 0/1 编码序列。机器数本身不含符号位以外的任何符号信息，它的正负完全由编码规则决定。

同一个机器数，按不同的编码规则解释会得到不同的真值；同一个真值，用不同的编码规则编码会得到不同的机器数。所以做题时必须先确认「本题用哪种码」或「按什么类型解释」。

机器数不带符号

写机器数时不能带负号，也不能带小数点。例如 $x=-0.75$，其原码要写成 `1.1100000`（8 位机器数，小数点只是约定位置），不能写成 `-0.11`。考试中把机器数写成带符号形式是本考点最低级的失分点。

（四）定点数的编码表示

定点数约定机器数中小数点的位置固定不变：小数点不使用符号表示，只是一个「约定」。计算机中只采用两种约定：

- 定点整数：小数点固定在最低位之后。n 位定点整数的真值是整数。
- 定点小数：小数点固定在符号位之后、最高数值位之前。n 位定点小数的真值绝对值小于 1。

下面统一约定「编码字长为 $n+1$ 位」，即 **1** 位符号位 + n 位数值位。$n+1$ 位编码的模是 2^{n+1}，其可表示的编码状态共 2^{n+1} 个。

*1.*原码

最高位是符号位（0 表示正，1 表示负），其余 n 位是真值的绝对值。

$$[x]_{原}=\begin{cases}x & 0\le x\le 2^n-1\\ 2^n+|x| & -(2^n-1)\le x\le 0\end{cases}$$

- 表示范围：$-(2^n-1)\sim+(2^n-1)$。
- **0** 的表示不唯一：$[+0]_{原}=0\underbrace{00\cdots0}_{n}$，$[-0]_{原}=1\underbrace{00\cdots0}_{n}$。
- 优点：与真值的对应关系直观，乘法时符号位与数值位可以分开处理。
- 缺点：符号位不能直接参与加减运算，加减法规则复杂；有两个零，给判零、比较带来麻烦。

现代计算机不用原码表示整数，但用原码表示浮点数的尾数（IEEE 754 的尾数就是原码形式，只是隐藏了规格化后的最高位 1）。

*2.*反码

正数的反码与原码相同；负数的反码是在原码基础上符号位不变、数值位全部取反。

$$[x]_{反}=\begin{cases}x & 0\le x\le 2^n-1\\ (2^{n+1}-1)+x & -(2^n-1)\le x\le 0\end{cases}$$

- 表示范围：$-(2^n-1)\sim+(2^n-1)$（与原码相同）。
- **0** 的表示不唯一：$[+0]_{反}=0\underbrace{00\cdots0}_{n}$，$[-0]_{反}=1\underbrace{11\cdots1}_{n}$。
- 反码的符号位可以参与运算，但最高位产生的进位必须循环进位（加回到最低位），硬件实现麻烦。它在现代计算机中几乎不用，只作为「原码求补码」的中间过渡形式。

*3.*补码

补码是整个计组的基石。先给定义，再解释为什么这样定义。

定义（从原码出发）：正数的补码与原码相同；负数的补码是符号位不变、数值位按位取反，末位加 **1**（即在反码基础上末位加 1）。

定义（从模运算出发，这才是本质）：$n+1$ 位补码是一个模 2^{n+1} 的模运算系统，其定义为

$$[x]_{补}=x \bmod 2^{n+1}=\begin{cases}x & 0\le x\le 2^n-1\\ 2^{n+1}-|x| & -2^n\le x\le 0\end{cases}$$

注意负数的定义式是 $2^{n+1}-|x|$，写成二进制恰好就是「$|x|$ 的全 $n+1$ 位按位取反后加 1」——这就是补码定义式与「取反加一」口诀的等价性来源。

- 表示范围：$-2^n\sim+(2^n-1)$。比原码、反码多表示一个最小负数 -2^n，因为 $+0$ 与 -0 的编码状态被合并，空出来的一种状态被赋予 -2^n。
- **0** 的表示唯一：$[0]_{补}=\underbrace{00\cdots0}_{n+1}$，全 0。
- 几个必须条件反射的编码：
  - 最小负数 -2^n 的补码是 $1\underbrace{00\cdots0}_{n}$（符号位 1，数值位全 0）。
  - 最大正数 2^n-1 的补码是 $0\underbrace{11\cdots1}_{n}$。
  - -1 的补码是 $\underbrace{11\cdots1}_{n+1}$，全 1。

为什么用补码：三条理由。

1. 加减法统一，符号位与数值位一同参与运算。在模 2^{n+1} 意义下，减法可以改写成加法：$[x-y]_{补}=[x]_{补}+[-y]_{补}\ (\text{mod } 2^{n+1})$。计算机中所有运算都基于加法器实现，这意味着一套加法器就能完成加、减两种运算，不需要为减法单独设计电路，也不需要额外判断「|x| 是否大于 |y|」来决定做加法还是减法。
2. 数 **0** 的表示唯一。判零只需要一个「结果是否全 0」的零标志 ZF，无需区分 $+0$ 和 -0。

3. 比原码多表示一个最小负数 -2^n，表示范围对称性更好，利用率更高。

正因为如此，50 年代以来所有计算机都用补码表示带符号整数，C 语言的 `char`、`short`、`int`、`long` 都是补码。

求 $[-x]_{补}$ 的唯一法则

将 $[x]_{补}$ 连同符号位在内全部按位取反、末位加 **1**，就得到 $[-x]_{补}$。注意「连同符号位」四个字：符号位也要取反。

例：$x=+0.75$，$[x]_{原}=[x]_{补}=0.1100000$（8 位）；$[x]_{补}$ 全部按位取反得 1.0011111，末位加 1 得 1.0100000，即 $[-0.75]_{补}=1.0100000$，真值 $-\left(1-2^{-1}\right)\times 2^{0}\cdots$ 按定点小数位权验证：1.0100000 的位权是 $-1\times 2^{0}+0\times 2^{-1}+1\times 2^{-2}=-1+0.25=-0.75$，正确。

原码与补码的互相转换，有两条不经过反码的速算法：

- 原码 $\rightarrow$ 补码（负数）：从右往左找到第一个 `1`，把这个 `1` 及其右侧的所有位保持不变，左侧的所有数值位按位取反（符号位保持 `1` 不变）。
- 补码 $\rightarrow$ 原码（负数）：同样的操作做一遍即可（因为补码的补码就是原码）。
- 补码 $\rightarrow$ 真值：按位权公式直接算，见下文「位权视角」。

例：$x=-52$ 的 8 位补码。$52=32+16+4=00110100B$，原码为 10110100。从右往左第一个 `1` 在最低位，它左侧的数值位 0110100 全部取反为 1001011……更稳妥的做法是取反加一：10110100 取反 11001011，加 1 得 11001100。验证：11001100 按位权算 $=-128+64+8+4=-52$，正确。

*4.*移码

移码（增码）是在真值上加上一个偏置值（**bias**）后按无符号数编码。$n+1$ 位编码中，偏置值通常取 2^n 或 2^n-1。

$$[x]_{移}=x+2^{n}\quad\text{（以偏置值}2^{n}\text{ 为例）}$$

移码与补码的关系：当偏置值取 2^n 时，$[x]_{移}=x+2^n$，而 $[x]_{补}=x \bmod 2^{n+1}$。两者在二进制上只差符号位是否取反：

$$[x]_{移}=[x]_{补}\text{ 的符号位取反}$$

验证：$x=+3$（$n=4$，5 位编码），$[x]_{补}=00011$，符号位取反得 $10011=19$；而 $x+2^4=3+16=19$，一致。$x=-3$，$[x]_{补}=11101$，符号位取反得 $01101=13$；而 $-3+16=13$，一致。

- 移码只用于表示定点整数，不能表示定点小数。
- 移码的 0 表示唯一：$[0]_{移}=100\cdots0$。
- 移码的核心价值：由于加上了偏置值，移码的全部编码在数轴上是从全 0 单调递增到全 1 的，因此移码可以按无符号数的方式直接比较大小，比较结果与真值的大小关系一致。这就是浮点数用移码表示阶码的原因——浮点数加减运算的对阶步骤要频繁比较两个阶码的大小，用移码比较可以省掉符号位单独判断的逻辑。

*5.*四种编码的横向对比

下表统一按「1 位符号位 + n 位数值位」共 $n+1$ 位编码理解。

编码	表示范围	最大数	最小数	**0** 的编码	主要用途
原码	$-(2^n-1)\sim+(2^n-1)$	$0,11\cdots1$	$1,11\cdots1$	两个：$0,00\cdots0$ 与 $1,00\cdots0$	浮点数尾数、乘除法符号处理
反码	$-(2^n-1)\sim+(2^n-1)$	$0,11\cdots1$	$1,00\cdots0$	两个：$0,00\cdots0$ 与 $1,11\cdots1$	原码与补码互转的中间形式
补码	$-2^n\sim+(2^n-1)$	$0,11\cdots1$	$1,00\cdots0$	唯一：$0,00\cdots0$	计算机中所有带符号整数
移码	$-2^n\sim+(2^n-1)$	$1,11\cdots1$	$0,00\cdots0$	唯一：$1,00\cdots0$	浮点数的阶码
无符号整数	$0\sim2^{n+1}-1$	$1,11\cdots1$	$0,00\cdots0$	唯一：$0,00\cdots0$	地址、编号、无负值的量

原码与反码的「最小数」为何不同

原码的最小负数对应数值位全 1、符号位 1，即 $-(2^n-1)$；而反码的负数越大（越接近 0）数值位越大，所以 $1,00\cdots0$ 才代表 $-(2^n-1)$，$1,11\cdots1$ 代表 -0。这两个编码的最小负数写法正好相反，是出题人最喜欢的陷阱点。

*6.*位权视角：算补码真值的最快方法

对 $n+1$ 位补码 $x_nx_{n-1}\cdots x_0$，最高位的位权是 -2^n，其余各位的位权与无符号数相同：

$$x=\sum_{i=0}^{n-1}x_i2^i+x_n\cdot(-2^n)$$

即「只有最高位（符号位）的位权是负的」。举例：

- $[1011]_{补}=1\times2^0+1\times2^1+0\times2^2+1\times(-2^3)=1+2-8=-5$
- $[0011]_{补}=1+2=3$

用位权视角可以直接推出补码的表示范围：最高位取 1、其余全 0 时，只剩下负权，取得最小值 -2^n；最高位取 0、其余全 1 时，只剩下全部正权，取得最大值 2^n-1。

无符号整数的位权全部为正：$x=\sum_{i=0}^{n}x_i2^i$。唯一差别就是最高位的位权，这是理解「同一串二进制按有符号/无符号解释结果不同」的关键。

2015 年统考真题

「由 3 个 1 和 5 个 0 组成的 8 位二进制补码，能表示的最小整数是（）。」答案是 -125。思路：要最小，符号位必须取 1（负权 $-2^7=-128$）；剩下 2 个 1 应放在位权最小的位置（2^0、2^1），使它们贡献的正值最小，得机器数 10000011，真值 $-128+2+1=-125$。这类题一律用位权法解，比逐个选项代入快得多。

*7.*补码比较大小的技巧

由位权视角可以直接得到两条结论，做选择题时很省时间：

- 同号（同在负半轴或同在正半轴）比较时，补码的真值随机器数（按无符号解释）单调递增。例如同为负数：$F340H<FF30H<FF40H$（真值分别为 -3264、-208、-192），同为非负数：$7010H<7500H<7740H$。

- 异号比较时，直接看符号位：符号位为 0 的一定大于符号位为 1 的，例如 $F300H$（负）$< 7070H$（正）。

另一个常用技巧是快速求「十六进制补码负数」的真值：先求与它互补（相加为全 **0** 且产生最高位进位）的那个正数，再取负号。例：$[x]_{补} = FF3AH$（16 位），因为 $FF3AH + 00C6H = (1)0000H$，而 $00C6H = 198$，所以 $x = -198$。

二、运算方法和运算电路

（一）基本运算部件

*1.*一位全加器

加法器的基本单元是一位全加器（**Full Adder, FA**），它有三个输入（被加数 A_i、加数 B_i、来自低位的进位 C_i）和两个输出（本位和 S_i、向高位的进位 C_{i+1}）。

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_iB_i + (A_i \oplus B_i)C_i$$

第二个式子可以这样理解：当且仅当「A_i 和 B_i 都为 **1**」（自己产生进位）或者「A_i 与 B_i 恰有一个为 **1** 且低位送来进位」（传递进位）时，本位才向高位进位。令 $G_i = A_iB_i$（进位产生函数），$P_i = A_i \oplus B_i$（进位传递函数），则 $C_{i+1} = G_i + P_iC_i$。

把 $n+1$ 个全加器串联，就得到 $n+1$ 位加法器。每位全加器的一位和公式和进位公式必须能默写，2022 年统考大题直接要求写出标志位的逻辑表达式，用的就是这两个式子。

A_i	B_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

*2.*串行进位与先行进位

串行进位（行波进位）加法器：把 $n+1$ 个全加器首尾相接，第 i 位的进位 C_i 必须等第 $i-1$ 位算完才能确定。最高位的进位要经过「进位链」逐级传播，最坏情况下的进位延迟与位数成正比，即 $O(n)$ 级门延迟。优点是结构极其简单，每一位只需要一个全加器。

先行进位（并行进位、超前进位）加法器：把进位公式展开，让每一级的进位直接由各位的 G_i、P_i 和最低位进位 C_0 算出，不再逐级等待：

$$
\begin{aligned}
C_1 &= G_0 + P_0C_0 \\
C_2 &= G_1 + P_1G_0 + P_1P_0C_0 \\
C_3 &= G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0 \\
&\vdots
\end{aligned}
$$

这样各位的进位同时产生，延迟从 $O(n)$ 降到接近常数（实际上受门电路扇入限制）。代价是进位逻辑的门电路数量随位数急剧增加，所以实际 CPU 采用分组先行进位：组内先行进位，组间可以串行或再先行。

做题要记住的三句话

① 全加器的一位和是三个输入的异或；② 进位是「产生的进位 + 传递的进位」；③ 先行进位的本质是用增加硬件换掉进位链的串行等待，ALU 的速度瓶颈就在这里。

*3.*算术逻辑部件（*ALU*）

ALU（Arithmetic Logic Unit，算术逻辑单元）的核心部件是带标志加法器，能完成多种算术运算（加、减、乘除的部分步骤、加 1、减 1 等）和逻辑运算（与、或、非、异或等）。

一个典型 ALU 的接口是：两个 n 位操作数输入端 X、Y，一个进位输入端 C_{in}，一个操作控制端 ALU_{op}，输出端 F 以及若干标志位。

- ALU_{op} 的位数决定操作的种类：k 位控制信号最多对应 2^k 种操作。例如 3 位控制端最多支持 8 种操作。
- 多路选择器 **MUX** 用来从多条输入通路或运算结果中选出一条输出。k 位控制信号对应 2^k 路输入。
- **ALU** 的操作数宽度（位数）通常与机器字长一致。

关于 ALU 的三个「重要认识」

① 计算机中所有运算都基于加法器实现；② 加法器不区分操作数是有符号数还是无符号数——它只是把两串 0/1 按位相加；③ 加法器不判定对错，它总是取低 n 位作为结果，同时生成标志信息，由指令去解释结果的正确性。这三句话是理解「同一套电路既能算无符号数又能算有符号数」的总纲。

（二）补码加/减运算器与标志位的生成

*1.*补码加减运算规则

补码加法：两个补码相加，符号位参与运算，两数和的补码 = 两数补码之和；符号位相加产生的进位直接舍去（模 2^{n+1} 运算）。

补码减法：$x - y = x + (-y)$，因此

$$
[x - y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}} \pmod{2^{n+1}}
$$

而 $[-y]_{\text{补}}$ 只需将 $[y]_{\text{补}}$ 连同符号位按位取反、末位加 1。

这个规则在 $y = -2^n$ 时失效：例如 8 位补码中 -128 的补码是 10000000，按规则求 $[-(-128)]_{\text{补}}$ 得到 10000000，即 -128，这显然不对，因为 8 位补码不能表示 $+128$。但只要结果不溢出，按此规则算出的补码减法结果仍然正确。例：$-7-(-128)$，$[-7]_{\text{补}} = 11111001$，$[-(-128)]_{\text{补}}$ 按规则算为 10000000，相加得 (1)01111001，舍去进位得 $01111001 = 121$，与 $-7+128 = 121$ 一致。这就是模运算系统的「回绕」特性。

*2.*加减运算的统一实现

减法在硬件上不单独实现，而是「把减数取反，加 1，再相加」：

- 加法：MUX 选择 0 那条通路，被加数 Y 原样送入加法器，低位进位 $C_0 = 0$。
- 减法：MUX 选择 1 那条通路，Y 先经过非门按位取反，同时减法控制信号 $Sub = 1$ 作为最低位的进位输入，等效于末位加 1，于是 Y 变成 $[-Y]_{补}$，减法变成加法。

因此无符号整数和带符号整数的加/减运算电路完全一样，这个电路称为整数加减运算部件，它基于带标志加法器实现。二者的区别只在于如何解释结果和看哪个标志位。

*3.*四个标志位

指令执行产生的标志位通常存放在状态寄存器 / 程序状态字寄存器（**PSW**）中。x86 把它们放在 `EFLAGS` 寄存器中，供条件转移指令使用；MIPS、RISC-V 没有专门的状态寄存器，条件分支指令直接根据 ALU 当前的状态标志决定是否转移。不论哪种结构，ALU 都会产生这些标志，区别只是「何时、如何利用」。

设加法器共 $n+1$ 位，最高位（符号位）产生的进位为 C_n（即 C_{out}），次高位（最高数值位）产生的进位为 C_{n-1}，Sub 是加减控制信号（$Sub = 1$ 表示减法，$Sub = 0$ 表示加法），则四个标志位的含义与判定如下。

标志	名称	含义	硬件判定
ZF	零标志	结果全 0 时为 1	结果的各位全部为 0
SF	符号标志	有符号结果的正负	直接取结果的最高位，$SF = F_n$
OF	溢出标志	有符号数加减结果溢出	$OF = C_n \oplus C_{n-1}$
CF	进/借位标志	无符号数加法的进位/减法的借位	$CF = Sub \oplus C_{out}$

逐条解释。

零标志 ZF（**Zero Flag**）：$ZF = 1$ 表示运算结果为 0。不论把操作数看作有符号数还是无符号数，ZF 都有意义，因为「结果是否为 0」与解释方式无关。它主要用于判断相等。

符号标志 SF（**Sign Flag**）：SF 表示有符号数加减运算结果的符号，直接取结果的最高位。对无符号数运算 SF 没有意义——无符号数没有负数，最高位也不代表符号。（有些教材把 SF 叫 `Signal Flag`，统考按「符号标志」理解。）

溢出标志 OF（**Overflow Flag**）：$OF = 1$ 表示有符号数加减运算溢出。硬件上由最高位进位 C_n 与次高位进位 C_{n-1} 求异或得到：

$$OF = C_n \oplus C_{n-1}$$

推导：补码减法已转化为加法，所以溢出只可能发生在两种情形。正数加正数：两个加数符号位都是 0，相加后结果的符号位变成 1（正数相加得负数），说明溢出；此时来自次高位的进位必定为 1，而符号位向外的进位必定为 0，$C_n \oplus C_{n-1} = 0 \oplus 1 = 1$。负数加负数：两个加数符号位都是 1，相加后结果符号位变成 0（负数相加得正数），说明溢出；此时来自次高位的进位必定为 0，而符号位向外的进位必定为 1，$C_n \oplus C_{n-1} = 1 \oplus 0 = 1$。两种情形合起来正好就是 $C_n \oplus C_{n-1}$。

OF 与操作数符号、运算种类的关系：

运算	符号关系	是否可能溢出
加法	两数异号	不会溢出
加法	两数同号	可能溢出（结果变号则溢出）
减法	两数同号	不会溢出
减法	两数异号	可能溢出（结果变号则溢出）

用一句话概括：「同号相加」「异号相减」才可能溢出；溢出发生时，结果的符号一定与两操作数的符号相反。

进/借位标志 CF（**Carry Flag**）：加法时 CF 是进位标志——若最高位产生进位，说明无符号数相加超过了 $2^{n+1}-1$；减法时 CF 是借位标志——若最高位没有产生进位，说明无符号数相减时被减数小于减数，需要向更高位借位。因此

$$CF = Sub \oplus C_{out}$$

- 加法（$Sub=0$）：$CF = C_{out}$，有进位则 $CF=1$。
- 减法（$Sub=1$）：$CF = \overline{C_{out}}$，无进位（即最高位没有产生进位）则 $CF=1$。

如果题目中没有单独的 Sub 控制信号，则用最低位的进位输入 C_{in}（C_0）代替，即 $CF = C_{in} \oplus C_{out}$，因为对加减运算器而言 C_0 就等于 Sub。

CF 与 OF 的区别是本章最核心的辨析

- OF 判断有符号数的溢出，CF 判断无符号数的溢出。两者的判定电路不同、适用对象不同，绝不能互换。
- 对同一个加法器，一次运算会同时产生两个标志：$A+B$ 时 OF 告诉你有符号解释下是否溢出，CF 告诉你无符号解释下是否溢出。至于该看哪个，由数据的类型决定——C 语言中 `int` 看 OF，`unsigned int` 看 CF。
- 加法时「有进位」、减法时「无进位（即需要借位）」都使 $CF=1$，两者形式相反但语义一致：都表示无符号数的结果超出了 $n+1$ 位能表示的范围。
- 无符号数运算产生的 SF 和 OF 并非「无意义」，它们的含义是：把参与运算的机器数和结果都解释为有符号数时，结果的符号和溢出情况。这是同一串 0/1 的另一种解读，不是电路出错。

*4.*标志位的使用：比较大小与条件转移

（**1**）无符号数比较大小：执行 $A-B$，用 ZF、CF 判断。

条件	标志位
$A=B$	$ZF=1$
$A\neq B$	$ZF=0$
$A<B$	$CF=1$（有借位）
$A>B$	$CF=0$ 且 $ZF=0$
$A\leq B$	$CF=1$ 或 $ZF=1$
$A\geq B$	$CF=0$

（**2**）有符号数比较大小：执行 $A-B$，用 ZF、SF、OF 判断。核心思想是用 $SF \oplus OF$ 表示「真实结果的正负」：当没有溢出（$OF=0$）时 SF 就是真实符号；当发生溢出（$OF=1$）时真实符

号与 SF 恰好相反。因此 $SF \oplus OF = 1$ 等价于「真实结果为负」。

条件	标志位
$A = B$	$ZF = 1$
$A \neq B$	$ZF = 0$
$A < B$	$SF \oplus OF = 1$
$A > B$	$SF \oplus OF = 0$ 且 $ZF = 0$
$A \leq B$	$SF \oplus OF = 1$ 或 $ZF = 1$
$A \geq B$	$SF \oplus OF = 0$

2011 年统考真题

「条件转移指令 `bgt`（无符号整数比较大于时转移）的转移条件是（）。」答案 $\overline{CF + ZF} = 1$，即 $CF = 0$ 且 $ZF = 0$。无符号数比较只看 CF 和 ZF，OF 与 SF 都不参与——这正是测试「CF 与 OF 分别管谁」的题。

2024 年统考真题（加法器的输出与标志位）

「若通用寄存器 `rs1`、`rs2` 的内容分别是 $87654321H$ 与 $98765432H$，执行 `add` 指令后 ALU 输出端 F、OF 和 CF 分别是什么？若该 `add` 指令处理的是无符号整数，则应根据哪个标志判断是否溢出?」

计算过程:$87654321H + 98765432H$ 按位相加得 $(1)1FDB9753H$,舍去最高位的进位,$F = 1FDB9753H$。两个加数的最高位都是 1（都解释为负数），而结果的最高位变为 0（正数），同号相加而结果变号，有符号数溢出，$OF = 1$。最高位产生了进位 $C_{out} = 1$，加法时 $CF = C_{out} = 1$，说明无符号解释下也越界，$CF = 1$。若按无符号整数处理，应看 CF 判断是否溢出。

2022 年统考真题（标志位的逻辑表达式）

「设 ALU 输入端 A、B 及输出端 F 的最高位分别为 A_{15}、B_{15}、F_{15}，写出 SF 与 OF 的逻辑表达式。」答案是:

$$SF = F_{15}$$

$$OF_{加} = A_{15}B_{15}\overline{F_{15}} + \overline{A_{15}}\,\overline{B_{15}}F_{15}, \qquad OF_{减} = A_{15}\overline{B_{15}}\,\overline{F_{15}} + \overline{A_{15}}B_{15}F_{15}$$

加法溢出只有两种可能: 两个正数 ($A_{15} = B_{15} = 0$) 相加得负数 ($F_{15} = 1$), 或两个负数 ($A_{15} = B_{15} = 1$) 相加得正数（$F_{15} = 0$）。减法溢出同理，只需把 B_{15} 换成 $\overline{B_{15}}$（因为减法已把减数取反）。

MIPS / RISC-V 与 x86 的差别（可考概念题）

x86 把 CF、ZF、SF、OF 等条件标志集中存放在 **EFLAGS** 标志寄存器（即程序状态字寄存器 PSW）中，条件转移指令读取它；而 **MIPS**、**RISC-V** 没有专门的状态寄存器，条件分支指令直接根据 **ALU** 当前产生的状态标志决定是否转移。不论哪种结构，ALU 都会产生这些标志，区别只在于何时、如何利用它们。

（三）乘/除法运算的基本原理

一句话原理

乘法和除法都可以用控制逻辑循环地执行加法和移位操作来实现。乘法 = 累加 + 右移；除法 = 加减 + 左移。理解了这一点，就能理解为什么「没有乘法指令的计算机也能完成乘法运算」——编译器把乘法编译成一段「比较、加法、移位」的循环即可。

*1.*原码一位乘法

符号与数值分开处理：乘积的符号位 = 两个操作数符号位的异或；乘积的数值部分 = 两个绝对值相乘。

绝对值（无符号）相乘的机器算法：被乘数每次与乘数的一位相乘得到一个部分积，部分积逐次右移并与下一个部分积累加。

- 乘数最低位为 1：部分积加上被乘数，然后右移一位。
- 乘数最低位为 0：部分积加 0，然后右移一位。
- 重复 n 次（n 为数值位数），最后乘积的高 n 位在部分积寄存器中、低 n 位在乘数寄存器中。

原理就是手算竖式：$X \times Y = X \times \sum y_i 2^i$，把 y_i 逐位取出决定「加不加 X」，每处理完一位就把累加结果右移（等价于把手算竖式里逐行左移的部分积改成结果右移）。

例：$X = 0.1101$（$13/16$），$Y = 0.1011$（$11/16$），求 $X \times Y$。

步骤	操作	部分积（P）	乘数（Y）
初始	$P_0 = 0$	0.0000	1011
第 1 步	Y 最低位为 1，$P + X$	0.1101	1011
	右移一位	0.0110	1101
第 2 步	Y 最低位为 1，$P + X$	1.0011	1101
	右移一位	0.1001	1110
第 3 步	Y 最低位为 0，$P + 0$	0.1001	1110
	右移一位	0.0100	1111
第 4 步	Y 最低位为 1，$P + X$	1.0001	1111
	右移一位	0.1000	1111

结果 $X \times Y = 0.10001111B = 143/256$，与真值 $13/16 \times 11/16 = 143/256$ 一致。（表中「右移」后乘数寄存器的高位补入部分积移出的最低位，这是把部分积和乘数拼成一个 $2n$ 位寄存器一起右移的写法。）

*2.*补码一位乘法（*Booth* 算法）

补码乘法不能像原码那样「符号与数值分开算」，因为补码的符号位带负权。Booth 算法的核心是重新组织展开式，把乘数按「相邻两位之差」重新编码：

$$[X \cdot Y]_{\text{补}} = [X]_{\text{补}} \times \left(-y_n + \sum_{i=1}^{n} y_{n-i} 2^{-i} \right) = \sum_{i=1}^{n} (y_{n-i+1} - y_{n-i}) [X]_{\text{补}} \cdot 2^{-i}$$

其中 y_{n+1} 是附加位（辅助位）。展开式的每一项 $(y_i - y_{i+1})$ 只可能是 0、$+1$、-1 三种值，因此每一步只需要在「加 X」「减 X」「什么都不加」之间选一个。

迭代规则：每一步考察乘数寄存器的最低位 y_i 与附加位 y_{i+1} 组成的两位，按下表决定操作，然后做算术右移一位。

y_i（乘数最低位）	y_{i+1}（附加位）	对部分积的操作
0	0	加 0（不变）
0	1	加 $[X]_{补}$
1	0	加 $[-X]_{补}$（即减 X）
1	1	加 0（不变）

记忆口诀：两位相同加 0；01 加被乘数；10 减被乘数。

完整流程（标准写法，务必按这个顺序操作）：

1. 附加位 y_{n+1} 初值置 **0**；部分积寄存器 A 初值置 0；乘数寄存器 Q 置乘数。
2. 判断 (y_i, y_{i+1})，按上表对 A 做加法或减法。
3. 把「A 与 Q 拼成的寄存器」整体算术右移一位：A 右移且最高位补 A 原来的符号位，Q 右移且最高位补入 A 移出的最低位，附加位补入 Q 移出的最低位。
4. 重复第 2、3 步共 n 次（n 为乘数的位数），不再做额外的加法。
5. 运算结束后，A（高 n 位）与 Q（低 n 位）拼接就是 $2n$ 位的乘积补码。若题目只保留 n 位结果，取 Q 的内容。

关于「最后一步不移位」的说法

不同教材对最后一次迭代的处理有两种写法：一种是「循环体 = 判断 → 加法 → 算术右移，共 n 次」；另一种是「前 $n-1$ 次移位，第 n 次只加不移位」。这两种写法的寄存器位数约定不同，得到的结果一致。做题时以题目给出的寄存器位数和步骤叙述为准，不要凭记忆套用。统考对 Booth 算法的要求是掌握「看两位 → 加减 → 算术右移」这个迭代结构和进位链的循环控制，不要求手算特定寄存器宽度下的每一位。

例一：$x = 1$，$y = -1$（**4** 位补码）——用最小规模验证规则。

$[x]_{补} = 0001$，$[y]_{补} = 1111$，$[-x]_{补} = 1111$。

步	$y_i y_{i+1}$	操作	A（**4** 位）	Q（**4** 位）	y_{i+1}
初始	—	—	0000	1111	0
1	1 0	减 x：$0000 + 1111$	0000	1111	1
2	1 1	加 0	0000	0111	1
3	1 1	加 0	0000	0011	1
4	1 1	加 0	0000	0001	1

结果 $A \cdot Q = 0000\,0001$，即 $+1$，验证 $1 \times (-1) = -1$ 的补码形式 11111111（低 4 位为 1111，A 位为符号扩展的 0000）。这个例子说明：当乘数为全 **1** 时，**Booth** 算法只在第一步做一次减法，后面全是移位，这正是「连续 1 可以合并操作」的体现。

例二：$x = -3$，$y = +3$（**4** 位补码）——一个可以逐行复算的完整算例。

$[x]_{补} = 1101$，$[-x]_{补} = 0011$，$[y]_{补} = 0011$。取部分积寄存器 A 为 $n+1 = 5$ 位，被乘数 x 按 5 位符号扩展为 11101（检验：$-16 + 8 + 4 + 1 = -3$，正确），$[-x]_{补} = 00011$；乘数 Q 为 4 位，附加位初值 0。

步	y_iy_{i+1}	操作	A（**5** 位）	Q（**4** 位）	附加位
初始	—	—	00000	0011	0
1	1 0	$A+X$：$00000+00011$	00011	0011	0
		算术右移一位	00001	1001	1
2	1 1	加 0	00001	1001	1
		算术右移一位	00000	1100	1
3	0 1	$A-X$：$00000+11101$	11101	1100	1
		算术右移一位	11110	1110	0
4	0 0	加 0	11110	1110	0
		算术右移一位	11111	0111	0

四步移位结束后运算终止。取 A 的低 **4** 位与 Q 拼接，得 1111 0111，按 8 位补码解释：符号位为 1，取反加一得 $00001001=9$，所以结果为 -9，与 $-3\times3=-9$ 一致。

手算 Booth 算法的三条铁律

① 部分积寄存器 A 比乘数多 **1** 位（补码乘法要留出符号扩展位），这样 A 的低 n 位与 Q 拼接正好是 $2n$ 位乘积；② 被乘数 X 要按 A 的位数做符号扩展后再参与加减，否则负数会被当成正数；③ 附加位初值 0，先加减后算术右移，共做 n 次。

Booth 算法的四个高频错误

① 附加位初值不是 **0**；② 判断的是「乘数最低位与附加位」这两位，而不是只看最低位；③ 先做加减再移位，顺序颠倒全盘皆错；④ 移位必须是算术右移（补符号位），用成逻辑右移会得到错误结果。

*3.*原码除法（恢复余数法）与加减交替法

原码除法同样把符号与数值分开：商的符号位 = 两个操作数符号位的异或，数值部分 = 两个绝对值相除。

恢复余数法：先在余数上减去除数，若余数为负说明「商这一位应该是 0」，此时必须把减掉的除数加回来（恢复余数），再左移；若余数为正，商这一位是 1，直接左移。这种方法每步可能多一次加法，故称「恢复余数」。

加减交替法（不恢复余数法）：观察到「恢复余数后左移再减」与「不恢复余数直接左移再加」是等价的，因此可以省掉恢复操作。规则是：

- 若当前余数为正：商上 1，左移一位，减除数。
- 若当前余数为负：商上 0，左移一位，加除数。

最后若余数为负，需要再加一次除数恢复出真正的余数。加法器的操作次数减少，硬件实现更快。

例（不恢复余数法，4 位）：$x=1101$（13），$y=0011$（3）。用 A（4 位余数）与 Q（4 位商）拼接，初始 $A=0000$，$Q=1101$。

步	操作	A	Q	商位
初始	—	0000	1101	—
1	左移一位	0001	1010	—

步	操作	A	Q	商位
	$A-y$：$0001-0011$，余数负	1110	1010	$q_0=0$
2	左移一位	1101	0100	—
	余数负，$A+y$：$1101+0011$	0000	0100	$q_0=1$
3	左移一位	0000	1000	—
	$A-y$：$0000-0011$，余数负	1101	1000	$q_0=0$
4	左移一位	1011	0000	—
	余数负，$A+y$：$1011+0011$	1110	0000	$q_0=0$

最终余数为负，再加一次除数：$1110+0011=0001$。商 $Q=0100=4$，余数 1，验证 $13=3\times4+1$，正确。

*4.*补码除法

补码除法的商和余数都用补码表示，符号位参与运算，规则由「商位与被除数、除数的符号关系」决定。统考要掌握的要点：

- 够减的判断依据是「余数与除数是否同号」：余数与除数同号表示够减，商上 1，下一步减除数；异号表示不够减，商上 0，下一步加除数。这正好与无符号除法「看余数正负」对应，只是判断方式改成了比较符号。
- 补码除法的商符在运算过程中自动形成，不需要像原码那样单独用异或求。
- 商的末位通常采用恒置 **1** 的方法来保证精度（末位商恒定为 1，误差不超过末位权重），这是补码除法的一个经典细节。
- 除法的合法性检查：除数不能为 **0**，且被除数的绝对值必须小于除数的绝对值（否则商溢出，无法用定点小数表示），硬件在除法开始前会做这个检查并可能产生「除零异常」。

（四）乘法电路和除法电路的基本结构

*1.*无符号数乘法电路

以 32 位无符号数乘法为例，逻辑结构包含如下部件：

部件	作用
被乘数寄存器 X	存放被乘数（32 位）
乘积寄存器 P	初值置 0（初始部分积 $P_0=0$）；结束时存放 64 位乘积的高 **32** 位
乘数寄存器 Y	初值置乘数；结束时存放 64 位乘积的低 **32** 位
进位触发器 C	保存加法器产生的进位
计数器 C_n	存放循环次数，初值 32，每循环一次减 1，减到 0 结束

部件	作用
ALU	核心部件，在控制逻辑控制下完成 $P+X$ 的加法

每次循环的三件事：① 控制逻辑根据 Y 的最低位决定是否把 X 加到 P（最低位为 1 则加，为 0 则不加）；② 在写使能控制下把运算结果写回 P，加法进位存入 C；③ 对 C、P、Y 三者同步逻辑右移一位——C 移入 P 的最高位，P 的最低位移入 Y 的最高位，Y 的最低位移出，C 补入 0。

于是 Y 的最低位依次被换成乘数的下一位，控制逻辑每次都从 Y 的最低位读取「当前该处理的乘数位」。循环 32 次后，乘积的高 32 位在 P、低 32 位在 Y。

*2.*有符号数（补码）乘法电路

以 32 位补码一位乘法（Booth）为例，与无符号乘法电路的主要区别：

- 乘数寄存器 Y 后面增加一个附加位（辅助位），初值 $Y_{-1}=0$。
- 控制逻辑每次根据 Y 的最后两位（最低位与附加位）决定执行「加 X」「加 $[-X]_{补}$」还是「加 0」，然后执行算术右移，结果写回寄存器。
- 计数器 C_n 控制循环次数，共执行 32 次。
- 因为要支持「减」，ALU 需要能完成加减运算，且需要一个能产生 $[-X]_{补}$ 的取反加一电路。

2020 年统考真题的两问

① 「基于 ALU、移位器、寄存器以及相应控制逻辑实现乘法指令时，控制逻辑的作用是什么?」答：控制加法和移位操作的次数，并发出控制信号来执行判断、加法、移位等操作。②「若指令系统中没有乘法指令，但有加法、减法和移位等指令，则在计算机上也能实现乘法运算，为什么?」答：乘法可以分解为加法和移位操作的循环，因此可以不断执行比较、加法（减法）、移位等指令来实现乘法的功能。

*3.*乘法运算的溢出判断

n 位乘 n 位的结果有 $2n$ 位。

- 若用 $2n$ 位保存乘积，一定不会溢出（$2n$ 位足以容纳两数之积）。
- 若只用 n 位保存结果（截取低 n 位），则可能溢出，需要判断。

类型	不溢出的机器判断	手算判断
无符号整数乘法	乘积的高 n 位全为 **0**	结果是否超出 $[0,2^n-1]$
带符号整数乘法	乘积的高 $n+1$ 位全为 **0** 或全为 **1**	结果是否超出 $[-2^{n-1},2^{n-1}-1]$

带符号乘法的判断要理解成：高 n 位必须恰好是低 n 位最高位（符号位）的符号扩展。若乘积为正，低 n 位最高位是 0，则要求高 $n+1$ 位全为 0；若乘积为负，低 n 位最高位是 1，则要求高 $n+1$ 位全为 1。这与 2020 年、2024 年统考大题的官方答案一致。

2010 年统考真题

「$r_1=FEH$，$r_2=F2H$，$r_3=90H$，$r_4=F8H$ 均为 8 位补码，若运算结果存放在 8 位寄存器中，会发生溢出的是（）。」先把补码转真值：$r_1=-2$、$r_2=-14$、$r_3=-112$、$r_4=-8$。逐项算：$r_1\times r_2=28$（不溢出）、$r_2\times r_3=1568$（8 位补码范围是 $-128\sim127$，远超出，溢出）、$r_1\times r_4=16$、$r_2\times r_4=112$。答案 B。

*4.*除法电路

除法电路的基本结构：

部件	作用
除数寄存器 Y	存放除数
余数寄存器 R	存放当前余数（部分余数）
商寄存器 Q	初值放被除数，运算过程中逐步写入商
ALU	执行除数 Y 与余数 R 的加减运算
控制逻辑	根据运算结果的符号（或与除数的符号关系）确定这一位商是 1 还是 0，并发出加减、移位控制信号
计数器	控制循环次数

每次循环：① ALU 完成 $R \pm Y$；② 控制逻辑根据结果确定商位并上商（把 Q 空出的最低位置为 1 或 0）；③ 寄存器 R、Q 同步左移一位，R 移出的高位丢弃、Q 空出的低位用来上商。除法电路通过循环执行加减法和左移来实现除法运算——这正是「乘法用右移、除法用左移」的对称性。

三、整数的表示和运算

（一）无符号整数

无符号整数指机器字长的全部二进制位都是数值位，没有符号位，其值就是各数值位的位权之和，相当于数的绝对值。n 位无符号整数的表示范围是 $0 \sim 2^n - 1$。

适用场合：地址运算、编号表示，以及一切「不会出现负值」的量。C 语言中用 `unsigned int`、`unsigned short` 等声明。

- 无符号数加法：n 位二进制直接按位相加，最高位产生的进位就是溢出标志（$CF = 1$）。
- 无符号数减法：$A - B$ 转化为 $A + [-B]_{\text{补}}$，即把 B 的 n 位全部按位取反、末位加 $\mathbf{1}$，再与 A 相加；若最高位没有产生进位，说明发生了借位（$CF = 1$），结果为负（以模 2^n 的形式回绕）。

（二）带符号整数

带符号整数最高位是符号位，其余各位是数值位，统一采用补码表示。n 位带符号整数的表示范围是 $-2^{n-1} \sim 2^{n-1} - 1$。

移位运算是最常用的「快速乘除」。C 语言中 `x<<k`、`x>>k` 分别表示左移、右移 k 位。

移位类型	适用对象	规则	说明
逻辑左移	无符号数	高位移出，低位补 0	移出的最高位为 1 时发生溢出
逻辑右移	无符号数	低位移出，高位补 0	相当于无符号数除以 2^k 向下取整
算术左移	有符号数（补码）	高位移出，低位补 0	与逻辑左移结果相同；移后符号位改变即溢出
算术右移	有符号数（补码）	低位移出，高位补符号位	相当于除以 2^k 向 $-\infty$ 取整

移位的三条关键结论

① 左移 k 位相当于乘以 2^k，右移 k 位相当于除以 2^k。② 左移可能溢出（正数可能变负数，负数可能变

正数），判断方法是「移位后符号位是否改变」；右移不会改变符号性，因此不会溢出。③ 算术移位时原来的符号位也一起移动——硬件做移位时不允许任何一位「保持不动」，所以右移必须动态地把符号位补进空出的高位。这也是「算术右移不能改变符号」的硬件原因。

右移的取整方向

算术右移是向 $-\infty$ 取整（对负数而言不是「向 0 取整」）。例：-7 的 8 位补码是 11111001，算术右移一位得 11111100，真值为 -4，而 $-7/2=-3.5$。若移出的位是 0（能整除），则相当于精确除以 2^k；若移出的是 1，则不能整除，结果向小方向取整。这一条在 C 语言程序题里常常是判断「某表达式能否编译成移位指令」的依据。

（三）零扩展与符号扩展

当把一个短整数赋给更长的整数类型时，需要在机器数的高位补齐若干位，补什么取决于源类型是无符号还是有符号。

扩展名	补位内容	适用场合
零扩展	高位补 0	无符号整数扩展，或把无符号数扩展为更长的无符号数
符号扩展	高位补源机器数的符号位（正数补 0，负数补 1）	带符号整数（补码）扩展

例：`short y = -9`，16 位机器数为 $FFF7H$。扩展为 32 位：

- 符号扩展（作为 `int`）：$FFFFFFF7H$。
- 零扩展（作为 `unsigned int`）：$0000FFF7H$。

定点小数与定点整数的扩展位置不同

定点整数在数值位之前（高位）扩展，定点小数在数值位之后（低位）扩展。原因很直观：定点整数要保住「哪些位是高位」，定点小数要保住「小数的位数」。混用会把数扩展错。

（四）无符号与有符号混合运算的陷阱

这几条几乎每年都会以选择题或程序题的形式出现。

（1）机器数不变、只改变解释方式（同宽度转换）

同宽度的 `short` 与 `unsigned short` 互转、`int` 与 `unsigned int` 互转，机器数一位都不变，变的只是「按哪种类型解释」。例如 `unsigned short usi = 65535; short si = usi;` 得到 `si = -1`，因为 $FFFFH$ 按补码解释是 -1。

（2）窄变量扩展为宽变量

- `short` → `int`：符号扩展。
- `unsigned short` → `int`（或 `unsigned int`）：零扩展（先补齐到目标宽度，目标为 `int` 时再按补码解释）。
- `short` → `unsigned int`：先符号扩展为 `int`，机器数不变，再按无符号解释。

（3）宽变量截断为窄变量

`int` → `short`：直接截取低 **16** 位，高位丢弃，机器数变了，真值可能完全不同。

例：`int i = 32777`，其 32 位机器数为 $00008009H$；`short si = i` 截取低 16 位得 $8009H$，符号

位为 1（负数），真值为 $-(2^{15}-9)=-32759$；`int j = si` 再符号扩展回 $FFFF8009H$，真值仍为 -32759。这就是 2024 年统考选择题的答案。

（4）有符号与无符号一起运算时，编译器默认把有符号数转换成无符号数

```
int a = -1;
unsigned int b = 1;
if (a > b) printf("yes");   /* 会输出 yes！ */
```

因为 a 的机器数 $FFFFFFFFH$ 被当作无符号数解释成 4294967295，远大于 1。这是 C 语言中最经典的坑。统考 2017 年大题中 `for（unsigned i = 0; i <= n - 1; i++）` 在 $n=0$ 时死循环，也是同一个原因：$n-1=0-1$ 在无符号意义下回绕成全 1（$2^{32}-1$），条件永真。

三条必须背下来的话

① 同一机器数按不同类型解释会得到不同的真值，转换时先看「机器数变不变」，再看「解释方式变不变」。② 扩展看源类型，截断丢高位；符号扩展补符号位，零扩展补 0。③ 有符号与无符号混合运算，结果按无符号解释，比较时尤其危险。

（五）有符号整数加减运算的溢出判断

统考反复考，这里给全三种方法。

方法一：根据操作数与结果的符号判断。若送至加法器的两个操作数符号相同，而结果的符号与它们不同，则结果溢出；否则不溢出。等价的说法：同号相加、异号相减才可能溢出。

方法二：根据最高位与次高位的进位判断。记符号位的进位为 C_n、最高数值位的进位为 C_{n-1}，则 $OF=C_n\oplus C_{n-1}$。两者不同说明溢出。

方法三：双符号位（变形补码、模 **4** 补码）判断。用两位符号位 K_1K_2 参加运算：

K_1K_2	含义
00	结果为正，未溢出
11	结果为负，未溢出
01	正溢出（结果本应为正，超出上界）
10	负溢出（结果本应为负，超出下界）

两位符号位不同即溢出，且最高位的符号位代表结果的真正符号。采用双符号位的好处是「溢出后结果仍可挽救」——右规（右移一位、阶码加 1）以后可以得到正确结果，这正是浮点数尾数运算采用双符号位的原因。

无符号整数加减的溢出用 CF 判断：加法时最高位产生进位、减法时最高位产生借位，则无符号数结果越界。

2013 年统考真题（标志位判断的经典）

「已知 8 位机器中整型变量 x、y 的机器数为 $[x]_{补}=11110100$、$[y]_{补}=10110000$，若 $z=2x+y/2$，则 z 的机器数是（）。」解析：$2x$ 为算术左移 1 位，$11110100\to11101000$（$-24\times2=-48$）；$y/2$ 为算术右移 1 位，$10110000\to11011000$（$-80\div2=-40$）；相加 $11101000+11011000=(1)11000000$，舍去进位得 11000000。答案 A。这类题的全部技巧就是「左移/右移 + 补码加法」，用十进制先估一下大小可以自查。

四、浮点数的表示和运算

（一）浮点数的一般格式

浮点数把定点整数与定点小数组合起来表示实数：

$$N = (-1)^S \times M \times 2^E$$

字段	名称	常用编码	作用
S	数符	1 位，0 正 1 负	表示整个浮点数的正负
E	阶码	补码或移码表示的定点整数	位数反映浮点数的表示范围；真值反映小数点的实际位置
M	尾数	原码或补码表示的定点小数	位数反映浮点数的精度；数符表示浮点数的正负

（二）浮点数的规格化

规格化的目的是让尾数的有效数字尽量占满尾数数位，从而在固定位宽下获得最高的精度。
规格化的判定条件：

- 原码表示的尾数：最高数值位必须为 **1**，即形如 $0.1\times\times\cdots$ 或 $1.1\times\times\cdots$。
- 补码表示的尾数：符号位与最高数值位必须相反，即形如 $0.1\times\times\cdots$（正）或 $1.0\times\times\cdots$（负）。

由此可以推出补码尾数的取值范围：最大正值为 $0.11\cdots1$，最小正值为 $0.10\cdots0$；最大负值为 $1.01\cdots1$，最小负值为 $1.00\cdots0$（真值 -1）。
两种规格化操作：

操作	触发条件	做法
左规	尾数最高数值位不是有效位（补码尾数符号位与最高数值位相同，形如 $00.0\times\cdots$ 或 $11.1\times\cdots$）	尾数算术左移一位，阶码减 **1**；可能要做多次
右规	尾数出现溢出（双符号位为 01 或 10）	尾数算术右移一位，阶码加 **1**；只做一次

浮点数的溢出与规格化是两回事：阶码小于最小允许值称为下溢，通常把结果置为机器零（$+0$ 或 -0）；阶码超过最大允许值称为上溢，机器产生异常、停止运算（有的机器置为 $\pm\infty$）。

（三）**IEEE 754** 标准

*1.*格式与偏移量

类型	总位数	符号位	阶码位	尾数位	偏移量（**bias**）
单精度 `float`	32	1	8	23	$2^{8-1}-1=127$

类型	总位数	符号位	阶码位	尾数位	偏移量（**bias**）
双精度 `double`	64	1	11	52	$2^{11-1}-1=1023$
扩展精度（现行标准）	128	1	15	112	$2^{15-1}-1=16383$

阶码采用移码表示，但偏移量取 $2^{n-1}-1$（而不是定点数移码常用的 2^n）。其中 n 是阶码的位数。单精度阶码 8 位、偏移量 127；双精度阶码 11 位、偏移量 1023。

阶码真值的求法：把阶码字段当作无符号数读出，再减去偏移量。

$$E_{\text{真}}=E_{\text{字段}}-127\quad(\text{单精度}),\qquad E_{\text{真}}=E_{\text{字段}}-1023\quad(\text{双精度})$$

例：8 位阶码字段 11000000，按无符号读是 192，所以真指数是 $192-127=65$。

尾数采用原码表示，且隐含一位最高位 **1**（规格化数的隐藏位）。尾数字段只存小数点后的 23（52）位，实际有效尾数是 1.字段。这样用 23 位字段换来 24 位有效精度（双精度是 53 位有效精度）。

*2.*规格化数的真值公式

$$\text{规格化数}=(-1)^S\times 1.f\times 2^{E_{\text{字段}}-\text{bias}},\qquad E_{\text{字段}}\in[1,254]\ (\text{单精度})$$

单精度规格化数的指数范围是 $-126\sim127$，因为阶码字段 0 和 255 被保留作特殊用途，可用的字段值只有 $1\sim254$。

尾数的隐含位为什么是 **1**：规格化数的尾数形如 $1.f$，最高位恒为 1，既然恒为 1 就可以省略不存，读取时自动补上。这就是「隐藏位」的含义，也是「规格化数的尾数取值范围是 $[1,2)$」的原因。

*3.*非规格化数与特殊值

IEEE 754 用阶码字段的全 **0** 和全 **1** 区分出四类取值（零、非规格化数、无穷大、非数），把它们记住是必考内容。

值的类型	符号位	阶码字段（单精度）	尾数字段	真值
正零	0	全 0	全 0	$+0$
负零	1	全 0	全 0	-0
非规格化数	0 或 1	全 0	不全为 **0**	$\pm(0.f)_2\times 2^{-126}$
正无穷	0	全 1（255）	全 0	$+\infty$
负无穷	1	全 1（255）	全 0	$-\infty$
非数	0 或 1	全 1（255）	不全为 **0**	`NaN`

非规格化数的两个特殊约定（必考）：

1. 隐含的最高位变成 **0**，即尾数形式是 $0.f$，不是 $1.f$。
2. 阶码真值固定视为 -126（双精度为 -1022），不是 $0-127=-127$。

这样设计的目的是实现逐渐溢出（**gradual underflow**）：非规格化数的最大值与规格化数的最小值平滑衔接——非规格化数的最大值为 $(0.\underbrace{1\cdots1}_{23}) \times 2^{-126} = (1-2^{-23}) \times 2^{-126}$，规格化数的最小值为 1.0×2^{-126}，两者只差一个 2^{-149} 的间隔。

单精度能表示的最小正非零非规格化数是尾数只有最低位为 1 的情形：$2^{-23} \times 2^{-126} = 2^{-149}$。双精度为 2^{-1074}。

四个「最小值/最大值」必须背下来（单精度）

- 规格化最小正数：2^{-126}（阶码字段 1，尾数全 0）
- 规格化最大正数：$(2-2^{-23}) \times 2^{127} = 2^{128} - 2^{104}$（阶码字段 254，尾数全 1）
- 规格化最小负数：$-(2-2^{-23}) \times 2^{127}$（符号 1、阶码字段 254、尾数全 1）
- 规格化最大负数：-2^{-126}（符号 1、阶码字段 1、尾数全 0）
- 非规格化最小正数：2^{-149}
- 最大正数（规格化）的机器数是 $7F7FFFFFH$，最小规格化正数是 $00800000H$，最大负数是 $80800000H$，最小负数是 $FF7FFFFFH$。

「最大负数」与「最小负数」别搞混

最大负数指「小于 0 的数中绝对值最小的那个」，即 -2^{-126}，它离 0 最近；最小负数指「小于 0 的数中绝对值最大的那个」，即 $-(2-2^{-23}) \times 2^{127}$。中文里「最大/最小」在负数区间最容易读反，读题时先在草稿纸上画一条数轴。

历年真题速览

- **2011** 年：$x = -8.25$，求 `float` 型寄存器内容（答案 `C1040000H`）。
- **2012** 年：`float` 能表示的最大正整数（答案 $2^{128} - 2^{104}$）。
- **2013** 年：`C6400000H` 的真值（答案 -1.5×2^{13}）。
- **2014** 年：两个 `float` 变量 `CC900000H` 与 `B0C00000H` 的大小关系（答案 $x < y$ 且符号相同）。
- **2018** 年：最小的规格化正数（答案 1.0×2^{-126}）。
- **2021** 年：不能用 IEEE 754 精确表示的数值（答案 1.2）。
- **2022** 年：-0.4375 的 IEEE 754 单精度表示（答案 `BEE00000H`）。
- **2023** 年：`80200000H` 的真值——阶码字段全为 0、尾数不全为 0，是非规格化数，尾数按 $0.f$ 还原为 $0.01B = \frac{1}{4}$，真值 $= -(0.01)_2 \times 2^{-126} = -2^{-2} \times 2^{-126} = -2^{-128}$（答案 -2^{-128}）。这道题的陷阱在于隐含位取 **0** 还是取 **1**：若误按规格化数处理（隐含位取 1），会算出 -1.25×2^{-126} 而选错。

*4.*真值 → *IEEE 754* 单精度：完整步骤

以「求 $+8.25$ 的 IEEE 754 单精度表示」为例，把每一步都摊开。

第一步：把十进制真值写成二进制小数。

$8.25 = 8 + 0.25 = 1000.01B$（$0.25 = 0.01B$ 由「乘 2 取整」得到：$0.25 \times 2 = 0.5$ 记 0，$0.5 \times 2 = 1.0$ 记 1）。

第二步：规格化，写成 $1.f \times 2^E$ 的形式。

小数点左移 3 位：$1000.01B = 1.00001B \times 2^3$，所以 $E = 3$，$f = 00001$。

第三步：求阶码字段。

$$E_{\text{字段}} = E + \text{bias} = 3 + 127 = 130 = 10000010B$$

第四步：确定数符与尾数字段。

$+8.25$ 为正数，$S=0$。尾数字段取 f 补齐到 23 位：00001000000000000000000。

第五步：拼接。

0 10000010 00001000000000000000000 → 按 4 位一组写成十六进制：0100 0001 0000 0100 0000 0000 0000 0000 → `41040000H`。

步骤	内容	结果
① 转二进制	$8.25=1000.01B$	1000.01
② 规格化	$1000.01=1.00001\times 2^3$	$E=3$, $f=00001$
③ 求阶码字段	$3+127=130$	10000010
④ 定符号与尾数	正数，尾数补足 23 位	$S=0$，$f=$ 00001000000000000000000
⑤ 拼接	0 10000010 00001000000000000000000	`41040000H`

5.IEEE 754 单精度 → 真值：完整步骤

以「求 `C0A00000H` 的真值」为例。

第一步：展开为二进制，按 **1/8/23** 切分。

`C0A00000H` = 1100 0000 1010 0000 ⋯ → $S=1$，阶码字段 = 10000001，尾数字段 = 01000000000000000000000。

第二步：判断类型。阶码字段既不是全 0 也不是全 1，是规格化数。

第三步：求阶码真值。$10000001B=129$，$E=129-127=2$。

第四步：还原尾数。隐含最高位 1，实际尾数 $=1.0100\cdots 0B=1.01B=1.25$。

第五步：合成真值。$(-1)^1\times 1.25\times 2^2=-5$。

字段	二进制	解读	结果
符号位 S	1	负数	$-$
阶码（8 位）	10000001	$129-127$	$E=2$
尾数（23 位）	0100000⋯	隐含 1 后为 $1.01B$	$M=1.25$
真值	—	$(-1)\times 1.25\times 2^2$	-5

（四）浮点数的加减运算

浮点数加减运算有固定的五步流程，统考大题几乎每年都从这里出，必须能背出每一步的做法和目的。

*1.*操作数检查

检查两个操作数是否为 0。若阶码和尾数均为 0，该浮点数为 0；若浮点数下溢，也被当作机器 0 处理。若加数、被加数或减数为 0，则结果直接是另一个操作数（被减数为 0 时结果是减数的相反数）。两个操作数都不为 0 时，才进入下面的流程。

*2.*对阶

求阶差 $\Delta E=E_1-E_2$。

- $\Delta E = 0$：阶码已相等，不需要对阶。
- $\Delta E \neq 0$：小阶向大阶看齐，把阶码小的那个数的尾数右移 $|\Delta E|$ 位，同时把它的阶码加 $|\Delta E|$，直到两数阶码相等。

为什么必须「小阶向大阶看齐」

如果反过来「大阶向小阶看齐」，需要把尾数左移，尾数高位会被移出，损失的是高位有效数字（严重损失精度）；而小阶向大阶看齐时尾数右移，移出的是低位，损失的只是精度最低的几位。另外，小阶向大阶看齐时阶码只会增大，因此对阶过程本身不会引起阶码上溢。

*3.*尾数加减

对阶后，两个数的阶码相同，按定点小数加减运算的规则对尾数求和或求差（若用补码，符号位参与运算；若用原码，需要先统一成补码）。运算后的结果若已经规格化，可直接进第 5 步。

*4.*结果规格化

右规：尾数相加后出现双符号位 01 或 10（尾数绝对值 ≥ 1，即发生了尾数溢出），把尾数算术右移一位、阶码加 **1**。右规只需要做一次，因为两个规格化尾数相加的结果绝对值不超过 2。

左规：尾数出现 $00.0\times\cdots$ 或 $11.1\times\cdots$（符号位与最高数值位相同，不是规格化形式），把尾数算术左移一位、阶码减 **1**，直到符号位与最高数值位相反为止。左规可能需要做多次。

左规的上限

尾数有 k 位数值位时，左规最多做 k 次；左规过程中阶码不断减 1，可能引起阶码下溢。若阶码减到最小允许值以下，说明结果太小，按机器零（± 0）处理。

*5.*舍入

在对阶右移和规格化右规的过程中，尾数的最低位会被移出。为了减少误差，移出的位通常保留下来参与运算，最后再统一舍入。常见方法：

方法	做法	特点
就近舍入（向偶数舍入）	见下	IEEE 754 默认模式
朝 0 舍入	直接截断多余位	最简单，误差单向累积
朝正无穷舍入	正数只要多余位不全为 0 就进 1；负数直接截尾	上界方向
朝负无穷舍入	负数只要多余位不全为 0 就进 1；正数直接截尾	下界方向
「0」舍「1」入	移出的最高位为 0 则舍，为 1 则末位加 1	类似十进制四舍五入
末位恒置 1	不论移出位是什么，末位恒置 1	误差累积小，损失一位精度

IEEE 754 的「就近舍入（向偶数舍入）」细则：

- 多余位大于最低位的一半（如多余位是 $1001\cdots$）：最低位进 1。
- 多余位小于最低位的一半（如多余位是 $0111\cdots$）：直接截断。
- 多余位恰好等于最低位的一半（多余位是 $1000\cdots$）：看最低位——为 **0** 则截断，为 **1** 则进 **1**，使最低位保持偶数。

*6.*判断溢出

在规格化与舍入过程中，阶码可能加 1 或减 1，因此需要检查阶码是否越界。用 e 表示真实指数，E 表示 IEEE 754 中存储的阶码字段（单精度）：

条件	含义
$E = 1 \sim 254$，即 $-126 \le e \le 127$	规格化数
$e > 127$	指数上溢，结果按 $\pm\infty$ 处理（具体与舍入模式有关）
$e < -126$	不能继续用规格化数表示，转入非规格化数处理
舍入后结果的绝对值小于 2^{-149}	结果变为 ± 0

三条容易被忽略的结论

① 对阶不会引起阶码上溢或下溢（阶码只会向大阶看齐，只增大不减小）。② 右规和尾数舍入都可能引起阶码上溢（阶码要加 1，若原本已是最大值则溢出为 ∞）。③ 左规可能引起阶码下溢；而尾数溢出不等于浮点数上溢——尾数溢出可以通过右规纠正，只有右规后阶码超过上限才是真正的上溢。

2015 年统考真题

「下列有关浮点数加减运算的叙述中，正确的是（）。Ⅰ. 对阶操作不会引起阶码上溢或下溢；Ⅱ. 右规和尾数舍入都可能引起阶码上溢；Ⅲ. 左规时可能引起阶码下溢；Ⅳ. 尾数溢出时结果不一定溢出。」四项全部正确，答案 D。这道题把上面三条结论全考了。

*7.*完整算例一：补码阶码与尾数的浮点加法（统考 *2009* 年真题型）

计算 $X = 2^7 \times \dfrac{29}{32}$ 与 $Y = 2^5 \times \dfrac{5}{8}$ 之和。已知浮点数的阶码 5 位、尾数 7 位，均含 **2** 位符号位（即变形补码：阶码 2 位符号位 + 3 位数值位，尾数 2 位符号位 + 5 位数值位）。

第 **0** 步：写出两个数的初始表示。

- 尾数：$\dfrac{29}{32} = 0.11101B$，写成 7 位尾数 00.11101；$\dfrac{5}{8} = 0.10100B$，写成 00.10100。
- 阶码：$[+7]_{\text{补}} = 00111$，$[+5]_{\text{补}} = 00101$。

① 对阶：阶差 $\Delta E = 7 - 5 = 2$，Y 的阶码小，把 Y 的尾数右移 **2** 位、阶码加 **2**：

$$[Y]_{\text{尾}} = 00.10100 \longrightarrow 00.00101, \qquad [Y]_{\text{阶}} = 00101 + 00010 = 00111\ (= +7)$$

② 尾数相加（变形补码加法，两个符号位都参与运算）：

项	双符号位尾数
$[X]_{\text{尾}}$	00.11101
$[Y]_{\text{尾}}$（对阶后）	00.00101
相加	**01.00010**

③ 规格化：结果的两个符号位是 01，表示尾数正溢出（尾数绝对值 ≥ 1），需要右规——尾数算术右移一位，阶码加 1：

$$[\text{尾}] = 01.00010 \longrightarrow 00.10001, \qquad [\text{阶}] = 00111 + 00001 = 01000\ (= +8)$$

④ 舍入：右规时尾数最低位移出的是 0，无需舍入。

⑤ 判断溢出：结果的阶码为 01000。阶码是变形补码，两个符号位为 01 说明它是正数，真值 $= +8$；而阶码的数值位只有 3 位，其最大正值是 $0111 = +7$，$+8$ 已经超界，因此阶码正溢出，结果无法表示。

最终结论：$X + Y$ 发生溢出。

用十进制核对：$X = 2^7 \times \dfrac{29}{32} = 128 \times \dfrac{29}{32} = 116$，$Y = 2^5 \times \dfrac{5}{8} = 32 \times \dfrac{5}{8} = 20$，$X + Y = 136$。而该格式能表示的最大正数约为 $2^7 \times 1 = 128$，$136 > 128$，确实溢出，与 ①～⑤ 的推导一致。

为什么「双符号位的 01」既可能是尾数溢出、也可能是阶码上溢

尾数出现 01 表示尾数相加后产生了进位，这个进位可以靠「右规」消化掉（尾数右移一位、阶码加 1）。阶码出现 01 却无法消化——阶码已经是变形补码的最高正值 $+8$（数值位只有 3 位），再加就没有位置了。所以尾数溢出可以挽救，阶码上溢才是真正的溢出。

*8.*完整算例二：*IEEE 754* 单精度浮点数的加法

计算 $0.5 + (-0.4375)$。

① 转成 **IEEE 754** 单精度格式：

$0.5 = 2^{-1} = 1.0 \times 2^{-1}$：$S = 0$，$E_{\text{字段}} = -1 + 127 = 126 = 01111110$，$f = 000\cdots0$（23 个 0），得 `3F000000H`。

-0.4375：$0.4375 = \frac{7}{16} = \frac{14}{32} = 0.01110B$，规格化得 1.11×2^{-2}：$S = 1$，$E_{\text{字段}} = -2 + 127 = 125 = 01111101$，尾数字段为 $110\cdots0$，得 `BEE00000H`。

② 对阶： $\Delta E = (-1) - (-2) = 1$，$-0.4375$ 的阶码小，尾数右移 **1** 位：

$$1.11000\cdots0 \to 0.11100\cdots0, \qquad E = -1$$

（为了后续加减，把两个操作数的尾数都写成 24 位定长形式，并把 x 的尾数也表示为 $0.f$ 形式以统一小数点位置。）

操作数	尾数（右对齐，**24** 位）
$x = 0.5$，$E = -1$	0.10000000000000000000000
$y = -0.4375$，对阶后，$E = -1$	−0.01110000000000000000000

③ 尾数相减（因为 y 为负，实际是 x 减去 $|y|$）：

$$0.10000000000000000000000 - 0.01110000000000000000000 = 0.00010000000000000000000$$

④ 规格化：结果尾数最高有效位不在规格化位置，需要左规——尾数左移 4 位、阶码减 4：

$$0.000100\cdots0 \to 1.00000000000000000000000, \qquad E = -1 - 4 = -5$$

⑤ 判断溢出与舍入：$E = -5$ 在 $-126 \sim 127$ 内，且尾数没有多余位需要舍入，结果有效。

⑥ 合成真值：

$$0.5-0.4375=1.0\times 2^{-5}=0.0625=2^{-4}$$

⑦ 写回 **IEEE 754** 格式：$S=0$，$E_{\text{字段}}=-5+127=122=01111010$，$f$ 全 0，得

$$0\ 01111010\ 00000000000000000000000=\texttt{3D800000H}$$

用 0.0625 验证：`0x3D800000` 确实是 0.0625，与 $0.5-0.4375$ 一致。

IEEE 754 加减法与补码浮点数加减法的差别

统考大题主流是补码表示阶码和尾数的浮点数加减运算（例一那种题型）。若用 IEEE 754 原码尾数做加减，尾数相减需要先做「原码减法」的符号处理，比补码尾数更繁琐。IEEE 754 的加减流程必须知道（步骤、对阶方向、隐藏位恢复、规格化后要重新加 127 还原阶码字段），但手算训练以补码浮点数为重点。

（五）浮点数加减运算的步骤汇总

步骤	做什么	关键点
① 操作数检查	判断是否为 0	为 0 则直接取另一操作数
② 对阶	求阶差，小阶向大阶看齐	小阶数的尾数右移，阶码加阶差；对阶不产生阶码上溢
③ 尾数加减	按定点小数规则运算	补码尾数符号位参与运算
④ 规格化	左规或右规	右规：尾数溢出（双符号位 01/10），移 1 位、阶码加 1；左规：符号位与最高数值位相同，逐位左移、阶码逐次减 1
⑤ 舍入	就近舍入等	只在对阶和右规时发生舍入
⑥ 判溢出	检查阶码	上溢为异常/∞，下溢按 0 或非规格化处理

本章小结表

（六）表一：各种编码对照（$n+1$ 位编码，**1** 位符号位 + n 位数值位）

编码	定点整数范围	定点小数范围	零的个数	符号位可否参与运算	用途
无符号	$0\sim 2^{n+1}-1$	—	1	无符号位	地址、编号
原码	$-(2^n-1)\sim 2^n-1$	$-(1-2^{-n})\sim 1-2^{-n}$	2	不能	浮点数尾数
反码	$-(2^n-1)\sim 2^n-1$	$-(1-2^{-n})\sim 1-2^{-n}$	2	可以（需循环进位）	转换中间形式
补码	$-2^n\sim 2^n-1$	$-1\sim 1-2^{-n}$	1	可以	计算机中的带符号整数
移码	$-2^n\sim 2^n-1$	不适用	1	可以	浮点数的阶码

（七）表二：四个标志位

标志	适用数据	判定式	为 **1** 的含义
ZF	所有	结果各位全 0	运算结果为 0
SF	有符号	$SF = F_n$（结果最高位）	有符号结果为负
OF	有符号	$OF = C_n \oplus C_{n-1}$	有符号加减溢出
CF	无符号	$CF = Sub \oplus C_{out}$	加法有进位 / 减法有借位

（八）表三：乘法与除法运算方法

方法	符号处理	迭代操作	结束条件	移位方向
原码一位乘	符号位异或，数值位相乘	依乘数最低位决定加被乘数或加 0	n 次	部分积与乘数右移
补码一位乘（Booth）	符号位自动参与	依乘数最低位与附加位决定加 X、加 $-X$、加 0	n 次	算术右移
原码除法（恢复余数/加减交替）	符号位异或	减除数，余数为负则恢复（或不恢复改为加）	n 次	余数与商左移
补码除法	符号位自动参与	依余数与除数是否同号决定加或减除数	n 次	余数与商左移

（九）表四：**IEEE 754** 关键数值（单精度）

项目	值	机器数（**16** 进制）
规格化最小正数	2^{-126}	`00800000H`
规格化最大正数	$(2-2^{-23}) \times 2^{127} = 2^{128} - 2^{104}$	`7F7FFFFFH`
规格化最大负数	-2^{-126}	`80800000H`
规格化最小负数	$-(2-2^{-23}) \times 2^{127}$	`FF7FFFFFH`
最小正非规格化数	2^{-149}	`00000001H`
最大非规格化数	$(1-2^{-23}) \times 2^{-126}$	`007FFFFFH`
$+0$ / -0	0	`00000000H` / `80000000H`
$+\infty$ / $-\infty$	—	`7F800000H` / `FF800000H`
阶码真值范围	$-126 \sim 127$	阶码字段 $1 \sim 254$

（十）表五：浮点数加减运算六步

步骤	操作	引起溢出的可能
① 检查	判 0、判特殊值	—
② 对阶	小阶向大阶看齐，小阶尾数右移、阶码加阶差	不会
③ 尾数加减	定点小数运算	尾数可能溢出（可右规挽救）

步骤	操作	引起溢出的可能
④ 规格化	左规（阶码减）或右规（阶码加）	左规可能下溢；右规可能上溢
⑤ 舍入	就近舍入等	舍入进位可能引发右规，进而上溢
⑥ 判溢出	检查阶码	上溢为异常，下溢按 0 或非规格化处理

易错清单

数制与编码

1. 机器数不带符号和小数点。写成 `-0.11`、`+1011` 都是错的，机器数是纯 0/1 序列。
2. 除基取余逆序读，乘基取整顺序读。顺序读反是高频错误。小数部分可能永远乘不完，要按题目精度截断。
3. 只有分母是 **2** 的幂的十进制小数才能被二进制精确表示。0.3、0.7、1.2 都是无限循环，据此判断「能否被 IEEE 754 精确表示」。
4. 原码、反码有两个零，补码、移码只有一个零。补码能多表示一个最小负数 -2^n，原码和反码不能。
5. 求 $[-y]_{补}$ 要连同符号位一起取反加一，不能只对数值位操作。
6. 移码与补码的关系是「符号位取反」，且移码只用于表示定点整数。注意 IEEE 754 阶码的偏移量是 $2^{n-1}-1$，不是定点移码常用的 2^n。
7. 位权法是算补码真值最快的方法：只有最高位（符号位）的位权是负的，为 -2^n。
8. 定点整数在高位扩展，定点小数在低位扩展。

加减运算与标志位

1. OF 管有符号，CF 管无符号，两者不能互换。每次加减运算两个标志都会同时产生，该看哪个由数据类型决定。
2. $OF = C_n \oplus C_{n-1}$（最高位进位异或次高位进位），不是 C_n 本身，也不是 $C_n \oplus Sub$。
3. $CF = Sub \oplus C_{out}$：减法时 $CF = \overline{C_{out}}$，即「最高位没有进位」才表示借位。
4. 同号相加、异号相减才可能溢出；溢出时结果符号与被加数符号相反。
5. 双符号位 01 是正溢出，10 是负溢出，最高位符号位代表真正的结果符号。
6. SF 对无符号数无意义，CF 对有符号数无意义，但不会报错——只是判据错。
7. 有符号比较大小要用 $SF \oplus OF$，不能只看 SF；无符号比较大小只看 CF 和 ZF。
8. 左移可能溢出（符号位改变即为溢出），算术右移不会溢出。算术右移是向 $-\infty$ 取整，$-7 >> 1 = -4$ 而不是 -3。

乘法与除法

1. 乘法用右移，除法用左移，两个方向不要记反。
2. **Booth** 算法的附加位初值必须为 **0**，判断的是「乘数最低位与附加位」两位，先加减后移位，移位是算术右移。
3. 无符号乘法不溢出的判据是乘积高 n 位全 **0**；有符号乘法不溢出的判据是高 $n+1$ 位全 **0** 或全 **1**（即高位是低位的符号扩展）。
4. 用 $2n$ 位保存乘积一定不溢出，只有截取低 n 位时才需要判溢出。
5. 除法开始前必须检查除数是否为 **0**、被除数绝对值是否小于除数，否则商溢出；除零会触发异常。

整数表示与运算

1. 同宽度的有符号/无符号互转，机器数不变，只改变解释方式。例如 $FFFFH$ 按 `short` 是 -1，按 `unsigned short` 是 65535。

2. 窄类型 → 宽类型看源类型：有符号做符号扩展，无符号做零扩展。
3. 宽类型 → 窄类型直接截断高位，真值可能完全变化（如 32777 截成 `short` 后是 -32759）。
4. 有符号与无符号混合运算，编译器把有符号数转换成无符号数，比较运算尤其危险（`-1 > 1u` 为真）。
5. `i <= n-1` 当 n 为无符号 **0** 时永真，会造成死循环。

浮点数

1. 规格化数的隐含位是 **1**，非规格化数的隐含位是 **0**，两者的尾数形式分别是 $1.f$ 和 $0.f$。
2. 非规格化数的阶码真值固定为 -126（双精度 -1022），不是 $0-127$。
3. 阶码字段全 **0** 且尾数全 **0** 是 ± 0；阶码全 **1** 且尾数全 **0** 是 $\pm\infty$；阶码全 **1** 且尾数非 **0** 是 `NaN`；阶码全 **0** 且尾数非 **0** 是非规格化数。四类要一一对应，不要混淆。
4. 阶码字段要减 **127**（**1023**）才是真指数，写完真值后要记得再加回来还原字段。
5.「最大负数」绝对值最小、「最小负数」绝对值最大，中文读题必须在数轴上确认。
6. 对阶一定不会引起阶码上溢或下溢；右规和舍入可能上溢，左规可能下溢。
7. 尾数溢出不等于浮点数上溢，尾数溢出可以由右规纠正，只有阶码越界才是真溢出。
8. 对阶一定是小阶向大阶看齐，反过来会造成高位有效数字丢失。
9. 规格化的补码尾数是「符号位与最高数值位相反」，原码尾数是「最高数值位为 1」，两种判据不能混用。

第三章存储器层次结构

本章考纲要求

> 第二部分计算机组成原理三、存储器层次结构（一）存储器的分类（二）层次化存储器的基本结构（三）半导体随机存取存储器
>
> 1. SRAM 存储器
> 2. DRAM 存储器
> 3. Flash 存储器
>
> （四）主存储器
>
> 1. DRAM 芯片和内存条
> 2. 多模块存储器
> 3. 主存和 CPU 之间的连接
>
> （五）外部存储器
>
> 1. 磁盘存储器
> 2. 固态硬盘（SSD）
>
> （六）高速缓冲存储器（Cache）
>
> 1. Cache 的基本原理
> 2. Cache 和主存之间的映射方式
> 3. Cache 中主存块的替换算法
> 4. Cache 写策略
>
> （七）虚拟存储器
>
> 1. 虚拟存储器的基本概念
> 2. 页式虚拟存储器：基本原理，页表，地址转换，TLB（快表）
> 3. 段式虚拟存储器的基本原理
> 4. 段页式虚拟存储器的基本原理

存储器层次结构是计组分值最重、题型最全的一章：选择题每年必考 2～4 道，应用题（第 43 题）几乎每隔一两年就落在这里。它的特点是概念题靠背、计算题靠公式、设计题靠流程，三块都不能瘸腿。

本章的主线只有一条：**CPU** 越来越快，存储器跟不上，于是用「层次 + 局部性」把它们拼成一个整体。所有具体技术都是这条主线的产物——Cache 是为了弥合速度差，虚拟存储器是为了弥合容量差，多模块交叉是为了让主存吞吐率跟上总线，SSD 是为了干掉寻道时间。抓住这条主线，本章的每个公式都能自己推出来，不用死背。

本手册约定：地址一律按字节编址讨论（题目另行说明时以题目为准）；公式中的「行」与「块」分别指 Cache 行（Cache Line）与主存块（Block）。

一、存储器的分类

（一）按存储介质分类

存储元件必须有两个截然不同的物理状态才能表示 0 和 1。按介质分三类：

介质	典型器件	说明
半导体存储器	RAM、ROM、寄存器、Cache、SSD、U 盘	分 MOS 型与双极型两类；速度最快
磁表面存储器	磁盘（HDD）、磁带	靠磁头与磁介质相对运动做电磁转换；非易失；速度慢
光存储器	CD-ROM、DVD-ROM	靠激光在盘面凹坑上读取；非易失

SSD 不是磁盘，CD-ROM 不是 ROM

这是两个一字之差、考频极高的概念：

- 固态硬盘 **SSD** 是半导体存储器，基于闪存 Flash 技术，本质属于电可擦除 ROM（EEPROM）。真正的「磁盘」是机械硬盘 HDD。
- **CD-ROM** 是光存储器（属于辅存），不是计算机术语里那个作为半导体芯片的 ROM。CPU 可以直接从 BIOS ROM 中取指执行，但绝不可能直接执行 CD-ROM 里的内容——辅存必须先调入主存。

（二）按存取方式分类

类型	全称	存取时间特点	典型器件
随机存取 RAM	Random Access Memory	与地址无关，是常数	内存、Cache
顺序存取 SAM	Sequential Access Memory	取决于信息所在位置；以「记录块」为单位编址	磁带
直接存取 DAM	Direct Access Memory	先直接定位到区域，再在区域内顺序查找	磁盘、光盘
相联存取 AM/CAM	Associative / Content Addressed Memory	按内容检索，不需知道地址	快表 TLB、全相联 Cache 的标记比较

三个反直觉结论

1. **ROM** 和 **RAM** 都采用「随机存取」方式。虽然习惯上把 ROM 与 RAM 并列，但 ROM 读取任一单元的时间同样与物理位置无关——都是地址译码后定位。区别只在于 ROM 存储区在程序执行中只能读，不能随意改写。
2. 并非所有半导体存储器都按地址访问。全相联 Cache 的比较器、快表 TLB 是按内容访问（CAM）的，所以「半导体存储器都采用随机存取方式」是错的。
3. 光盘（**CD-ROM**）属于直接存取，不是随机存取，也不是顺序存取。

（三）按信息可更改性与易失性分类

- 可读可写存储器：RAM 芯片。

- 只读存储器：ROM 芯片。名称属于历史遗留，现代 ROM（E^2PROM、Flash）都能重写，只是写入慢、不能随意写。
- 易失性存储器：断电后信息消失。RAM、Cache。
- 非易失性存储器：断电后信息保持。ROM、磁表面存储器、光存储器。

（四）按作用与存取速度分类

层次	实现器件	位置与特点
寄存器存储器	触发器	CPU 内部，字长与机器字长相同；速度与 CPU 匹配，容量极小
高速缓冲存储器 **Cache**	**SRAM**	CPU 与主存之间；现代集成于 CPU 内；容量小、价格高
主存储器 **MM**	主要用 **DRAM**	CPU 可直接访问；可与 Cache、辅存交换数据
辅助存储器	磁盘、SSD	**CPU** 不能直接访问；容量极大、速度慢、位价格低

主存不等于「全是 DRAM」

现代 PC 的主存由 **RAM** 区 + **ROM** 区组成：RAM 区存放运行中的程序和数据，由 DRAM 实现；ROM 区存放 BIOS 等固化程序，由 ROM 芯片实现。所以「系统的主存都用 DRAM 芯片实现」是错误的。某些嵌入式系统的主存甚至全由 ROM 元件构成。

（五）存储器的性能指标

1.存取时间与存取周期

- 存取时间（**Access Time**）T_A：启动一次存储器操作到该操作真正完成所经历的时间。读、写时间可能不同（DRAM 读慢写快，闪存读快写慢）。
  - 读出时间：从收到有效地址和读命令，到数据稳定地送上数据总线。
  - 写入时间：从收到有效地址、数据和写命令，到数据被可靠写入。
- 存取周期（**Memory Cycle Time**）T_C：存储器进行两次独立的、连续的存取操作之间的最小时间间隔。它关注的是连续工作的吞吐能力。

$$T_C = T_A + T_R$$

其中 T_R 为恢复时间（**Recovery Time**）。DRAM 是破坏性读出，读出后必须把数据重写回去并让位线电压复位；地址译码器、读写控制逻辑翻转后也需要时间稳定，这些就是恢复时间的来源。

存取周期 > 存取时间，用哪个要看清题目

- 问「读一次要多久」「单次访问延迟」→ 用存取时间 T_A。
- 问「连续读取的最小时间间隔」「最大带宽」「流水线启动下一体的间隔」→ 必须用存取周期 T_C。

2.存储器带宽

存储器带宽（**Bandwidth**）：单位时间内存储器能传输的连续最大数据量，单位 B/s、MB/s、GB/s。

$$B_m = \frac{W}{T_C}$$

W 为数据位宽（字节数），T_C 为存取周期。

算例：某主存存取周期 50 ns、存取时间 40 ns、数据总线宽 64 位。

- 位宽 $W = 64/8 = 8$ B；连续访问的最小间隔受存取周期约束，取 $T_C = 50$ ns。
- $B_m = 8\ \text{B}/(50 \times 10^{-9}\ \text{s}) = 160 \times 10^6\ \text{B/s} = 160\ \text{MB/s}$。
- 题干中的「存取时间 40 ns」是干扰项。

提高带宽有两条路：缩短 T_C（提高工作频率，如 DDR 代际演进）与增加 W（双通道/四通道内存，位宽翻倍）。若 W 固定，则带宽的极限就是总线的传输能力，与存储器内部能并行准备多少个字无关。

（六）存储器的层次化结构

*1.*为什么需要分层

理想存储器要同时满足容量大、速度快、价格低，但任何单一技术都做不到：SRAM 快而贵且小，磁盘大而便宜却慢，且 CPU 性能提升速度长期快于存储器，形成「性能鸿沟」。分层存储把不同速度、容量、成本的器件组合成金字塔，使整个系统速度接近最高层、容量和单位成本接近最低层。

从 CPU 往外，自上而下：寄存器 → Cache → 主存 → 辅存（磁盘/SSD）→ 远程存储。四条规律：越往下，访问速度越慢、容量越大、位价格越低、被访问频次越低。寄存器、Cache、主存统称内部存储器，CPU 可直接访问；辅存及以下为外部存储器，信息必须先调入主存。

*2.*访问与调度规则

- 子集副本原则：上一层内容是下一层内容的副本。Cache 中的数据在主存中，主存中的数据在辅存中（采用写回策略时，Cache 中的新数据可能暂时还不在主存里）。
- 逐层查找原则：寄存器 → Cache → 主存 → 辅存。Cache 未命中访主存，主存没有访辅存。
- 数据传送只在相邻两层之间进行，单位是定长块。辅存中找到的数据不能直接送 CPU，必须经历：辅存 → 主存 → Cache → 寄存器/CPU。
- 特例：CPU 可以和主存直接交换信息（绕过 Cache）；主存是与 Cache、辅存同时打交道的枢纽。

*3.Cache-*主存层次与主存-辅存层次

对比维度	**Cache-**主存层次	主存-辅存层次
位置与访问权限	最靠近 CPU，CPU 可直接访问两者	CPU 不能直接访问辅存
引入目的	解决速度问题	解决容量问题，形成虚拟存储器
交换单位	主存块 **Block**，通常 8 ~ 128 B	页 **Page**，通常 4 KB ~ 64 KB
缺失处理	硬件自动完成	操作系统（软件）完成
映射方式	直接、全相联、组相联均可	一律全相联
映射记录介质	Cache 行的 Tag 字段	主存中的页表
写策略	直写或回写均可	一律回写
透明性	对所有程序员透明	对应用程序员透明，对系统程序员不透明

两者有四点共性：都基于程序访问的局部性；都要解决地址映射；都要解决替换；缺失时都要把包含目标信息的块从慢速层调入快速层。

缺页代价远大于 Cache 缺失，所以页大、全相联、只回写

主存比 Cache 慢 10 ~ 100 倍，辅存比主存慢约 10000 倍。所以：页面远大于主存块（4 KB 对 32 B）；主存-辅存层次必须用全相联把冲突缺失降到最低；写操作只能回写——每次改主存都同步写磁盘是不可承受的。

（七）程序访问的局部性原理

在较短时间间隔内，程序产生的地址往往集中在存储空间的很小范围内，这叫程序访问的局部性，是分层存储能生效的全部前提。

- 时间局部性（**Temporal Locality**）：某个存储单元在被访问后，短时间内很可能再被访问。来源：循环体、循环变量、高频调用的函数。
- 空间局部性（**Spatial Locality**）：某个存储单元的邻近单元在短时间内很可能被访问。来源：顺序执行的指令、数组的连续元素。

步长 k：访问连续数组时每隔 k 个元素访问一次。步长为 1 即顺序引用模式，是空间局部性最重要的来源；步长越大，空间局部性越差。

三条速判原则

1. 变量重复原则：对同一变量有重复访问 → 时间局部性好。
2. 步长最小原则：步长越小 → 空间局部性越好，步长为 1 最佳。
3. 循环体原则：循环体一般时间和空间局部性都好；循环体越小、迭代次数越多，局部性越好。

算例（一维数组遍历）

```
for (i = 0; i < N; i++)
    sum += v[i];
```

- 标量 `sum`、`i`：每次迭代都被引用 → 时间局部性极好；单变量，讨论空间局部性无意义。
- 数组元素 `v[i]`：按存储顺序逐个读取，步长为 1 → 空间局部性极好；但每个元素只访问一次 → 时间局部性差。

算例（行优先 vs 列优先二维数组）

数组按行优先存放，`M = N = 2048`，每元素 4 B。按行遍历 `a[i][j]`（内层 j 递增）时步长为 1，每取一个元素都能顺带命中同块内后续元素，空间局部性好；按列遍历（内层 i 递增）时相邻访问间隔 $N \times 4 = 8192$ B，远超一个 Cache 块，每次访问几乎都缺失，空间局部性极差。同样一份数据、同样的计算量，仅交换循环次序就可能让 Cache 缺失率相差十倍以上。

局部性的统考考法

【2017-14】给出嵌套循环 `for(i=0..9) { temp=1; for(j=0..i) temp*=a[j]; ... }`，问数组 a 的访问局部性。答案是时间局部性与空间局部性皆有：内层循环顺序访问 `a[0..i]` 步长为 1（空间局部性好），且靠前的元素被外层循环反复重复访问（`a[0]` 访问 10 次、`a[1]` 访问 9 次，时间局部性好）。做题时要把「重复访问 → 时间」「按序访问 → 空间」两条分别对着代码数一遍，不要只凭感觉。

二、半导体随机存取存储器

（一）半导体存储芯片的内部结构与译码方式

一个半导体存储芯片由存储矩阵、译码驱动电路、读写控制电路三部分组成，并对外引出四类信号线：

信号线	方向	作用
地址线	单向（入）	送入地址，经译码器选中单元
数据线	双向	读出数据或写入数据
片选线 CS	单向（入）	选中该芯片；多片组成系统时用于字扩展
读写控制线 WE	单向（入）	下达读/写命令

相关术语：记忆单元（存储元/位元）是能表示 0/1 的物理器件；存储单元是具有相同地址的一组位，即一个编址单位；存储体（存储矩阵/阵列）是所有存储单元的集合。

*1.*单译码结构（线选法）

n 位地址直接送入一个 n–2^n 译码器，每根输出线作为一条字线，被选中的字线激活整行存储单元，其上的所有位构成一个「字」被同时读写。这种芯片称为字片式芯片。

致命缺陷：地址位数多时输出线爆炸。例如 $n = 16$ 时需要 16–65536 译码器，输出 65536 根字线，硅片上布线密集、寄生电容极大、良率骤降，物理上不可实现。所以大容量芯片不用一维单译码。

*2.*双译码结构（重合法）

地址译码器分成 X、Y 两组。以 4096 个单元、12 位地址为例：$A_0 \sim A_5$ 送 X 译码器产生 64 条行选择线，$A_6 \sim A_{11}$ 送 Y 译码器产生 64 条列选择线，行列交叉唯一确定一个单元。若地址为 `000000000001`，则 X 译码器第 2 根输出线 x_1 有效、Y 译码器第 1 根输出线 y_0 有效，选中单元 $(1, 0)$。

被选中的行列交叉点上只有一位，所以这种芯片称为位片式芯片。若存储阵列采用三维结构（多个位平面），则不同位平面在同一行列交叉点上的多位构成一个存储字，被同时读写——这本质上就是位扩展。

$$\text{双译码结构译码输出线总数} = \text{行地址线数} + \text{列地址线数}$$

单译码 vs 双译码

- 单译码（线选）：输出线 $= 2^n$，线多、速度快，只适合小容量。
- 双译码（重合）：输出线 $= 2^{n/2} \times 2$，线少，是大容量芯片的必然选择。
- 记忆锚点：双译码的译码输出线总数 = 行地址线数 + 列地址线数，而不是二者乘积，更不是 2^n。

（二）基本存储元件

存储单元电路必须具备四个条件：两种稳定状态；两状态可由外部信号控制互相转换；能被读出；无外部原因时信息能长期保存。

1.SRAM：六管静态 *MOS* 存储元件

6 个 MOS 管中，4 个构成一对正负反馈门电路，即双稳态触发器；另外 2 个像「守门员」，控制数据进出。只要不断电，互相拉扯的平衡状态就永远保持，不需要任何额外动作维持——这就是「静态」的含义。

- 优点：速度快（直接读触发器状态）；不用刷新；读写简单、控制逻辑少。
- 缺点：面积大（6 个管子）、集成度低、成本高、功耗较大。
- 应用：**Cache（L1/L2/L3）**、DRAM 的行缓冲。

2.DRAM：单管动态 *MOS* 存储单元

只由 **1** 个 **MOS** 管 **+ 1** 个电容 C_S 构成，用电容上有无电荷表示 1/0。写 1 给电容充电，写 0 放电。读出时 C_S 上的电荷被稀释放电，电位下降，所以是破坏性读出，读后必须重写（再生）。电容会漏电，几毫秒内电荷就漏光，所以必须定期刷新。

- 优点：面积极小、集成度高、成本低、静态功耗小。
- 缺点：速度慢（读写要充放电）；必须刷新；断电或不刷新即丢数据。
- 应用：主存（**DDR** 内存条）、显存。

特性	**SRAM**（六管）	**DRAM**（单管）
存储原理	双稳态触发器（电路自锁）	电容充放电（保存电荷）
核心元件	6 个 MOS 管	1 个 MOS 管 + 1 电容
读出后是否需重写	不需要	需要（破坏性读出）
是否需要刷新	不需要	必须定期刷新
读写速度	快（纳秒级）	较慢（需充放电）
集成度	低	高
成本 / 功耗	高 / 较大	低 / 小
典型应用	Cache	主存、显存

SRAM 与 DRAM 的三组混淆点

1. 「静态/动态」指的是维持状态的方式：静态靠电路自锁不需干预，动态靠电荷会衰减、需动态刷新。不是说 SRAM 的数据不变。
2. 两者都是易失性的。只是 DRAM 更「脆弱」——不断电但停止刷新同样会丢数据。
3. **DRAM** 的「读操作」和「刷新操作」不是一回事。刷新本质上是一次没有第 **5** 步（数据输出）的读操作：行地址由芯片内部刷新计数器给出，不发列地址，不向 CPU 输出数据。读操作则必须给行、列地址并把数据送上数据总线。

（三）DRAM 的刷新

*1.*刷新的基本规则

- 按行进行：每次刷新的单位是一行，该行所有列同时完成「破坏性读出 + 内部重新充电恢复」。因此刷新只需要行地址，不需要列地址。
- 占用一个存取周期：刷新一行的耗时等于一次正常读写的时间，记为 t_c。
- 地址由内部计数器生成：刷新行地址不是 CPU 提供的，而是 DRAM 芯片内部（或内存控制器）

的刷新地址计数器自动产生并逐行加 1。芯片内部有一个多路选择器 MUX，正常读写时切到外部行地址缓冲器，刷新周期时切到刷新计数器，屏蔽外部地址。

- 与 **CPU** 争用：刷新期间不能响应 CPU 的读写请求，这就产生了「内存争用」问题。
- 优化方向：为减少刷新总开销，芯片设计趋向减少行数、增加列数（如 128 行 ×2048 列而非对称矩阵），因为刷新开销只与行数有关。

*2.*三种刷新方式

统一算例：DRAM 为 128 行 ×128 列，读写周期 $t_c = 0.5\ \mu$s，最大刷新间隔 2 ms。

2 ms 内需完成全部 128 行刷新；$2\ \text{ms}/0.5\ \mu\text{s} = 4000$ 个时隙，从中选出 128 个时隙，每时隙刷新一行即可。

方式	原理	优点	缺点
集中刷新	前 $4000 - 128 = 3872$ 个周期全部用于读写，最后 128 个周期集中刷新所有行	读写期间不受刷新打扰，存储器速度快	存在长时间死区，期间 CPU 完全无法访存；行数越多死区越长
分散刷新	把存取周期拉长一倍：前半段读写或保持，后半段强制刷新一行，$t_s = 0.5 + 0.5 = 1\ \mu$s	彻底消除集中死区	刷新过于频繁，严重拖慢系统速度（每 1 μs 刷新一行，2 ms 内刷新约 15 遍，远超必要）
异步刷新	把 128 行均匀平摊到 2 ms 内：相邻两行刷新间隔 $= 2\ \text{ms}/128 = 15.625\ \mu$s，每次耗时 0.5 μs	既保证每行在 2 ms 内刷新一次，又把死区打散成 0.5 μs 的微小停顿	需要刷新定时逻辑；现代内存最常用的方式

刷新计算三个必背量

设芯片有 r 行，存取周期 t_c，最大刷新间隔 T_{max}：

- 刷新总次数 $= r$；刷新总耗时 $= r \cdot t_c$。
- 集中刷新死区时间 $= r \cdot t_c$，死区率 $= \dfrac{r \cdot t_c}{T_{\text{max}}}$。
- 分散刷新下存取周期变为 $2t_c$，有效访存速度降为原来的一半。
- 异步刷新下相邻两行刷新间隔 $= \dfrac{T_{\text{max}}}{r}$，每次停顿 t_c。

统一算例：$r = 128$，$t_c = 0.5\ \mu$s，$T_{\text{max}} = 2$ ms。集中刷新死区 $= 128 \times 0.5 = 64\ \mu$s，死区率 $= 64/2000 = 3.2\%$；异步刷新间隔 $= 2000/128 = 15.625\ \mu$s。

刷新与引脚数的统考考法

【2015-17】工作期间需要刷新的是（）。答案 **B. SDRAM**——凡基于电容存储的 DRAM 家族（含 SDRAM、DDR）都需要刷新；SRAM、ROM、Flash 都不需要。【2018-17】2K × 1 位 DRAM，要求地址引脚数最少且刷新开销尽量小，求行数 r、列数 c。答案 **C.（32，64）**。推导分两步：① DRAM 地址复用，地址引脚数 $= \max(\log_2 r, \log_2 c)$，要使它最小须让 r、c 尽量接近，$2\text{K} = 2048 = 2^{11}$，故 $r \cdot c = 2048$，选项 B（64×32）与 C（32×64）的引脚数都是 6，满足「最少」；② 刷新开销只与行数有关，r 越小越好，故选 $r = 32$。

（四）DRAM 的地址复用

DRAM 芯片引脚数直接决定封装成本，所以采用行列地址分两次传送：同一组地址引脚先接收行地址（由 $\overline{RAS}$ 行地址选通信号锁存），再接收列地址（由 $\overline{CAS}$ 列地址选通信号锁存）。

$$\text{逻辑地址位数} = \log_2(\text{存储单元数}) = \text{行地址位数} + \text{列地址位数}$$

$$\text{物理地址引脚数} = \max(\text{行地址位数},\ \text{列地址位数}) \approx \left\lceil \frac{\log_2 M}{2} \right\rceil$$

其中 M 为芯片的存储单元数（字数）。

算例 **1**：4M × 8 位 DRAM 芯片的地址引脚与数据引脚总数是多少?

- 逻辑地址位数 $= \log_2(4\text{M}) = \log_2(2^{22}) = 22$ 位。
- 复用后物理地址引脚 $= 22/2 = 11$ 根（行地址 11 位 + 列地址 11 位分时复用同一组 11 根引脚）。
- 数据引脚 $= 8$ 根（芯片位宽为 8）。
- 总数 $= 11 + 8 = 19$ 根。（本题即【2014-15】，答案是 19。）

算例 **2**：256K × 8 位 DRAM 芯片。

- 逻辑地址 $= \log_2(2^{18}) = 18$ 位 $\rightarrow$ 物理地址引脚 $18/2 = 9$ 根。
- 数据引脚 8 根。总数 $= 17$ 根。

算例 **3**：8192 × 8192 × 8 位 DRAM 芯片。

- 逻辑地址 $= 13 + 13 = 26$ 位；但行列已明确各 13 位，故物理地址引脚 $= \max(13, 13) = 13$ 根（不是 26 根！）。
- 数据引脚 8 根，总数 $= 21$ 根。芯片内行缓冲容量 $= 8192 \times 8$ 位（激活一整行时导出到读出放大器的数据量）。

DRAM 引脚计算的三个坑

1. 只要题目出现「**DRAM**」，地址引脚就必须除 **2**（或取行列最大值）；SRAM 不复用地址，引脚数就是完整逻辑地址位数。4M × 8 的 SRAM 需要 22 根地址引脚，DRAM 只需 11 根。
2. 问的是「芯片」的引脚数，与系统总容量无关。「256MB 存储器由若干 4M × 8 位 DRAM 芯片构成」这个总容量只用于算芯片片数，不影响单片引脚数。
3. 行列不等时要取最大值。32 × 64 的阵列，行地址 5 位、列地址 6 位，复用后引脚数是 $\max(5, 6) = 6$ 根，而不是 $11/2 = 5.5 \rightarrow 6$ 以外的任何数。

（五）只读存储器 ROM 与 Flash 存储器

1.ROM 的演进

类型	写入方式	擦除方式	特点
MROM（掩膜式）	厂家制造时写入，固定掩膜	不可擦除	内容出厂固定，可靠性高，灵活性最差
PROM（可编程）	用户用编程器写入（熔丝/反向二极管击穿）	不可擦除	只能一次性编程，写错即报废
EPROM（可擦除可编程）	编程器写入，浮置栅 MOS 管	紫外线照射，约 8～20 分钟	可多次修改，但必须整片擦除

类型	写入方式	擦除方式	特点
EEPROM / E^{2}PROM（电可擦除）	加电写入	电擦除	可精确到字节删除/重写，写入时间远长于读取
Flash 闪存	在线擦除和重写（写命令到命令寄存器）	电擦除	以「块 **Block**」为基本单位擦除；读快写慢
SSD 固态硬盘	基于闪存芯片阵列	电擦除	广义属于 ROM，但已无「只读」限制

记忆口诀：名字带 **E** 的都能改写（E 代表 Erasable）；有两个 **E**（**EEPROM**）说明用最方便的电来擦除。擦除粒度：EPROM 整片 > Flash 按块 > EEPROM 按字节。

2.Flash 存储器的特点

Flash 兼有 RAM 和 ROM 的优点：高密度、非易失、功耗低、集成度高、不需后备电源。它沿用 EPROM 的浮栅/热电子注入写入方式，又具备 E^{2}PROM 的可擦除特点，因此又叫快擦型电可擦除重编程 **ROM**。常见应用：BIOS、U 盘、存储卡、SSD。

工作方式有读、编程、擦除、功耗下降四种，编程与擦除通过写命令到命令寄存器来管理。

Flash 最核心的一条：读快写慢，且写前必须先擦

- 读直接感应浮栅 MOS 管的导通/截止，速度与半导体 RAM 相当（纳秒级）。
- 写必须先擦除（按块施加高压使电子隧穿），再编程（注入电子），速度与硬盘相当（微秒～毫秒级）。
- 因此「信息可读可写，并且读写速度一样快」是错误的表述（【2012-16】的答案就是这一句）。
- 擦除单位是块，块内所有页一起被擦；擦干净的块中每页可写一次、读无限次。

3.NOR Flash 与 *NAND Flash*

维度	**NOR Flash**	**NAND Flash**
读出方式	随机存取，可直接按地址取任一字节	按页读出，随机存取粒度粗
读出速度	快，支持片内执行（XIP）	较慢，但顺序/页读吞吐高
写入与擦除速度	慢	快
集成度与位价格	低、贵	高、便宜
耐久性（擦写次数）	约 $10^4 \sim 10^5$ 次	早期约 $10^3 \sim 10^4$ 次，现代 3D NAND 显著提高
典型用途	存放固件代码（BIOS、嵌入式 boot）	U 盘、SSD、存储卡等大容量存储

Flash 的擦写寿命与「先擦后写」

Flash 的每个块可擦除次数有限，反复写同一个块会把它写坏（机械硬盘的扇区不会因为写得多而损坏）。此外，要写某页信息时必须先擦除该页所在的整块；若该块内其他页有数据，得先把它们复制到一个擦除过的新块中，再写入目标页。这就是 SSD 写放大与需要「垃圾回收 + 磨损均衡」的根本原因。

三、主存储器

（一）DRAM 芯片与内存条

现代内存条由多片 DRAM 芯片按低位交叉编址 + 同时启动的方式组装，并配以 64 位（或更宽）的存储器总线。

典型结构算例：用 8 片 16M $\times$ 8 位 DRAM 芯片扩展成一个 128 MB 内存条，按字节编址。每片内部是 $4096 \times 4096 \times 8$ 位阵列，行列地址各 12 位（$2^{12} = 4096$），含 8 个位平面。

- 主存地址共 27 位（128 MB $= 2^{27}$ B），高 24 位为行、列地址，低 3 位用于选择 8 片中的哪一片。
- 读数据时，高 12 位行地址同时送到 8 片芯片，把对应行读入各自的行缓冲；再把列地址同时送到 8 片，各取 8 位，组合成 **64** 位送上总线。
- CPU 再根据主存地址低 3 位，从存储控制器缓冲区中取出自己需要的那一个字节。

这一结构直接解释了数据对齐的必要性：一次访存固定取出第 $0 \sim 7$、$8 \sim 15$、……、$8k \sim 8k+7$ 号单元。一个 `int`（4 B）若从地址 **8** 开始存放（对齐，起始地址是 4 的倍数），只需访问主存 **1** 次；若跨在地址 **6**、**7**、**8**、**9** 上（不对齐），则跨越了两个 8 字节块，需访问 **2** 次。

（二）存储器容量的扩展

当单片芯片的字数或字长不够时，需要在位（字长）和字（地址空间）两个方向扩展。

$$\text{位扩展片数} = \frac{\text{目标数据位宽}}{\text{单片数据位宽}}, \qquad \text{字扩展组数} = \frac{\text{目标地址空间}}{\text{单片地址空间}}$$

$$\text{总片数} = \text{位扩展片数} \times \text{字扩展组数} = \frac{\text{目标总容量}}{\text{单片容量}}$$

以「目标 $M \times N$ 位，单片 $m \times n$ 位」为通用记号：

$$\text{位扩展片数} = \frac{N}{n}, \qquad \text{字扩展组数} = \frac{M}{m}, \qquad \text{总片数} = \frac{N}{n} \times \frac{M}{m} = \frac{M \times N}{m \times n}$$

*1.*位扩展法

适用：芯片数据位宽 < 目标数据位宽（地址空间已够）。目的：加宽字长，地址空间不变。

信号线	连接方式
地址线	全部并联，接系统地址总线
片选 CS	全部并联，同时选中
读写控制线	全部并联
数据线	各芯片单独引出，分别接数据总线的不同位

核心特征：所有芯片同时工作，共同提供一个完整的字。

算例：用 4 片 4K $\times$ 1 位 SRAM 构成 4K $\times$ 4 位存储器。

- 位扩展片数 $= 4/1 = 4$ 片；地址空间不变，仍为 4K，需 12 根地址线 $A_0 \sim A_{11}$。
- 4 片地址线并联、片选并联、WE 并联；第 1 片的数据线接 D_0，第 2 片接 D_1，第 3 片接 D_2，第 4 片接 D_3。

*2.*字扩展法与片选方式

适用：芯片字数 < 目标字数（位宽已够）。目的：扩大地址空间，每单元位数不变。

信号线	连接方式
地址线低位	全部并联，作片内寻址
地址线高位	接译码器（或直接接片选），作片间选择
数据线	全部并联
读写控制线	全部并联
片选 CS	各芯片独立，由高位地址译码后分别控制

核心特征：各芯片分时工作，同一时刻只有一片被选中。

片选方式一：线选法

取字选之外的高位地址线直接接到各芯片的 CS 端。规定某根高位地址线为 0 时选中对应芯片，其余高位必须为 1。

- 优点：不需译码器，电路简单。
- 缺点：地址空间不连续，存在大量空洞；片选线必须满足「每次只有一位有效」。

算例：用 4 片 8K × 8 位 SRAM 组成 32K × 8 位，采用线选法。

- 字扩展 4 片，数据位宽已满足。芯片地址线 13 根（$2^{13} = 8\text{K}$）。
- 若只用 4 根高位线 $A_{13} \sim A_{16}$ 分别接 4 片的 CS，且要求同一时刻只有一根为低，则 CPU 需要 17 根地址线。
- 各片地址范围（地址不连续）：

芯片	高位 $A_{16}A_{15}A_{14}A_{13}$	最低地址	最高地址
第 1 片	111**0**	1110 0000 0000 0000	1110 1111 1111 1111
第 2 片	11**0**1	1101 0000 0000 0000	1101 1111 1111 1111
第 3 片	1**0**11	1011 0000 0000 0000	1011 1111 1111 1111
第 4 片	**0**111	0111 0000 0000 0000	0111 1111 1111 1111

片选方式二：译码片选法（主流）

把高位地址线送入地址译码器，由译码器输出产生片选信号。

- 优点：地址空间连续，充分利用地址资源。
- 缺点：需要译码器，电路略复杂。

算例：用 4 片 8K × 8 位 SRAM 组成 32K × 8 位，采用译码片选法。

- CPU 需 15 根地址线（$2^{15} = 32\text{K}$）、8 根数据线；芯片 13 根地址线、8 根数据线。
- 高位地址线 $= 15 - 13 = 2$ 根（A_{14}、A_{13}），接 **2-4** 译码器。
- $A_0 \sim A_{12}$ 并联到 4 片芯片作片内寻址；译码器输出 0/1/2/3 分别接 4 片的 CS。

芯片	$A_{14}A_{13}$	地址范围（十六进制）
第 1 片	00	0000H ~ 1FFFH
第 2 片	01	2000H ~ 3FFFH
第 3 片	10	4000H ~ 5FFFH
第 4 片	11	6000H ~ 7FFFH

*3.*字位同时扩展法

适用：字数和字长都不够。分两步走：

```
第一步：先按【位扩展】把若干芯片组成一个" 逻辑组"，满足数据位宽要求
第二步：再按【字扩展（译码片选）】把多个" 逻辑组" 连接，满足地址空间要求
```

- 组内：地址线、片选线、控制线并联，数据线按位独立引出（位扩展特征）。
- 组间：片选由高位地址译码后各自控制，同一时刻只有一组工作（字扩展特征）。

算例：用 8 片 8K × 4 位 RAM 构成 32K × 8 位存储器。

- 位扩展片数 $= 8/4 = 2$ 片一组（两片拼成 8K × 8 位）。
- 字扩展组数 $= 32\text{K}/8\text{K} = 4$ 组。
- 总片数 $= 2 \times 4 = 8$ 片 ✓。
- 组内：芯片 ①② 为第 1 组，地址线、CS、WE 并联，① 的数据线接 $D_0 \sim D_3$、② 接 $D_4 \sim D_7$。
- 组间：CPU 高位 A_{14}、A_{13} 接 2-4 译码器，4 个输出分别控制 4 组 CS；$A_0 \sim A_{12}$ 并联到所有芯片。

各组地址分配：第 0 组 0000H ~ 1FFFH，第 1 组 2000H ~ 3FFFH，第 2 组 4000H ~ 5FFFH，第 3 组 6000H ~ 7FFFH。

三种扩展方式对比：

扩展方式	解决的问题	数据线	地址线低位	地址线高位	片选线	时序
位扩展	数据位宽不够	各自独立引出	全部并联	全部并联	全部并联	同时工作
字扩展	地址空间不够	全部并联	全部并联	接译码器	各自独立	分时工作
字位同时	两者都不够	组内并联、按位独立	全部并联	接译码器	组内并联、组间独立	组内同时、组间分时

（三）主存与 CPU 之间的连接

主存与 CPU 之间通过三类总线连接：

- 地址总线 $A_0 \sim A_{k-1}$（单向）：低若干位送片内地址、高若干位送译码器。
- 数据总线 $D_0 \sim D_{w-1}$（双向）：位扩展时按位分组接不同芯片。
- 控制总线（单向）：读/写命令 WE、访存控制信号 MREQ、片选信号 CS。CPU 访问 I/O 与访问主存共用地址线与数据线，靠 MREQ 区分。

*1.*存储器扩展设计题的完整步骤

已知：单片芯片规格 $m \times n$ 位，目标主存规格 $M \times N$ 位（或给定地址范围）。

```
Step 1  算芯片总数与组织方式
        位扩展片数 = N / n          （片/组）
        字扩展组数 = M / m          （组数）
        总片数 = 位扩展片数 * 字扩展组数

Step 2  分配地址线
        CPU 地址线总根数 = log2(M)（或按题目给定地址空间取 log2(地址空间大小)）
        片内地址线根数   = log2(m)       -> 接各芯片地址引脚（全部并联）
        片选译码所需位数 = log2(组数)    -> 接译码器输入

Step 3  分配数据线
        每组内各芯片数据线独立引出，拼满 N 位

Step 4  选片选方式
        组数少、对地址连续性要求低 -> 线选法
        组数多、地址需连续          -> 译码片选法（主流）

Step 5  画连接图并写出各片地址范围
        组内：地址/CS/WE 并联，数据线按位独立
        组间：低位地址并联，高位地址-> 译码器-> 各组 CS
        地址范围 = [组号<<log2(m) , (组号 +1)<<log2(m) - 1]
```

通用模板题

目标 $M \times N$ 位，单片 $m \times n$ 位（M、m 为字数，N、n 为位宽，均取 2 的幂）：

$$\text{总片数} = \frac{N}{n} \times \frac{M}{m}, \quad \text{片内地址线} = \log_2 m, \quad \text{片选译码位数} = \log_2 \frac{M}{m}, \quad \text{CPU 地址线} = \log_2 M$$

算例：32K × 8 位由 8K × 4 位芯片组成。总片数 $= (8/4) \times (32\text{K}/8\text{K}) = 2 \times 4 = 8$ 片；片内地址线 $= \log_2 8\text{K} = 13$ 根；片选译码 $= \log_2 4 = 2$ 位（用 2-4 译码器）；CPU 地址线 $= \log_2 32\text{K} = 15$ 根。

扩展类真题只考几件事

【2009-15】主存 64 KB，ROM 区 4 KB、其余为 RAM 区，用 2K × 8 位 ROM 与 4K × 4 位 RAM 设计。答：**D**（**2**、**30**）。ROM 片数 $= 4\text{KB}/2\text{KB} = 2$；RAM 区 $= 64 - 4 = 60$ KB，4K × 4 位需 2 片一组凑 8 位，$60\text{KB}/4\text{KB} = 15$ 组，共 $15 \times 2 = 30$ 片。【2010-15】若干 2K × 4 位芯片组成 8K × 8 位，地址 0B1FH 所在芯片的最小地址。答：**D**（**0800H**）。位扩展 2 片一组 → 2K × 8；字扩展 4 组。0B1FH = 0000 1011 0001 1111，高 2 位 01 决定组号 1，该组最小地址 = 0100 0000 0000 0000 = 0800H。【2011-15】按字节编址，主存地址空间 64 MB，实际装 4M × 8 位 RAM 共 32 MB，问 MAR 位数。答：**D**（**26** 位）。**MAR** 位数由主存地址空间决定，与实际安装容量无关：$\log_2 64\text{M} = 26$。【2016-16】64 KB 按字节编址，4000H ~ 5FFFH 为 ROM 区，其余 RAM，用 8K × 4 位 SRAM。答：**C**（**14** 片）。ROM 区 = 2000H = 8 KB，RAM 区 = 56 KB；8K × 4 位需 2 片一组，$56/8 = 7$ 组，共 14 片。【2021-15】24 位地址线、32 位数据线、按字编址、字长 32 位，000000H ~ 3FFFFFH 为 RAM，用 512K × 8 位芯片。答：**C**（**32** 片）。先注意「按字编址」：地址范围 400000H = 2^{22} 个字 × 4 B = 16 MB；片数 $= 16\text{MB}/512\text{KB} = 32$。【2023-15】CPU 30 根地址线，按字节编址，须占满全部地址空间，RAM:ROM = 3 : 1，RAM 在低地址、ROM 在高地址。答：**C**（**30000000H～3FFFFFFFH**）。总空间 $2^{30} = 1$ GB，ROM = 256 MB 位于最高段，起始地址 $= 1\text{GB} - 256\text{MB} = 768\text{MB} = 30000000\text{H}$。

扩展题高频失分点

1. 「按字编址」还是「按字节编址」看漏。按字编址时地址单元数 × 每字字节数 = 总容量，与按字节编址的换算完全不同。

2. 把「主存地址空间」当成「实际安装容量」。MAR 位数（以及 CPU 地址线根数）由地址空间决定，装多少内存不影响 MAR。
3. 算完总容量就忘了位扩展。4K × 4 位芯片装 8 位宽的内存，必须先 2 片一组，否则永远算少一半。
4. 地址范围算错。RAM 在低地址段时从 0000H 起；ROM 在高地址段时从「总空间 − ROM 容量」起。把两者顺序搞反直接全错。
5. 译码器位数算成组数。n 个组需要 $\log_2 n$ 根高位地址线，接 n 输出的译码器。

（四）多模块存储器

CPU 速度远快于主存。多模块（多体）存储器是一种空间并行技术：用多个结构完全相同的存储模块并行工作，提高吞吐率，而不改变单个模块的存取速度。

*1.*两种编址方式

对比维度	连续编址（高位交叉 / 顺序编址）	交叉编址（低位交叉）
地址结构	[高位：模块号][低位：模块内地址]	[高位：模块内地址][低位：模块号]
相邻地址	在同一模块内	在不同模块中
体号公式	模块号 $=\lfloor$地址$/m\rfloor$（高位）	体号 = 地址 mod m（低位）
能否并行	不能，无法提高吞吐率	能，可实现流水线并行访问
适用场景	扩充容量（内存条插槽）	突发顺序访问（CPU 取指、DMA、Cache 调块）

高位交叉为什么并行不了：程序具有局部性和连续性，连续执行的指令和数据基本落在同一个存储体里，该体访问频繁、其余体空闲，多体形同虚设。

低位交叉为什么有效：连续地址分散在不同体，各体有独立的读写电路，可以不用等前一个体准备好就开始下一个体，各体并行准备数据。

编址方式与扩展方式不是一回事

- 高位交叉 = 顺序编址，等价于字扩展的思路（分时工作）。
- 低位交叉的「同时启动」方式在连接上就是位扩展（多个芯片同时工作，各自提供总线的一部分位）。名字侧重不同：位扩展强调容量扩展，多模块交叉强调编址方式。

*2.*低位交叉的流水存取时间

设存储周期为 T、总线传输周期为 τ、交叉模块数为 m。访存分三个阶段：T_1 送地址与命令、T_2 存储器准备数据（即存取时间）、T_3 总线传输一次数据（τ）。

教材与真题默认忽略恢复时间，令存储周期 ≈ 存取时间，即经过一个 T 后数据才送上数据线。

轮流启动方式：每个模块一次读写的位数 = 总线数据位数（每次只传一个模块的数据），流水条件是

$$T = m\tau$$

即每隔 τ 启动下一个模块。

$$t_{\text{准备}}(n) = T + (n-1)\tau$$

$$t_{总}(n) = \underbrace{\tau}_{\text{送首地址与命令}} + \underbrace{T + (n-1)\tau}_{\text{存储器准备}n\text{ 个字}} + \underbrace{\tau}_{\text{传送最后一个字}} = T + (n+1)\tau$$

逐步推导（以 $m = 4$、$T = 4\tau$、连续读 n 个存储字为例）：

1. t_0：CPU 发出首地址与读命令，耗时 τ。
2. $t_1 = \tau$：启动体 M_0；此后每间隔 τ 依次启动 M_1、M_2、M_3、M_0……
3. M_0 经过完整存储周期 $T = 4\tau$ 后，在 $t_2 = \tau + T$ 时刻把第一个数据送上数据线并开始传送。
4. 从此进入流水线稳定状态：每经过一个 τ 就有一个数据准备好送出。第 n 个字在 $\tau + T + (n-1)\tau$ 时刻准备好。
5. 最后加上传送第 n 个字所需的一个 τ。

$$t_{总}(n) = \tau + T + (n-1)\tau + \tau = T + (n+1)\tau$$

平均准备一个字的时间：

$$\bar{t} = \frac{T + (n-1)\tau}{n} \xrightarrow{n\to\infty} \tau$$

即 n 很大时，存储器平均每个存储周期（每 τ）就能向 CPU 提供一个字——稳定流水时一个存储周期内向 **CPU** 提供 m 个数据。

对比高位交叉（连续编址）：连续地址都在同一个体，两次读之间必须间隔 T，故

$$t_{准备}(n) = nT, \qquad \bar{t} = T \quad (\text{与} n \text{ 无关})$$

统一算例：$m = 4$ 体低位交叉，τ 为总线传输周期，$T = 4\tau$，连续读取 5 个地址连续的存储字 $W_0 \sim W_4$。

- 低位交叉：$t_{准备} = T + (5-1)\tau = 4\tau + 4\tau = 8\tau$；总耗时 $= \tau + 8\tau + \tau = 10\tau = 2.5T$。
- 高位交叉：$t_{准备} = 5T = 20\tau$；总耗时 $= \tau + 20\tau + \tau = 22\tau = 5.5T$。
- 加速比约 2.2；若 n 更大，低位交叉趋近于每 τ 出一个字（加速比趋近 $m = 4$）。

低位交叉计算必背四条

1. 流水条件：$T = m\tau$（每隔 τ 启动一个体）。
2. 准备 n 个字的时间：$T + (n-1)\tau$。
3. 含总线开销的总时间：$T + (n+1)\tau$（送首地址 τ + 准备 + 送尾数据 τ）。
4. 稳定流水时：一个存储周期内提供 m 个字，平均每字耗时 τ。

低位交叉的统考考法

【2015-18】4 体交叉编址，主存地址序列 `8005,8006,8007,8008,8001,8002,8003,8004,8000`，问可能发生访存冲突的地址对。答：**D**（**8000** 和 **8004**）。体号 = 地址 mod 4：`8004` 体 0、`8008` 体 0 但相隔 4 步（不冲突）；`8000` 与 `8004` 同为体 0 且相邻（相隔 1 步 < 4），必然冲突。【2017-13】4 片 64M × 8 位 DRAM 交叉编址构成主存，总线宽 32 位，每次最多读写 32 位，`double` 型变量 x 地址为 `804001AH`，读取 x 需要几个存储周期。答：**C**（**3**）。每片 8 位、4 片并行正好 32 位，符合同时启动条件。`double` 占 8 B，起始地址低 2 位为 `10`（体号 2），8 个字节跨越 $8000 \sim 8007$、$8008 \sim 800F$ 等 8 字节边界，需 **3** 次并行读取。【2013-43】32 位机，主存 8 体交叉，每体存储字长 32 位、存储周期 40 ns，总线宽 32 位、时钟 200 MHz（周期 5 ns），突发传送 32 B。一次读突发事务 = 5 ns（送地址）+ 40 ns（准备首个数据）+ 8×5 ns（传送 8 次数据）= 85 ns。注意「准备数据」只算第一个数据的时间，后续数据靠流水线补上。

*3.*访存冲突

冲突条件：m 体交叉存储器中，两次连续访问落在同一个体，且时间间隔小于存储周期 T（即访问步数差 $< m$）。

判断步骤：

1. 对每个地址求 mod m，得到体号；
2. 在访问序列中找体号相同的地址对；
3. 检查它们在序列中的间隔步数是否 $< m$。

无冲突的理想条件：相邻 m 个访问地址分别属于 m 个不同的体（体号连续不重复）。

冲突题的两个陷阱

1. 只看体号相同就判冲突是错的，必须同时满足「间隔步数 < m」。8004 与 8008 都是体 0，但序列中相隔 4 步（$= m$），不冲突。
2. 「可能」二字很关键。题目问「可能发生冲突的地址对」，只要存在某种时刻安排让其冲突即可，不需要它在给定序列中真的紧邻。

四、外部存储器

（一）磁盘存储器

*1.*组成与物理结构

磁盘存储器由磁盘控制器、磁盘驱动器、盘片三大部分组成。

- 磁盘控制器：磁盘与主机的 I/O 接口，含控制逻辑、时序电路、「并 → 串」与「串 → 并」转换电路。
- 磁盘驱动器：读写电路、读写转换开关、读写磁头与磁头定位伺服系统。
- 盘片与盘面：每个盘片的两个面各有一个磁头，因此磁头号 = 盘面号。

关键术语：

- 磁道（**Track**）：磁头与盘片相对运动形成的同心圆。磁道从外向里编号，最外圈为 0 号磁道。
- 柱面（**Cylinder**）：多个盘片上相同编号的磁道构成一个柱面，所以磁道号 = 柱面号。
- 扇区（**Sector**）：每个磁道被分成的若干等长区域，磁盘读写的最小单位（传统 512 B，现代常用 4 KB）。

写入顺序：写完一个柱面上所有磁道后才移动到下一个柱面。

*2.*磁盘的物理地址与读写过程

地址结构：[驱动器号][柱面（磁道）号][盘面（磁头）号][扇区号]。

为什么是「柱面号在前、盘面号在后」

若地址结构是（盘面号，柱面号，扇区号），读完盘面 0 的整条磁道后，要继续读相邻磁盘区域就必须启动磁头臂移动到下一个磁道；而（柱面号，盘面号，扇区号）下，同一柱面的不同盘面柱面号相同，只需激活相邻盘面的磁头即可，不必移动磁头臂，省下大量寻道时间。这就是柱面号必须放在盘面号之前的根本原因。

磁盘读写共三步：寻道 → 旋转等待 → 读写。

- 寻道：磁盘控制器把盘地址送到驱动器后，启动磁头定位伺服系统，按磁头号与柱面号把指定磁头移到指定柱面，完成后发出寻道结束信号。
- 旋转等待：盘片旋转，扇区计数器每收到一个扇区标志脉冲加 1，与地址寄存器中的扇区地址比较，一致时输出扇区符合信号。
- 读写：写操作把数据送写入电路生成写电流脉冲；读操作由读出放大电路读出内容送控制器。

磁盘既不能像 RAM 那样按地址随机访问任一单元，也不是完全按顺序存取，而是先定位磁道、再在磁道内顺序查找，因此属于直接存取（**DAM**）。

*3.*性能指标

（**1**）记录密度

- 道密度：沿磁道分布方向上单位长度内的磁道数目。
- 位密度：沿磁道方向上单位长度内存放的二进制信息数目。

低密度存储方式下所有磁道扇区数相同 → 每个磁道位数相同 → 内道位密度比外道高；高密度存储方式下各磁道位密度相同 → 外道扇区数多于内道 → 整盘容量大得多。

（**2**）存储容量

$$\text{非格式化容量（未格式化容量）} = \text{位密度} \times \text{内圈周长} \times \text{总磁道数}$$

$$\text{格式化容量} = \text{每扇区字节数} \times \text{每磁道扇区数} \times \text{磁道数（柱面数} \times \text{盘面数）}$$

$$\text{磁盘总容量} = \text{柱面数} \times \text{磁头数（盘面数）} \times \text{每磁道扇区数} \times \text{每扇区字节数}$$

非格式化容量 > 格式化容量：低级格式化要在每扇区加头部（扇区号、同步信息）、尾部（ECC 纠错码）以及扇区间隔，这些都要占用磁介质空间。

算例：某磁盘 8 个柱面、4 个盘面、每磁道 16 个扇区、每扇区 512 B。

格式化容量 $= 8 \times 4 \times 16 \times 512\ \text{B} = 256\ \text{KB}$。

（**3**）平均存取时间

磁盘出队后要经历寻道、旋转等待、读写数据传输三个过程：

$$\text{响应时间} = \text{排队延迟} + \text{控制器时间} + \text{寻道时间} + \text{旋转等待时间} + \text{数据传输时间}$$

$$\boxed{\text{存取时间} = \text{寻道时间} + \text{旋转（等待）延迟} + \text{传输时间}}$$

- 寻道时间 $T_s = p + m \cdot n$：p 为启动磁头臂的时间，m 为每跨越一个磁道的耗时，n 为需跨越的磁道数。
- 旋转延迟（平均）：

$$T_r = \frac{1}{2} \times \frac{1}{r} = \frac{1}{2r} \quad (r：\text{转/秒}), \qquad T_r = \frac{1}{2} \times \frac{60}{r} = \frac{30}{r}\ \text{s} \quad (r：\text{转/分})$$

定位过程中最多转 1 圈、最少 0 圈，平均需半圈，所以除以 2。

- 传输时间（读 b 字节，每磁道 N 字节，转速 r 转/秒）：

$$T_t = \frac{b}{N} \times \frac{1}{r} = \frac{b}{rN}$$

因为读写一条完整的磁道恰好需要转一圈的时间 $1/r$。等价形式：访问一个扇区的传输时间 $= \dfrac{\text{一圈时间}}{\text{每磁道扇区数}} = \dfrac{\text{扇区容量}}{\text{数据传输率}}$。

旋转延迟只有两种算法，别混

- 转速单位是 **r/min**（如 7200 rpm）：$T_r = \dfrac{60}{2r} = \dfrac{30}{r}$ s。
- 转速单位是 **r/s**：$T_r = \dfrac{1}{2r}$ s。

题目给「r 秒/转」这种反着写的单位要特别小心——先统一成转/秒或转/分再算。

算例 **A**（【2013-21】）：转速 10000 rpm，平均寻道 6 ms，传输速率 20 MB/s，控制器延时 0.2 ms，读一个 4 KB 扇区。

- 转一圈 $= 60/10000 = 6$ ms $\rightarrow$ 平均旋转延迟 $= 6/2 = 3$ ms。
- 传输时间 $= 4\text{KB}/20\text{MB/s} \approx 0.2$ ms。
- 总时间 $= 6 + 3 + 0.2 + 0.2 = 9.4$ ms。（答案 B）

算例 **B**（【2015-20】）：转速 7200 rpm，平均寻道 8 ms，每磁道 1000 个扇区，访问一个扇区。

- 转一圈 $= 60/7200 = 8.333$ ms $\rightarrow$ 平均旋转延迟 $= 4.167$ ms。
- 一个扇区的传输时间 $= 8.333/1000 \approx 0.0083$ ms（可忽略）。
- 总时间 $= 8 + 4.167 + 0.0083 \approx 12.2$ ms。（答案 B）

（4）数据传输率

$$\text{内部数据传输率} = \frac{\text{转速 (转/分)}}{60} \times \text{内圈周长} \times \text{最内道位密度}$$

因为同一时刻只有一个磁头在读写，所以内部传输率由内圈（周长最小、位密度最高）决定。

磁盘的两个易错断言

1. 「磁盘的最小读写单位是一字节」错误，最小读写单位是扇区。哪怕只读 1 个字节，也要把包含它的整个扇区读出来（【2019-20】答案）。
2. 「格式化容量比非格式化容量小」正确。格式化要写扇区头、尾和间隔，占用空间；非格式化容量是按道密度与位密度算出的全部磁化单元总数。
3. 磁盘是机械设备，读写串行：不可能同一时刻既读又写，也不可能同时读两组数据。

（5）减少延迟的硬件手段

对一个盘面内的扇区采用交替编号，对不同盘面的扇区采用错位命名（均在低级格式化时完成），可以显著减少连续读写多条记录时的磁头等待时间。软件层面则靠磁盘调度算法减少寻道时间。

4.磁盘阵列 RAID

RAID（廉价冗余磁盘阵列）把多个独立物理磁盘组成一个逻辑盘，数据交叉存储、并行访问。基本思想三点：组织成阵列以增加容量；用类似主存多模块交叉的技术让多盘并行工作提高传输率；用冗余技术提高可靠性。

级别	定义	要点	类别
RAID0	无冗余、无校验	条带化交替存放、并行读写，容量与速度都提升，无容错	条带化
RAID1	镜像磁盘阵列	两盘同写互为备份，可靠性高，容量减半	镜像
RAID2	海明码纠错阵列	多个冗余校验盘，冗余开销大，已不再使用	并行访问
RAID3	位交叉奇偶校验	单个冗余盘 + 小条带交叉	并行访问
RAID4	块交叉奇偶校验	大数据块交叉，各盘操作独立	独立访问
RAID5	无独立校验盘的奇偶校验	校验块分布在各个磁盘中，成本不高效率高，广泛使用	独立访问

RAID 的考法

【2013-20】用于提高 **RAID** 可靠性的措施有：Ⅰ磁盘镜像 ✓、Ⅱ条带化 ×（只提速不提供冗余）、Ⅲ奇偶校验 ✓、Ⅳ增加 Cache ×（只提性能，写缓存掉电还可能丢数据）。答案 **B**（仅Ⅰ、Ⅲ）。

（二）固态硬盘 SSD

*1.*结构与读写特性

SSD 基于闪存 Flash 技术，属于电可擦除 ROM（EEPROM）。组成：

- 闪存翻译层：负责把逻辑块号翻译成物理页地址，相当于磁盘控制器。
- 存储介质：闪存颗粒。多个闪存芯片（**Flash Chip**） → 每芯片含多个块（**Block**） → 每块含多个页（**Page**）。

SSD 的四条读写特性

1. 以页为单位读写（页相当于磁盘的扇区）；以块为单位擦除。
2. 写某页前必须先擦除该页所在的整块。若块内其他页有数据，要先把它们复制到另一个已擦除的空块，再写入目标页——这就是 SSD 的「写放大」。
3. 支持随机访问：给定逻辑地址，闪存翻译层可通过电路迅速定位物理地址，没有寻道与旋转延迟。
4. 读快写慢：擦干净的块中每页可写一次、读无限次；读速度与 RAM 相当，写速度与硬盘相当。

*2.*与机械硬盘的对比

维度	机械硬盘 **HDD**	固态硬盘 **SSD**
访问定位方式	移动磁臂 + 旋转盘片（电路控制）	纯电路译码定位
寻道时间 / 旋转延迟	有，且是主要开销	没有
随机访问性能	差	极好
顺序读写性能	中等	高
噪音 / 抗震 / 功耗	有噪音、怕摔、功耗高	安静、耐摔抗震、功耗低
单位容量价格	低	高
寿命特性	扇区不因写入次数多而损坏	块擦除次数有限，反复写同一块会写坏

*3.*磨损均衡

思想：把「擦除」操作平均分布到各个块上，避免某些块被反复擦写而提前失效，从而延长整盘寿命。

- 动态磨损均衡：写入数据时，优先选择累计擦除次数少的新闪存块。
- 静态磨损均衡：SSD 主动监测并自动做数据分配与迁移——让老旧的块承担以读为主的存储任务，让较新的块承担更多的写任务。效果更优，是现代 SSD 的标配。

SSD 不一定比 HDD 快——这句话要看场景

SSD 的随机访问性能碾压 HDD（没有寻道和旋转延迟），但写操作因必须先擦块、可能触发数据搬迁（垃圾回收），写入延迟可能反而高于顺序写磁盘。所以「SSD 读写都比磁盘快」不严谨；正确的说法是「SSD 随机访问性能高、读快，写涉及擦除会慢」。

五、高速缓冲存储器（Cache）

（一）Cache 的基本原理

Cache 是 CPU 与主存之间的小容量高速缓冲层，用来弥补二者速度鸿沟。它由 **SRAM** 构成（无需刷新），L1/L2 集成在 CPU 核内，L3 为多核共享。

核心工作逻辑：程序运行时，硬件基于局部性原理把即将用到的、主存中相邻的一批数据/指令预先成批拷贝到 **Cache**。**Cache** 的内容是主存部分内容的副本（镜像），因此不会增加系统总容量。

术语澄清：主存块（**Block**）$\neq$ 页（**Page**）。主存块是 Cache-主存层次的传输单位（8 ~ 128 B），纯硬件实现；页是虚存-主存层次的单位（4 KB ~ 64 KB），面向操作系统内存管理。

Cache 行除数据区外还有两个核心标识位：

- 有效位 **V**（**Valid Bit**）：标识该行数据是否合法有效。系统上电/复位时所有行 V 清零；装入合法主存块后置 1；淘汰时清 0。**V=0** 时无论 **Tag** 是否匹配都判为缺失。
- 标记位 **Tag**：标识该行存放的是哪个主存块。只有 **Tag** 匹配且 **V=1** 才算命中。采用写回策略时还需 **1** 位脏位。

访问流程：CPU 给出主存地址 $\rightarrow$ 按映射规则定位候选行 $\rightarrow$ 比较 Tag 与 V $\rightarrow$ 命中则从 Cache 取数并按块内地址选出字/字节；缺失则访问主存取块，同时把整块调入 Cache（必要时先替换）。

整个流程——判命中、取数、调块、替换——必须由硬件在一条指令的执行周期内完成，对程序员完全透明。因此 **Cache** 缺失不会引发 **CPU** 陷入内核，不会引起进程切换；而缺页必须由操作系统处理。

*1.*性能指标

- 命中率 h：$h = \dfrac{\text{命中次数}}{\text{总访存次数}}$，$0 \le h \le 1$。
- 缺失率 $= 1 - h$。
- 命中时间 t_c：Cache 命中的访问延迟（即 Cache 自身的访问时间）。
- 主存访问时间 t_m：t_m 通常是 t_c 的几十到上百倍。

平均访问时间：两种模型写法不同，看题目是串行还是并行访问。

串行访问模型（先查 Cache，未命中再访主存）：

$$T_a = h \cdot t_c + (1-h)(t_c + t_m) = t_c + (1-h)\,t_m$$

并行访问模型（同时访问 Cache 与主存，命中则立即取消主存请求）：

$$T_a = h \cdot t_c + (1-h)\,t_m$$

访问效率：

$$e = \frac{t_c}{T_a}$$

两个平均访问时间公式不能乱用

教材常写 $T_a = t_c + (1-h)t_m$（串行：无论命中与否都要花一个 t_c）；也有题目按 $T_a = ht_c + (1-h)t_m$（并行：同时发起，命中就赢）。区别在于缺失时是否还要额外计入 t_c。读完题先确认组织方式，再选公式；如果 $t_c \ll t_m$，两者差别不大，但选项往往就卡在这个差别上。

算例：$t_c = 4$ ns，$t_m = 100$ ns。

- 若 $h = 95\%$，串行模型：$T_a = 4 + 0.05 \times 100 = 9$ ns，$e = 4/9 \approx 44.4\%$。
- 并行模型：$T_a = 0.95 \times 4 + 0.05 \times 100 = 8.8$ ns。

两级 **Cache** 算例：L1 命中率 95%、访问时间 2 ns；L1 缺失时访 L2，L2 命中率 80%、访问时间 10 ns；L2 也缺失则访主存，主存访问时间 100 ns。

$$T_a = 2 + 0.05 \times 10 + 0.05 \times 0.20 \times 100 = 2 + 0.5 + 1 = 3.5 \text{ ns}$$

三级分解检验：L1 命中贡献 $0.95 \times 2 = 1.9$；L2 命中贡献 $0.05 \times 0.8 \times (2+10) = 0.48$；主存贡献 $0.05 \times 0.2 \times (2+10+100) = 1.12$；合计 3.5 ns ✓。

2.Cache 设计要考虑的四类问题

1. 地址映射与变换：主存块按什么规则放到 Cache 行？如何快速定位？（直接映射、全相联、组相联）
2. 块替换：Cache 满时淘汰哪一行？（FIFO、LRU、LFU、随机）
3. 写一致性：写操作如何保证 Cache 与主存一致？（写直达、写回、写分配、非写分配）
4. 地址划分：如何把主存地址切成 `Tag | Index | Offset` 供硬件并行匹配。

（二）Cache 与主存之间的映射方式

三种映射方式在命中率（可选位置越多越高）、查找速度（可选位置越多越慢）、冲突率（可选位置越少越高）之间权衡。

主存块大小的统一约定：设块内地址占 w 位，则块大小 $= 2^w$ 字节。块内地址位数 $w = \log_2(\text{块大小})$，永远是地址的最低位段。

*1.*直接映射

映射规则：

$$\text{Cache 行号} = \text{主存块号 mod Cache 行数}$$

等价说法：把主存按 Cache 大小分区，每个区内的块数与 Cache 行数相同；主存地址分成标记 **Tag**（区地址）+ 行号 **Index**（区内行索引）+ 块内地址 **Offset** 三部分。Index 字段就是上式中的余数。

地址字段位数（设主存地址 n 位，块大小 2^w B，Cache 行数 $C = 2^c$）：

$$w = \log_2(\text{块大小}), \qquad c = \log_2(\text{Cache 行数}), \qquad t = n - c - w$$

$$\text{Cache 行数} = \frac{\text{Cache 数据区容量}}{\text{块大小}}$$

特点：每个主存块只有唯一确定的 1 个可放位置；只需一个共享的比较器；需要行号译码器；硬件成本最低；不需要替换算法（冲突时直接替换）；命中率最低、冲突率最高（Cache 未满也可能冲突）。适用于大容量 Cache。

*2.*全相联映射

映射规则：一个主存块可装入 Cache 的任意一行。

地址只有标记 **Tag** + 块内地址 **Offset** 两个字段（没有行号/组号）：

$$w = \log_2(\text{块大小}), \qquad t = n - w$$

$$t = \log_2(\text{主存块数}) = \log_2\left(\frac{\text{主存容量}}{\text{块大小}}\right)$$

特点：Tag 最长；利用率最高、命中率最高、冲突率最低（只有 Cache 全满才需要替换）；需要每一行一个比较器并行比较，硬件成本最高；替换算法复杂。适用于小容量 Cache，如快表 **TLB**。

全相联的硬件实现要点：主存 Tag 广播到所有行的比较器，比较结果与各行的有效位 V 送入与门（Tag 相等且 V=1 才输出 1），所有与门输出送入或门产生全局 `hit`；命中行打开三态门，数据经 MUX 按块内地址选出字/字节输出；`hit=0` 时阻断 MUX 输出，CPU 转而访问主存。缺失时需从主存取块、按替换算法选牺牲行、写入并更新 Tag 与 V。

*3.*组相联映射

映射规则：Cache 分成若干组，每组 k 行，称 k-路组相联。主存块先用直接映射定位到固定的组，再在组内全相联映射到任意一行：

$$\text{Cache 组号} = \text{主存块号 mod Cache 组数}$$

地址分三段：标记 **Tag** + 组索引 **Index** + 块内地址 **Offset**。

$$\text{组数} = \frac{\text{Cache 总行数}}{\text{路数}k} = \frac{\text{Cache 数据区容量}}{\text{块大小} \times k}$$

$$d = \log_2(\text{组数}), \qquad w = \log_2(\text{块大小}), \qquad t = n - d - w$$

查找流程：用组索引经译码器定位到唯一一组（这一步把查找范围大幅缩小）→ 组内 k 路的 Tag 与有效位并行比较（k 个比较器）→ 命中路的与门打开三态门 → MUX 按块内地址选出数据。

组相联统一了另外两种映射：

- $k=1$（每组 1 行，共 m 组）→ 退化为直接映射（只有 1 个比较器）。
- $m=1$（只有 1 组，共 $k=n$ 行）→ 退化为全相联映射（无需组号译码器）。

组索引位数只由组数决定，与路数无关

这是最容易错的一点。2^d 组就占 d 位，不论每组有几路。增加路数 k 会减少组数，从而减少组索引位数、增加 Tag 位数，但不会改变组索引与路数的直接对应关系。

*4.*三种映射方式对比

对比项	直接映射	全相联映射	k-路组相联
映射规则	行号 = 块号 mod 行数	任意行	组号 = 块号 mod 组数，组内任意
可放位置数	1（唯一确定）	n（全部行）	k（组内）
地址字段	Tag + 行号 + 块内	**Tag** + 块内	Tag + 组号 + 块内
Tag 长度	最短	最长	居中
比较器	**1** 个（共享）	n 个（每行一个）	k 个（每组 k 个）
需要译码器	需要（行号）	不需要	需要（组号）
硬件成本	最低	最高	居中
命中率 / Cache 利用率	低	最高	较高
冲突率	最高（未满也可能冲突）	最低（全满才冲突）	居中
替换算法	不需要	复杂	仅组内，需要
适用	大容量 Cache	小容量（TLB）	最常用（L1/L2/L3）

关联度（路数）与性能的关系（Cache 大小、块大小固定时）：

- 关联度越低 → 命中率越低、判命中开销越小、命中时间越短、Tag 额外空间开销越小。
- 直接映射命中时间最短、额外开销最小；全相联命中时间最长、额外开销最大。

这解释了现代 CPU 为何普遍使用 4 路、8 路、16 路组相联：路数增加时命中率迅速逼近全相联，而硬件成本仍可控；路数为 1 冲突率过高，路数等于总行数则硬件代价过大。

*5.*地址划分与容量计算的解题流程

```
Step 1  由【Cache 数据区容量 / 块大小】求 Cache 总行数
Step 2  由【总行数 / 路数 k】求组数（直接映射时组数 = 行数，全相联时组数 = 1）
Step 3  Offset 位数 w = log2(块大小)
        直接映射: Index 位数 c = log2(行数)
        组相联  : Index 位数 d = log2(组数)
        全相联  : 无 Index
Step 4  Tag 位数 t = 主存地址位数 n - Index 位数 - w
Step 5  每行总位数 = 数据位数 + Tag 位数 + 有效位 (1)
                   + 脏位（写回时要算，直写时不算）
Step 6  Cache 总容量 = 行数 * 每行总位数
```

通用容量公式（设块大小 2^w 字节、Cache 行数 n、主存地址位数由 w 与 Tag 位数 s 决定）：

$$\text{直接映射：}\quad C = n \times \left(1 + t_{\text{tag}} + 8 \times 2^{w}\right) \text{ 位},\quad n = 2^{c},\ t_{\text{tag}} = s - c$$

$$\text{全相联：}\quad C = n \times (1 + s + 8 \times 2^{w}) \text{ 位}$$

$$k \text{ 路组相联：}\quad C = kn \times (1 + s - d + 8 \times 2^{w}) \text{ 位},\quad \text{组数} = n = 2^{d}$$

（若采用写回策略，括号内还要再加 1 位脏位。）

算例 1（直接映射）：主存地址 32 位，按字节编址，块大小 16 B，Cache 数据区容量 64 KB，直接映射。

- 行数 $= 64\text{KB}/16\text{B} = 2^{12} = 4096$ 行 $\rightarrow c = 12$ 位；$w = \log_2 16 = 4$ 位。
- Tag $= 32 - 12 - 4 = 16$ 位。
- 每行 $= 1$ 位 V + 16 位 Tag + 4 个 32 位字 $= 1 + 16 + 128 = 145$ 位。
- Cache 总容量 $= 4096 \times 145 = 580$ Kbit $= 72.5$ KB；数据占比 $= 64/72.5 \approx 88.3\%$。

算例 2（直接映射 + 写回，即**【2015-15】**）：主存地址 32 位、按字节编址、块大小 4 个字（每字 32 位 = 4 B，故块 16 B）、写回、能存 4K 字数据，求 Cache 总位数。

- 数据容量 $= 4\text{K} \times 4\text{B} = 16$ KB；行数 $= 16\text{KB}/16\text{B} = 1024 \rightarrow c = 10$；$w = 4$。
- Tag $= 32 - 10 - 4 = 18$ 位。
- 每行 $= 1(\text{V}) + 1(\text{脏}) + 18(\text{Tag}) + 128(\text{数据}) = 148$ 位。
- 总容量 $= 1024 \times 148 = 148$ Kbit。（答案 C）

算例 3（直接映射 + 写回，即**【2021-16】**）：主存地址 32 位、按字节编址、Cache 数据区 32 KB、块大小 32 B、直接映射、写回，求每行位数。

- 行数 $= 32\text{KB}/32\text{B} = 1024 \rightarrow c = 10$；$w = 5$；Tag $= 32 - 10 - 5 = 17$。
- 每行 $= 1 + 1 + 17 + 32 \times 8 = 275$ 位。（答案 A）

算例 4（组相联，即**【2022-16】**）：主存地址 32 位、按字节编址、Cache 数据区 32 KB、块大小 64 B、**8** 路组相联，问比较器个数与位数。

- 行数 $= 32\text{KB}/64\text{B} = 512$；组数 $= 512/8 = 64 \rightarrow d = \log_2 64 = 6$；$w = \log_2 64 = 6$。
- Tag $= 32 - 6 - 6 = 20$ 位。
- 访问时用组索引选定一组，组内 **8** 路并行比较 $\rightarrow$ 比较器个数 $= 8$，每个宽度 = Tag 位数 $= 20$ 位。（答案 A）

地址划分题的五个高频错点

1. 把「**Cache** 容量」当成「数据区容量」。题目给的行数/组数由数据区容量算；Tag 字段与总容量才需要加元数据位。
2. 组索引位数按路数算。2^d 组就 d 位，与每组几路无关。
3. 写回忘加脏位、直写多加了脏位。看写策略决定这一位要不要算。
4. 「每行位数」与「总容量」问法不同。前者只乘一次，后者乘行数。
5. 主存块大小单位是「字」时忘记换算成字节。4 个字 × 32 位 = 16 B，$w = 4$ 而不是 $\log_2 4$。

命中与映射的统考考法

【2009-21】访存 1000 次、缺失 50 次，命中率？$h=(1000-50)/1000=95\%$（答案 D）。【2009-14】Cache 16 块、2 路组相联、块大小 32 B、按字节编址，主存 129 号单元所在块装入哪个组号？块号 $=\lfloor 129/32\rfloor=4$，组数 $=16/2=8$，组号 $=4 \bmod 8=4$（答案 C）。也可直接看地址 129 = `100 00001`：低 5 位是块内地址、接着 3 位是组号 $100=4$。【2022-16】同算例 4；【2021-16】同算例 3；【2015-15】同算例 2。这一类型是必拿分项，公式背熟 + 单位换算细致即可。

（三）Cache 中主存块的替换算法

Cache 行数远少于主存块数，多个主存块会映射到同一行/组。新块调入而位置已满时必须淘汰一个。适用性提醒：

- 直接映射不需要任何替换算法——每个块只有唯一位置，调入即覆盖。
- 全相联的替换涉及全部 Cache 行。
- 组相联的替换只在特定组内进行。

统一算例：某 **4** 路组相联 Cache 的第 0 组初始为空，依次访问映射到该组的块号序列

$$1,\ 2,\ 3,\ 4,\ 1,\ 2,\ 5,\ 1,\ 2,\ 3,\ 4,\ 5$$

前 4 次访问把组内 4 个空位填满，状态为 `[1，2，3，4]`；接着访问 1、2 均命中；访问 **5** 时冲突，需按算法淘汰一个。

算法	核心思想	实现机制	本例淘汰谁	优点	缺点
FIFO 先进先出	谁先进入谁先被淘汰（认老不认新）	计数器或循环队列	淘汰 **1**（最早进入）	硬件最简单	性能差，可能产生 **Belady** 异常；不反映局部性
LRU 最近最少使用	淘汰最久未被访问的块	每行一个计数器/时间戳：命中行清零，其余加一；替换时选值最大者	淘汰 **3**（访问历史…3, 4, 1, 2 中最久）	最符合时间局部性，命中率最高，主流采用	硬件开销大，每次访问都要更新计数器
LFU 最不经常使用	淘汰过去被访问次数最少的块	每行一个访问计数器，命中加 1；替换时选值最小者	3 与 4 次数并列（各 1 次），需配合 FIFO 打破平局 → 淘汰 **3**	体现长期高频热点	受「历史包袱」影响：曾经狂热的块赖着不走；成本高
随机替换	随机挑一个淘汰	随机数发生器	任意（如摇到 2 就淘汰对应行）	硬件最简单、速度最快	完全不用局部性，命中率不稳定

LRU 位位数：

$$\text{每块的 LRU 替换位} = \log_2(\text{组内块数})\ \text{bit}$$

2 路组相联需 1 位，4 路需 2 位，8 路需 3 位。

LRU 手工模拟的标准做法（必考题型）：在组内维护一个「最近访问顺序」队列，每次访问（无论命中还是缺失）都执行两步：

1. 命中：把被命中的块移到队列最前端（标记为最新）。
2. 缺失：淘汰队列最末端的块，把新块放到最前端。

初始时若组内有空位，直接填空位，不算替换。判断最终命中次数时，逐项走一遍即可。

替换算法模拟的三个坑

1. 命中的访问也要更新 **LRU** 顺序。很多同学只在缺失时动队列，导致后续淘汰对象选错。
2. **FIFO** 与 **LRU** 在「命中是否影响顺序」上完全不同：FIFO 的进入顺序不受命中影响；LRU 受。
3. **LFU** 的计数器只在命中时加 **1**，缺失调入的新块计数器从 1（或 0）开始，且并列时必须说明用什么规则打破平局。题目没给就按 FIFO 处理。

（四）Cache 写策略

Cache 中存放的是主存的副本。写操作若只改 Cache 不改主存，就会出现不一致：DMA 设备读到旧数据、多核中另一个核读到过期值、程序崩溃后主存状态错误。解决靠写策略，分写命中与写缺失两种情形。

*1.*写命中时的策略

写直通法（**Write-Through**，又称全写法、写穿法）

- 写操作同时更新 **Cache** 与主存，任何时刻两者保持一致。
- 优化：配置写缓冲（**Write Buffer**）队列。CPU 写完 Cache 后把写请求放入写缓冲，由写缓冲异步更新主存，CPU 不必等待主存写完。

维度	说明
一致性	始终一致，主存任何时刻都是最新值
写速度	慢（每次写都要访问慢速主存，靠写缓冲缓解）
实现复杂度	简单，不需要脏位
总线带宽	占用多（每次写都产生主存访问）

写回法（**Write-Back**，又称回写法）

- 写操作只更新 **Cache**，不立即写主存；只有当该行被替换时才写回主存。
- 关键机制：脏位（**Dirty Bit**），每行 1 位。
  - 脏位 = 1：该行被 CPU 修改过，与主存不一致，淘汰前必须先写回主存。
  - 脏位 = 0：该行与主存一致，可直接覆盖，无需写回，节省主存带宽。

维度	说明
一致性	不能保证实时一致，存在「脏数据窗口」
写速度	快（绝大多数写操作只访问 Cache）
实现复杂度	较复杂，需要维护脏位
总线带宽	占用少（只有替换脏块时才写主存）

两种写命中策略对比

对比维度	写直通法	写回法
Cache 与主存一致性	实时一致	存在不一致窗口
每次写是否访问主存	必须访问	通常不访问
是否需要脏位	不需要	需要
适用场景	对一致性要求高（多核共享、DMA 频繁）	对性能要求高（主流 **CPU**）
是否配写缓冲	可以	不需要

*2.*写缺失时的策略

写分配法（**Write-Allocate**）：写缺失时先把目标块从主存调入 **Cache**，再在 Cache 中执行写。设计动机是时间局部性——这个地址被写了，接下来很可能还会被读或再次被写。通常搭配写回法。

非写分配法（**No Write-Allocate**）：写缺失时直接写主存，不把该块调入 Cache。若写完之后短期内不再访问该地址，调块进来反而浪费空间、挤占更有价值的块。通常搭配写直通法。

对比维度	写分配法	非写分配法
写缺失时的处理	调块入 Cache 再写 Cache	直接写主存，不动 Cache
后续读命中率	更高（块已在 Cache 中）	较低
Cache 空间利用	可能引入不常用块	不引入新块，节省空间
通常搭配	写回法	写直通法

*3.*策略组合

实际系统中两种策略成对使用：

组合	逻辑链	定位
写回法 + 写分配法	写回法要把写操作留在 Cache 里 → 写命中直接改 Cache；写缺失就先把块调进来再改	主流 **CPU**，性能优先
写直通法 + 非写分配法	写直通法已保证每次写都到达主存，调块进来的意义很小	一致性优先

写策略的统考考法

【2015-16】指令 `add xaddr, 3`，页式虚存 + TLB，Cache 采用直写，完成该指令至少访问主存几次？答：**B**（**1** 次）。取指时若 TLB 命中且 Cache 命中，访主存 0 次；取操作数同样假设 TLB 与 Cache 都命中，访主存 0 次；但写回结果时直写策略要求同步写主存，故至少 1 次。这正是「写直达法无法避免主存访问」的直接体现。

写策略四条易错

1. 写直通法不需要脏位，写回法必须要有脏位。算 Cache 每行位数时，脏位是否计入完全取决于写策略。
2. 脏位 = **1** 才需要写回；脏位 = 0 的行被淘汰时直接覆盖，不产生主存写。
3. 写分配与写回、非写分配与写直通是「通常」搭配，不是强制。理论上可以交叉，但那样会自相矛盾（写回 + 非写分配意味着写缺失时既不更新 Cache 又要等替换才写回，逻辑失效）。
4. 写缓冲只解决「**CPU** 等待」问题，不改变一致性语义。它让 CPU 不必阻塞，但主存最终一定会被更新。

六、虚拟存储器

（一）虚拟存储器的基本概念

虚拟存储器不是独立的物理硬件，而是一套硬件与操作系统协同实现的内存管理技术。它要解决三个痛点：

1. 突破物理主存容量限制：为程序员提供远超物理主存大小的连续、统一的编程地址空间。
2. 弥合 **CPU** 与外存的效率鸿沟：以主存（DRAM）作为外存的高速缓存，减少对低速外存的直接依赖。
3. 支撑多进程安全并发：为每个进程提供私有、隔离的虚拟地址空间。

核心构件：

- 虚拟地址 **VA**（**Virtual Address**，逻辑地址）：应用程序视角的地址。
- 物理地址 **PA**（**Physical Address**）：硬件实际用来寻址内存单元的地址。
- **MMU**（**Memory Management Unit**，存储管理部件）：集成在 CPU 内，负责地址转换与合法性校验。
- 页表基址寄存器 **PTBR**：记录页表在主存中的首地址。进程切换时只需改 **PTBR**，即可完成页表切换。多级页表下 PTBR 存的是最顶层页表（页目录）的物理基址。

「地址转换由谁完成」必须分清

- **OS**（软件）负责：创建页表、写入映射关系；修改 PTBR；处理缺页异常。
- **MMU**（硬件）负责：每次访存时查 TLB、翻译地址、做权限检查；TLB 缺失时遍历页表（硬件页表漫游器）。

一句话：页表是软件写的，但「查页表」这个动作每次都是硬件做的。所以「地址转换由 MMU 硬件完成」是正确的表述。

两类异常：

- 缺页异常：目标数据未载入主存。MMU 检测到后触发异常，CPU 从用户态陷入内核，由缺页处理程序完成外存 → 主存的页面调入并更新页表，之后重新执行触发缺页的那条指令（不是下一条！）。
- 存储保护异常：地址越界、访问权限不符，由操作系统处理（终止进程或报错）。

虚拟页与物理页：虚拟页 VP（虚页/逻辑页）与物理页 PP（实页/页框、页帧）大小完全相等，故两者页内偏移位数相同。因虚拟存储器容量大于主存，虚拟页号位数 > 物理页号位数。

$$\text{虚拟地址} = \text{虚拟页号 VPN} + \text{页内偏移}, \qquad \text{物理地址} = \text{物理页号 PPN} + \text{页内偏移}$$

MMU 转换过程中「不会」检测的事件

【2024-18】下列事件不是在 MMU 地址转换过程中检测的是（）。答：**B. Cache** 缺失。MMU 负责虚 → 实转换，会检测访问越权、页面缺失、TLB 缺失。**Cache** 缺失是在 **MMU** 完成地址翻译、**CPU** 拿着物理地址访问主存数据时才检测的，二者不在同一环节。

（二）页式虚拟存储器

*1.*基本原理

主存与虚拟地址空间都被划分为大小相等的页，内外存以页为单位交换信息。采用请求分页：只把活跃页面调入主存，非活跃页面留在辅存；访问到不在主存的页面时触发缺页异常。

页式虚拟存储器采用全相联映射（每个虚拟页可放到主存中任意一个空闲物理页框），因为缺页代价极高，必须把冲突缺失降到最低。

Cache-主存与主存**-**辅存的定量对比：Cache 容量 ≪ 主存 ≪ 辅存；Cache 访问速度 ≫ 主存 ≫ 辅存（主存比 Cache 慢 10 ~ 100 倍，辅存比主存慢约 10000 倍）。因此页面（4 KB ~ 64 KB）远大于主存块（16/32/64 B），主存-辅存层次必然用全相联 + 写回。

*2.*页表与页表项

每个进程由操作系统建立独立的页表，记录虚拟页到物理页/磁盘位置的映射。页表每一行是一个页表项 **PTE**：

字段	别名	作用
有效位（装入位）	存在位	=1：虚页已调入主存，存放位置字段即物理页号；=0：未调入，若存放位置为 NULL 表示未分配，否则记录该页在磁盘上的起始地址
脏位（修改位）	—	=1：页面已被修改，替换时需写回辅存；=0：可直接替换
引用位（使用位）	替换控制位	记录页面访问情况，LRU 算法中作为 LRU 位
访问权限位（保护位）	保护位	定义可读可写 / 只读 / 可执行，实现存储保护

*3.*地址转换过程

页表项地址的生成：

$$\text{页表项地址} = \text{PTBR} + \text{虚拟页号} \times \text{页表项大小}$$

完整过程（页面命中）：

1. CPU 送出指令中的虚拟地址，交 MMU 转换；
2. MMU 用 PTBR 与虚拟页号生成页表项地址，访问主存中对应页表项；
3. 主存向 MMU 返回页表项；
4. 有效位 = 1，确认页面在主存，把物理页号 + 页内偏移拼接成物理地址，访问主存对应数据；
5. 主存向 CPU 返回目标数据。

页面缺失（缺页）：

1. 步骤 1 ~ 3 同上；
2. 有效位 = 0，MMU 触发缺页异常，调用操作系统缺页异常处理程序；
3. 主存已满时按替换算法选定待淘汰页面；若其脏位 = 1，需先写回磁盘，否则直接丢弃；
4. 缺页处理程序从磁盘把目标页面调入主存，更新页表项；
5. 返回原进程，重新执行触发缺页的指令，此次按命中流程完成访存。

虚实地址转换算例（【2022-15】）：主存地址 24 位，虚拟地址空间 4 GB，页大小 4 KB，按字节编址。访问虚拟地址 00082840H。

- 页大小 4 KB $= 2^{12}$ → 页内偏移 **12** 位；虚拟地址 32 位（4 GB）→ 虚页号 **20** 位。
- 拆 00082840H：低 12 位 840H 是页内偏移，高 20 位 00082H 是虚页号。
- 00082H $= 8 \times 16 + 2 = 130$。查表：虚页号 130 的存在位 = 1，实页号 = 018H。
- 主存地址 24 位，实页号占高 $24 - 12 = 12$ 位；查得的实页号位数正好，高 4 位补 0，拼接得 018H + 840H = **018840H**。（答案 C）

再算一例（【2013-16】）：主存地址空间 256 MB，虚拟地址空间 4 GB，页大小 4 KB，TLB 全相联 4 项。虚拟地址 03FFF180H。

- 页内偏移 12 位 → 虚页号 = 03FFFH。
- 在 TLB 中找标记为 03FFFH 的表项：有效位 = 1，页框号 = 0153H。
- 物理地址 = 页框号 × 4KB + 偏移 = ‘0153180H‘。（答案 A）

虚实转换题的三步固定套路

1. 先由页大小定页内偏移位数 $= \log_2$(页大小)，其余高位是虚页号；
2. 拆地址、算虚页号、查表（十六进制先转十进制或直接按位看，注意别数错位数）；
3. 物理地址 = 页框号 « 偏移位数 + 原偏移，同时要检查「页框号位数 + 偏移位数」是否等于物理地址位数，不足补 0、多余则本身信息量不够（此时题目通常会设计得刚好）。

缺页的返回点：缺页属于内部异常中的故障（**Fault**），处理完成后必须重新执行原指令，这一点【2019-14】考过——选项「回到发生缺页的指令的下一条指令执行」是错的。

*4.*使用快表 *TLB* 加速地址转换

Cache 虽能缓存部分页表块，但块粒度较大，不能充分利用虚存访问的局部性。于是处理器维护转换旁路缓冲区 **TLB**（**Translation Look-aside Buffer**），专门缓冲经常访问的页表项。

$$\text{TLB 的关键字} = \text{虚拟页号 VPN}$$

TLB 本质是一个容量很小的 Cache：

- 大多采用全相联或组相联方式，且多采用随机替换算法。
- TLB 离 CPU 更近、更快，称快表；主存中的页表称慢表。
- 典型容量 32 ~ 1024 项（1 ~ 2 级）。

全相联 **TLB** 的工作逻辑：虚拟地址分为「虚拟页号 + 页内偏移」。MMU 把虚拟页号与 TLB 内所有表项的 **Tag** 并行比较，若 Tag 匹配且有效位 = 1 则命中，直接取物理页号与页内偏移拼接。

组相联 **TLB** 的工作逻辑（以 2 路为例）：虚拟地址的 VPN 部分划分为「**TLB** 标记 + **TLB** 组索引」。MMU 用组索引定位到对应组（每组 2 项），把 TLB 标记与组内 2 项的标记并行比较，结果与有效位做逻辑与，生成选择信号，选中的物理页号与页内偏移拼接。

TLB 表项的位数计算

$$\text{VPN 位数} = \text{虚拟地址位数} - \log_2(\text{页大小})$$

$$\text{组索引位数} = \log_2\left(\frac{\text{TLB 表项数}}{\text{路数}}\right)$$

$$\text{TLB 标记位数} = \text{VPN 位数} - \text{组索引位数}$$

$$\text{物理页号位数} = \text{主存地址位数} - \log_2(\text{页大小})$$

算例（【2024-17】）：虚拟地址 32 位，主存地址 30 位，页大小 1 KB，TLB 共 32 个表项，**4** 路组相联，求 TLB 标记字段至少几位。

- 页内偏移 $= \log_2 1\text{KB} = 10$ 位 $\rightarrow$ **VPN** $= 32 - 10 = 22$ 位。
- 组数 $= 32/4 = 8 \rightarrow$ 组索引 **3** 位。
- **TLB** 标记 $= 22 - 3 = 19$ 位。（答案 C）

5.TLB、页表、*Cache* 的组合访问

主存数据分两类：存储页表内容的页表块与存储程序数据的数据块。页表块同样会被 Cache 缓存（Cache 硬件不区分内容，只按访问频度缓存）。

三种典型情形：

- 理想情况（不访问主存）：页表项在 Cache 中命中，数据也在 Cache 中命中。只需 2 次 Cache 访问（第一次拿页表项，第二次拿数据）。
- 较糟糕情况（访问主存 **2** 次）：无缺页（页在主存），但页表项在 Cache 中缺失、数据也在 Cache 中缺失。需 2 次主存访问，并把两个块分别调入 Cache。
- 最糟糕情况：在最糟糕情况基础上，访问目标数据触发缺页，需要访问磁盘。

TLB、页、**Cache** 命中/缺失组合的可行性：

场景	**TLB**	页	**Cache**	可行性	说明
最好：不访主存	命中	命中	命中	可能	TLB 命中则页必命中
访主存 1 次	命中	命中	缺失	可能	数据在主存但不在 Cache
访主存 1 次	缺失	命中	命中	可能	TLB 缺失但页仍在主存
访主存 2 次	缺失	命中	缺失	可能	两次主存：查页表 + 取数据
最糟：访磁盘	缺失	缺失	缺失	可能	页不在主存，数据必然不在 Cache
不可能	命中	缺失	缺失	不可能	页缺失说明信息不在主存，TLB 中不可能有有效表项
不可能	命中	缺失	命中	不可能	同上
不可能	缺失	缺失	命中	不可能	页缺失则数据不在主存，必然不在 Cache

一句话记住「不可能」：TLB 命中 ⇒ 页必命中

TLB 是页表的缓存，只有当一个页面已经调入主存（页命中）时，它的有效映射才可能被缓存到 TLB 中。所以「**TLB** 命中而页缺失」永远不可能出现。同理，页缺失 ⇒ **Cache** 必然缺失（数据根本不在主存）。

6.TLB 与 *Cache* 的对比

对比维度	**Cache**	**TLB**
缓存对象	主存中的数据/指令块	内存页表中的页表项
查询 key	物理地址（块号部分）	虚拟页号 **VPN**
输出结果	数据内容	物理页号 **PPN** + 权限
解决的问题	CPU 与主存的速度差	每次地址翻译都要访主存查页表的开销
位置	CPU 芯片内（L1/L2/L3）	MMU 内部
典型容量	32 KB ~ 32 MB（多级）	32 ~ 1024 项（1 ~ 2 级）
映射方式	直接 / 组相联 / 全相联	全相联或组相联
缺失处理	硬件自动完成	硬件或 **OS**（视架构）
组成器件	**SRAM**	**SRAM / CAM**
进程切换	无需刷新（key 是物理地址，全局唯一）	需要刷新（key 是虚拟地址，不同进程同 VA 映射不同 PA）

一句话区分：**Cache** 加速「取数据」，**TLB** 加速「找地址」。

TLB 与 Cache 的统考考法

【2010-17】不可能发生的命中组合？答 **D**（**TLB** 命中、**Cache** 命中、**Page** 未命中）——理由同上。【2020-15】关于 TLB 和 Cache 的错误叙述？答 **D**（都由 **DRAM** 组成）。两者都由 **SRAM/CAM** 组成（追求与 CPU 匹配的速度）；只有主存才由 DRAM 组成。其余 A（命中率都与局部性有关）、B（缺失后都要访主存）、C（缺失处理都可由硬件实现）都正确。【2024-18】见前文「MMU 不检测 Cache 缺失」。

7.多级页表

单级页表要求整张页表在主存中连续存放，当虚拟地址空间很大而实际使用的页面很少时会浪费大量主存。多级页表把页号分级，每级页表自身也存成一个页面，按需分配。

虚页号位数与页表大小：设虚拟地址 n 位、页大小 2^p 字节、页表项 e 字节：

$$\text{虚页号位数} = n - p, \qquad \text{页表项数} = 2^{n-p}, \qquad \text{单级页表大小} = 2^{n-p} \times e \text{ 字节}$$

$$\text{一级页号位数} = p - \log_2 e, \qquad \text{页表级数} = \left\lceil \frac{n-p}{p - \log_2 e} \right\rceil$$

算例：虚拟地址 32 位，页大小 4 KB，页表项 4 B。

- 虚页号 $= 32 - 12 = 20$ 位 $\rightarrow$ 页表项数 $= 2^{20} = 1\text{M}$ 项 $\rightarrow$ 单级页表 $= 1\text{M} \times 4\text{B} = 4$ MB，且必须连续分配 4 MB。
- 采用两级页表：每级页表自身占一个 4 KB 页面 = 1024 个页表项 $\rightarrow$ 每级页号 **10** 位（$10+10=20$ ✓）。
- 一级页表（页目录）只占 4 KB；每个二级页表也只占 4 KB，只为实际用到的虚页号区间分配二级页表。

算例：虚拟地址 48 位、页大小 4 KB、页表项 8 B。$p = 12$，$\log_2 e = 3$，每级页号 $= 12 - 3 = 9$ 位；虚页号 $= 48 - 12 = 36$ 位；级数 $= \lceil 36/9 \rceil = 4$ 级。这与 x86-64 实际使用的 4 级页表一致。

多级页表的两点澄清

1. 多级页表降低的是「页表本身占用的连续主存空间」，不是「一次地址转换的访存次数」。相反，级数越多、TLB 缺失时遍历页表的访存次数越多（4 级页表在 TLB 缺失时最多需 4 次读主存）。实际中靠 TLB 命中 + Cache 缓存页表块把开销压下来。
2. 能装下页面的最小单位就是「页」。页表项数不得超过一页能装下的项数，即每级页号位数 $= p - \log_2 e$，因为一级页表必须正好占一个页面。

（三）段式虚拟存储器

基本原理：按程序的逻辑结构（函数、代码段、数据段、堆、栈等）把虚拟地址空间划分为长度不等的段。每段从 0 开始编址，段内地址连续。

虚拟地址结构：`[段号 S][段内偏移 W]`。

段表：每个进程一张，每个段一个段表项。段表项含：

字段	含义
段长	该段的长度（用于越界检查）
段基址	该段在主存中的起始物理地址
有效位 / 装入位	该段是否已调入主存
访问权限	可读 / 可写 / 可执行
修改位、访问位	供替换使用

地址转换：

$$\text{物理地址} = \text{段表项中的段基址} + \text{段内偏移}$$

转换时必须检查段内偏移是否超过段长，超过则触发越界异常（这是段式存储管理的核心保护机制，页式则不需要，因为页内偏移位数天然不会越界）。

优点：按逻辑单位划分，便于共享（多个进程段表项指向同一段基址）、便于保护（按段设权限）、便于动态链接与动态增长（段长可变）。缺点：段长不等导致主存分配复杂，易产生外部碎片；需要拼接段基址与段内偏移。

（四）段页式虚拟存储器

基本思想：先按逻辑结构分段，再把每段分页。兼取段式「便于共享保护」与页式「无外部碎片、分配简单」的优点。

虚拟地址结构：`[段号 S][段内页号 P][页内偏移 W]`。

查找过程（两次查表）：

1. 用段号查段表，得到该段页表的起始地址与段长（用于越界检查）；
2. 用段内页号查页表，得到物理页号（页框号）；
3. 物理页号与页内偏移拼接成物理地址。

地址转换效率问题：

$$\text{段页式一次访存的主存访问次数} = 3 \text{ 次（查段表} + \text{查页表} + \text{取数据）}$$

所以段页式必须配 TLB。**TLB** 中直接缓存「段号 + 段内页号 → 物理页号」的完整映射，命中时一次即可得到物理页号，避免两次查表。

段页式题目的两条注意

1. 地址字段是三段，不是两段。只写「段号 + 页内偏移」是错的，必须包含段内页号。
2. 越界检查在段这一级做（用段表中的段长），页这一级不需要。

虚拟存储器的统考考法汇总

年份题号	考点	答案要点
2010-17	TLB/页/Cache 组合	不可能：TLB 命中而页缺失
2013-16	TLB 全相联查表	`03FFF180H` → `0153180H`
2015-16	虚存 + 直写访存次数	至少 1 次（写回结果必须写主存）
2019-14	缺页处理	返回原指令重新执行
2020-15	TLB 与 Cache 对比	都由 SRAM/CAM 组成，不是 DRAM
2022-15	页式地址转换	页内偏移 12 位，`00082H`=130 → `018840H`
2024-16	两个层次的对比	主存-外存层次是全相联，不是直接映射
2024-17	TLB 标记位数	VPN 22 位 − 组索引 3 位 = 19 位
2024-18	MMU 检测的事件	Cache 缺失不在 MMU 阶段检测

Cache-主存与主存-辅存四条对照（【2024-16】的核心）

1. 交换单位：Cache-主存用块，主存-辅存用页 ✓
2. 实现者：Cache-主存的替换由硬件实现，主存-辅存的替换由软件（**OS**）实现 ✓
3. 写策略：Cache-主存可回写或直写；主存-辅存必然回写 ✓
4. 映射方式：Cache-主存可用直接/组相联/全相联；主存-辅存通常用全相联——所以说「主存-外存层次通常采用直接映射方式」是错的。

本章小结表

知识点	核心结论
按存取方式分类	RAM 随机、SAM 顺序（磁带）、DAM 直接（磁盘/光盘）、CAM 按内容（TLB、全相联 Cache）
ROM 与 RAM	都采用随机存取方式；ROM 是非易失半导体存储器；CD-ROM 是光存储器，不是 ROM
SSD 归属	半导体存储器，基于 Flash（EEPROM），不是磁盘
存取时间 vs 存取周期	$T_C = T_A + T_R$；连续访问与带宽必须用 T_C
带宽	$B_m = W/T_C$；提高靠缩短 T_C 或加宽 W
层次化四条规律	越往下速度越慢、容量越大、位价格越低、访问频次越低
两个层次的缺失处理	Cache 缺失由硬件处理；缺页由操作系统处理
局部性	时间局部性 = 重复访问；空间局部性 = 邻近访问；步长越小越好
SRAM	6 管双稳态触发器；快、贵、集成度低、不需刷新；用作 **Cache**

知识点	核心结论
DRAM	1 管 1 电容；慢、便宜、集成度高、需刷新；用作主存；破坏性读出需再生
DRAM 刷新	按行、不需列地址、地址由内部计数器产生；集中/分散/异步三种
异步刷新间隔	$T_{\max}$/行数，如 2ms/128 = 15.625 μs
集中刷新死区	= 行数 $\times\, t_c$；行数越多死区越长
DRAM 地址复用	行列分两次传送；物理地址引脚 = max(行位数, 列位数)；4M $\times$ 8 芯片共 19 引脚
Flash	读快写慢；按页写、按块擦；写前必须先擦块；寿命有限
芯片扩展	位扩展片数 $= N/n$，字扩展组数 $= M/m$，总片数 $= (N/n)(M/m)$
位扩展连接	地址/CS/WE 全并联，数据线各自独立，各片同时工作
字扩展连接	数据/低位地址/WE 全并联，片选各自独立，各片分时工作
线选法	无译码器、地址空间不连续、有空洞
译码片选法	需译码器、地址空间连续、主流方案
MAR 位数	由主存地址空间决定，与实际安装容量无关
多模块编址	高位交叉（顺序）：相邻地址同体，不能并行；低位交叉：相邻地址异体，可流水
低位交叉体号	体号 = 地址 mod m
低位交叉时间	准备 n 个字：$T + (n-1)\tau$；总时间 $T + (n+1)\tau$；流水条件 $T = m\tau$
低位交叉吞吐	稳定流水时一个存储周期提供 m 个字，平均每字 τ
访存冲突	同体且序列间隔步数 $< m$
磁盘组成	磁盘控制器 + 磁盘驱动器 + 盘片；磁头号 = 盘面号，磁道号 = 柱面号
磁盘地址结构	（驱动器号，柱面号，盘面号，扇区号）；柱面号在前以省寻道
平均存取时间	寻道时间 + 旋转延迟 + 传输时间（+ 控制器延迟）
旋转延迟	$T_r = 1/(2r)$（r 转/秒）$= 30/r$ 秒（r 转/分）
传输时间	$T_t = b/(rN)$，b 为字节数、N 为每磁道字节数、r 转/秒
容量	格式化容量 < 非格式化容量；格式化容量 = 柱面 $\times$ 盘面 $\times$ 每道扇区 $\times$ 每扇区字节
磁盘读写单位	扇区，不是字节
RAID	镜像与奇偶校验提高可靠性；条带化只提性能，不提供冗余
SSD 读写	以页读写、以块擦除；写前必擦；随机访问快、无寻道与旋转延迟
磨损均衡	动态：优先选擦除次数少的块；静态：自动迁移数据，老块承担读任务（更优）
Cache 组成	SRAM；内容为主存副本，不增加系统总容量；对程序员透明
Cache 标识位	有效位 V + 标记 Tag（+ 写回时脏位）
命中率与平均访问时间	串行：$T_a = t_c + (1-h)t_m$；并行：$T_a = ht_c + (1-h)t_m$
访问效率	$e = t_c/T_a$
直接映射	行号 = 块号 mod 行数；Tag 最短；1 个比较器；无需替换算法
全相联	任意行；只有 Tag + 块内地址；n 个比较器；命中率最高
组相联	组号 = 块号 mod 组数，组内任意；k 个比较器；最常用
退化关系	$k = 1$ 退化为直接映射；组数 $= 1$ 退化为全相联
地址划分	$c = \log_2$ 行数（或 $d = \log_2$ 组数），$w = \log_2$ 块大小，$t = n - c - w$

知识点	核心结论
每行位数	Tag + 有效位 1 + 数据（+ 写回时脏位 1）
FIFO	淘汰最早进入者；简单；可能 Belady 异常
LRU	淘汰最久未访问者；命中率最高；LRU 位 $=\log_2$ 组内块数
LFU	淘汰访问次数最少者；有历史包袱；并列需打破平局
写直达	同时写 Cache 与主存；不需脏位；可配写缓冲
写回	只写 Cache，替换时写回；需要脏位；主流
写分配 / 非写分配	写缺失时调块（配写回）/ 不调块直接写主存（配直写）
页式虚存映射	全相联；交换单位是页（4 KB ~ 64 KB）
虚实转换	页表项地址 = PTBR + VPN × 页表项大小；物理地址 = 页框号 + 页内偏移
缺页返回点	重新执行原指令，不是下一条
TLB	VPN 作关键字；全相联或组相联；随机替换；进程切换需刷新
TLB 标记位数	VPN 位数 − 组索引位数
TLB 与 Cache	Cache 用物理地址查、加速取数；TLB 用虚拟地址查、加速找地址
不可能组合	TLB 命中而页缺失；页缺失而 Cache 命中
多级页表	每级页号位数 $= p - \log_2 e$；级数 $= \lceil (n-p)/(p-\log_2 e) \rceil$；按需分配、节省连续主存
段式	段长可变；地址 = 段号 + 段内偏移；需越界检查；便于共享保护，有外部碎片
段页式	段号 + 段内页号 + 页内偏移；查段表、页表两次，需 TLB 加速

易错清单

本章最容易被扣分的地方

1. 以为 **ROM** 采用顺序存取。ROM 与 RAM 都采用随机存取方式，差别只在能否随意改写。
2. 把「半导体存储器都采用随机存取」当成正确。全相联 Cache 的比较器、快表 TLB 是按内容访问（CAM）。
3. 把 **CD-ROM** 归入 **ROM**、把 **SSD** 归入磁盘。CD-ROM 是光存储器；SSD 是半导体 Flash（EEPROM）。计算机术语中的 ROM 特指半导体芯片。
4. 混淆存取时间与存取周期。算带宽、算连续读取最小间隔、算流水线启动间隔，必须用存取周期 T_C。
5. 把 **MAR** 位数与实际安装容量挂钩。MAR 位数由主存地址空间决定，与装了多少内存无关（【2011-15】）。
6. **DRAM** 地址引脚忘了除以 **2**。4M × 8 位 DRAM 只有 11 根地址引脚，不是 22 根；行列不等时取 max。
7. **DRAM** 引脚数被系统总容量干扰。题目问「芯片」引脚数时，总容量只用于算片数。
8. 认为刷新需要列地址。刷新按行进行，只需行地址，且由芯片内部刷新计数器给出，不需要 CPU 提供。
9. 集中刷新的死区与行数关系记反。行数越多（矩阵越「高」），死区越长；所以设计上倾向减少行数、增加列数。
10. 把分散刷新的存取周期当成不变。分散刷新把存取周期拉长一倍（读写 + 刷新各半），有效访存速度减半。
11. 字扩展时忘记位扩展。4K × 4 位芯片装 8 位宽内存必须先 2 片一组，否则片数少一半。
12. **SRAM** 与 **DRAM** 的地址引脚混算。SRAM 不复用地址，引脚数就是完整逻辑地址位数。
13. 低位交叉的体号算成高位。体号 $=$ 地址 mod m；若按地址除以 m 取整，就变成高位交叉了。
14. 低位交叉时间公式漏掉总线开销。准备 n 个字是 $T + (n-1)\tau$，含送首地址与传送末字的总时间是 $T + (n+1)\tau$，两者别混用。

15. 访存冲突只比较体号。必须同时满足「同体」且「序列间隔步数 $< m$」。
16. 磁盘旋转延迟的单位算错。转速给 r/min 时用 $30/r$ 秒；给 r/s 时用 $1/(2r)$ 秒。题目用「r 秒/转」这类反向单位时要格外小心。
17. 把磁盘最小读写单位当成字节。是扇区（传统 512 B）。
18. 认为格式化容量大于非格式化容量。格式化要在每扇区加头、尾与间隔，所以格式化容量更小。
19. **Cache** 映射时把「行数」与「组数」搞混。组索引位数由组数决定，与路数无直接关系：2^d 组就是 d 位。
20. 写回忘加脏位、直写多算了脏位。脏位只在写回策略下存在。
21. 地址位数计算时把 **Cache** 总容量当成数据区容量。行数由数据区容量算；Tag 与元数据位只在算总容量/每行位数时才加。
22. **LRU** 模拟时命中不更新顺序。每次访问（无论命中与否）都要把被访问块移到「最新」位置。
23. 以为直接映射需要替换算法。每个主存块只有唯一位置，调入即覆盖，不需要任何替换算法。
24. 认为 **TLB** 缺失与缺页是同一回事。TLB 缺失只是快表里没有该表项，页可能仍在主存（此时只需查慢表）；缺页是页根本不在主存，必须访磁盘。
25. 写出「**TLB** 命中而页缺失」的组合。这不可能发生——TLB 命中意味着该页已在主存。
26. 以为多级页表降低了一次地址转换的访存次数。恰恰相反，级数越多，TLB 缺失时遍历页表的访存次数越多；多级页表省的是页表本身占用的连续主存空间。
27. 段式虚存忘了越界检查。段长可变，必须检查段内偏移是否超过段长；页式不需要。
28. 段页式地址漏掉「段内页号」。结构是段号 + 段内页号 + 页内偏移三段。
29. 认为缺页处理完成后执行下一条指令。缺页属于故障（**Fault**），处理完必须重新执行原指令。
30. 以为 **Cache** 缺失会陷入内核。Cache 缺失完全由硬件处理，对程序员透明，不引起进程切换；只有缺页才由操作系统处理。
31. 说「**Cache** 增加了系统存储容量」。Cache 只是主存的副本，不增加总容量；增加容量是虚拟存储器（主存-辅存层次）的目标。
32. 主存-辅存层次误以为用直接映射。因缺页代价极高，主存-辅存层次采用全相联，且写操作只回写（【2024-16】）。
33. 认为 **SSD** 的读和写都比磁盘快。SSD 随机访问性能与读性能优异，但写操作要先擦块，还可能触发数据迁移，不一定快。
34. 把「透明」理解成「看得见」。体系结构中透明 = 看不见 = 不需要管。Cache 对程序员透明，是指程序员不用写任何调度 Cache 的代码。

本章历年统考考点分布（按知识点）

- 存储器分类与性能指标：2010-16（RAM/ROM 性质判断）、2011-14（不采用随机存取方式的器件，考 CD-ROM 属直接存取）。
- 半导体存储器：2014-15（4M × 8 DRAM 引脚数 19）、2015-17（需要刷新的是 SDRAM）、2012-16（Flash 读写速度不一样快）、2018-17（2K × 1 DRAM 行列数取 32×64）。
- 主存储器：2009-15（ROM/RAM 芯片数 2、30）、2010-15（`0B1FH` 所在芯片最小地址 `0800H`）、2011-15（MAR 26 位）、2016-16（SRAM 14 片）、2021-15（按字编址，32 片）、2023-15（RAM:ROM=3:1，ROM 地址范围）、2015-18（4 体交叉冲突对）、2017-13（3 个存储周期）、2013-43（8 体交叉 + 突发传送事务时间 85 ns）。
- 外部存储器：2013-20（RAID 可靠性措施）、2013-21（平均存取时间 9.4 ms）、2015-20（12.2 ms）、2019-20（磁盘最小读写单位不是字节）。
- **Cache**：2009-21（命中率 95%）、2009-14（组号 4）、2014-16（指令/数据 Cache 分离的目的）、2015-15（Cache 总容量 148 Kbit）、2016-15（缺失率 12.5%）、2021-16（每行 275 位）、2022-16（比较器 8 个、20 位）、2015-16（直写至少访主存 1 次）。
- 虚拟存储器：2010-17（不可能的命中组合）、2013-16（TLB 查表得 `0153180H`）、2019-14（缺页返回原指令）、2020-15（TLB/Cache 由 SRAM 组成）、2022-15（`018840H`）、2024-16（主存-辅存层次全相联）、2024-17（TLB 标记 19 位）、2024-18（Cache 缺失不在 MMU 检测）。
- 局部性：2017-14（数组 a 的时间与空间局部性皆有）。

七、本章公式速查

公式	表达式	说明
存取周期	$T_C = T_A + T_R$	恢复时间 T_R
存储器带宽	$B_m = W/T_C$	W 为数据位宽（字节）
硬件元数据	元数据比例 $= \frac{\text{每行位数} - \text{数据位数}}{\text{每行位数}}$	衡量 Tag 开销
DRAM 地址引脚	$p = \lceil \log_2 M/2 \rceil$ 或 $\max(\log_2 r, \log_2 c)$	M 为字数
集中刷新死区	$T_{\text{死区}} = r \cdot t_c$	r 为行数
异步刷新间隔	$\Delta = T_{\max}/r$	$T_{\max}$ 通常 2 ms 或 64 ms
芯片总数	$N_{\text{片}} = (N/n)(M/m)$	目标 $M \times N$、单片 $m \times n$
片内地址线	$\log_2 m$	并联到所有芯片
片选译码位数	$\log_2(M/m)$	接译码器
低位交叉准备时间	$T + (n-1)\tau$	流水条件 $T = m\tau$
低位交叉总时间	$T + (n+1)\tau$	含送首地址与传送末字
高位交叉准备时间	nT	与并行无关
磁盘旋转延迟	$1/(2r)$ 秒（r 转/秒）；$30/r$ 秒（r 转/分）	平均半圈
磁盘传输时间	$b/(rN)$	N 为每磁道字节数
平均存取时间	$T_s + T_r + T_t$	寻道 + 旋转 + 传输
格式化容量	柱面 × 盘面 × 每道扇区 × 每扇区字节	—
命中率	$h =$ 命中次数/总访存次数	—
平均访问时间（串行）	$T_a = t_c + (1-h)t_m$	先查 Cache 再查主存
平均访问时间（并行）	$T_a = h\,t_c + (1-h)t_m$	同时发起
访问效率	$e = t_c/T_a$	—
直接映射行号	$=$ 主存块号 mod Cache 行数	—
组相联组号	$=$ 主存块号 mod Cache 组数	—
Cache 行数	= 数据区容量/块大小	—
组数	= 总行数/路数	—
地址字段	$w = \log_2$ 块大小，c 或 $d = \log_2$ 行数/组数，$t = n - c - w$	—
每行位数	$t + 1 +$ 数据位$(+1_{\text{写回}})$	—
LRU 位	$\log_2$(组内块数)	—
页内偏移位数	$\log_2$(页大小)	—
物理地址位数	PPN 位数 + 页内偏移位数	—
页表项地址	PTBR + VPN × 页表项大小	—

公式	表达式	说明
单级页表大小	$2^{n-p} \times e$ 字节	n 虚拟地址位、p 页内偏移位、e 项大小
多级页表每级页号位数	$p - \log_2 e$	每级页表恰占一页
TLB 标记位数	VPN 位数 $-\log_2$(组数)	组数 = 表项数/路数

第四章指令系统

本章考纲要求

第二部分计算机组成原理［考查目标］

1. 掌握指令集体系结构的基本知识和基本实现方法，对计算机硬件相关问题进行分析，并能够对相关部件进行设计。
2. 理解计算机系统的整机概念，能够综合运用计算机组成的基本原理和基本方法，对高级编程语言（C 语言）程序中的相关问题进行分析，具备软硬件协同分析和设计能力。

四、指令系统（一）指令系统的基本概念（二）指令格式（三）寻址方式（四）数据的对齐和大/小端存放方式（五）CISC 和 RISC 的基本概念（六）高级语言程序与机器级代码之间的对应

1. 编译器、汇编器和链接器的基本概念
2. 选择结构语句的机器级表示
3. 循环结构语句的机器级表示
4. 过程（函数）调用对应的机器级表示

指令系统是「软件与硬件的交界面」：向上，它是编译器生成代码时唯一能使用的「词汇表」；向下，它决定了数据通路与控制器必须实现哪些功能。本章在统考中的分量很重，并且呈现出非常鲜明的两极化特征——一类是纯计算题（扩展操作码能表示多少条指令、相对寻址的转移目标地址、结构体成员地址与填充字节），另一类是纯概念辨析题（CISC 与 RISC 的特征归属、编译器/汇编器/链接器的产物、寄存器的保存责任）。计算题套路固定、可以练到不失分；概念题则要求把「谁负责保存」「谁负责修改」这类归属关系记牢，因为出题人恰恰喜欢把 RISC 的特征安到 CISC 头上。

本章的一条主线必须始终在脑子里：指令做的事，无非是「算出有效地址 → 去某个地方取操作数 → 运算 → 写回」。寻址方式解决的是第二步，栈帧解决的是「过程调用时这些数据放在哪里」，对齐解决的是「放的时候地址有什么约束」。

一、指令系统的基本概念

（一）指令与指令系统

指令（**Instruction**）是计算机执行某种操作的命令，是 CPU 能够直接识别并执行的最小功能单位。一条指令由操作码和若干地址码组成，前者说明「做什么」，后者说明「对谁做」。

指令系统（**Instruction Set**），又称指令集，是一台计算机中所有指令的集合。指令系统是计算机硬件与软件之间的接口规范：硬件必须实现指令系统中每一条指令的功能，而任何软件最终都必须翻译成指令系统中的指令序列才能运行。指令系统一经确定，通常在一系列机型中保持兼容（例如 x86 系列不断扩充新指令但绝不删除老指令），这就是为什么它「只能加不能减」。

（二）指令集体系结构（ISA）

指令集体系结构（**Instruction Set Architecture**，**ISA**）是机器语言程序员或编译程序编写程序时所看到的计算机系统的抽象，它规定了：

- 指令的格式、类型与功能；
- 操作数的寻址方式；
- 可访问的寄存器组与主存空间大小、编址方式；
- 数据类型与格式（整数、浮点数及其长度）；
- 指令执行的顺序控制方式（顺序、转移、中断与异常）。

ISA 是「软硬件之间的契约」。同一个 **ISA** 可以有很多种不同的硬件实现：同样是 x86-64 指令系统，Intel 与 AMD 的微架构完全不同（流水线级数、Cache 大小、执行单元数量都不同），但对程序员而言程序行为一致。反过来，ISA 不同意味着机器语言不兼容。统考里凡是问「计算机的层次结构中，属于软件与硬件界面的是什么」，答案就是 ISA。

ISA 属于硬件与软件的交界面

ISA 在程序员眼中就是「这台机器能做哪些事」的完整描述；在硬件设计者眼中则是「必须实现的功能清单」。它不包括具体的微架构实现（流水线级数、Cache 容量、控制器是微程序还是硬布线）。

（三）指令字长与机器字长

这三个长度必须分清，考试常把它们放进同一条选择题里：

名称	定义	决定谁
机器字长	CPU 一次整数运算能处理的二进制位数，实际上就是定点运算数据通路的位数（ALU、通用寄存器、加法器的宽度）	通用寄存器位数、ALU 宽度
存储字长	一个存储单元（存储字）的二进制位数	主存数据线宽度、MDR 位数
指令字长	一条指令的二进制位数	**IR** 的位数；按字节编址时决定一条指令占几个字节

关键结论：

1. 机器字长一般等于通用寄存器的位数，也等于 **ALU** 的宽度。常说「32 位机器 / 64 位机器」，指的就是机器字长。
2. 指令字长与机器字长之间没有任何约束关系。机器字长强调的是参与 ALU 运算的操作数位数，而有些指令根本没有参与 ALU 运算的操作数（如 `nop`、`jmp`）。所以「指令字长一定等于机器字长」是错的。
3. 指令字长决定 **IR** 的位数：若指令字长最大为 32 位，IR 至少 32 位。
4. 机器字长与存储字长的关系：机器字长通常等于存储字长，也可以是存储字长的整数倍（2 倍、4 倍），即一次能处理多个存储字。
5. 指令字长与存储字长的关系：一条指令可以占半个、一个、两个存储字，分别称为半字长指令、单字长指令、双字长指令。若指令字长是存储字长的 2 倍，取一条指令需要访存 2 次。

「机器字长」不等于「字」

在 80x86 系列中，一个「字」的宽度被约定为 16 位，所以 32 位叫「双字」；而在 IBM 303X 系列中一个

字是 32 位。「字」是计量单位，其宽度由具体机器约定，不必然等于机器字长。

（四）定长指令字与变长指令字

- 定长指令字：所有指令位数相同。优点是译码简单、便于流水线实现（取指阶段一次即可取到完整指令）。RISC 普遍采用定长指令字，典型为 32 位。
- 变长指令字：不同指令长度不同。优点是编码紧凑、能用短指令表示常用操作；缺点是译码复杂、难流水。CISC 普遍采用变长指令字。

二、指令格式

（一）操作码与地址码

一条指令在机器中表示为：

$$\text{指令} = \underbrace{\text{操作码 OP}}_{n\text{ 位}} \;\Big\|\; \underbrace{\text{地址码}A_1\|A_2\|\cdots\|A_k}_{k\text{ 个地址字段}}$$

- 操作码字段（**OP**）：指出指令应执行什么操作，如加、减、传送、转移。操作码的位数 n 决定了该机器最多能有 2^n 条指令（定长操作码时）。
- 地址码字段：给出操作数或操作数地址的信息。地址码的个数决定了指令是几地址指令，也决定了「取操作数需要访存几次」。

若指令字长为 L 位、操作码占 n 位、共有 k 个地址字段，则每个地址字段平均可以分到 $\frac{L-n}{k}$ 位（定长操作码、等长地址字段的情形）。

（二）零地址 / 一地址 / 二地址 / 三地址指令

按地址码个数划分，指令格式有以下几种，它们各自隐含了「操作数从哪来、结果到哪去」的约定：

格式	形式	操作数来源与结果去向	典型指令
零地址	`OP`	无显式地址码，操作数隐含（如在堆栈顶）	`nop`、`halt`、堆栈机的 `add`
一地址	`OP A`	只有一个显式操作数，另一操作数隐含（通常是累加器 ACC）	`inc A`、`neg A`；堆栈机的单操作数指令
二地址	`OP A1, A2`	两个操作数，结果通常写回 A_1（目的操作数）	`add A1, A2`（含义 $A_1 \leftarrow A_1 + A_2$）
三地址	`OP A1, A2, A3`	两个源操作数、一个目的地址，三者互不重叠	`add A3, A1, A2`（$A_3 \leftarrow A_1 + A_2$）

地址码个数与访存次数的关系

设指令字长与存储字长相等、取指令本身需要访存 **1** 次，且操作数均在主存中、结果写回主存，则取操作数并写回的总访存次数为：

- 零地址指令：1 次（只取指令，操作数隐含在寄存器/栈顶）；
- 一地址指令：$1+1=2$ 次（取指令 + 取操作数；结果可能写回累加器，不访存）；
- 二地址指令：取指令 1 次 + 取两个操作数 2 次 + 写回 1 次，最多 4 次；
- 三地址指令：取指令 1 次 + 取两个源操作数 2 次 + 写结果 1 次，共 4 次。

要点是：访存次数取决于「有几个操作数在内存里」以及「是否需要写回内存」，而不是简单地等于地址码个数。若某个地址字段采用寄存器寻址，则该操作数不访存。

二地址与三地址的访存次数为何常常相同

二地址指令把结果写回 A_1，只需 1 次写回；三地址指令把结果写到 A_3，也是 1 次写回。二者都是「读两个操作数 + 写一个结果」。真正影响访存次数的是源操作数的个数与它们的位置，不是地址码的个数本身。

常见出题角度

给出「指令字长 16 位、操作码固定 n 位、有 4 种寻址方式、单地址指令中形式地址多少位」这类条件，问直接寻址的可寻址范围（典型年份：2020 年统考第 16 题）。解题时把每一条指令的位宽像预算一样逐项扣减：总位数 − 操作码位数 − 寻址方式位数 = 形式地址位数，再按「无符号地址」把 2^m 算成范围。

（三）定长操作码

定长操作码：所有指令的操作码长度相同，均为 n 位。此时最多可定义 2^n 条指令。

设计要点：若指令系统共有 m 条指令，则操作码位数必须满足

$$n \geq \lceil \log_2 m \rceil$$

例如指令系统有 48 条指令，则 $\lceil \log_2 48 \rceil = 6$ 位（$2^5 = 32 < 48 \leq 64 = 2^6$）。

定长操作码的优点是译码简单、译码时间短，缺点是当实际指令数远小于 2^n 时编码空间浪费严重。

（四）扩展操作码（可变长操作码）

当指令字长固定、地址码个数又要变化时（零地址、一地址……共存），就使用扩展操作码：操作码长度随地址码个数的减少而变长。核心思想是把短操作码中用不到的编码留作「扩展标志」，供长操作码使用。

*1.*编码方法

以「零地址、一地址、二地址、三地址四种格式共存」为例。设指令字长为 16 位，每个地址字段 4 位，操作码起始长度为 4 位。

- 三地址指令：格式为 `OP(4 位) | A1(4 位) | A2(4 位) | A3(4 位)`。4 位操作码共 16 个编码，若三地址指令用了 m 条（$m < 16$），则剩下 $16 - m$ 个编码保留不用，作为二地址指令操作码的扩展前缀。
- 二地址指令：格式为 `OP(8 位) | A1(4 位) | A2(4 位)`。其操作码由 8 位构成，但高 4 位必须是被保留的「扩展标志」，通常是 `1111`。于是可用编码数为 $16 - m$ 种（对应低 4 位的所有取值）。若二地址指令用了 k 条，则剩余 $(16 - m) - k$ 个编码继续保留。
- 一地址指令：格式为 `OP(12 位) | A1(4 位)`。可用编码数为 $\big[(16 - m) - k\big] \times 16$ 种，用掉 j 条后剩余继续保留。
- 零地址指令：格式为 `OP(16 位)`。可用编码数为 $\big[(16 - m) - k\big] \times 16 - j$ 种（每保留一个 12 位前缀，就能展开出 16 个 16 位编码）。

计数公式（必背）

设 p 为地址字段位数、q 为操作码起始位数（三地址时的操作码位数），逐级向下：

- 三地址最多 2^q 条；

- 二地址最多 $(2^q - m) \times 2^p$ 条，其中 m 为已定义的三地址指令数；
- 一地址最多 $[(2^q - m) \times 2^p - k] \times 2^p$ 条，其中 k 为已定义的二地址指令数；
- 零地址最多 $[(2^q - m) \times 2^p - k] \times 2^p \times 2^p - j$ 条，其中 j 为一地址指令数。

口诀：每减少一个地址字段，操作码就多出 p 位，可用编码数「乘以 2^p」，再减去已用掉的条数。

扩展操作码的三个易错点

1. 保留的是「编码」，不是「位」。是短操作码的未使用编码成为长操作码的前缀，而不是操作码字段本身变短。
2. 每一级都要先「展开」再「扣减」。正确顺序是：先由上一级的剩余编码数乘以 2^p 得到本级的全部可用编码，再减掉本级已定义的指令数。
3. 零地址指令不占地址字段，所以它的可用编码数不需要再乘 2^p，直接就是上一级剩余的前缀数。

*2.*完整算例一：扩展操作码的设计与验证

题目：某机器指令字长 16 位，采用扩展操作码，地址字段宽度为 4 位，要求支持三地址、二地址、一地址、零地址四种格式。现已知三地址指令 6 条、二地址指令 120 条、一地址指令 200 条，问零地址指令最多还能定义多少条?

第一步：确定三地址指令的操作码位数。

三地址指令 6 条，起始操作码只需 $q = 4$ 位即可（$2^4 = 16 \geq 6$），且必须预留扩展编码，因此取 $q = 4$，地址字段各 4 位，校验位数：

$$4 + 4 + 4 + 4 = 16 \text{ 位} \quad \checkmark$$

三地址指令用掉 6 个编码，剩余 $16 - 6 = 10$ 个编码保留作扩展。

第二步：展开二地址指令的可用编码数。

二地址操作码为 8 位（高 4 位是保留标志，低 4 位可自由取值），可用编码数为

$$(16 - 6) \times 2^4 = 10 \times 16 = 160 \text{ 条}$$

二地址指令需要 120 条，$120 \leq 160$，容量够用。剩余 $160 - 120 = 40$ 个编码保留扩展。

第三步：展开一地址指令的可用编码数。

一地址操作码为 12 位，可用编码数为

$$40 \times 2^4 = 640 \text{ 条}$$

一地址指令需要 200 条，$200 \leq 640$，容量够用。剩余 $640 - 200 = 440$ 个编码保留扩展。

第四步：展开零地址指令的可用编码数。

零地址操作码即为完整的 16 位，不再消耗地址字段，可用编码数为

$$440 \times 2^4 = 7040 \text{ 条}$$

结论：零地址指令最多能定义 **7040** 条。

验证（自洽性检验）：把四类指令的编码区间写出来，检查有没有重叠。若规定三地址操作码取 `0000~0101`（6 条），保留 `0110~1111`（10 个）；二地址高 4 位取 `0110~1111`、低 4 位任意，其中前 120 个（即从 `0110 0000` 开始的 120 个）已用，其余 40 个保留；一地址由这 40 个前缀各展开 16 个编码，前

200 个已用，其余 440 个保留；零地址由这 440 个前缀各展开 16 个编码。逐级区间互不相交，且总数 $16\times16\times16\times16=65536$ 恰好等于 16 位操作码的全部编码空间：$6\times16^3+120\times16^2+200\times16+7040=24576+30720+3200+7040=65536$ ✓

反向验证：什么时候必须加长操作码

若上题改成「三地址指令 13 条、二地址指令 300 条」，取 $q=4$ 时三地址剩余 3 个编码，二地址只能展开 $3\times16=48$ 条，远远不够 300 条，说明操作码起始位数取 **4** 位不够，必须加长到 **5** 位。这就是 2017 年统考第 16 题的考法：先假设操作码位数，逐级验证容量是否够用，不够就加大再验，最后把「操作码位数 + 地址字段总位数」按字节对齐向上取整得到指令字长。

*3.*完整算例二：层级指令条数的直接计算

题目（16 位定长指令字，操作用扩展编码，地址码 6 位，含零地址、一地址、二地址三种格式）：若二地址指令有 12 条、一地址指令有 254 条，则零地址指令最多有多少条？

解答：

- 二地址格式为 `OP(4 位) | A(6 位) | A(6 位)`，共 $4+6+6=16$ 位 ✓。4 位操作码共 16 个编码，用了 12 条，剩余 4 个。
- 一地址格式为 `OP(10 位) | A(6 位)`，可用编码数 $4\times2^6=256$ 条。用了 254 条，剩余 $256-254=2$ 个前缀。
- 零地址格式操作码为 16 位，可用编码数 $2\times2^6=128$ 条。

答案：**128** 条。

验证（用 **16** 位操作码的全部编码空间核对）：二地址指令占 12 条 $\times\,2^{12}$（两个 6 位地址字段）$=49152$ 个编码；一地址指令占 254 条 $\times\,2^6=16256$ 个编码；零地址指令占 128 条 $\times\,2^0=128$ 个编码；剩余未用的编码属于「保留扩展位」，不属于任何已定义指令。三者相加 $49152+16256+128=65536=2^{16}$，恰好把 16 位操作码的编码空间切分完毕，说明结果自洽。

另一种核对方式：零地址指令数 =（一地址剩余前缀数）$\times\,2^{\text{零地址可用的额外位数}}$，即 $(256-254)\times2^6=2\times64=128$ ✓

为什么零地址指令还能「再乘 6 位」

因为零地址指令虽然没有地址字段，但它的操作码可以扩展到占满整个 16 位指令字长。每保留一个 10 位的一地址前缀，其后的 6 位就有 $2^6=64$ 种取值，这 64 种取值都可以是相互不同的零地址指令操作码。漏掉这一步「再乘 2^p」是最常见的失分原因。

三、寻址方式

（一）指令寻址与数据寻址

指令寻址解决的是「下一条要执行的指令在哪里」，由 PC（程序计数器，x86 中称为 EIP）负责。它只有两种基本方式：

- 顺序寻址：$(PC)+$“1”，这里的「1」指一条指令的长度（按字节编址时按字节数增加）。
- 跳跃寻址：由转移指令给出目标地址，修改 PC。

数据寻址（操作数寻址）解决的是「本条指令的操作数在哪里」。指令中的地址字段给出的往往不是操作数的真实地址，而是形式地址（**A** 或 **D**）；CPU 依据寻址方式字段（寻址特征），对形式地址做运算，得到有效地址（**Effective Address**，**EA**）。

$$\text{操作数} = (EA)$$

其中括号表示「取该地址单元的内容」。当需要嵌套取值时（间接寻址），就写成 $((A))$ 这样的多重括号。

形式地址 vs 有效地址

形式地址是指令字里写着的那个数，可能是一个偏移量、一个寄存器编号、一个立即数；有效地址是经过寻址方式换算后，操作数在主存（或寄存器）中的真实位置。整个寻址方式这一节，本质上就是问「EA $= f$(形式地址, 寄存器) 中的 f 是什么」。

（二）各寻址方式详解

*1.*立即寻址

- **EA**：无（不访存取操作数，操作数就在指令里）。
- 形式地址：形式地址字段 A 就是操作数本身。操作数 $= A$。
- 访存次数：取操作数阶段 **0** 次（只需取指令 1 次）。
- 寻址范围：由地址字段的位数 n 决定。无符号数为 $0 \sim 2^n - 1$；带符号补码为 $-2^{n-1} \sim 2^{n-1} - 1$。
- 特点：速度最快；数值范围受限；通常用于给寄存器赋初值、与寄存器内容做常数运算。汇编写法如 `addi $t0,$t1,100`、`MOV EAX,2026H`。

*2.*寄存器寻址

- **EA**：$EA = R_i$（形式地址字段给出寄存器编号，操作数就在该寄存器中）。
- 访存次数：取操作数阶段 **0** 次。
- 寻址范围：由通用寄存器个数决定，n 位寄存器编号可指定 2^n 个寄存器。
- 特点：速度极快、指令字长短（寄存器编号只需 3~5 位）；寄存器资源稀缺，不能放太多数据。汇编写法如 `add $t0,$t1,$t2`。

*3.*直接寻址

- **EA**：$EA = A$。形式地址就是操作数在主存中的真实地址。
- 访存次数：取操作数 **1** 次（取指令 1 次另计）。
- 寻址范围：2^n，其中 n 是形式地址字段位数。例如形式地址 8 位、按字节编址时可寻址 $0 \sim 255$；16 位可寻址 64 KB。
- 特点：简单，不需要加法器；但地址被硬编码在指令中，程序浮动后失效，且寻址空间受形式地址位数限制。

*4.*间接寻址

- **EA**：$EA = (A)$。形式地址 A 是主存中某个单元的地址，该单元里存放的才是操作数的有效地址。操作数 $= ((A))$。
- 访存次数：取操作数需要 **2** 次（第一次取回真实地址，第二次取回操作数）。若是多重间址则次数更多，因此每得到一个地址就需要 1 个「间址标志位」标明本次取回的是地址还是数据。
- 寻址范围：由主存单元的字长决定，而与形式地址字段位数无关——这正是间接寻址最大的价值：用很短的形式地址访问很大的存储空间。
- 特点：便于实现指针、子程序返回参数等；但速度最慢，现代处理器一般不提供一到两条指令就完成的纯数据间接寻址。

*5.*寄存器间接寻址

- **EA**：$EA=(R_i)$。形式地址给出寄存器编号 R_i，该寄存器里存放的是操作数的有效地址（不是操作数本身）。操作数 $=((R_i))$。
- 访存次数：取操作数 **1** 次（地址在寄存器中，取地址不访存）。
- 寻址范围：由寄存器的位数决定（如 32 位寄存器可覆盖 4 GB 地址空间）。
- 特点：兼顾速度与寻址范围，是高级语言中指针的底层实现。汇编写法如 `MOV EAX,[EBX]`、`lw $t0,0($t1)`。

直接 / 间接 / 寄存器间接：看清「谁存的是地址」

- 直接寻址：指令里直接给出地址。
- 间接寻址：指令给出「存放地址的地址」（一次访存换回地址）。
- 寄存器间接寻址：寄存器里存放地址（不需要访存换地址，省一次访存）。

括号的层数就是访存的层数：A 是地址，(A) 取一次，$((A))$ 取两次。

*6.*相对寻址

- **EA**：$EA=(PC)+D$。D 是形式地址字段给出的位移量，用补码表示，计算时须符号扩展到与 PC 同宽。
- 访存次数：若用于取操作数，取操作数 1 次；若用于确定转移目标，不访存。
- 寻址范围：由位移量位数 n 决定，可前后跳转的范围为 $-2^{n-1}\sim 2^{n-1}-1$（相对于 PC 而言，按字节或按字，取决于题目单位）。
- 用途：① 公共子程序的浮动（保证指令与数据的相对位置不变，程序整体搬家后仍能取到同一数据）；② 转移指令的目标地址计算（x86 的 `jmp/jcc`、MIPS 的 `bne` 都是相对寻址）。

相对寻址最大的坑：(PC) 已经自增

当 CPU 完成取指阶段后，PC 已经自动加上了本指令的长度，此时 (PC) 指向的是「下一条指令的地址」，而不是当前指令的地址。所以

$$\text{转移目标地址}=\text{当前指令地址}+\text{当前指令长度}+D$$

写成公式就是 $EA=(PC)+D$，但要记住这里的 (PC) 是已自增的值。题目若写「每取一个字节 PC 自动加 1」「取指后 PC 加 2」一类的话，就是在提示你按哪种方式自增。

相对寻址转移目标计算（经典题型）

题目：某机器主存按字节编址，转移指令采用相对寻址，由 2 个字节组成（第 1 字节为操作码，第 2 字节为相对位移量）。取指时每取 1 个字节 PC 自动加 1。若转移指令所在地址为 2000H，位移量字段为 06H，求转移成功后的目标地址。

步骤：

1. 该指令占 2 字节，取指结束后 PC 已自增：$(PC)=2000\text{H}+2=2002\text{H}$。
2. 位移量 06H 为正数（最高位为 0，无需符号扩展为负），$D=+6$。
3. 目标地址 $=(PC)+D=2002\text{H}+06\text{H}=2008\text{H}$。

答案：**2008H**。

变式一（负位移量）：若指令地址为 2000H、占 2 字节、位移量字段为 `FAH`。`FAH = 1111 1010B`，是补码，符号扩展为 32 位是 `FFFFFFFAH`，其值为 -6。目标地址 $=2002\text{H}+(-6)=2002\text{H}-0006\text{H}=1\text{FFCH}$。

变式二（反求位移量）：若指令地址为 2000H、长 2 字节，要求转移到 2010H，求位移量字段。$D=2010\text{H}-2002\text{H}=000\text{EH}$，写成 8 位补码即 `0EH`。

变式三（**PC** 按指令条数自增）：MIPS 的 bne 采用相对寻址，定长 32 位，按字节编址，则 (PC) 已加 4，且偏移量字段以「指令条数」为单位，需左移 2 位：

$$\text{目标地址} = (PC + 4) + (\text{符号扩展后的 OFFSET} \times 4)$$

*7.*基址寻址

- **EA**：$EA = (B) + A$。B 为基址寄存器，A 为形式地址（在此扮演偏移量的角色）。
- 访存次数：取操作数 1 次。
- 寻址范围：由基址寄存器位数决定（可覆盖整个主存空间），形式地址只用来表示块内偏移。
- 用途：① 程序重定位——由操作系统在装入时为每个程序分配基地址写入基址寄存器（如重定位寄存器、页表基址寄存器），用户程序不可修改；② 扩展有限长度指令的寻址空间——用短地址码访问大空间，只要改变基址寄存器内容，操作数的地址空间就移到另一个区间。
- 寄存器由谁修改：这是容易混淆的地方——用于「多道程序重定位、虚拟存储器」时由操作系统在内核态修改，用户不能改；用于「扩大寻址范围」时由通用寄存器（如 x86 的 EBP、EBX、MIPS 的 $sp）承担，用户程序必须自己修改。

*8.*变址寻址

- **EA**：$EA = (I) + A$。I 为变址寄存器，此处形式地址 A 充当基准地址（如数组首地址，编译时固定），变址寄存器内容充当偏移量/下标。
- 访存次数：取操作数 1 次。
- 寻址范围：由变址寄存器位数 + 形式地址位数共同决定。
- 用途：面向用户，专用于遍历数组、字符串、线性表等成批数据。循环中形式地址（数组首地址）不变，只需让变址寄存器按元素长度递增（如 int 数组每次加 4），即可依次访问每个元素。这是「最适合按下标顺序访问一维数组元素」的寻址方式。

对比项	基址寻址	变址寻址
公式	$EA = (B) + A$	$EA = (I) + A$
寄存器里存什么	基准地址（程序/数据块起点）	偏移量（数组下标）
形式地址 A 是什么	偏移量	基准地址（数组首地址）
谁修改寄存器	操作系统（重定位场景）/ 用户（扩址场景）	用户 / 程序员
执行过程中寄存器内容	通常保持不变	不断变化（随循环递增）
主要应用	程序重定位、短地址访问大空间	数组、线性表、字符串的遍历

基址与变址的「谁在前谁在后」

两者公式形式相同，区别在于谁是基准、谁是变化量。统考按「基址寻址面向系统、变址寻址面向用户」来理解：基址寄存器存基准地址且执行中不变，变址寄存器存偏移量且执行中可变。若题目只说「某寄存器内容 + 形式地址」，就要看题干给的场景来判断。

*9.*堆栈寻址

堆栈按后进先出（**LIFO**）的方式存取数据。寻找存放在堆栈中的操作数地址的方法称为堆栈寻址，操作数总是位于栈顶，因此不需要在指令中给出地址（堆栈寻址本质上属于隐含寻址）。

类型	实现	是否需要堆栈指针	栈顶是否移动	数据是否移动
存储器堆栈（硬堆栈/软堆栈）	在主存中开辟栈区	需要，设置堆栈指针 SP 指向栈顶	移动（SP 随进出栈变化）	不移动
寄存器堆栈（硬堆栈）	用一组寄存器作栈区	不需要	固定不动	移动（数据在寄存器间挪动）

存储器堆栈的操作（以按字节编址、以字节为单位进出栈、栈向低地址增长为例）：

$$\text{入栈：}\ SP \leftarrow SP-1,\quad M[SP] \leftarrow R$$

$$\text{出栈：}\ R \leftarrow M[SP],\quad SP \leftarrow SP+1$$

若以 32 位字为单位进出栈，则上式中的 1 应换成 4。寄存器堆栈速度快但容量有限，存储器堆栈容量大但速度慢；寄存器堆栈必须用专用堆栈指令控制，存储器堆栈则不一定。

（三）寻址方式完整对比表

寻址方式	有效地址 **EA**	取操作数访存次数	操作数所在位置	寻址范围由谁决定	典型用途
立即寻址	无 EA，操作数 $=A$	0	指令中	形式地址字段位数	常数运算、赋初值
寄存器寻址	$EA=R_i$，操作数 $=(R_i)$	0	通用寄存器	寄存器编号位数	寄存器间运算
直接寻址	$EA=A$	1	主存	形式地址字段位数	访问固定地址的变量
间接寻址	$EA=(A)$，操作数 $=((A))$	2（每重间址 +1）	主存	主存单元字长	指针、参数传递
寄存器间接寻址	$EA=(R_i)$，操作数 $=((R_i))$	1	主存	寄存器位数	指针、数组遍历
相对寻址	$EA=(PC)+D$（PC 已自增）	1（取操作数时）	主存 / 指令	位移量位数（补码）	转移目标、程序浮动
基址寻址	$EA=(B)+A$	1	主存	基址寄存器位数	程序重定位、扩址
变址寻址	$EA=(I)+A$	1	主存	变址寄存器位数 + 形式地址位数	数组、循环
堆栈寻址	隐含：$EA=(SP)$	1	主存栈区 / 寄存器组	栈区容量	过程调用、表达式求值

偏移寻址的三种形式

相对寻址、基址寻址、变址寻址在计组中被统称为偏移寻址，它们的数学逻辑完全一致，都是「寄存器

的内容 + 形式地址」。区别在于参与相加的寄存器是谁、由谁控制、用在什么场景。因此「下列不属于偏移寻址的是」这类题的答案通常是间接寻址（间接寻址是「取内容」而不是「相加」）。

变址寻址与地址计算（2018 年统考第 18 题思路）

按字节编址的计算中，`double` 型数组 A 的首地址为 2000H，用变址寻址和循环访问数组，变址寄存器存放的是下标，每次循环取一个元素后自增 1，而实际参与地址计算的是「变址值 × `sizeof(double)`」。若某次取到的元素地址为 2100H，则

$$\text{偏移字节数} = 2100\text{H} - 2000\text{H} = 100\text{H} = 256$$

$$\text{变址寄存器内容} = 256 \div 8 = 32$$

要点：变址寄存器里放的是下标还是字节偏移，必须看题目明确的约定，两者相差一个 sizeof(元素) 因子。

四、数据的对齐和大/小端存放方式

（一）大端方式与小端方式

大端方式（**Big Endian**）：最高有效字节（**MSB**）存放在最低地址，低地址 → 高地址依次存放从高字节到低字节。小端方式（**Little Endian**）：最低有效字节（**LSB**）存放在最低地址。

设 32 位机器数 M = 1234 5678H，存放在首地址 1000H 开始的 4 个字节中：

地址	大端方式	小端方式
1000H	**12H**（MSB）	**78H**（LSB）
1001H	34H	56H
1002H	56H	34H
1003H	**78H**（LSB）	**12H**（MSB）

由此可以得到两条好用的结论：

1. 大端：一个数据的首地址就是它的最高有效字节地址；末地址是最低有效字节（LSB）的地址。
2. 小端：一个数据的首地址就是它的最低有效字节地址；末地址是最高有效字节（MSB）的地址。

求 LSB 地址的题，先算 EA 再判端序

题目若问「该操作数的 LSB（最低有效字节）所在地址」，要先按寻址方式算出首地址（即最低地址），再根据端序判断 LSB 落在首地址还是末地址。

- 大端：MSB 在首地址，LSB 在首地址 +（数据字节数 − **1**）。
- 小端：LSB 在首地址。

以 32 位操作数为例，「首地址 + 3」就是大端时 LSB 的地址。

综合算例（基址寻址 + 大端 + 符号扩展）

题目：某计算机采用大端方式，按字节编址。某指令中操作数的机器数为 `1234FF00H`，采用基址寻址，形式地址（补码表示）为 `FF12H`，基址寄存器内容为 `F0000000H`，求该操作数 **LSB** 所在的地址。

步骤：

1. 求有效地址。形式地址 `FF12H` 是 16 位补码，最高位为 1，为负数，需符号扩展到 32 位：`FFFFFF12H`。

2. 相加：EA = F0000000H + FFFFFF12H = 1 EFFF FF12H。32 位运算丢弃最高位进位，得 EA = EFFF FF12H。这就是该 4 字节操作数的首地址。
3. 判端序。大端方式下 MSB 在最低地址，LSB 在最高地址。
4. 求 **LSB** 地址：首地址 $+3$ = EFFF FF12H + 3 = EFFF FF15H。

答案：**EFFF FF15H。**

要点提醒：① 补码形式地址参与 32 位加法时一定要先符号扩展；② 加法产生的最高位进位要丢弃；③ 大端时 LSB 不在首地址。

大小端与程序正确性：大小端只影响多字节数据的字节排放顺序，不影响单字节数据，也不改变数值本身。同一份程序在大端机和小端机上运行结果可能不同，典型场景是：① 通过指针按字节读取一个多字节整数；② 网络协议的报文字段；③ 直接按字节比较两个数值的存储内容。x86 采用小端，MIPS 传统上采用大端。

看机器指令的立即数字节序判端序

给定机器代码 `C7 45 FC 00 00 00 00` 表示 `int i = 0`，若另一个立即数 -64（32 位补码为 `FFFFFFC0H`）要写成机器代码，则在小端下立即数的 4 个字节为 `C0 FF FF FF`，在大端下为 `FF FF FF C0`。反过来，若题目给出机器代码让你判断端序，就把其中的多字节字段（立即数、偏移量、地址）按两种端序各还原一次，看哪一种结果合理。

（二）边界对齐与不对齐存储

边界对齐要求：一个占 k 字节的数据，其起始地址必须是 k 的整数倍。

- `char`（1 字节）：任意地址都对齐；
- `short`（2 字节）：起始地址必须是 **2** 的倍数；
- `int`、`float`（4 字节）：起始地址必须是 **4** 的倍数；
- `double`、`long long`（8 字节）：起始地址必须是 **8** 的倍数（32 位机器上通常按 4 字节对齐）。

为什么要求对齐？因为主存通常是按字（或按 Cache 行）访问的。若一个 4 字节的 `int` 起始地址为 2（不对齐），它跨越了两个存储字，一次访存取不全，CPU 需要两次访存再拼接，还可能跨越 Cache 行边界造成额外缺失；而在某些 RISC 架构上，不对齐访存会直接触发对齐异常。对齐的代价是浪费少量填充字节，换来的是访存次数的减少与硬件实现简化。

填充字节（**padding**）：为了满足对齐要求，编译器会在成员之间、以及结构体末尾插入无意义的填充字节。

（三）结构体大小的计算规则

设结构体中最大成员的对齐要求为 M（即最大成员的字节数，且不超过机器的最大对齐值）。计算步骤：

1. 从偏移 0 开始，依次为每个成员分配空间；
2. 若当前偏移不是该成员对齐要求的整数倍，就插入填充字节，把偏移抬到最近的整数倍位置；
3. 该成员占用 k 字节后，偏移增加 k；
4. 所有成员分配完毕后，若结构体总大小不是 M 的整数倍，则在末尾补齐到 M 的整数倍（这是为了保证结构体数组中每个元素的地址仍然满足对齐要求）。

$$\text{sizeof(struct)} = \left\lceil \frac{\text{最后一个成员的结束偏移}}{M} \right\rceil \times M$$

*1.*完整算例一：*int / char / short* 结构体

题目：某计算机按字节编址，采用小端方式，int 占 32 位、short 占 16 位、char 占 8 位，数据按边界对齐存储。

```
struct {
    int a;
    char b;
    short c;
} record;
record.a = 273;
```

若 record 的首地址为 0xC008，求地址 0xC008 中的内容，以及 record.c 的地址。

第一步：算各成员偏移与地址。

成员	类型	字节数	对齐要求	起始偏移	实际地址	说明
a	int	4	4	0	0xC008	0 是 4 的倍数，直接放
b	char	1	1	4	0xC00C	紧随 a
填充	—	1	—	5	0xC00D	c 需 2 字节对齐，偏移 5 不是 2 的倍数
c	short	2	2	6	0xC00E	偏移 6 是 2 的倍数 ✓

结构体总大小 = 8 字节，是最大成员对齐要求（4）的整数倍，无需末尾填充。故 record.c 的地址为 **0xC00E**。

第二步：算 record.a 的字节内容。

273 = 0x00000111，即机器数为 00 00 01 11H，4 个字节从高到低是 00H，00H，01H，11H。小端方式下最低有效字节放在最低地址，于是：

地址	内容	对应字节
0xC008	**11H**	LSB
0xC009	01H	
0xC00A	00H	
0xC00B	00H	MSB

答案：地址 **0xC008** 中的内容为 0x11，record.c 的地址为 **0xC00E**。

填充字节的位置与内容

填充字节的内容是不确定的（编译器不保证填 0），考试只考它在哪里、占几字节。另外，填充既可能出现在成员之间（如本例 b 与 c 之间），也可能出现在结构体末尾（当总大小不是最大对齐值的整数倍时），末尾填充最容易被漏掉。

*2.*完整算例二：*short + int* 结构体

题目：按字节编址、小端方式，按边界对齐为下列结构体变量 a 分配空间，a 的首地址为 2020FE00H，成员 x2 的机器数为 12340000H，求其中 34H 所在的存储单元地址。

```
struct record {
    short x1;
    int x2;
} a;
```

第一步：算偏移。

- x1（short，2 字节，对齐要求 2）：偏移 0，地址 2020FE00H ~ 2020FE01H。
- 下一个可用偏移是 2，但 x2（int，4 字节，对齐要求 4）要求偏移是 4 的倍数，因此偏移 2、3 处填充 **2** 个字节。
- x2：偏移 4，地址 2020FE04H ~ 2020FE07H。
- 总大小 8 字节，是 4 的整数倍 ✓。

第二步：算 **x2** 的字节内容。机器数 12340000H 的 4 个字节为 12H，34H，00H，00H（从高到低）。小端方式下低地址放低字节：

地址	内容
2020FE04H	00H（LSB）
2020FE05H	00H
2020FE06H	**34H**
2020FE07H	12H（MSB）

答案：34H 所在的存储单元地址是 2020FE06H。

*3.*完整算例三：含 *double* 的复杂结构体

题目：32 位机、小端、按边界对齐，char 1 字节、short 2 字节、int 4 字节、double 8 字节。

```
struct Node {
    char   tag;      /* 1 字节 */
    int    id;       /* 4 字节 */
    short  flag;     /* 2 字节 */
    double value;    /* 8 字节 */
    char   tail;     /* 1 字节 */
};
```

求各成员的偏移与结构体总大小。

成员	字节数	对齐要求	起始偏移	填充字节数	说明
tag	1	1	0	0	
填充	—	—	1~3	3	使 id 从 4 的倍数开始
id	4	4	4	0	
flag	2	2	8	0	8 是 2 的倍数

成员	字节数	对齐要求	起始偏移	填充字节数	说明
填充	—	—	10~15	6	使 value 从 8 的倍数开始
value	8	8	16	0	16 是 8 的倍数
tail	1	1	24	0	
末尾填充	—	—	25~31	7	总大小须为 8 的倍数

结果：tag 偏移 0、id 偏移 4、flag 偏移 8、value 偏移 16、tail 偏移 24，结构体总大小为 **32** 字节。若改为不按边界对齐（紧凑/打包）存储，则所有成员连续排列：tag 偏移 0、id 偏移 1、flag 偏移 5、value 偏移 7、tail 偏移 15，总大小为 16 字节。

对比结论：对齐存储把结构体从 16 字节撑到 32 字节，浪费了一倍空间，但每个成员都能一次访存取到；打包存储省空间，却可能让 id、value 跨越存储字边界，需要多次访存。这就是「空间换时间」在指令系统与存储系统交界处的体现。

一句话记住结构体大小算法

每个成员从「自己的对齐要求的最小整数倍偏移」开始放，中间不够就补；全部放完后，总大小向上取整到「最大成员的对齐要求」的整数倍。

对齐类真题的三种问法

1. 问某成员地址（如 record.c 的地址）：按上面三步走即可。
2. 问某字节所在地址（如 34H 在哪个单元）：先定位成员，再按端序映射到具体字节。
3. 问某个存储单元里的内容（如 0xC008 中是什么）：先定位该地址属于哪个成员的第几个字节，再按端序还原。

五、CISC 和 RISC 的基本概念

（一）CISC 的基本概念与特点

CISC（Complex Instruction Set Computer，复杂指令集计算机）的设计思路是用一条硬件指令完成复杂任务，通过增强指令功能、增加指令条数来缩短「高级语言语义」与「机器指令语义」之间的差距。它产生的原因有两条：一是硬件成本下降而软件成本上升，增加复杂指令可以简化编译器设计；二是向后兼容——同一系列的新机器只能扩充指令、不能删除老指令，导致指令集不断膨胀。

CISC 的核心特征：

- 指令数量多，一般 200 条以上，甚至数百条；
- 指令字长不固定，指令格式和寻址方式种类繁多；
- 访存不受限制：除了专门的取数/存数指令外，运算类指令也可以直接访问主存（如 add eax, [ebp-8]）；
- 控制器以微程序控制为主：指令太复杂，难以用纯硬件逻辑直接实现，控制存储器往往占 CPU 芯片面积 50% 以上；
- 各指令执行时间相差很大，多数复杂指令需要多个时钟周期，流水线实现困难；
- 各指令使用频度相差很大，约 80% 的时间只用到 20% 最简单的指令（**80-20** 规律）；
- 难以用优化编译生成高效目标代码；

- 典型代表：**x86** 系列（Intel Core i5/i7/i9、AMD Ryzen）。

（二）RISC 的基本概念与特点

RISC（**Reduced Instruction Set Computer**，精简指令集计算机）的设计思路基于 80-20 规律：只保留使用频率最高、最简单的指令，复杂功能由简单指令组合实现。

RISC 的核心特征：

- 严格的 **Load/Store** 架构：只有取数（**Load**）和存数（**Store**）指令可以访问主存，所有运算都必须在寄存器之间进行；
- 面向通用寄存器的结构：配置数量庞大的通用寄存器（几十到上百个），以减少访存；
- 指令「长相」高度统一：指令长度固定，指令格式和寻址方式种类很少，译码逻辑大为简化；
- 控制器以硬布线（组合逻辑）控制为主，控制部件通常只占 CPU 芯片面积 10% 左右；
- 深度依赖指令流水线：绝大多数指令在一个时钟周期内完成；
- 极其重视编译优化：把复杂操作推给编译器，靠编译器调度指令顺序以填满流水线；
- 兼容性包袱小但兼容性差：因为精简了指令系统，大多数 RISC 机器无法与老机器保持软件兼容；
- 典型代表：**ARM**、**MIPS**、**RISC-V**。

（三）CISC 与 RISC 的完整对比

对比项目	**CISC**	**RISC**
指令系统	复杂、庞大	简单、精简
指令数量	一般大于 **200** 条	一般小于 **100** 条
指令长度	不固定（变长）	固定（定长，典型 32 位）
寻址方式	种类多（常达十几种）	种类少（通常仅寄存器、立即数、基址/偏移）
访存指令	不加限制，运算指令也能访存	只有 **Load/Store** 可访存
通用寄存器数量	较少	多（几十至上百个）
控制器实现	绝大多数为微程序控制	绝大多数为硬布线（组合逻辑）控制
指令执行时间	相差较大，多数需多个时钟周期	绝大多数在一个时钟周期内完成
流水线难易	难（指令变长、执行时间不一）	易（定长、规整，天然适合流水线）
编译优化	难以用优化编译生成高效目标代码	采用优化编译可生成较为高效的代码
各指令使用频度	相差很大	都比较常用
软件兼容性	大多能向后兼容（高档机含低档机全部指令）	大多数不能与老机器兼容
典型代表	x86（Intel Core、AMD Ryzen）	ARM、MIPS、RISC-V

（四）走向融合

从结构合理性与执行效率看，RISC 的思路更科学，但现代 CISC 处理器早已不是「纯粹」的 CISC：它们在微架构层面把复杂的 CISC 指令打碎成简单的微操作来执行，大量融入了 RISC 的技术成分。因此两者的性能差距正在缩小。判断一台机器是 **CISC** 还是 **RISC**，不能只看品牌，而要看指令字长是否固定、是否严格限制只有 **Load/Store** 访存——这正是 2017 年统考大题判断「计算机 M 是 RISC 还是 CISC」的依据。

CISC/RISC 概念题的高频陷阱

1. 「普遍采用微程序控制器」是 **CISC** 的特征，不是 **RISC** 的。RISC 普遍采用硬布线控制。这是最高频的错误选项。
2. 「通用寄存器数量不多」是 **CISC** 的特征，不是 RISC 的。
3. 「严格限制只有 **LOAD/STORE** 指令能访存」是 **RISC** 的特征，说成 CISC 就是错的。
4. 「大多数 **RISC** 架构能与老机器实现软件兼容」是错的，CISC 才具备这种兼容性。
5. 「指令长度固定」是 **RISC** 的；CISC 指令长度不固定。
6. 注意「**RISC** 大多数指令在一个时钟周期内完成」这句话是正确的。RISC 追求的是单周期完成 + 流水线重叠，而不是「多周期完成」。

真题考法

概念辨析选择题几乎每年一条，形式是「下列关于 RISC/CISC 的叙述中，错误的是」。解题时只需对着上表中的加粗项逐条核对。此外，RISC 的「指令字长固定、寻址方式少、只有 Load/Store 访存」这三条也会在大题里作为判据出现：给一段机器级代码，只要观察到不同指令占用的字节数不同（如 `push` 1 字节、`cmp` 3 字节），或出现了直接对内存操作数做运算的指令，就可以判定为 CISC。

六、高级语言程序与机器级代码之间的对应

（一）编译器、汇编器和链接器的基本概念

从高级语言源程序到可执行目标文件，依次经过预处理、编译、汇编、链接四个阶段：

阶段	输入	输出	主要工作
预处理	`.c` 源程序	`.i` 预处理后的源程序	处理 `#define` 宏展开、`#include` 文件包含、`#ifdef` 条件编译、删除注释
编译	预处理后的源程序	`.s` 汇编语言源程序	词法分析、语法分析、语义分析、中间代码生成、代码优化、生成汇编代码
汇编	`.s` 汇编语言源程序	`.o` 可重定位目标文件（机器语言）	把汇编指令翻译成机器指令，生成符号表与重定位信息
链接	多个`.o` 目标文件 + 库文件	可执行目标文件	符号解析（把每个符号引用与一个符号定义关联）、重定位（把符号地址合并到最终地址空间）、合并段

四个阶段的产物必须严格对应。注意「编译」和「汇编」的分界：编译的产物是汇编语言程序（文本），汇编的产物是机器语言目标文件（二进制）。

「编译」和「汇编」不要混

「预处理 → 编译 → 汇编 → 链接」这个顺序中，把「汇编」和「编译」调换位置是最常见的错误选项（2022 年统考第 20 题就考查了这个顺序）。简记：编译产出汇编代码，汇编产出目标文件。

另外注意：可执行文件与源程序之间不是一一对应的——一个可执行文件可能由多个源文件、多个目标文件和若干库文件链接而成。

链接器与重定位

链接器要解决两个问题：符号解析（`printf` 在哪里定义?）和重定位（每个符号最终被安排在哪个地址?）。这也解释了为什么「编译时地址」和「运行时地址」可能不同，以及为什么直接寻址不利于程序浮动、而相对寻址和基址寻址有利于程序浮动。

（二）选择结构语句的机器级表示

机器级表示中没有「**if**」这条指令。`if-else` 是靠条件测试 + 条件转移来实现的，思路有两种等价写法：

- 正向跳转：条件成立就跳到 Then 块；
- 反向跳转（编译器实际采用的主流写法）：测试条件的反面，条件不成立就跳到 Else 块，否则顺序执行 Then 块。

编译器之所以采用反向跳转，是为了让「条件成立」这条最可能的分支顺序执行，从而提高指令预取与分支预测的命中率。

*1.*条件码与测试指令

|x86 的算术/逻辑运算都会设置程序状态字（EFLAGS）中的条件码：

标志	名称	含义
ZF	零标志	结果为 0 时置 1
SF	符号标志	结果为负时置 1（即结果最高位）
OF	溢出标志	针对有符号数，补码运算溢出时置 1
CF	进位/借位标志	针对无符号数，最高位产生进位或借位时置 1

两条只设置标志位、不改变操作数的指令：

- `cmp`：底层执行减法 $A-B$，只更新标志位，不保存差。用于比较大小、判断相等。
- `test`：底层执行按位与 $A\&B$，只更新标志位。常用于 `test eax,eax` 判断寄存器是否为 0（`test` 两个相同操作数时，ZF 反映该数是否为 0，SF 反映其符号）。

条件转移指令按操作数类型分为三组：

类别	指令	转移条件（逻辑表达式）	适用场景
相等/零	`je` / `jz`	$ZF=1$	相等、结果为零
相等/零	`jne` / `jnz`	$ZF=0$	不相等、结果非零
符号	`js` / `jns`	$SF=1$ / $SF=0$	判断正负
溢出	`jo` / `jno`	$OF=1$ / $OF=0$	溢出检查
无符号数	`ja`（大于）	$CF=0$ 且 $ZF=0$	unsigned 大于
无符号数	`jae`（大于等于）	$CF=0$	unsigned 大于等于
无符号数	`jb`（小于）	$CF=1$	unsigned 小于
无符号数	`jbe`（小于等于）	$CF=1$ 或 $ZF=1$	unsigned 小于等于
有符号数	`jg`（大于）	$SF=OF$ 且 $ZF=0$	int 大于
有符号数	`jge`（大于等于）	$SF=OF$	int 大于等于

类别	指令	转移条件（逻辑表达式）	适用场景
有符号数	`jl`（小于）	$SF \neq OF$	int 小于
有符号数	`jle`（小于等于）	$SF \neq OF$ 或 $ZF = 1$	int 小于等于

有符号用 `jg/jl`，无符号用 `ja/jb`

同一个比较，操作数是有符号还是无符号，编译出的跳转指令完全不同。原因是减法结果相同但「谁大谁小」的解释不同：-1 作为有符号数是负数（小于 0），作为无符号数却大于所有正数。当 `cmp` 的两个操作数中有一个是 `unsigned` 时，必须用 `ja/jb` 系列（只看 CF 和 ZF），不能用 `jg/jl` 系列（要看 SF 和 OF）。

*2.*示例：*`if-else`* 的机器级表示

```
int abs_delta(int x, int y) {
    int result;
    if (x > y)
        result = x - y;
    else
        result = y - x;
    return result;
}
```

假设参数 `x` 在 `[ebp+8]`、`y` 在 `[ebp+12]`：

```
abs_delta:
    push ebp
    mov  ebp, esp
    mov  eax, [ebp+8]       ; eax = x
    mov  edx, [ebp+12]      ; edx = y
    cmp  eax, edx           ; 计算 x - y，设置 ZF/SF/OF/CF
    jle  .L_else            ; x > y 的" 反面" 是 x <= y，成立就跳去 Else
    ; ---- Then 分支 (x > y) ----
    sub  eax, edx           ; result = x - y
    jmp  .L_done            ; Then 执行完必须无条件跳过 Else
.L_else:
    ; ---- Else 分支 (x <= y) ----
    sub  edx, eax           ; y - x
    mov  eax, edx
.L_done:
    pop  ebp
    ret
```

这段代码的三个要点：① `cmp` 只设标志位、不改操作数；② 跳转条件用的是 `x > y` 的反面 `x <= y`（反向跳转）；③ Then 块末尾必须有 `jmp` 跳过 Else 块，否则会「贯穿」执行 Else。

（三）循环结构语句的机器级表示

循环在机器级就是「条件转移（向后跳出）+ 无条件转移（向前跳回）」的组合。编译器先把循环改写成 `goto` 形式，再翻译成测试与跳转。三种循环的差别只在条件判断的位置。

1.do-while 循环（后测试）

```
do
    body_statement
while (test_expr);
```

翻译为 goto：

```
loop:
    body_statement
    t = test_expr;
    if (t) goto loop;
```

特点：循环体在条件判断之前执行，因此循环体至少执行一次。汇编结构最简洁——循环头就是循环体入口。

```
do_while:
    push ebp
    mov  ebp, esp
    mov  eax, 0            ; sum = 0
    mov  ecx, 1            ; i = 1
loop:
    add  eax, ecx          ; sum += i
    add  ecx, 1            ; i++
    cmp  ecx, [ebp+8]      ; 比较 i 与 n
    jle  loop              ; 条件成立，跳回循环体
    pop  ebp
    ret
```

2.while 循环（前测试）

```
while (test_expr)
    body_statement;
```

编译器有两种标准翻译方法：

方法一：**jump to middle**（跳转到中间）——先无条件跳到条件测试处，条件成立才进入循环体：

```
    goto test;
loop:
    body_statement
test:
    t = test_expr;
    if (t) goto loop;
```

方法二：**guarded-do**（带保护的 **do-while**）——先测试一次条件，不成立直接跳过；成立则进入 do-while 结构，循环体末尾再测试：

```
    t = test_expr;
    if (!t) goto done;
```

```
loop:
    body_statement
    t = test_expr;
    if (t) goto loop;
done:
```

特点：条件在循环体之前判断，初始条件为假时循环体一次都不执行。若把 while 翻译成不带前置判断的 do-while，会多执行一次循环体，这是编译错误。

3.for 循环

```
for (init_expr; test_expr; update_expr)
    body_statement;
```

`for` 在语义上完全等价于：

```
init_expr;
while (test_expr) {
    body_statement;
    update_expr;
}
```

翻译为 `goto` 形式：

```
    init_expr;          /* 只执行一次 */
    t = test_expr;
    if (!t) goto done;  /* 初始条件为假则直接跳过 */
loop:
    body_statement
    update_expr;        /* 更新循环变量 */
    t = test_expr;
    if (t) goto loop;
done:
```

特点：`init_expr` 只执行一次；`test_expr` 在每次循环体之前求值；`update_expr` 在每次循环体之后、条件判断之前求值。

*4.*三种循环的对比

循环	条件判断位置	循环体最少执行次数	跳回方式	关键特征
`do-while`	循环体之后	**1** 次	条件成立则向后跳回循环头	结构最简，无前置判断
`while`	循环体之前	**0** 次	需先跳到测试处或先做一次保护性判断	必须有一次前置测试
`for`	循环体之前	**0** 次	与 `while` 相同，多一段初始化	初始化只做一次；更新在体后

循环类题的两个陷阱

1.「至少执行一次」只属于 `do-while`。`while` 和 `for` 初始条件为假时循环体一次都不执行。
2. 条件跳转的方向：循环条件成立时要向后跳回循环头（形成闭环），而跳出循环的条件跳转是向前跳。若题目给的汇编把成立与不成立的条件写反了，就一定是错的。

循环的真题考法

一是给一段 C 循环代码和对应的汇编，问「哪条指令是条件转移」「每轮循环执行几次某条指令」。二是给一个二维数组赋值循环（如 `for(i=0;i<24;i++) for(j=0;j<64;j++) a[i][j]=10;`），问嵌套循环的机器级实现。三是把 `sum += a[i];` 这样的语句翻译成指令序列，考查「变址计算要用移位代替乘法」（`int` 数组左移 2 位、`short` 数组左移 1 位）。

（四）过程（函数）调用对应的机器级表示

过程调用是本章最综合的考点，它把指令系统、寻址方式、栈、寄存器约定全部串起来。

*1.*运行时栈与栈帧

运行时栈（**run-time stack**）是一块从高地址向低地址增长的内存区域，用来存放过程调用时需要保存和恢复的信息。栈指针 ESP（MIPS 中为 `$sp`）始终指向栈顶（当前最低地址）。

每个正在执行的过程在栈上占有一块区域，称为栈帧（**Stack Frame**）。以 x86-32 为例，一个典型栈帧从高地址到低地址依次为：

顺序	内容	由谁压入	说明
1	调用者的实参	调用者（Caller）	C 约定从右至左压栈，第一个参数在最低地址
2	返回地址	`call` 指令自动压入	`call` 的下一条指令的地址
3	旧的 **EBP**	被调用者（Callee）	`push ebp`，保存上一层栈帧的基址
4	被调用者保存的寄存器	Callee	如 `EBX`、`ESI`、`EDI`
5	局部变量	Callee	`sub esp, N` 开辟的空间

EBP 与 **ESP** 的分工：

- **EBP**（基址指针，**Base Pointer**）：指向当前栈帧的底部（高地址端，紧邻返回地址与旧 EBP）。在函数执行期间保持不变，因此可以用固定的偏移量稳定访问数据：
  - `[ebp + 8]`、`[ebp + 12]`、`[ebp + 16]` ……依次是第 1、2、3 个参数（`[ebp+4]` 是返回地址，`[ebp]` 是旧 EBP）；
  - `[ebp - 4]`、`[ebp - 8]` ……是第 1、2 个局部变量。
- **ESP**（栈指针，**Stack Pointer**）：始终指向栈顶（低地址端），随 push/pop 动态变化。

为什么需要 EBP，只用 ESP 行不行

ESP 在函数执行过程中会因为压栈、弹栈而不断变化，若用 ESP 加偏移访问参数，偏移量必须随每次压栈而调整，编译器难以静态确定。而 EBP 在整个函数执行期间不变，参数与局部变量的偏移量都是编译期常量，寻址更高效、更稳定。若编译器能完全静态算出相对 ESP 的偏移（如叶子函数），也可以省略 EBP 以节省一次压栈和一次弹栈。

*2.*调用与返回指令

- `call 目标地址`：隐含两步——① 把返回地址（`call` 的下一条指令的地址）压栈；② 把目标地址送入 EIP，跳转到被调用过程。
- `ret`：从栈顶弹出返回地址送到 EIP，控制权交回调用者。`ret` 本身不清理参数空间，参数由调用者用 `add esp，M` 清理（cdecl 约定）。

*3.*过程调用的完整流程

调用前（**Caller**）：

1. 若 EAX/ECX/EDX 在调用后还需要用，先把它们压栈保存（调用者保存寄存器）；
2. 把参数从右至左依次压栈；
3. 执行 `call Q`。

进入被调用者（**Callee**）：

1. `push ebp` ——保存调用者的 EBP；
2. `mov ebp，esp` ——建立新栈帧，EBP 指向新栈帧底部；
3. `sub esp，N` ——为局部变量开辟 N 字节空间；
4. `push ebx/esi/edi` ——保存需要用到的被调用者保存寄存器。

执行主体：通过 `[ebp+8]`、`[ebp+12]` 访问参数，通过 `[ebp-4]`、`[ebp-8]` 访问局部变量，结果通常放在 EAX。

返回前（**Callee**）：

1. 把返回值放入 EAX；
2. `pop ebx/esi/edi` 恢复被调用者保存寄存器；
3. `leave`（等价于 `mov esp，ebp` + `pop ebp`）撤销栈帧；
4. `ret` 弹出返回地址并跳回。

返回后（**Caller**）：

1. 从 EAX 取返回值；
2. `add esp，M` 清理参数（M 为参数总字节数）；
3. 恢复之前保存的调用者保存寄存器。

*4.*寄存器使用约定

类别	**x86-32** 寄存器	**MIPS** 寄存器	谁负责保存
调用者保存（Caller-saved，易失）	EAX、ECX、EDX	`$t0~$t9`、`$a0~$a3`、`$v0~$v1`	调用者自己压栈，否则被调用者可随意覆盖
被调用者保存（Callee-saved，非易失）	EBX、ESI、EDI、EBP、ESP	`$s0~$s7`、`$sp`、`$fp`	被调用者若使用则必须先保存、返回前恢复

类别	x86-32 寄存器	MIPS 寄存器	谁负责保存
特殊	EIP（不可直接访问）、EFLAGS	$ra（返回地址，由 jal 自动写入）、$zero（恒 0）	—

返回值的存放：x86 放 EAX，MIPS 放 $v0。参数传递：x86 默认全部压栈（fastcall 约定才用寄存器），MIPS 前 4 个参数放 $a0~$a3、多余的压栈。

「调用者保存」到底是谁保存

这个术语描述的不是义务方，而是被保护的对象。「调用者保存寄存器」的意思是：如果调用者希望调用后这些寄存器的值不变，那么调用者必须自己保存它们；被调用者使用它们不需要事先保存。反之，「被调用者保存寄存器」的意思是：被调用者若要用，就必须先保存、返回前恢复。答题时务必写清「谁在什么时机保存/恢复」。

*5.*完整算例：栈帧布局与递归的栈变化

题目：x86-32，按字节编址，int 占 4 字节。

```
int calculate(int x, int y, int z) {
    int diff = x - y;
    return diff * z;
}
int caller(void) {
    return calculate(10, 5, 3);
}
```

（1）栈帧布局与各成员地址

```
caller:
    push ebp
    mov  ebp, esp
    push 3              ; 参数 z
    push 5              ; 参数 y
    push 10             ; 参数 x
    call calculate      ; 自动压入返回地址
    add  esp, 12        ; 清理 3 个参数 (3*4)
    pop  ebp
    ret

calculate:
    push ebp            ; 保存旧 EBP
    mov  ebp, esp       ; 建立新栈帧
    sub  esp, 4         ; 为 diff 开辟 4 字节
    mov  eax, [ebp+8]   ; x
    sub  eax, [ebp+12]  ; x - y
    mov  [ebp-4], eax   ; diff = x - y
    mov  eax, [ebp-4]   ; 取 diff
    imul eax, [ebp+16]  ; diff * z
    mov  esp, ebp       ; 释放局部变量空间 (leave 的前半)
    pop  ebp            ; 恢复旧 EBP
    ret
```

以 `calculate` 执行时的 EBP 为基准，栈中内容为：

地址（相对 **EBP**）	内容	属于
`[ebp+16]`	`z = 3`	参数 3（最后压栈，最高地址）
`[ebp+12]`	`y = 5`	参数 2
`[ebp+8]`	`x = 10`	参数 1（最先入栈的参数，最低地址）
`[ebp+4]`	返回地址	`call` 自动压入
`[ebp]`	旧的 EBP	`push ebp`
`[ebp-4]`	`diff`	局部变量

要点：参数压栈顺序是「从右至左」，所以第一个参数 `x` 的地址最小（`[ebp+8]`），最后一个参数 `z` 的地址最大（`[ebp+16]`）。返回地址在 **`[ebp+4]`**，紧接着就是第一个参数，这正是 `[ebp+8]` 成为第一个参数的原因：`[ebp]` 是旧 EBP、`[ebp+4]` 是返回地址，各占 4 字节。

（**2**）递归的栈变化

以 `int f(int n) { return n < 2 ? 1 : n * f(n-1); }` 为例，`f(3)` 的调用过程：

时刻	动作	栈帧数量	当前栈顶内容
T0	`main` 调用 `f(3)`	1	`f(3)` 的栈帧：参数 3、返回地址、旧 EBP
T1	`f(3)` 调用 `f(2)`（参数 `n-1=2` 已压栈）	2	`f(2)` 的栈帧叠在 `f(3)` 栈帧下方（更低地址）
T2	`f(2)` 调用 `f(1)`	3	`f(1)` 栈帧
T3	`f(1)` 满足 `n<2`，返回 1	$3 \to 2$	弹出 `f(1)` 栈帧，回到 `f(2)` 的 `imul` 之后
T4	`f(2)` 算出 $2 \times 1 = 2$，返回	$2 \to 1$	弹出 `f(2)` 栈帧，回到 `f(3)`
T5	`f(3)` 算出 $3 \times 2 = 6$，返回	$1 \to 0$	弹出 `f(3)` 栈帧，回到 `main`

结论：递归深度为 d 时，同一时刻栈上最多同时存在 d 个栈帧；递归调用次数为 d 次（`f(10)` 需要调用 10 次，`n` 依次为 $10, 9, \cdots, 1$）。每层栈帧的大小由该过程的参数个数、局部变量个数、需要保存的寄存器个数决定，编译期即可确定，因此递归的总栈空间与递归深度成正比。

递归栈类真题的三个问法

1. 问调用次数：`f(10)` 需要调用 `f1` 共 10 次（`n` 从 10 递减到 1），要数清「最后一次 `n=1` 不再递归」。
2. 问哪条指令递归调用：答案就是 `call f1` 这一条；再问「哪几条指令一定会使程序跳转」，则要答 `call`（无条件过程调用）、`jmp`（无条件跳转）、`ret`（无条件返回）——`jle` 这类条件转移不在其列，因为条件不成立时它不跳。
3. 问栈帧偏移与参数访问：参数 `n` 在 `[ebp+8]`；若某变量在 `[ebp-8]`，说明它是第 2 个 4 字节局部变量。

（五）MIPS 汇编基础

MIPS 是最典型的 RISC 机器，也是统考大题（尤其是流水线题）最常出现的指令系统。它的设计把 RISC 的特征体现得淋漓尽致。

*1.*体系结构与寄存器约定

- 指令长度固定 **32** 位，按字节编址，采用大端方式，要求按字边界对齐（字的地址必须是 4 的倍数）。

- **Load/Store** 架构：只有 `lw/sw` 能访存，运算指令的操作数必须来自寄存器。
- 32 个 32 位通用寄存器，编号 0~31，各占 5 位。

寄存器	编号	用途	保存责任
`$zero`	0	恒为 0，写入被丢弃	—
`$v0~$v1`	2~3	过程返回值	调用者保存
`$a0~$a3`	4~7	前 4 个参数	调用者保存
`$t0~$t9`	8~15、24~25	临时变量	调用者保存
`$s0~$s7`	16~23	局部变量/长期数据	被调用者保存（用了必须压栈）
`$k0~$k1`	26~27	内核保留	—
`$gp`	28	全局指针	—
`$sp`	29	栈指针（栈向低地址增长）	被调用者保存
`$fp`	30	帧指针（指向栈帧底部）	被调用者保存
`$ra`	31	返回地址（`jal` 自动写入）	被调用者保存（非叶子过程必须压栈）

*2.*三种指令格式

格式	字段划分（共 **32** 位，从左到右）	用途	寻址方式
R 型	`OP`(6) → `rs`(5) → `rt`(5) → `rd`(5) → `shamt`(5) → `func`(6)	算术、逻辑、移位运算	寄存器寻址
I 型	`OP`(6) → `rs`(5) → `rt`(5) → 立即数 / 偏移量 (16)	立即数运算、`lw/sw`、条件分支	立即数、基址、相对寻址
J 型	`OP`(6) → 目标地址 (26)	`j`、`jal` 无条件跳转	伪直接寻址

R 型指令的 `OP` 恒为 `000000`，具体运算由 6 位的 `func` 字段决定；`shamt` 用于移位指令指定移位位数，非移位指令该字段为 0。

I 型访存指令的有效地址：$EA = (rs) + \text{SignExt}(\text{立即数})$——这就是基址寻址，也是 MIPS 唯一原生的访存寻址方式。

I 型分支指令的目标地址：

$$\text{目标地址} = (PC + 4) + (\text{SignExt}(\text{offset}) \times 4)$$

即 PC 已自增 4，且 16 位偏移量以「指令条数」为单位，需左移 2 位换算成字节地址。

J 型指令的目标地址：把 26 位直接地址左移 **2** 位（末尾补两个 0，因为指令按字对齐）得到 28 位，再与 PC 高 4 位拼接：

$$\text{Target_PC} = PC[31{:}28] \parallel (\text{target} \ll 2)$$

*3.*常用指令

类别	指令	汇编举例	含义
算术	add	add $s1,$s2,$s3	$s1 = $s2 + $s3（带溢出检查）
	sub	sub $s1,$s2,$s3	$s1 = $s2 - $s3
	addi	addi $s1,$s2,100	$s1 = $s2 + 100（立即数符号扩展）
访存	lw	lw $s1,100($s2)	$s1 = M[$s2 + 100]
	sw	sw $s1,100($s2)	M[$s2 + 100] = $s1
逻辑	and / or / nor	and $s1,$s2,$s3	按位与 / 或 / 或非
	andi / ori	andi $s1,$s2,100	立即数零扩展后按位运算
移位	sll / srl	sll $s1,$s2,2	逻辑左移 / 右移（shamt 字段）
条件分支	beq	beq $s1,$s2,L	if ($s1 == $s2) goto L
	bne	bne $s1,$s2,L	if ($s1 != $s2) goto L
	slt / slti	slt $s1,$s2,$s3	if ($s2 < $s3) $s1 = 1; else $s1 = 0
跳转	j	j L	无条件跳转
	jal	jal L	$ra = PC+4; goto L（过程调用）
	jr	jr $ra	goto $ra（过程返回）

MIPS 没有状态码

x86 靠标志位（ZF/SF/OF/CF）判断条件，**MIPS** 没有状态码寄存器。它判断条件只有两条路：① 用 beq/bne 直接比较两个寄存器；② 需要比较大小时，先用 slt/slti 把结果（0 或 1）放进临时寄存器，再用 beq/bne 与该寄存器或 $zero 比较。「先 **slt** 再 **beq**」是 **MIPS** 分支的两步法。

*4.*简单 *C* 语句的 *MIPS* 翻译

（1）if-else 的翻译

```
if (check_val == target_val)
    score = score + bonus;
else
    score = score - penalty;
```

```
    bne  $s0,$s1,label_else   ; 用" 反面" 条件跳过 IF 块
    add  $s2,$s2,$s3          ; score = score + bonus
    j    label_exit
label_else:
    sub  $s2,$s2,$s4          ; score = score - penaltylabel_exit:
```

若条件涉及大小比较（如 if （x > y） max = x; else max = y;），先把 x > y 转成等价形式 y < x：

```
    slt  $t0,$s1,$s0          ; 若 y < x 则 $t0 = 1，即 x > y 成立
    beq  $t0,$zero,label_else ; $t0 == 0 说明 x <= y，跳去 Else
    add  $s2,$s0,$zero        ; max = x
    j    label_exit
```

```
label_else:
    add  $s2,$s1,$zero        ; max = ylabel_exit:
```

（2）循环的翻译（含数组变址）

```
while (i < size) {
    sum = sum + src[i];
    i++;
}
```

MIPS 没有自动变址寻址，lw/sw 的唯一形式是 offset(base) 且 offset 必须是立即数。因此访问 src[i]（int 数组）必须显式算出地址：

$$EA = \text{src 首地址} + i \times 4$$

```
Loop:
    slt  $t0,$s0,$s1          ; i < size ?
    beq  $t0,$zero,Exit       ; 不成立则跳出
    sll  $t1,$s0,2            ; $t1 = i * 4   （左移 2 位代替乘法）
    add  $t1,$t1,$s3          ; $t1 = src + i*4，即 src[i] 的地址
    lw   $t2,0($t1)           ; 取出 src[i]
    add  $s2,$s2,$t2          ; sum += src[i]
    addi $s0,$s0,1            ; i++
    j    Loop                 ; 跳回循环头
Exit:
```

数组变址的移位规律

int（4 字节）数组：下标乘以 4 → sll 左移 **2** 位；short（2 字节）数组：下标乘以 2 → sll 左移 **1** 位；char 数组：下标即字节偏移，不移位。看到 sll rX，rY，2 就应该反应出「这是在算 int 数组元素的地址」。

（3）过程调用的翻译：叶子过程与非叶子过程

叶子过程（内部不再调用其他过程）：不需要栈帧，因为 $ra 不会被覆盖，$t 系列寄存器又属于调用者保存。

```
int leaf_calc(int a, int b) { return (a + b) * 4; }
```

```
leaf_calc:
    add  $t0,$a0,$a1      ; $t0 = a + b
    sll  $v0,$t0,2        ; $v0 = $t0 << 2  （左移代替乘以 4）
    jr   $ra              ; 直接返回
```

非叶子过程（内部还要调用其他过程）：必须建立栈帧并保存 $ra。

```
int non_leaf(int x, int y, int z) {
    int sum = leaf_calc(x, y);
    return sum + z;
}
```

```
non_leaf:
    addi $sp,$sp,-8       ; 开辟 8 字节栈帧
    sw   $ra,4($sp)       ; 必须保存返回地址：后面的 jal 会覆盖 $ra
    sw   $s0,0($sp)       ; 保存被调用者保存寄存器 $s0
    addu $s0,$a2,$zero    ; 把 z 挪到 $s0 暂存（$a2 可能被子过程破坏）
    jal  leaf_calc        ; 调用子过程，返回值在 $v0
    add  $v0,$v0,$s0      ; sum + z
    lw   $s0,0($sp)       ; 恢复 $s0
    lw   $ra,4($sp)       ; 恢复返回地址
    addi $sp,$sp,8        ; 释放栈帧
    jr   $ra              ; 返回
```

非叶子过程必须保存 $ra 的原因

执行 jal 时硬件会自动把下一条指令的地址写入 $ra，覆盖原来的值。如果非叶子过程不在开头把 $ra 压栈，本函数原本的返回地址就永久丢失，程序会陷入死循环或跳飞。这是「为什么必须用栈」的核心原因。

本章小结表

考纲条目	核心结论	关键公式 / 数字	常考题型
指令系统的基本概念	ISA 是软硬件界面；机器字长 = ALU/通用寄存器宽度；指令字长与机器字长无约束关系；指令字长决定 IR 位数	机器字长 = 定点数据通路位数	概念选择
指令格式	操作码 + 地址码；定长操作码 $n \geq \lceil \log_2 m \rceil$；扩展操作码逐级「展开再扣减」	每减少一个地址字段，可用编码乘 2^p	扩展操作码计数
零/一/二/三地址指令	访存次数取决于操作数位置与是否写回，不取决于地址码个数	三地址取两源写一结果，最多 4 次访存	位宽分配、寻址范围
寻址方式	偏移寻址 = 寄存器 + 形式地址（相对/基址/变址）；间接寻址不是偏移寻址	相对 $EA = (PC) + D$，**PC** 已自增	转移目标地址计算
各寻址方式访存次数	立即/寄存器 0 次；直接/寄存器间接/相对/基址/变址 1 次；间接 2 次起	括号层数 = 访存层数	选择、计算
大小端	大端：MSB 在低地址；小端：LSB 在低地址	大端 LSB 地址 = 首地址 + 字节数 − 1	字节序列题
边界对齐	每成员从自身对齐要求的整数倍偏移开始；末尾补齐到最大对齐值的整数倍	填充可出现在成员间与结构体末尾	结构体大小/成员地址
CISC 与 RISC	指令数、字长、访存限制、寄存器数、控制器、流水线、编译优化、兼容性	CISC 微程序控制；RISC 硬布线控制	概念辨析选择
编译/汇编/链接	预处理 → 编译 → 汇编 → 链接	编译产出汇编代码；汇编产出目标文件	顺序选择
选择结构	cmp/test 设标志位（不改操作数）+ jcc 反向跳转	有符号 jg/jl（SF/OF）；无符号 ja/jb（CF/ZF）	汇编分析
循环结构	do-while 后测试（至少执行一次）；while/for 前测试	条件成立向后跳回，不成立向前跳出	汇编分析

考纲条目	核心结论	关键公式 / 数字	常考题型
过程调用	调用者保存 vs 被调用者保存；`call` 压返回地址、`ret` 弹出	`[ebp+8]` 为第 1 个参数，`[ebp-4]` 为第 1 个局部变量	栈帧布局、递归
MIPS	32 位定长、Load/Store、无状态码、`$ra` 存返回地址	I 型分支：$(PC+4)+\text{signExt}(off)\times 4$	大题（常与流水线结合）

易错清单

逐条核对，每一条都是统考反复设置的陷阱

1. 扩展操作码的第一级要先「展开」再「扣减」。正确顺序是「上一级剩余编码数 $\times 2^p$ − 本级已用条数」，顺序颠倒会得到完全不同的答案。
2. 零地址指令的可用编码数等于上一级剩余前缀数 $\times 2^p$。漏掉最后这一步乘法是最常见的失分点。
3. 操作码位数取定后必须逐级验证容量。某一级不够用（如二地址需要 300 条但只展开出 48 个编码），说明起始操作码位数取小了，必须加长重算。
4. 相对寻址公式里的 (PC) 是已自增后的值，等于「当前指令地址 + 当前指令长度」，不是当前指令的地址。漏加指令长度会差出整整一条指令的长度。
5. 相对寻址的位移量是补码，为负数时要先符号扩展再做加法，不能当无符号数直接加。
6. 「偏移寻址」只包含相对寻址、基址寻址、变址寻址三种，间接寻址不属于偏移寻址。
7. 基址寻址与变址寻址公式相同但角色相反：基址寄存器存基准地址（执行中不变），变址寄存器存偏移量（执行中递增）；形式地址在基址寻址中是偏移量，在变址寻址中是基准地址。
8. 变址寄存器的内容是下标还是字节偏移，必须看题目约定。两者相差一个 `sizeof(元素)` 因子（`int` 数组差 4 倍）。
9. 间接寻址的寻址范围由主存单元字长决定，与形式地址位数无关；寄存器间接寻址的寻址范围由寄存器位数决定。
10. 大端方式下 **LSB** 不在首地址。求「LSB 所在地址」时，大端是「首地址 + 字节数 − 1」，小端才是「首地址」。
11. 基址寻址中补码形式地址要符号扩展，32 位相加后最高位的进位要丢弃。
12. 填充字节可能出现在结构体末尾。总大小必须是「最大成员对齐要求」的整数倍，这一步最容易被漏掉。
13. 填充字节的内容是不确定的，考试只考位置和字节数，不要假设它是 0。
14. 结构体的对齐值取「最大成员的对齐要求」，不是「所有成员字节数之和」，也不是机器的最大对齐值（取两者中较小者）。
15. 「**RISC** 普遍采用微程序控制器」是错误的。RISC 普遍采用硬布线（组合逻辑）控制，CISC 才以微程序控制为主。
16. 「通用寄存器数量不多」是 **CISC** 的特征，RISC 的通用寄存器数量多。
17. 「严格限制只有 **LOAD/STORE** 指令能访存」是 **RISC** 的特征，CISC 对访存指令不加限制。
18. 「大多数 **RISC** 架构能与老机器软件兼容」是错误的，只有 CISC 具备这种向后兼容性。
19. 「**RISC** 大多数指令在一个时钟周期内完成」是正确的，不要因为「复杂功能要多条指令组合」就误判为错误。
20. 编译与汇编的产物不要颠倒：编译产出汇编语言程序，汇编产出目标文件（机器语言，尚不可执行）；完整顺序是「预处理 → 编译 → 汇编 → 链接」，链接之后才是可执行目标文件。
21. `cmp` 和 `test` 不改变操作数的值，只更新标志位。`cmp` 底层做减法，`test` 底层做按位与。
22. 有符号比较用 `jg/jl`（看 **SF** 与 **OF**），无符号比较用 `ja/jb`（看 **CF** 与 **ZF**）。混用会在负数或大数上出错。
23. `if-else` 的汇编常用「反面条件」跳转：条件不成立才跳到 Else，条件成立时顺序执行 Then 块；Then 块末尾必须有 `jmp` 跳过 Else。

24. 只有 `do-while` 的循环体至少执行一次，`while` 和 `for` 初始条件为假时循环体一次都不执行。
25. 循环条件的条件跳转是「成立则向后跳回」，跳出循环的跳转是向前跳。方向写反即为错误。
26. 参数压栈顺序是「从右至左」，因此第一个参数的地址最小（`[ebp+8]`），最后一个参数的地址最大。
27. `[ebp+4]` 是返回地址，`[ebp]` 是旧的 **EBP**，所以第一个参数在 `[ebp+8]`，第二个在 `[ebp+12]`。这个 8 的来由必须能推出来。
28.「调用者保存」说的是「调用者若想保值就得自己保存」，不是「调用者有义务为被调用者保存」；被调用者可以随意覆盖调用者保存寄存器。
29. 非叶子过程必须在开头把 `$ra` 压栈（MIPS）/ `call` 会自动覆盖返回地址（x86），因为过程中的下一次调用会覆盖返回地址。
30. 递归深度决定栈帧数量：深度为 d 时同一时刻栈上最多有 d 个栈帧，递归总栈空间与深度成正比。`f(10)` 需要调用 10 次，不是 11 次。
31. **MIPS** 没有状态码寄存器，比较大小时必须「先 `slt` 再 `beq/bne`」两步走，不能用 x86 的 `jg/jl` 思路去套。
32. **MIPS** 的 **I** 型分支指令偏移量以「指令条数」为单位，目标地址要算 $(PC+4)+\text{SignExt}(off)\times 4$，漏乘 4 就会算错。
33. **MIPS** 访问数组必须显式计算地址（`sll` 算字节偏移 + `add` 加首地址），没有 x86 那种「基址 + 变址 × 比例」的一条指令完成自动变址。
34. 指令字长与机器字长没有任何约束关系，不要因为「指令字长 16 位」就推断「机器字长也是 16 位」。IR 的位数由指令字长决定，MAR 的位数由地址线位数决定，MDR 的位数由数据线位数决定。

第五章中央处理器

本章考纲要求

五、中央处理器（CPU）（一）CPU 的功能和基本结构（二）指令执行过程（三）数据通路的功能和基本结构（四）控制器的功能和工作原理（五）异常和中断机制

1. 异常和中断的基本概念
2. 异常和中断的分类
3. 异常和中断的检测与响应

（六）指令流水线

1. 指令流水线的基本概念
2. 指令流水线的基本实现
3. 结构冒险、数据冒险和控制冒险的处理
4. 超标量和动态流水线的基本概念

（七）多处理器基本概念

1. SISD、SIMD、MIMD、向量处理器的基本概念
2. 硬件多线程的基本概念
3. 多核（multi-core）处理器的基本概念
4. 共享内存多处理器（SMP）的基本概念

CPU 是计组分值最重、也最容易出综合大题的一章。本章的考察呈「三条主线」：结构主线（寄存器与数据通路的组成、位数、控制信号）、时序主线（指令周期—机器周期—时钟周期的层级与各周期的微操作序列）、数量主线（数控制存储器的容量、数数据通路需要多少时钟周期、数流水线的总周期与吞吐率）。

只要把「微操作 → 控制信号 → 时钟周期数」这条链条打通，本章的选择题与应用题就都是同一套方法。以下按考纲条目顺序展开。

一、CPU 的功能和基本结构

（一）CPU 的功能

CPU（Central Processing Unit）由运算器和控制器两大部分构成。现代 CPU 还把 Cache、MMU（存储管理部件）、浮点运算部件、异常与中断处理逻辑集成到片内，但按「数据通路 + 控制部件」的两分法理解，任何复杂的 CPU 都不例外。

CPU 的四个基本功能：

- 指令控制：控制指令的执行顺序。程序是指令的有序集合，CPU 必须保证指令按序取、按序执行，由 PC 保证。
- 操作控制：产生每条指令的操作控制信号序列（即微命令序列），送到各执行部件的控制端。

- 时间控制：对各种操作信号进行时间调制，规定每条控制信号的产生时刻与持续时间，即「什么时候送操作数、什么时候发运算控制、什么时候送结果」。
- 数据加工：对数据进行算术运算、逻辑运算与逻辑测试。

一句话记住四条功能

指令控制（顺序）+ 操作控制（命令）+ 时间控制（时序）+ 数据加工（算术逻辑）。

（二）运算器的组成与各部件的作用

运算器的核心任务是「对数据进行加工」，由下列部件组成：

部件	英文/缩写	作用
算术逻辑单元	ALU	完成算术运算、逻辑运算；输出运算结果与标志
通用寄存器组	GPRs / GR	存放操作数、中间结果、地址信息；对汇编程序员可见
暂存寄存器	X、Y、Z 等	暂存 ALU 的输入操作数或输出结果；对程序员透明
累加寄存器	ACC	通用寄存器之一，常作为 ALU 的一个输入端与结果的落点
移位器	—	对操作数或运算结果进行移位（左移、右移、算术/逻辑移位）
程序状态字寄存器	PSW / PSR / FLAGS	保存 OF、SF、ZF、CF 等状态标志与中断、系统状态信息
计数器	—	控制乘、除运算的操作步数

需要特别说明的是暂存寄存器为什么必需。在单总线数据通路中，ALU 是纯组合逻辑部件，做一次运算时必须保证两个输入端同时有效。若第一个操作数从主存读出后直接放在总线上，等第二个操作数再经总线送出时前一个操作数已被总线上的新数据破坏，因此必须先把先到的操作数锁存到暂存寄存器 X（另一端固定接 ALU 输入）。同理，ALU 的输出端不能直接连总线，否则输出会经总线反馈到输入端形成竞争，所以运算结果要先锁存到暂存寄存器 Z（或 Y）。

- 暂存寄存器对应用程序员透明，不参与指令系统，不能用汇编指令直接访问。
- 累加寄存器 **ACC** 是通用寄存器，与「暂存寄存器」性质不同：ACC 程序员可见，暂存寄存器不可见。
- 移位器不是所有运算器都必备，有的机器把移位功能做进 ALU，或让 ALU 输出直接接数据总线/地址总线。做题时必须按题目给的框图判断，不能想当然。

PSW 的作用有两个层次：

1. 保存算术逻辑运算或测试指令所产生的状态信息——溢出标志 **OF**、符号标志 **SF**、零标志 **ZF**、进位/借位标志 **CF**、辅助进位 **AF**、奇偶标志 **PF**。条件转移指令正是根据 PSW 的内容判定是否转移。
2. 保存中断与系统工作状态信息，如中断允许标志 IF、陷阱标志 TF、方向标志 DF、处理器工作状态位（内核态/用户态）。操作系统靠它判断 CPU 当前所处状态。

PSW 的两个高频陷阱

- **PSW** 可能只在某些指令执行后改变。普通传送指令（如 `MOV`）通常不改变标志位，因此「某指令执行后 ZF=1」这类判断要先看该指令是否影响标志。
-「标志位由 ALU 产生」并不等于「ALU 是时序元件」。**ALU** 仍是组合逻辑元件，标志位只是它的组合输出，需要由 PSW 这个时序元件锁存才得以保存。

（三）控制器的组成与各部件的作用

控制器的任务是「根据指令、时序信号和状态条件产生控制信号」。

部件	英文/缩写	作用
程序计数器	PC / IP	存放下一条待执行指令的地址
指令寄存器	IR	存放当前正在执行的指令，其位数 = 指令字长
指令译码器	ID	把 IR 中的操作码翻译成译码信号，同一时刻只有一路有效
存储器地址寄存器	MAR / AR	存放将要访问的主存单元地址，经地址总线送主存
存储器数据寄存器	MDR / DR	存放从主存读出或将要写入主存的数据，是 CPU 与主存间的缓冲
微操作信号发生器	CU	综合译码信息、时序信号与状态条件，产生微命令序列
时序系统	—	产生时钟脉冲、机器周期、节拍电位等定时信号

各部件的工作脉络是：**PC** → **MAR** → 主存 → **MDR** → **IR** → 译码器 → 微操作信号发生器 → 各部件控制端。IR 中的地址码部分还要经地址生成逻辑（按寻址方式）加工，产生的地址按需要送入 PC、MAR 或运算部件。

（四）各寄存器的位数与地址、字长的关系

这是选择题每年都可能出现的一类题，必须区分清楚「哪个寄存器由谁决定」。

PC 与 **MAR** 由主存地址位数决定。

- **MAR** 的位数等于主存地址总线的位数，即 MAR 位数 $= \log_2$(可寻址单元总数)。它反映的是最多能寻址多少个存储单元。无论取指令还是取操作数，地址都必须先送 MAR。
- **PC** 的位数一般与主存地址总线位数相等，因为 PC 保存的就是一个主存地址。

但如果题目问的是「至少需要多少位」，就要看指令在内存中的存放方式：

地址对齐如何压缩 PC 位数

若按字节编址、指令字长 32 位（4 B）且按字边界对齐存放，则任何一条指令的起始字节地址的低 2 位恒为 00。此时 PC 只需保存「对齐位置」的编号共 2^{30} 个，即 **30** 位；形成真实字节地址时在低位补两个 0 即可。而 **IR** 必须完整保存一条 **32** 位指令，至少需要 **32** 位。

这就是「主存 4 GB、字长 32 位、按字节编址、32 位定长指令字、按字边界对齐」时答案取 **PC = 30** 位、**IR = 32** 位的原因。

MDR/IR 与机器字长、存储字长的关系。

- **IR** 的位数 = 指令字长。定长指令系统中它等于机器字长（或者说指令字长）；变长指令系统中 IR 要能装下最长的指令。
- **MDR** 的位数 = 存储字长，因为 MDR 是 CPU 与主存之间一次数据交换的宽度。常见配置中存储字长与机器字长相等，所以有「MDR 位数 = 存储字长 = 机器字长」的说法；但这一等式并不必然成立。若主存的存储字长与机器字长不等（例如机器字长 64 位、存储字长 32 位），则 MDR 只需 32 位。

位数关系的四个易错点

- **MAR** 只看「可寻址单元个数」，不看编址方式本身。按字节编址的 4 GB 主存，地址 32 位，MAR 就是 32 位；若按字（4 B）编址，则地址位数减 2，MAR 也随之减 2。
- **PC** 与 **MAR** 一般都等于地址总线位数，但在「至少」类问题中 **PC** 可以更短（靠地址对齐省掉低位）。
- **MDR** 由存储字长决定，不由机器字长决定，两者只是常常相等。
- 通用寄存器的位数 = 机器字长（反映一次能处理的数据位数），与地址位数无关。32 位机中 GPR 32 位，但主存可能有 4 GB 即地址 32 位，两者数值相同只是巧合；若主存是 64 GB，则地址 36 位而 GPR 仍是 32 位。

真题考法

- 「汇编语言程序员可见的寄存器」——PC、通用寄存器、基址/变址寄存器、PSW 可见；**MAR**、**MDR**、**IR**、微指令寄存器、暂存寄存器、控制存储器（**CM**）不可见。可见性的判据是「程序能否通过指令语义观察或改变它的状态」，而不是「能否把它直接当操作数写在汇编里」。
- 「PC 和 IR 的位数至少是多少」——按上面对齐法算。
- 「时钟脉冲信号」的性质：时钟周期由机器脉冲源经整形分频产生，其倒数是主频；同步时序电路要求的时钟周期不短于「寄存器之间最长组合逻辑路径传播时间 + 寄存器建立时间」。但「每来一个时钟脉冲就执行一条新指令」是错的：多周期 CPU 中一条指令占多个时钟周期，流水线被阻塞时也可能没有新指令进入。

二、指令执行过程

（一）指令周期的层级：指令周期、机器周期、时钟周期

一条指令从主存取出到执行完毕所需的全部时间称为指令周期（**instruction cycle**）。它是 CPU 执行一条指令的时间，是本章时序分析的最高层级。指令周期可以划分为若干机器周期（**CPU** 周期），每个机器周期又可以划分为若干时钟周期（节拍脉冲 / 振荡周期）。三者是层层包含的关系：

$$\text{指令周期} \supset \text{机器周期} \supset \text{时钟周期}, \qquad T_{\text{指令}} = \sum_i T_{\text{机器}i}, \qquad T_{\text{机器}} = \sum_j T_{\text{时钟}j}$$

层级	定义	能完成的工作	由什么决定
指令周期	取出并执行一条指令的全部时间	一条完整指令	指令功能、寻址方式、是否发生中断
机器周期	从主存读出一个指令字所需的最短时间	一个较复杂的基本操作（如一次存储器读或写）	通常以主存工作周期为基础确定
时钟周期	主频的倒数（$T = 1/f$）	一次微操作	机器脉冲源经整形分频

关于机器周期的宽度，考纲与统考的口径是：机器周期的宽度通常以主存工作周期为基础来确定。原因是主存的读/写周期比 CPU 内部的一次运算操作长得多，若以 CPU 内部操作为准，则存储器根本来不及响应。

不同指令的指令周期长度不同，这是理解取指、间址、执行、中断四周期的关键：

- 空操作指令 NOP：只包含取指周期（但仍会改变 PC 与 IR）。
- 无条件转移指令 JMP：包含取指周期 + 执行周期。
- 采用间接寻址的指令：多出一个间址周期。
- 在指令结束时若检测到中断请求：多出一个中断周期。

因此，一个完整的指令周期可能包含取指周期、间址周期、执行周期、中断周期四个阶段，其中执行周期可以有 0 ~ n 个机器周期。

三条必背结论

1. 任何指令的指令周期都至少包含一个机器周期，也至少包含一个时钟周期，即 $T_{指令} \geq T_{机器} \geq T_{时钟}$。因此「指令周期一定大于或等于一个 CPU 时钟周期」是正确的。
2. 只要不采用 **Cache** 和不采用指令预取，每条指令的指令周期中 **CPU** 至少访问内存一次（取指）。
3. 指令周期的长度由功能决定，不由时钟周期数固定。单周期 CPU 中所有指令都占 1 个时钟周期，但那个时钟周期必须覆盖最慢指令的完整通路，所以周期很长、主频很低。

（二）取指周期的流程与数据通路

取指周期的任务是「按 PC 从主存取出指令送入 IR，并形成下一条指令地址」。用寄存器传送语言（RTL，Register Transfer Language）描述为：

```
(PC)   -> MAR          ; 现行指令地址送地址寄存器
(MAR)  -> MDR  等价于 M(MAR) -> MDR   ; 按地址读出指令字
(MDR)  -> IR           ; 指令送指令寄存器
(PC)+1 -> PC           ; 形成下一条指令地址（"1" 表示一条指令的长度）
```

对应的控制信号序列（以单总线数据通路为例）：

时钟	数据通路（微操作）	控制信号
C1	(PC) → MAR	PCout，MARin
C2	M(MAR) → MDR；(PC)+1 → PC	read，MDRinE，PCinc（+1）
C3	(MDR) → IR	MDRout，IRin

操作控制器在取指周期要发出 `PCout`、`MARin`、`read`、`MDRinE`、`MDRout`、`IRin`、`PCinc` 等信号。注意第三步不能与第二步合并：数据从主存读出并稳定地出现在 MDR 上需要时间，同一时钟周期内不能既完成访存又完成经总线向 IR 的传送。

（三）间址周期的流程与数据通路

间址周期的任务是「取出操作数的有效地址」。它在取指周期结束后，由译码器分析 IR 确定该指令使用了间接寻址才进入。以 `MOV R1,(05H)` 这类源操作数采用间址寻址的指令为例：

```
Ad(IR) -> MAR          ; 把形式地址（05H）送地址寄存器
1      -> R            ; 发读命令
M(MAR) -> MDR          ; 从主存读出有效地址
(MDR)  -> Ad(IR)  或者  (MDR) -> MAR   ; 有效地址送回地址码字段或直接送 MAR
```

时钟	数据通路（微操作）	控制信号
C4	Ad(IR) → MAR	AD(IR)out，MARin
C5	M(MAR) → MDR	read，MDRinE

时钟	数据通路（微操作）	控制信号
C6	(MDR) → MAR	MDRout，MARin

间址周期每增加一次间接寻址，就要多访问一次主存。一次间址需要一个完整的机器周期（因为必须访存一次）。

取指微程序与间址微程序的关系

取指周期的微程序是公共的，对每条机器指令都相同，微程序控制器直接提供；而间址周期与执行周期的微程序是在取指结束后、由指令译码器分析操作码一次性生成的，然后在各自阶段的时钟到来时分别启动。这一点是理解微程序控制器「下地址逻辑」的关键。

（四）执行周期的流程与数据通路

执行周期的任务是「根据 IR 中指令字的操作码和操作数，经 ALU 处理后产生执行结果」。不同指令的执行周期操作不同，没有统一的数据流向，这是它与其他三个周期的根本区别。

仍以「`MOV R1,(05H)`，源操作数间址」为例：间址周期已取得有效地址并放入 MAR，执行周期要把该地址中的数据写入 R1：

时钟	数据通路（微操作）	控制信号
C7	M(MAR) → MDR	read，MDRinE
C8	(MDR) → R1	MDRout，R1in

各类指令在执行周期中的典型微操作（单总线结构、只涉及单次传送的示例）：

- 寄存器之间传送：`(R1) → R2`，一个时钟周期，`R1out`、`R2in`。
- 加法运算并写回：`(R1)→Y`；`(ACC)+(Y)→Z`；`(Z)→ACC`，需 **3** 个时钟周期，因为任何时刻总线上只能有一个数据。
- 存数：`(R1)→MDR`；`(R2)→MAR`；`MDR→M(MAR)`，通常 3 个时钟周期。
- 转移：`Ad(IR) → PC`，一个时钟周期，`IRout`、`PCin`。

（五）中断周期的流程与数据通路

中断周期发生在一条指令执行结束、且检测到中断请求并允许响应时，任务是保存断点并转去中断服务程序。若将断点保存在栈中、SP 指向栈顶、进栈操作为「先修改指针后送入数据」，则：

```
(SP)-1 -> SP          ; 栈指针先减 1（若约定 SP 指向栈顶空单元则改为 (SP)->MAR 后再 (SP)-1->SP）
(SP)   -> MAR         ; 断点要存入的存储单元地址送 MAR
1      -> W           ; 发写命令
(PC)   -> MDR         ; 把断点（当前 PC）送 MDR
(MDR)  -> M(MAR)      ; 断点写入主存
向量地址 -> PC        ; 中断服务程序入口地址送 PC，转向服务程序
```

中断周期的微操作可概括为：关中断、保存断点、引出中断服务程序。这三件事由中断隐指令完成（详见异常与中断机制一节）。

（六）各周期微操作的通用分析方法

综合题的答题步骤可以固定为：

1. 列时序：先确定这条指令要经历哪几个周期（取指？间址？执行？中断？）。
2. 标数据流：每个周期写出所有必须在同一时刻完成的 RTL 微操作。
3. 查冲突：同一时钟周期内检查三条「一」规则——同一寄存器只出现一次、同一总线只使用一次、主存/ALU 只访问一次。
4. 数周期：每个时钟周期完成一次不冲突的数据传送，周期数即为时钟数。

指令执行过程的常见陷阱

- 「空操作指令的指令周期中任何寄存器内容都不改变」是错的。NOP 不完成显式数据运算，但取指仍会改变 PC（自增）和 IR。
- 「每个时钟周期都开始执行一条新指令」是错的（多周期、流水线阻塞都不成立）。
- 外中断在指令执行结束后检测，内中断（异常）在指令执行过程中检测。混用这两句几乎必错。

三、数据通路的功能和基本结构

（一）数据通路的定义与组成元件

数据通路（**datapath**）：指令执行过程中数据所经过的路径，以及路径上涉及的部件。数据通路由操作元件和状态元件两类元件组成，二者必须分清：

类别	别名	特性	典型部件
组合逻辑元件	操作元件	输出只取决于当前输入，不受时钟控制，无记忆	ALU、加法器、移位器、多路选择器 MUX、译码器、扩展器
时序逻辑元件	状态元件	能保存状态，受时钟驱动，有记忆	PC、IR、MAR、MDR、通用寄存器组、暂存寄存器、PSW、流水段寄存器、存储器

判据

「这个部件会不会记住上一次的结果」——会记住的是状态元件，不记住的是操作元件。

- ALU、MUX、扩展器、加法器：操作元件。
- PC、GPRs、暂存寄存器 X/Z、流水段寄存器：状态元件。
- 通用寄存器组的读操作是组合逻辑（给地址就有数据输出），写操作是时序逻辑（要时钟沿触发），因此 GPRs 可以在同一个时钟周期内完成一次「读 + 写」。

数据通路由控制部件产生的控制信号控制：寄存器的写使能、MUX 的选择端、ALU 的功能选择、存储器的读写控制，都来自控制器。

容易混淆的两条

- 生成控制信号的控制部件本身不属于数据通路。所以「指令流水线的数据通路包含生成控制信号的控制部件」是错的。
- 数据通路要包含（或配合）异常事件的检测与响应电路：如运算溢出检测、地址越界/对齐检测、以及异常入口所需的状态保存和 PC 选择通路。所以「数据通路不包含用于异常事件检测及响应的电路」是错的。

（二）单总线结构的数据通路

把 ALU 和所有寄存器都连到一条内部公共总线上，就是单总线结构。

- 由于单总线上某一时刻只能有一个部件输出数据，运算器必须配置暂存寄存器 **X**（暂存 ALU 的一个操作数）与 **Z**（暂存运算结果）；ALU 的另一个操作数直接来自内总线。
- 所有向总线输出的部件都通过三态门控制，防止总线冲突；`PCout`、`MDRout`、`IRout`、`Rout`、`Zout` 等是互斥的。
- 单总线结构不能用来实现单周期处理器：一条稍复杂的指令需要多次总线传送，一个时钟周期内根本做不完。因此单周期 CPU 必须使用专用数据通路。

在单总线结构中，「某一时刻总线上只有一个数据有效」决定了每一组必须前后相继的传送各占一个时钟周期。

（三）多总线结构与专用数据通路

- 多总线结构：CPU 内部有两条或更多条总线。三条总线时，总线 A、B 分别传送两个源操作数，总线 C 传送结果，因此暂存寄存器 **Y**、**Z** 可以取消（三条总线各传不同数据，不会冲突）。配合双口通用寄存器组，`R[R3] ← R[R1] op R[R2]` 只需一个时钟周期。
- 专用数据通路：按指令执行过程中数据与地址的流动方向安排专门连接，不使用共享总线。优点是避免资源竞争、性能高、便于实现流水线；缺点是硬件量大。现代处理器普遍采用专用数据通路。

在专用通路中，若要把一个寄存器的内容传送到另一个寄存器，通常借道 **ALU** 完成（控制 ALU 做「直送」操作）。

真题考法

「单周期处理器为什么不能采用单总线结构」几乎每年都以不同形式出现。答题要点是：单总线一次只能完成一次数据传送，而单周期要求所有指令在一个时钟周期内完成（CPI=1），两者矛盾。单周期的代价是时钟周期必须按最慢指令的关键路径确定，主频低；其好处是控制信号在整条指令执行期间保持不变，不需要复杂的时序控制。

（四）数据通路的数据传送过程与时钟周期数计算

这是主频最高的大题考点。核心方法论只有一句：一个时钟周期完成一次不发生冲突的数据传送；凡是必须在不同时刻发生的传送，就要各占一个时钟周期。

给出两条硬性规则（可直接用于排节拍）：

1. 同一节拍中，同一寄存器只能出现一次（不能既读又写同一寄存器作为源和目的）。
2. 同一节拍中，同一条总线只能使用一次；同一节拍中只访问主存一次、只访问 **ALU** 一次。

完整算例：单总线数据通路执行 `MOV R1,(R2)`（源操作数间址寻址），计算所需时钟周期数。

设按字节编址、指令字长 32 位、按边界对齐，PC 自动加 1（表示加一个指令字长）。执行这条指令的全部微操作序列为：

阶段	节拍	微操作	控制信号	说明
取指	T1	(PC) → MAR	PCout，MARin	送指令地址

阶段	节拍	微操作	控制信号	说明
取指	T2	M(MAR) → MDR；(PC)+1 → PC	read，MDRinE，PCinc	访存取指，同时改 PC
取指	T3	(MDR) → IR	MDRout，IRin	指令入 IR
间址	T4	Ad(IR) → MAR	AD(IR)out，MARin	形式地址送 MAR
间址	T5	M(MAR) → MDR	read，MDRinE	取出有效地址
间址	T6	(MDR) → MAR	MDRout，MARin	有效地址留在 MAR
执行	T7	M(MAR) → MDR	read，MDRinE	按有效地址取操作数
执行	T8	(MDR) → R1	MDRout，R1in	写入 R1

结论：单总线数据通路执行该指令共需 **8** 个时钟周期。其中取指 3 拍、间址 3 拍、执行 2 拍。

对照：若采用三总线专用数据通路，且执行周期中的取数与写回都能并行安排，则取指仍为 3 拍（受主存访问限制），但间址可压缩为 2 拍、执行可压缩为 1 拍，总计约 6 拍。这说明数据通路结构直接决定指令所需的时钟周期数，做题必须按题目框图上「哪些部件之间有直接通路」逐拍判定。

时钟周期数计算的四句口诀

1. 数访存次数：每次主存读/写各占一拍（或题目规定的存储器等待周期数）。
2. 数总线传送次数：单总线上每次「一个源 → 一个目的」占一拍，且要检查是否冲突。
3. 数 **ALU** 运算次数：涉及 ALU 的运算另占一拍（输入已就绪的前提下）。
4. 能并行就并行：多总线结构下同一拍可以同时完成多次传送，能省则省，但受上面两条硬性规则约束。

四、控制器的功能和工作原理

（一）时序与控制

控制器要在正确的时刻发出正确的控制信号，所以必须有时序系统。时序信号的作用是对微操作控制信号进行时间调制，严格规定每条信号的产生时刻与持续时间。基本层次为：时钟脉冲（节拍）→ 机器周期 → 指令周期。

- 同步控制方式：所有操作由统一的时钟节拍控制，实现简单，但机器周期长度必须按最慢操作确定，效率较低。
- 异步控制方式：不给操作规定固定节拍，靠「回答信号」串联，效率高但控制复杂。
- 联合控制方式：多数操作按同步方式，少数慢操作（如访存）用异步回答，是实际机器的常见做法。

（二）硬布线控制器

硬布线控制器（组合逻辑控制器）：用专门的组合逻辑电路直接产生控制信号。它的输入是指令译码信号、时序信号（机器周期/节拍电位）和状态条件，输出是各部件所需的微命令。

设计思路（可按此复述）：

1. 分析微操作命令：对每条机器指令、每个机器周期，列出所有微操作，确定每条微操作对应哪个微命令、在哪个节拍发出。

2. 列出逻辑表达式：把「微命令 = 该微命令在哪些指令、哪个机器周期、什么条件下需要发出」写成布尔表达式。可用与阵列（译码器）—或阵列结构或可编程逻辑阵列 PLA 实现。
3. 时序：用节拍电位（机器周期状态）+ 节拍脉冲共同控制，保证信号按正确的时间顺序出现。

特点：

比较项	硬布线控制器	微程序控制器
控制信号来源	组合逻辑电路直接产生	执行控制存储器中的微指令产生
速度	快（省去了读微指令、译码微指令的时间）	慢（每条微命令要多一次访存 CM 和译码）
规整性	差，是一个多输入/多输出的巨大逻辑网络	规整，逻辑上与「程序设计」类似
修改与扩展	困难（要重新设计硬件逻辑）	容易（只需改写控制存储器中的微程序）
适用场合	指令系统简单规整（典型是 RISC、MIPS）	指令系统复杂的 CISC 处理器
实现方式	组合逻辑（可用有限状态机描述）	存储技术 + 程序设计技术，属「软件时序」

（三）微程序控制器

基本思想：仿照程序设计的方法，把每条机器指令的执行过程分解为若干步，每一步的微命令序列预先编制好并存放起来；执行指令时按顺序取出这些「微命令集合」去控制硬件。这就是「存储程序」思想在控制器设计上的应用。

一组术语必须分清（这是选择题高频区）：

- 微命令（**micro-command**）：控制部件向执行部件发出的控制信号，如 `PCout`、`MARin`、`ADD`、`read`。
- 微操作（**micro-operation**）：执行部件收到微命令后所进行的操作，如 PC 把内容送上总线、ALU 完成加法。微命令与微操作一一对应。
- 相容性 / 互斥性微操作：能在同一个节拍内并行执行的微操作为相容，不能并行为互斥。例如 `read` 与 `write` 互斥，各部件的「输出到总线」控制信号互斥（同一时刻总线上只能有一个数据），而各寄存器的「从总线锁存数据」使能信号可以相容（可同时有多个模块接收）。
- 微指令（**micro-instruction**）：同一节拍内发出的、相容的一组微命令的集合，即「一个状态」的全部控制信号。
- 微程序（**micro-program**）：完成一条机器指令所需的一组微指令序列。每条机器指令对应一段微程序。
- 控制存储器（**Control Storage**，**CS/CM**）：位于 CPU 内部、存放全部微程序的只读存储器。**CS** 按地址访问，不是按内容访问。
- 微地址：微指令在控制存储器中的地址。
- 微指令寄存器（μ**IR**）：在微指令执行期间暂存微指令。
- 微地址寄存器（μ**AR** / μ**PC**）：存放将要执行的微指令的微地址。
- 微周期（**micro-cycle**）：从控制存储器中读取并执行一条微指令所需的全部时间，通常为一个时钟周期。

微命令 / 微指令 / 微程序的三层关系

微命令（**1** 个控制信号）→ 微指令（同拍内相容的微命令集合）→ 微程序（实现一条机器指令的微指令序列）

由此得到两条极重要的等价关系：

- 一条机器指令需要多少个时钟周期，它的微程序就包含多少条微指令（反之亦然）。
- 机器指令与微指令是两个层次的东西。CPU 能直接理解并执行的是机器指令（以及微程序控制器内部的微指令）；伪指令、汇编指令都必须先由汇编器翻译成机器指令。
- 只有相容的微命令才能出现在同一条微指令中，这直接决定了微指令的并行能力。

*1.*控制存储器与主存的比较

项目	主存储器 **MM**	控制存储器 **CS**
存放内容	机器指令与数据	微程序（微指令）
位置	CPU 外部	CPU 内部
存储容量	大	小
存储字长	存储字长（常等于机器字长）	微指令长度
存取速度	慢	快（与 CPU 同周期量级）
实现介质	ROM 与 RAM 均可	一般是 ROM（只读型）
访问方式	按地址访问	按地址访问

CS 与 MM 的常见错误说法

「控制存储器按内容访问」是错的——它和主存一样按地址访问。按内容访问的是相联存储器（如 TLB 的实现方式）。

（四）微指令的格式与编码方式

微指令的基本格式包含三部分：

1. 操作控制字段（微命令字段）：给出本条微指令要发出的所有微命令。
2. 顺序控制字段（下地址字段 / 转移控制字段）：给出下一条微指令的微地址，或指明如何形成下一条微地址。
3. 判别测试字段（P 字段）：表示按什么条件进行分支判断，如按指令操作码无条件分支（P0）、按 PSW 的相等标志条件分支（P1）。

*1.*按操作控制字段的组织方式分类

- 水平型微指令：一次能定义并执行多个可并行的微命令。微指令字较长，微程序短，并行能力强、执行速度快，但微指令字长浪费大、编写困难。
- 垂直型微指令：一条微指令只能完成一个微操作（或 1 ~ 2 个），类似机器指令的格式，要用微操作码 + 操作数地址。微指令字短、编码空间利用率高，但微程序长、并行能力差、执行速度慢。

水平型微指令的并行能力

「水平型微指令并行操作能力强」这一结论的根据是——它把同一条微指令中的操作控制字段分成若干段，同一段内的微命令互斥、不同段之间的微命令相容，因此一次可以同时发出多条相容微命令。判断一条微指令能发出几个微命令，看的就是它的操作控制字段里「能同时置为有效」的位数。

*2.*微指令的编码方式

直接编码（直接控制法）：操作控制字段中每一位对应一个微命令，置 1 发该微命令，置 0 不发。

- 优点：不必译码，控制信号直接产生，速度最快，微命令可任意组合（并行能力最强）。

- 缺点：机器微命令个数很多时微指令字极长，而一条微指令中有效的微命令很少，编码空间利用率低。

字段直接编码（字段直接编码法）：把操作控制字段划分成若干小字段，互斥的微命令编在同一个字段，相容的微命令编在不同字段。每个字段经译码后产生该字段内的一个微命令。

- 关键规定：每个字段必须能够表示「本字段不发出任何微命令」这一状态，因此含 n 个互斥微命令的字段至少需要 $\lceil \log_2(n+1) \rceil$ 位。
- 总位数公式：

$$W_{\text{op}} = \sum_{i=1}^{m} \lceil \log_2 (n_i + 1) \rceil$$

其中 m 为字段（互斥类）个数，n_i 为第 i 个字段内的互斥微命令数。

- 优点：在保持一定并行能力的同时显著缩短微指令字长。
- 缺点：要经过译码，速度比直接编码慢一点。

字段间接编码（字段间接编码法）：一个字段中的微命令含义还要由另一个字段（或另一个字段的某几位）决定，简称「间接编码」。它能进一步缩短微指令字长，但译码电路更复杂、微指令并行能力进一步下降。统考只要求知道它的定义与「进一步缩短字长、并行性更差」这一结论。

完整算例：字段直接编码的位数计算（对照 2012 年统考真题）

某计算机采用微程序控制方式，操作控制字段采用字段直接编码法，共有 33 个微命令，构成 5 个互斥类，分别包含 7、3、12、5、6 个微命令，求操作控制字段至少多少位。

$$\lceil \log_2(7+1) \rceil + \lceil \log_2(3+1) \rceil + \lceil \log_2(12+1) \rceil + \lceil \log_2(5+1) \rceil + \lceil \log_2(6+1) \rceil$$

$$= 3 + 2 + 4 + 3 + 3 = 15 \text{ 位}$$

注意 $\lceil \log_2 13 \rceil = 4$（因为 $2^3 = 8 < 13 \le 16 = 2^4$），$\lceil \log_2 7 \rceil = 3$。答案 **15** 位。若误以为不需要「全 0 不操作」状态，就会算成 $3+2+4+3+3$ 之外的错误结果或漏掉一位；若误用直接编码则得到 33 位。

（五）微地址的形成方式

1. 计数器方式（增量方式）：微地址寄存器带计数功能，微指令按 μPC+1 顺序执行；遇到转移时用转移地址字段修改 μPC。
   - 优点：微指令中不必设置下地址字段，字长较短。
   - 缺点：不能实现多路条件分支，只能二路分支；编制微程序时不灵活。
2. 断定方式（下地址字段法）：微指令中直接给出下一条微指令的地址（下地址字段），由判别测试字段决定是否按该字段转移。
   - 优点：可实现多路分支，微程序编制灵活。
   - 缺点：每条微指令都要带下地址字段，微指令字较长。
3. 结合方式（混合方式）：把以上两者结合，顺序执行时用计数器，需要分支时用下地址字段，兼顾字长与灵活性。

微程序入口地址的形成（结合前面「转移微指令」的概念）：

- 取指微程序是所有指令公共的，其入口地址固定（如 0000）。取指结束后的第一条微指令是转移微指令，它的作用就是「根据机器指令的操作码，跳转到该指令对应微程序的入口地址」。

- 转移微指令通常通过一个专门的小型 ROM（记为 ROM1）实现：以 IR 中的操作码为输入（即索引），输出即该指令微程序的入口地址。
- 同一入口下再分支的情形（如 `lw`/`sw` 首地址相同但后续流程不同）可用第二个 ROM（ROM2）实现，也可用断定法的硬件逻辑实现。例如下地址字段给出 `1010`，当机器指令操作码的某一位为 1 时用异或门把它变成 `1001`：$1010 \oplus 0011 = 1001$，即可用极少的硬件实现二路分支。
- 微程序的最后一条微指令通常取「取下一条机器指令的取指微程序首地址」，从而回到取指，形成循环。

判别测试字段（P 字段）的作用

P 字段本身的位数由「有多少种判别测试条件」决定，因此 P 字段位数 $= \lceil \log_2(\text{测试条件数} + 1) \rceil$。若微程序共有 25 个状态（微指令），则下地址字段至少 $\lceil \log_2 25 \rceil = 5$ 位。

（六）微指令条数与控制存储器容量的计算

这是「数量主线」的核心题型。三条计算步骤：

1. 算微指令条数：

$$N_\mu = N_{\text{公共取指微程序}} + N_{\text{各指令微程序总和}}$$

1. 算微指令字长（位数）：$W_\mu = W_{\text{操作控制字段}} + W_{\text{判别测试字段}} + W_{\text{下地址字段}}$。
2. 算控制存储器容量：$C = N_\mu \times W_\mu$（单位 bit）或 $C = \dfrac{N_\mu \times W_\mu}{8}$（单位 B）；反之，下地址字段位数 $\geq \lceil \log_2 N_\mu \rceil$。

完整算例一（对照 **2014** 年统考真题）：某计算机采用微程序控制器，共有 32 条机器指令，公共取指微程序包含 2 条微指令，各指令对应的微程序平均由 4 条微指令组成，采用断定法（下地址字段法）确定下条微指令地址，求下地址字段至少多少位。

- 各指令微程序微指令数：$32 \times 4 = 128$ 条。
- 控制存储器至少容纳：$2 + 128 = 130$ 条。
- $2^7 = 128 < 130 \leq 256 = 2^8$，故下地址字段至少 $\lceil \log_2 130 \rceil = 8$ 位。

完整算例二（自编，用于练容量）：某机有 60 条机器指令，公共取指微程序 3 条微指令，各指令微程序平均 6 条微指令；微指令字长为操作控制字段 24 位、判别测试字段 2 位、下地址字段（由算例一的方法确定为 9 位）。求控制存储器容量。

- 微指令条数：$3 + 60 \times 6 = 363$ 条。
- 微指令字长：$24 + 2 + 9 = 35$ 位。
- 控制存储器容量：$363 \times 35 = 12705$ 位 $= 12705/8 \approx 1588.1$ B，取整为 **1589 B**（工程上按整数个存储字，即 363 字 $\times$ 35 位）。

微程序计算题的三个陷阱

- 公共取指微程序只算一次，不要按指令条数重复计算。
- 下地址字段位数由「微指令总条数」决定，不是由机器指令条数决定。先算总条数再取对数。
- 单位：题目问「多少位」就不要答字节；问「多少字节」时要注意是否进位取整。

五、异常和中断机制

（一）异常和中断的基本概念与区别

异常（**exception**），又称内中断：CPU 在执行当前指令的过程中，由 CPU 内部检测到的、与当前指令同步的意外事件。

中断（**interrupt**），又称外中断：CPU 外部的设备或部件提出的、与当前指令异步的服务请求。

对比项	内部异常（**exception**）	外部中断（**interrupt**）
来源	CPU 内部，由当前指令或其执行过程引起	CPU 外部设备或部件（定时器、键盘、网卡……）
与当前指令的关系	同步：由当前指令引起	异步：与当前指令无关
检测时机	在指令执行过程中实时检测	在一条指令执行结束后采样中断请求引脚
能否屏蔽	不能用中断屏蔽位屏蔽	分为可屏蔽中断与不可屏蔽中断
典型例子	除数为 0、非法操作码、缺页、越权访问、自陷、溢出	键盘输入、定时器到时、网络数据包到达、打印机缺纸

判断外部中断的唯一标准

问自己一句：「这个事件是不是由当前这条指令同步引起的?」

- 「键盘输入」→ 外部中断；「除数为 0」→ 内部异常。
- 「访存缺页」→ 发生在访存指令的地址转换过程中，属内部故障，不是外部中断。
- 「浮点运算下溢」→ 由当前浮点指令引起，属内部异常。

不要用「事件发生在内存里还是设备外」来判断——缺页发生在内存管理部件里，但它仍是内部异常。

为什么外部中断必须在指令结束后才能检测? 若在指令执行中途响应中断，指令被部分执行，CPU 无法保存一个「精确且可恢复」的中间状态；而异常发生的时刻和引发它的指令是确定的，处理完后可以准确地重新执行或跳过该指令，所以异常可以在指令执行过程中检测并响应。

（二）异常和中断的分类

*1.*内部异常的三类

类型	英文	含义	指令完成情况	返回位置	典型例子
故障	fault	指令执行中发现的、可能被修复的错误	指令尚未成功完成	返回本条指令重新执行	缺页、缺段、访问权限违例、某些对齐错误
自陷	trap	程序主动执行陷阱指令（如 `INT n`、`syscall`）引发	指令已经完成	返回下一条指令	系统调用、断点、单步跟踪、调试
终止	abort	严重且通常不可恢复的硬件错误	发生严重错误，程序状态不可靠	一般不返回原程序	硬件故障、总线错误、电源故障

三类异常的分辨要点

- 自陷（**trap**）属于内部异常，不是外部中断。「自陷是通过陷阱指令预先设定的一类外部中断事件」是错误说法——它与当前指令同步。
- 不能说「内部异常处理后都返回发生异常的指令继续执行」。fault 返回原指令重执行，trap 返回下一条指令，abort 不返回。三者返回位置不同。
- 自陷可在用户态由程序主动执行陷阱指令触发（如 `INT n` 产生软中断），转而进入内核态执行相应处理程序——这是「用户态 → 内核态」的一条正规途径。

*2.*外部中断的两类

- 可屏蔽中断（**maskable interrupt**，`INTR`）：主要来自 I/O 设备。CPU 可通过在中断控制器中设置屏蔽字来屏蔽或不屏蔽它；被屏蔽的中断请求信号不会送到 CPU。一般外设中断都属于可屏蔽中断。
- 不可屏蔽中断（**non-maskable interrupt**，`NMI`）：通常是非常紧急的硬件故障，如电源掉电、内存校验错、硬件看门狗超时、时钟中断、硬件线路故障。它通过专用 `NMI` 请求线提出，不受中断允许标志 **IF** 和中断屏蔽字控制，任何情况下都会被送到 CPU。

不可屏蔽中断（NMI）的四条结论

1. **CPU** 处于关中断状态（**IF=0**）时，仍能响应 **NMI** 请求。
2. **NMI** 的优先级高于所有可屏蔽中断，通常仅低于硬件复位。
3. NMI 同样需要保存断点、同样需要通过中断向量表（或 IDT）寻址服务程序（x86 中向量号固定为 2）。所谓「NMI 没有中断响应周期」是错误说法。
4. 响应 NMI 时 CPU 会自动屏蔽可屏蔽中断（等效于隐式 `CLI`），但不屏蔽 **NMI** 自身，即 NMI 可以嵌套；这种屏蔽持续到执行中断返回指令为止。

统考中若题目未提 NMI，外部中断一律按可屏蔽中断处理；讨论「多重中断」时也只针对可屏蔽中断。

中断优先级的两个层次（务必区分）：

- 响应优先级（中断判优）：由中断查询程序或硬件判优电路决定的优先级，反映「多个中断同时请求时先响应谁」。它由硬件连线或固定逻辑决定，软件不易改动。经验顺序为：不可屏蔽中断 > 内部异常 > 可屏蔽中断；内部异常中，硬件故障 > 软件中断；I/O 设备按速度高低排序，速度高的优先级高（因为速度高的设备数据容易丢失）。
- 处理优先级（中断屏蔽）：由各自的中断屏蔽字动态设定，反映「正在处理某个中断时，允许谁打断它」。可以用中断屏蔽字改变的是处理优先级，不是响应优先级。

（三）异常和中断的检测与响应

可屏蔽中断的响应条件（三条同时满足）：

1. 有中断请求（中断请求信号有效，且该请求未被屏蔽）；
2. CPU 处于开中断状态（IF=1）；
3. 当前指令执行结束，且没有更高优先级的请求正在处理。

「请求有效」不等于「立即响应」

「一旦可屏蔽中断请求信号有效，CPU 将立即响应」是错误的。中断响应是「异步但不立即」的：即使信号有效，若 IF=0、或当前指令尚未执行完、或有更高优先级事件正在处理，CPU 都不会响应。

响应时机与流程：CPU 在每条指令执行结束、取下一条指令之前，采样中断请求引脚（如 `INTR`）。若发现有效且满足响应条件，则进入中断响应过程。中断控制器根据各设备的中断请求和屏蔽情

况，结合中断响应优先级，识别出当前请求的中断类型号，并通过数据总线送给 CPU。

中断隐指令：CPU 在中断响应过程中由硬件自动执行的一系列操作。它不是一条真实的机器指令（在指令系统中找不到它，也无操作码），但功能上等价于一条指令，称为「隐指令」。其内容为：

1. 关中断（将中断允许触发器清 0），防止在保存现场时被新的可屏蔽中断打断；
2. 保存断点——保存 PC（有的机器还同时保存 PSW）到栈或专用寄存器（如 MIPS 的 EPC）；
3. 引出中断服务程序——根据中断类型号从中断向量表取出中断向量（中断服务程序的入口地址）送 PC。

硬软件分工的一句话

硬件负责：关中断、保护断点、引出服务程序（中断隐指令）软件负责：保护现场（保存通用寄存器）、设置新屏蔽字、中断事件处理、恢复现场、开中断、中断返回

中断向量与中断向量表：中断向量是中断服务程序的入口地址（有的体系结构中还包含初始 PSW）；中断向量表是存放所有中断向量的表格，位于内存固定区域，以中断类型号为索引。「中断类型号 → 查表 → 中断向量 → 送 PC」是硬件完成的寻址过程。

断点保存位置：为支持嵌套处理，多数处理器把断点保存在栈中（如 80x86）；若不支持多重中断嵌套，可以把断点保存在专用寄存器中（如 MIPS 用 EPC 存放断点）。后者开销小，因为访问寄存器比访问栈（实际是访问主存）快。

单重中断的中断服务程序执行顺序（因为题目问的是「服务程序内」，硬件已在隐指令阶段完成关中断与保存断点）：

```
保护现场 -> 中断事件处理 -> 恢复现场 -> 开中断 -> 中断返回
```

支持多重中断（中断嵌套）时的完整顺序为：

```
关中断（隐指令）-> 保存断点（隐指令）-> 引出服务程序（隐指令）
-> 保护现场和屏蔽字 -> 设置新屏蔽字 -> 开中断 -> 中断事件处理
-> 关中断 -> 恢复现场和屏蔽字 -> 开中断 -> 中断返回
```

断点与现场不是一回事

- 保护断点：保存 PC（及 PSW），由硬件（中断隐指令）完成。
- 保护现场：保存通用寄存器等用户可见工作寄存器的内容，由中断服务程序（软件/操作系统）完成。

  因此「由操作系统保存的」是通用寄存器的内容，不是 PC；「中断隐指令完成的」不包括「保存通用寄存器」。

（四）多重中断与中断屏蔽

单重中断：CPU 在执行中断服务程序过程中，对新的中断请求一律不予响应。多重中断（中断嵌套）：CPU 暂停现行中断服务程序，转去处理新的更高优先级的中断请求。

CPU 要具备多重中断功能，必须满足两个条件：

1. 在中断服务程序中提前设置开中断指令（在保护现场之后开中断）；
2. 优先级别高的中断源有权中断优先级别低的中断源。

中断屏蔽技术：每个中断源配一个屏蔽触发器，所有屏蔽触发器组合成一个屏蔽字寄存器，其内容称为屏蔽字。CPU 可通过执行相应的 I/O 指令修改屏蔽字的内容，从而动态改变中断处理的先后次序。

屏蔽字的约定与规律：

- 一般约定 **1** 表示屏蔽（禁止），**0** 表示开放（允许申请）。
- 每个中断源对应一个屏蔽字——即处理该中断源的中断服务程序运行期间，屏蔽字寄存器中的内容。
- 屏蔽字中 **1** 越多，该中断的处理优先级越高。
- 每个屏蔽字中至少要有 **1** 个 **1**（至少要能屏蔽自身，防止同一中断源反复打断自己）。
- 屏蔽字中各位的顺序要与中断源一一对应（题目通常会说明第几位对应哪个中断源）。

「改变中断处理次序」的屏蔽字构造方法（可直接照做）：

1. 把中断源按题目要求的处理优先级从高到低排成一队，例如 $A > B > C > D$。
2. 写出第 X 个中断源的屏蔽字时：自身位填 **1**；处理优先级比它低的所有中断源，对应位填 **1**；处理优先级比它高的所有中断源，对应位填 **0**。
3. 换个说法：某中断源 S 的屏蔽字中，凡是「处理优先级不高于 S 」的中断源（含 S 自己）对应的位都是 **1**。

完整算例：按改变后的处理次序写出屏蔽字

题设：某中断系统有 4 个中断源 `1#`、`2#`、`3#`、`4#`，其中断响应优先级为 $1\# > 2\# > 3\# > 4\#$。现要求把中断处理优先级改成 $1\# > 4\# > 3\# > 2\#$。设屏蔽字中一位对应一个中断源，位序自左至右为 1#2#3#4#，**1** 表示屏蔽，**0** 表示开放。求各中断源对应的屏蔽字。

解：按处理优先级 $1\# > 4\# > 3\# > 2\#$，逐个来写。

- `1#` 的处理优先级最高，它不仅屏蔽自身，还必须能屏蔽所有处理优先级比它低的 $4\#, 3\#, 2\#$：$\Rightarrow$ **1111**。
- `4#` 处理优先级第二。比它高的只有 `1#`（对应位为 0），它屏蔽自身及更低的 $3\#, 2\#$（对应位为 1）：$\Rightarrow$ **0111**。
- `3#` 处理优先级第三。比它高的有 `1#`、`4#`（两位为 0），它与更低的 `2#` 为 1：$\Rightarrow$ **0011**。
- `2#` 处理优先级最低。比它高的有 `1#,4#,3#`（三位为 0），只有自身为 1：$\Rightarrow$ **0001**。

验算：每个屏蔽字中 1 的个数依次是 4、3、2、1，与处理优先级从高到低严格对应，符合「1 越多优先级越高」的规律；且每个屏蔽字都至少含一个 1（都不为 0），满足「必须能屏蔽自身」的要求。

接着算「**CPU** 完成各中断服务程序的过程」：设 CPU 执行主程序时，`1#`、`3#`、`4#` 同时发生请求，随后在执行 `3#` 的中断服务程序过程中，`2#` 又提出请求。

- 主程序：三请求同时到达，按响应优先级 $1\# > 3\# > 4\#$ 选 `1#` 先响应。运行 `1#` 时屏蔽字为 1111，其余三者全被屏蔽。
- `1#` 服务程序结束 $\rightarrow$ 恢复旧屏蔽字，剩下 `3#`、`4#` 待响应，按响应优先级选 `3#`。运行 `3#` 时屏蔽字为 0011，`4#`、`2#` 都被屏蔽。
- 此时 `2#` 提出请求但被 0011 屏蔽，不能打断 `3#`。
- `3#` 服务程序结束 $\rightarrow$ 恢复旧屏蔽字，此时 `2#`、`4#` 都已请求且未被屏蔽；按处理优先级 $4\# > 2\#$（`2#` 的屏蔽字 0001 允许 `4#` 打断），标为「`3#`$\rightarrow$ 恢复点」后先响应 `4#`。
- `4#` 运行期间屏蔽字 0111，`2#` 被屏蔽，不能嵌套。
- `4#` 结束 $\rightarrow$ 再按处理优先级响应 `2#`，`2#` 运行期间屏蔽字 0001。
- `2#` 结束 $\rightarrow$ 返回到最早被挂起的 `3#` 服务程序的断点处继续执行剩余部分。
- `3#` 结束 $\rightarrow$ 返回主程序。

最终处理先后次序为：`1#` 全部 $\rightarrow$ `3#` 前半 $\rightarrow$ `4#` 全部 $\rightarrow$ `2#` 全部 $\rightarrow$ `3#` 后半 $\rightarrow$ 主程序。

若把处理优先级改成 1# > 2# > 3# > 4#（与原响应优先级一致），则屏蔽字变为：

中断源	处理优先级 **1#>2#>3#>4#** 时的屏蔽字	处理优先级 **1#>4#>3#>2#** 时的屏蔽字
1#	1111	1111
2#	0111	0001
3#	0011	0011
4#	0001	0111

对照可见：只有 **2#** 与 **4#** 的屏蔽字互换，正是这一次互换把两者的处理次序颠倒过来。这也说明——不改屏蔽字时处理优先级就等于响应优先级；改屏蔽字只能改变处理次序，不能改变响应优先级本身。

多重中断的四条易错点

- 「中断处理期间 **CPU** 一直处于关中断状态」是错的。那是单重中断；多重中断在保护现场之后要开中断。
- 「中断屏蔽字用于确定中断响应的优先级」是错的。屏蔽字改变的是处理优先级（能否被打断），响应优先级由硬件判优电路固定决定。
- 「只有在用户态才能检测和响应中断」是错的。内核态同样可以响应中断，这正是中断嵌套的机制。
- 每个屏蔽字必须至少有一个 **1**（屏蔽自身），写屏蔽字题时若某行为全 0，一定是错的。

六、指令流水线

（一）指令流水线的基本概念

指令流水线（**instruction pipeline**）：把一条指令的执行过程划分为 k 个功能段（流水段），让多条指令在不同功能段中同时工作、重叠执行，从而提高单位时间内完成的指令数。它利用的是时间并行性。

经典五段划分（MIPS 风格）：

流水段	缩写	主要工作	主要功能部件
取指	IF	按 PC 从指令存储器取指令，同时计算 PC+4	指令存储器 IM、PC、加法器、MUX
译码/取数	ID	译码，按 Rs、Rt 从寄存器堆读出操作数	寄存器堆读口、控制器
执行	EX	ALU 运算、计算访存有效地址、分支比较与目标地址	ALU、扩展器、加法器
访存	MEM	读/写数据存储器；分支指令在此更新 PC	数据存储器 DM
写回	WB	把结果写回寄存器堆	寄存器堆写口、MUX

流水寄存器（流水段寄存器）：在相邻流水段之间插入的状态元件，保存上一段的中间结果供下一段使用。它们的作用是把各段的工作隔离开，使得不同指令可以同时在各自的功能段中工作。

- **IF/ID**：保存取出的指令和 **PC+4**（分支指令后面要用 PC+4 计算相对转移地址）。
- **ID/EX**：保存 $R[R_s]$、$R[R_t]$、R_t、R_d、$imm16$、$func$、**PC+4** 以及译码产生的控制信号。指令本身不必再保存（需要的信息已经提取出来）。
- **EX/MEM**：保存 ALU 结果（或有效地址）、Zero 标志、Overflow 标志、要写回的目的寄存器号、要存入 DM 的数据 $R[R_t]$、以及继续往后的控制信号。

- **MEM/WB**：保存从 DM 读出的数据与 ALU 结果、目的寄存器号、以及写回所需的控制信号。

为什么指令系统要在意规整性（有利于流水的指令系统特点）：指令格式规整且长度一致（如 MIPS 32 位定长，PC+4 计算方便）、指令和数据按边界对齐存放、只有 **Load/Store** 指令才能访问存储器、源寄存器位置固定（指令未完成译码即可开始读寄存器）。这四点使各段的行为统一、延迟稳定，便于流水。

流水线设计的两个原则：流水段个数以最复杂指令所用的功能段个数为准；流水段长度以最复杂的操作所花时间（关键路径）为准。这也解释了为什么流水线会「延长单条指令的执行时间」——所有指令都要按最慢指令的节奏走，还要加上流水段寄存器的开销。

二阶段流水线为什么不能成倍提速

① 执行阶段因各指令耗时不同，必须以最慢者为准，故执行阶段很长；② 为统一节拍，取指阶段也要拉长到与最慢执行阶段同长；③ 流水段寄存器的读写增加额外延迟；④ 各种冒险会引入阻塞。所以流水线深度不能无限增加，当段间缓冲开销占比过大时再加深就没有意义。

（二）指令流水线的性能计算

设流水线有 k 段，每段执行时间为 Δt，连续执行 n 条指令。

时钟周期：若忽略流水段寄存器延时，则 $T_{clk} \geq \max(t_1, t_2, \ldots, t_k)$；若段间寄存器延时为 t_r，则 $T_{clk} \geq \max(t_1, \ldots, t_k) + t_r$。

总时间：不留空隙时，第一条指令用 $k\Delta t$，此后每个 Δt 流出 1 条结果，共流出 $n-1$ 条：

$$T = k\Delta t + (n-1)\Delta t = (k+n-1)\Delta t$$

若无阻塞时的理想周期数为 $k+n-1$，而实际额外插入了 s 个阻塞周期，则 $T = (k+n-1+s)\Delta t$。

吞吐率（单位时间完成的指令数）：

$$TP = \frac{n}{T} = \frac{n}{(k+n-1)\Delta t}$$

当 $n \to \infty$ 时，理想最大吞吐率 $TP_{\max} = \dfrac{1}{\Delta t}$，即流水线充满后每周期完成一条指令。

加速比（相对非流水线串行执行；串行执行 n 条指令需 $nk\Delta t$）：

$$S = \frac{nk\Delta t}{(k+n-1)\Delta t} = \frac{nk}{k+n-1}$$

当 $n \to \infty$ 时 $S \to k$，即理想最大加速比等于流水段数 k。

效率（流水线设备的实际利用率。k 个段共 $k \cdot T$ 的时空区，完成 n 条指令占用 nk 个「段 · 周期」）：

$$E = \frac{nk}{k(k+n-1)} = \frac{n}{k+n-1}, \qquad E = \frac{S}{k}$$

完整算例：五段流水线的四项指标

题设：某处理器采用 5 段指令流水线，每段执行时间为 $\Delta t = 2$ ns，段间流水寄存器延时已计入这 2 ns 内。连续执行 100 条指令，执行过程中无任何阻塞。求总时间、吞吐率、加速比、效率。

- 总周期数：$k+n-1 = 5+100-1 = 104$ 个。
- 总时间：$T = 104 \times 2\ \text{ns} = 208\ \text{ns}$。

- 吞吐率：$TP = \dfrac{100}{208\ \text{ns}} \approx 0.4808 \times 10^9$ 条/秒 $\approx 4.81 \times 10^8$ 条/秒。
- 理想最大吞吐率：$\dfrac{1}{2\ \text{ns}} = 5 \times 10^8$ 条/秒（因为 $n = 100$ 有限，实际吞吐率略低于它）。
- 加速比：$S = \dfrac{nk}{k+n-1} = \dfrac{100 \times 5}{104} \approx 4.81$（理想上限为 $k = 5$）。
- 效率：$E = \dfrac{n}{k+n-1} = \dfrac{100}{104} \approx 96.2\%$，验证 $E = S/k = 4.81/5 \approx 96.2\%$。

变式：若每条指令之间都额外插入 1 个阻塞周期（$s = 99$），则周期数变为 $104 + 99 = 203$，$T = 406$ ns，$TP \approx 2.46 \times 10^8$ 条/秒，$S \approx 2.46$。

性能计算的四个陷阱

- 单位：Δt 常用 ns、ps，吞吐率是「条/秒」，别把 $1/\Delta t$（s^{-1}）与「条/秒」混算但忘记换算 10^{-9}。
- 理想最大加速比是 k，但有限 n 条指令时的实际加速比一定小于 k（因为流水线要填充，有 $k-1$ 个周期是「装填开销」）。
- n 很大时才能近似用 $TP_{\max} = 1/\Delta t$ 与 $S = k$。选择题常给「某机主频 1.03 GHz、4 级流水、执行 100 条指令」这类小数据，必须老老实实算 $k + n - 1$。
- 理想 **CPI**：单周期 **= 1**，基本流水线 **= 1**，超流水线 **= 1**，多周期 **> 1**，超标量 **< 1**。别把「超流水线 CPI<1」写对（那是错的，超流水线不改变 CPI，它靠提高主频提速）。

（三）结构冒险、数据冒险和控制冒险的处理

冒险（**hazard**）是指使流水线不能在预定的时钟周期内继续推进（需要阻塞）的情形。

*1.*结构冒险（*structural hazard*）

成因：多条指令在同一时钟周期争用同一硬件资源。典型情形：

- 单存储器时，Load 指令在 MEM 段取数据，同时 IF 段要取指令，两者争用同一个存储器；
- 寄存器堆只有单端口时，ID 段读寄存器与 WB 段写寄存器冲突；
- ALU 未流水化时，多条指令同时要用 ALU。

处理方法：

1. 规整功能段的划分原则：每个功能部件在每条指令中只使用一次，且只能在特定的流水段中使用（例如 ALU 只在 EX 段用、寄存器写口只在 WB 段用）。
2. 增加或分离资源：把寄存器堆的读口和写口独立开来（利用时钟上升沿写、下降沿读，使前半周期写、后半周期读）；把指令存储器 **IM** 与数据存储器 **DM** 分开——这正是指令 **Cache** 与数据 **Cache** 分离的主要目的：减少指令流水线的资源冲突，而不是为了提高命中率。
3. 阻塞（**stall**）：资源不足时让后到的指令等待。代价是流水线出现气泡。

*2.*数据冒险（*data hazard*）

成因：后面指令要用前面指令的结果，但该结果尚未产生。

三类数据相关：

类型	全称	含义	基本按序流水线中是否出现
RAW	写后读（真相关）	前指令写、后指令读同一位置	会，是必须处理的类型

类型	全称	含义	基本按序流水线中是否出现
WAR	读后写（反相关）	前指令读、后指令写同一位置	不会（按序执行不颠倒次序）
WAW	写后写（输出相关）	两条指令写同一位置	不会（按序写回不颠倒次序）

在按序发射、按序执行、按序写回的经典五段流水线中只可能出现 RAW；WAR 与 WAW 只会在乱序执行、多发射的处理器中成为真正的冒险。

处理方法：

1. 转发 / 旁路（**forwarding / bypassing**）：流水段寄存器中已经有了后续指令需要的值，通过增加 MUX 把 EX/MEM 或 MEM/WB 寄存器中的结果直接送回 ALU 输入端，无需等写回。转发是解决冒险的手段，本身不会引起阻塞。
2. 暂停、插入气泡（**stall / bubble**）：硬件检测到相关时冻结 PC 和 IF/ID 寄存器，让后续指令重复一次或插入 nop，直到数据可用。代价是 s 个阻塞周期。
3. 编译调度（软件方法）：由编译器把无关指令调到相关指令之间填充空隙；若无合适指令则插入 `nop`。

Load-use 冒险：转发解决不了的那一类

`load` 指令的数据在 **MEM** 段末才从 DM 读出，而紧随其后的指令在同一周期就要在 **EX** 段使用它，时间上来不及，因此即使有转发也必须至少阻塞 **1** 个周期。这就是典型的「装入—使用数据冒险（load-use data hazard）」。

结论：转发不能解决所有数据冒险，所以「所有数据冒险都可以通过加入转发电路解决」是错误的。与之相对，「所有数据冒险都能通过调整指令顺序和插入 nop 解决」是正确的（前提是保持程序语义）。

相关指令对的快速判定法（用于「下列指令对中不存在数据冒险的是」这类题）：把每条指令写成「写哪些寄存器、读哪些寄存器」，若「前一条写」与「后一条读」有交集，则为 RAW 真相关；若后一条只是写了同一寄存器而并不读它，则不构成 RAW。相邻的两条指令若只共享「被写」的寄存器而不存在「先写后读」关系，就不是数据冒险。此外，两条指令访问同一主存地址但处于基本按序流水线中时不构成冒险（较早的 load 必然先于较晚的 store 到 MEM 段，访存次序不会颠倒）。

*3.*控制冒险（*control hazard*）

成因：遇到会改变指令执行顺序的情形（各类转移指令、调用与返回指令、异常和中断），PC 的更新值来不及在取下一条指令之前准备好，导致流水线中已取出的若干条指令是错的。

延迟损失时间片 C：由于流水线阻塞而带来的延迟执行周期数。它由「分支结果和目标地址在第几段才确定」决定——越早算出并送回 **PC**，C 越小。在经典数据通路中分支在 MEM 段才更新 PC，$C = 3$（要丢弃/填充 3 个周期）；若在 ID 段就完成译码并送回转移目标地址，$C = 1$；若在 IF 段就能算出目标地址（如 `j` 指令按 PC+4 高 4 位与 26 位立即数拼接），则 $C = 0$。

处理方法：

1. 阻塞 / 插入气泡或 **nop**：在分支指令后插入 C 个气泡或 C 条 `nop`。效率极低，一般不单独采用。
2. 分支预测：
   - 静态预测：总是预测「条件不满足（not taken）」，即继续执行顺序指令。若转移概率约 50%，正确率也只有约 50%。
   - 动态预测：根据程序执行的历史动态调整预测，正确率较高，现代处理器几乎都采用。核

心结构是分支历史记录表 **BHT**（记录分支的历史方向）与分支目标缓冲 **BTB**（缓存分支目标地址，使 IF 段就能得到目标地址），每个表项用分支指令地址的低位作索引，因此在 IF 阶段就能取到预测位，从而不必等分支结果算出。

3. 延迟槽 / 分支延迟时间片的调度（静态调度）：由编译程序把分支指令前面与分支无关的指令调到分支指令后面执行，用来填充延迟时间片（称为分支延迟槽，**branch delay slot**），不够时用 nop 填充。
4. 预取两个方向的目标指令、加快并提前形成条件码、提高转移方向的猜准率。

真题考法

- 「不会引起流水线阻塞的是」——数据旁路（转发）。数据相关、条件转移、资源冲突都会引起阻塞。
- 「采用指令 Cache 与数据 Cache 分离的主要目的」——减少指令流水线的资源冲突（结构冒险），不是提高命中率，也不是降低缺失损失。
- 「无转发机制的五段流水线中，存在数据冒险的指令对是」——按上文的「写/读集合」判定，且注意没有转发时，距离 **2** 条以内的 **RAW** 都需要阻塞。
- 「采用转发处理数据冒险、硬件阻塞处理控制冒险时，哪些指令发生阻塞」——关键是识别出 **load-use** 必须阻塞 **1** 拍，以及分支指令之后的指令必须等待分支结果。

（四）超标量和动态流水线的基本概念

指令级并行（**Instruction Level Parallelism**，**ILP**）：单一指令流内部存在的、可同时执行的指令之间的并行性。提高 ILP 有两条基本途径：增加流水线级数（超流水线）和在一个周期内发射多条指令（多发射）。

超流水线（**super-pipelining**）：通过增加流水线级数，把每段的工作量切得更小，从而缩短时钟周期、提高主频。它不改变 **CPI**（理想仍为 **1**），靠主频提速；代价是流水段寄存器开销增大、分支预测失败的代价更大、气泡的负面影响更明显。

超标量（**superscalar**）：处理器内部设置多个独立的功能部件，使得一个时钟周期内可以同时发射并执行多条指令，理想 CPI <1（即 IPC>1）。它常与动态调度技术结合以提高指令执行的并行性。要注意：超标量的本质是「一个周期发射多条指令」，它并不缩短流水线功能段的处理时间（那是加深流水线或提频要做的事）。

多发射的两种实现方式：

比较项	静态多发射（超长指令字 **VLIW**）	动态多发射（超标量）
指令打包由谁决定	编译器在编译时完成（生成超长指令字，「发射包」）	硬件在执行时动态决定
冒险处理	完全或主要由编译器通过代码调度、插入 nop、静态分支预测消除	硬件检测数据冒险并阻塞，结合动态分支预测
可移植性	差：编译结果与具体处理器结构紧密相关，换机器往往要重新编译	强：编译后的代码可在不同超标量处理器上运行
硬件复杂度	简单（把最难的调度交给编译器）	复杂

推测（**speculation**）：由编译器或处理器猜测指令的执行结果，并据此调整指令执行顺序，使指令执行达到最大可能的并行。它依赖于「指令间相关性分析」和「分支方向预测」，一旦推测错误必须有检测与回滚机制，因此推测错误会增加额外开销。

动态调度与乱序执行：动态流水线的重要思想是「在等待解决阻塞时，到后面找指令提前执行」。

其通用模型为：

- 一个指令预取和译码单元：有序发射；
- 多个并列执行的功能单元：乱序执行；
- 一个提交单元：有序提交（保证异常精确、程序状态正确）。

为什么有了编译器调度还需要硬件动态调度：并非所有阻塞都能事先确定（如 **Cache** 缺失就是不可预见的阻塞）；动态分支预测必须依据真实执行情况；动态调度还能把处理器细节屏蔽起来，使同一份编译结果在不同处理器上都可用。

寄存器重命名（**register renaming**）：把指令中引用的体系结构寄存器（逻辑寄存器）映射到一组物理寄存器上，从而消除 **WAR** 与 **WAW** 这类「假相关」，只保留真正的 RAW 相关，让乱序执行能挖掘更多并行性。

推测执行（**speculative execution**）：在分支结果未确定时，按预测方向继续取指并执行，把结果暂时保存在不被提交的位置；预测正确则提交，预测错误则丢弃并回滚。

超标量的三条易错说法

- 「超标量能缩短流水线功能段的处理时间」——错。
- 「超流水线的 CPI 小于 1」——错，超流水线 CPI 仍为 1。
- 「超标量通过一个周期发射多条指令使 CPI 可以小于 1」——对。

七、多处理器基本概念

（一）Flynn 分类法与向量处理器

1966 年 Flynn 提出的分类法按指令流（**instruction stream**）数量和数据流（**data stream**）数量两个维度划分计算机体系结构：

类型	指令流	数据流	核心原理	典型硬件	并行层次
SISD	单	单	顺序处理	传统单核 CPU、微控制器	无显式并行（内部靠 ILP）
SIMD	单	多	数据级并行	GPU、CPU 向量单元（AVX/NEON）、向量处理器	数据级并行
MISD	多	单	冗余或流水线处理	容错系统（如航天飞机飞控计算机）；袁书认为现实中不存在这种计算机	—
MIMD	多	多	线程级 / 任务级并行	多核 CPU、多处理器、集群、超级计算机	线程级及以上

要点：

- **SISD**：仅包含一个处理器和一个存储器，一段时间内只执行一条指令流，每条指令最多处理两个数据（双目运算）或一个数据（单目运算）。现代单核 CPU 在宏观上仍是 SISD，但在微观上通过流水线、超标量、超流水线、**VLIW** 等技术挖掘指令级并行，从而在 SISD 框架内压榨单指令流的性能。
- **SIMD**：一个指令控制部件 + 多个处理单元 + 多个（局部）存储器，所有处于执行状态的处理单元同时执行同一条指令，但各自处理不同的数据。适合循环体向量化、图像/音频处理、图形学渲染、科学计算、神经网络等高度规整的数据级并行任务。

- **MIMD**：一定包含多个计算机或多个处理器，是目前大多数并行处理系统的处理方式。它可以同时支持数据并行（同一程序的多个副本处理不同数据，即 SPMD）和任务并行。

向量处理器（**vector processor**）属于 **SIMD**，专门面向向量型数据的并行计算。它与普通 SIMD 的差别在于：

1. 拥有专门的向量指令集：一条 `VADD` 指令即可完成两个含几十到上百个元素的向量相加，而普通 CPU 必须写循环。
2. 拥有大容量向量寄存器，一个向量寄存器存放一整个向量。
3. 计算单元是高度流水化的向量功能单元，可以每周期送入一组数据并每周期产出一组结果，吞吐率极高。
4. 拥有专门优化的 Load/Store 单元，可在主存与向量寄存器之间流式传输整块连续数据。

向量处理器多采用流水线方式（也有阵列方式，以流水线为主），主存储器容量限定了机器的解题规模，因此需要大容量集中式公共存储器，并常用多端口同时读取的交叉多模块存储器来维持高速运算所需的数据供给。

MISD 在实际中极为罕见，其价值不在加速计算而在提高可靠性与容错：用多个处理器以不同的算法/实现处理同一个关键数据流，再通过表决机制比较输出，从而屏蔽单机的硬件瞬时故障或软件缺陷。

（二）硬件多线程

问题背景：现代处理器经常要等待较慢的存储器。当遇到 Cache 缺失或长流水线暂停时，处理器进入停顿（**stall**），成百上千个时钟周期被浪费。

硬件多线程（**hardware multithreading**）：允许单个物理处理器核心同时管理多个线程的状态，操作系统会把这一个物理核心看成多个「逻辑核心」。当一个线程因停顿时无法继续时，核心快速切换到另一个已就绪的线程，用原本会被浪费的周期执行它的指令。其核心思想是用线程级并行（**TLP**）掩盖延迟和停顿，提高核心资源利用率与整体吞吐量。

硬件上的改造：复制线程状态（为每个线程提供独立的 PC、通用寄存器等）；共享执行资源（ALU、FPU、Cache、流水线等由所有线程共享）。

特性	细粒度多线程 **FGMT**	粗粒度多线程 **CGMT**	同时多线程 **SMT**
线程切换时机	每个时钟周期都切换	只在发生高成本长停顿（如 L2/L3 缺失）时切换	不切换，同一周期执行多个线程的指令
是否清空流水线	不涉及	需要清空流水线，切换开销较大	—
主要优点	能掩盖所有类型的延迟	单线程性能几乎不受影响，实现相对简单	资源利用率与吞吐量最大化
主要缺点	降低单线程执行速度，必须轮流分享执行周期	无法掩盖短延迟（数据依赖、分支预测失败等）	硬件设计最复杂，存在资源竞争与安全问题
典型实现	—	—	Intel 超线程（HT）、AMD SMT

硬件多线程的三条易错点

- 硬件多线程不只用于多核处理器。它是核心内部的技术，单核处理器同样可以采用。所以「硬件多线程技术只可用于多核处理器」是错的。
- **SMT** 下操作系统看到的是多个逻辑核心（如 4 核 8 线程的 CPU，OS 认为有 8 个可用处理器），由操作系统把软件线程调度到这些逻辑核心上，硬件再在微观层面执行。
- 细粒度多线程与粗粒度多线程不能混淆：「每个时钟周期轮流」是细粒度；「只在长停顿时才切换」是粗粒度。

（三）多核处理器

多核（**multi-core**）处理器：把多个处理器核集成在同一个芯片上。每个核有自己独立的控制部件、运算部件、寄存器和私有 Cache（L1，通常还有 L2），多个核共享片上的 L3 Cache、内存控制器等资源，并通过片上互连网络通信。

- 多核处理器中，各核可以执行不同的指令流、处理不同的数据，因此多核处理器属于 **MIMD** 结构。
- 多核的并行主要是线程级并行，需要操作系统的调度与并行编程配合；而流水线、超标量挖掘的是指令级并行，对软件透明。
- 多核相比多处理器系统（多个独立芯片）的优势：通信延迟低、带宽高、共享资源、功耗与体积更优。
- 多核带来的新问题：**Cache** 一致性、共享资源的竞争与同步、以及编程模型上对任务分解与负载均衡的要求。

（四）共享内存多处理器（SMP）与 UMA/NUMA、Cache 一致性问题

共享内存多处理器：多个处理器共享同一物理主存，通过互连网络（总线、交叉开关、网络）连接。

对称多处理器（**SMP**）：将多个相同类型的 CPU 通过总线互连，并以等同地位共享系统所有资源，多个 CPU 对称工作，无主次或从属关系，共享单一物理地址空间。因为每个 CPU 访问内存中任何地址所需时间相同，**SMP** 就是一种 **UMA** 结构。

按「处理器访问存储单元的时间是否一致」划分：

结构	全称	特征	说明
UMA	一致性内存访问	所有处理器访问任何存储单元的时间一致	也就是 SMP。所有 CPU 通过共享总线/交叉开关连到主存。实现简单，但 CPU 个数多时会产生访存冲突、总线成为瓶颈，可扩展性受限
NUMA	非一致性内存访问	处理器对不同存储单元的访问时间可能不一致，取决于位置：访问本地存储器快，访问其他处理器的存储器慢	把物理内存分布到各处理器附近形成「节点」。所有共享存储模块统一编址、形成单一地址空间，非本地内存也用 Load/Store 访问，因此 NUMA 上运行 UMA 程序无需改动，但同主频下性能不如 UMA

NUMA 的两个细分形态：

- **NC-NUMA**（No Cache NUMA）：处理器不带高速缓存。
- **CC-NUMA**（Cache-Coherent NUMA）：处理器带一致性高速缓存，必须解决 Cache 一致性，最常见的是基于目录（**directory**）的 **Cache** 一致性机制——每个存储器中有一个存放目录信息的

存储区，每个主存块对应一个目录项，目录项中有有效位等字段，记录该主存块与各 Cache 行的对应关系。

Cache 一致性问题：在共享内存多处理器中，同一主存块可能同时驻留在多个处理器的私有 Cache 中。若某处理器修改了自己 Cache 中的副本，而其他处理器 Cache 中的旧副本仍然有效并被读取，就会读到过期数据。这就是 **Cache 一致性（cache coherence）**问题。

解决思路：

- 监听 / 总线侦听（**snooping**）协议：各 Cache 通过共享总线监听其他处理器发出的访存事务；某个 Cache 中的副本被修改时，将修改信息广播到所有核的 Cache，若其他核的 Cache 中有该数据则将其置为无效，同时把主存中的数据同步更新。**MESI** 协议就是这类协议的典型代表（每个 Cache 行处于 M/E/S/I 四种状态之一）。
- 基于目录的协议：适用于互连网络不是共享总线、处理器数量较多的情形（如 CC-NUMA），由目录记录每个主存块被哪些 Cache 缓存、是否被修改，从而只向相关节点发送一致性维护消息，避免广播。

多计算机系统（**multicomputer**）与多处理器系统的区别：多计算机的每个节点有自己私有的地址空间，「共享内存」是靠消息传递在软件层模拟的。因为没有共享内存总线瓶颈和 Cache 一致性问题，多计算机系统具有极高的可扩展性，可扩展到成千上万个节点，当代顶级超级计算机几乎都采用这种架构。

真题考法

「下列关于并行处理技术的叙述中，不正确的是」——注意四个判据：多核处理器属于 **MIMD**（对）；向量处理器属于 **SIMD**（对）；硬件多线程只能用于多核（错，单核也可）；**SMP** 中所有处理器共享单一物理地址空间（对）。

本章小结表

主题	核心公式 / 结论	关键判据
寄存器位数	$W_{MAR} = W_{PC} =$ 地址位数；$W_{IR} =$ 指令字长；$W_{MDR} =$ 存储字长；$W_{GPR} =$ 机器字长	「至少」类问题：按边界对齐可省掉地址低位（PC 30 位 vs 地址 32 位）
寄存器可见性	PC、GPR、PSW 可见；MAR、MDR、IR、暂存器、μIR、CM 不可见	能否通过指令语义观察/改变
周期层级	指令周期 ⊃ 机器周期 ⊃ 时钟周期；机器周期以主存工作周期为基准	$T_{\text{指令}} \geq T_{\text{机器}} \geq T_{\text{时钟}}$
取指 RTL	(PC)→MAR；M(MAR)→MDR；(PC)+1→PC；(MDR)→IR	3 个节拍（单总线）
间址 RTL	Ad(IR)→MAR；M(MAR)→MDR；(MDR)→MAR	每次间接寻址增加一次访存
中断周期 RTL	(SP)-1→SP；(SP)→MAR；1→W；(PC)→MDR；(MDR)→M(MAR)；向量地址 →PC	关中断 + 保护断点 + 引出服务程序
操作元件 vs 状态元件	ALU、MUX、加法器、扩展器 = 操作；PC、GPR、暂存器、流水寄存器 = 状态	是否记忆状态
微命令/微指令/微程序	微命令（1 信号）⊂ 微指令（同拍相容命令集）⊂ 微程序（一条机器指令）	微指令条数 = 该指令的时钟周期数

主题	核心公式 / 结论	关键判据
字段直接编码	$W=\sum_{i=1}^{m}\lceil\log_2(n_i+1)\rceil$，必须留一个「全 0 不操作」状态	7,3,12,5,6 → 3+2+4+3+3 = **15** 位
控制存储器容量	$C=N_\mu\times W_\mu$；下地址字段 $\geq\lceil\log_2 N_\mu\rceil$	32 条指令 × 4 条 + 公共 2 条 = 130 条 → 8 位
硬布线 vs 微程序	硬布线：快、难改；微程序：慢、易改、规整	见控制器对比表
异常 vs 中断	异常：同步、指令执行中检测；中断：异步、指令结束后检测	是否由当前指令同步引起
三类异常返回位置	fault 返回原指令重执行；trap 返回下一条；abort 不返回	指令是否已完成
中断隐指令	关中断、保存断点、引出中断服务程序（全由硬件完成）	通用寄存器由软件保存
屏蔽字	1 屏蔽、0 开放；自身位为 1；屏蔽所有处理优先级不高于自己的源；1 越多处理优先级越高	不改屏蔽字则处理优先级 = 响应优先级
流水线时间	$T=(k+n-1+s)\Delta t$	无阻塞时为 $k+n-1$ 个周期
流水线吞吐率	$TP=\frac{n}{T}$，理想 $TP_{\max}=\frac{1}{\Delta t}$	单位统一换算到秒
流水线加速比	$S=\frac{nk}{k+n-1}\to k$	有限 n 时 $S<k$
流水线效率	$E=\frac{n}{k+n-1}=\frac{S}{k}$	—
时钟周期	$T_{clk}\geq\max(t_i)$；有段间寄存器则为 $\max(t_i)+t_r$	2018 年真题：80+20=100 ps
三类冒险	结构（分资源/阻塞）、数据（转发/气泡/调度）、控制（预测/延迟槽/阻塞）	转发不能解决 load-use
Flynn 分类	SISD / SIMD / MISD / MIMD	多核 = MIMD；向量处理器 = SIMD；MISD 多认为不存在
硬件多线程	FGMT 每周期切换；CGMT 长停顿时切换；SMT 同周期多线程	单核也能用硬件多线程
存储模型	UMA = SMP（访问时间一致）；NUMA 本地快远程慢；CC-NUMA 用目录协议	Cache 一致性问题源于私有 Cache 的副本

易错清单

必须逐条自检的易错点

1. **MAR** 由可寻址单元个数决定，不由机器字长决定；MDR 由存储字长决定，不必然等于机器字长。
2. **PC** 在「至少多少位」的题目里可以不等于地址位数（按字边界对齐时省掉低位，4 GB 主存 + 32 位定长指令 → PC 30 位、IR 32 位）。
3. 汇编程序员可见的是 PC、通用寄存器、基址/变址寄存器、PSW；**MAR**、**MDR**、**IR**、微指令寄存器、暂存寄存器、控制存储器不可见。
4. 空操作指令的指令周期中 **PC** 和 **IR** 都会改变，不能说「任何寄存器都不改变」。
5. 机器周期通常以主存工作周期为基准，而不是以 CPU 内部运算时间为基准。
6. **ALU** 是组合逻辑（操作）元件，PSW、GPR、暂存器是时序逻辑（状态）元件；数据通路不包含控制部件，但包含（或配合）异常检测与响应电路。

7. 单周期处理器不能采用单总线结构；单周期的 CPI=1，多周期 CPI>1，基本流水线 CPI=1，超标量 CPI<1，超流水线 CPI=1。
8. 微命令与微操作一一对应；只有相容的微命令能出现在同一条微指令中；一条微指令的并行能力由操作控制字段决定。
9. 控制存储器按地址访问，不是按内容访问；CS 在 CPU 内、用 ROM、存微指令、字长 = 微指令长度。
10. 字段直接编码必须为每个字段留出「全 **0** 不操作」编码，故含 n 个互斥微命令的字段要 $\lceil \log_2(n+1) \rceil$ 位。
11. 下地址字段位数由微指令总条数决定（含公共取指微程序），不是由机器指令条数决定。
12. 公共取指微程序的微指令只计入一次，不要按指令条数重复累加。
13. 异常是同步的、在指令执行过程中检测；外部中断是异步的、在指令执行结束后检测。
14. 自陷（**trap**）是内部异常，不是外部中断；fault 返回原指令重执行，trap 返回下一条，abort 不返回。
15. 中断隐指令 = 关中断 + 保护断点 + 引出中断服务程序，由硬件完成；保存通用寄存器是软件（**OS**）的事。
16. 可屏蔽中断请求有效 $\neq$ 立即响应；必须同时满足「有未屏蔽请求 + 开中断 + 当前指令执行结束」。
17. **NMI** 在关中断状态下也能响应，且优先级高于所有可屏蔽中断；「NMI 没有中断响应周期 / 不需要保存断点」都是错的。
18. 中断屏蔽字改变的是处理优先级，不是响应优先级；每个屏蔽字必须至少含一个 **1**（屏蔽自身）；某中断源的屏蔽字中，所有「处理优先级不高于它」的源对应位均为 1。
19. 多重中断中 **CPU** 不是一直关中断：保护现场前后要按「开中断—处理—关中断」的顺序配合。
20. 转发（旁路）是解决冒险的手段，本身不引起阻塞；但 **load-use** 冒险即使有转发也必须阻塞至少 **1** 拍，所以「所有数据冒险都能靠转发解决」是错的。
21. 指令 **Cache** 与数据 **Cache** 分离的主要目的是减少流水线的结构冒险（资源冲突），不是提高命中率。
22. 基本按序流水线中只会出现 **RAW**；WAR、WAW 只在乱序/多发射中成为真正冒险，靠寄存器重命名消除。
23. 超流水线不改变 **CPI**（仍为 **1**）；超标量的本质是一周期发射多条指令，不缩短功能段处理时间。
24. 动态调度与乱序执行的模型是「有序发射、乱序执行、有序提交」，有序提交是保证异常精确与程序状态正确的关键。
25. 硬件多线程不限于多核，单核也能实现；SMT 让操作系统看到多个逻辑核心。
26. 多核处理器属于 **MIMD**，向量处理器属于 **SIMD**，**MISD** 通常认为现实中不存在。
27. **SMP = UMA**（访问任何地址时间一致）；NUMA 中访问本地存储器快于远程；**CC-NUMA** 用基于目录的协议维护 Cache 一致性。
28. **Cache** 一致性问题来自私有 Cache 中同一主存块的多个副本被分别读写；解决途径是侦听（如 MESI）或基于目录的协议。

第六章总线和输入/输出系统

一、总线和输入/输出系统

本章考纲要求

（一）总线

1. 总线的基本概念
2. 总线的组成及性能指标
3. 总线事务和定时

（二）I/O 接口（I/O 控制器）

1. I/O 接口的功能和基本结构
2. I/O 端口及其编址

（三）I/O 方式

1. 程序查询方式
2. 程序中断方式：中断的基本概念，中断响应过程，中断处理过程，多重中断和中断屏蔽的概念
3. DMA 方式：DMA 控制器的组成，DMA 传送过程

本章是计组里「概念多、计算题密」的一章：总线带宽、查询开销、中断开销、DMA 传送时间四类计算几乎年年出现，而总线的分类、定时方式、接口寄存器、三种 I/O 方式的对比又几乎年年出选择题。学这一章的正确顺序是先抓住四组对比（串行/并行、同步/异步、统一编址/独立编址、中断/DMA），再把四个公式族练熟。下面按考纲条目顺序展开。

二、总线

（一）总线的基本概念

总线（**bus**）是连接多个部件、供各部件之间交换信息的一组共享传输线。它的两个本质特征是：

- 分时：同一时刻只允许一个部件（主设备）向总线发送信息，其他部件只能等待属于自己的时间段。
- 共享：总线上挂接的多个部件可以在不同时刻分别获得使用权；从设备可以在同一时刻共同接收总线上广播的信息。

总线是为了替代早期「分散连接」而出现的。分散连接下，部件两两之间都要有专门连线，连线数量随部件数平方级增长，且同一时刻每个部件只能与一个部件通信，既难扩展又难维护。改成总线后，所有部件挂在一组公共线上，新增设备只需挂到总线上，可扩展性大幅提升，代价是同一时刻只能有一对主从设备通信，需要总线仲裁来解决争用。

> **总线 = 共享 + 分时**
>
> 共享决定「大家都能用」，分时决定「同一时刻只有一个能发」。这两句话能推出后面所有结论：为什么需要仲裁、为什么需要定时、为什么带宽要按「每次传送多少字节 × 每秒传送多少次」算。

*1.*总线的分类

按数据传送方式分：串行总线与并行总线。

- 并行总线：含多条双向数据线，一次可以同时传送多位。优点是单位时间内传送的数据量大；缺点是各条数据线的电气特性很难完全一致，线长增加后各位可能无法同时到达接收端（时钟偏斜），相邻线之间还会串扰，因此并行总线只适合近距离通信。大多数系统总线属于并行总线。
- 串行总线：只有一条双向数据线或两条单向数据线，逐位按序传送。优点是线少、抗干扰好、传输距离长；缺点是同样时钟频率下单次传送量少。大多数通信总线属于串行总线。

「并行一定比串行快」是错的

并行总线能同时传多位，在时钟频率较低时确实比串行快；但当频率提高到一定程度，并行线之间的串扰和偏斜会让数据无法恢复，反而提不上频率。串行总线线少、串扰小，容易把时钟频率做高，还能用多通道（多条链路）叠加带宽，所以高速串行总线的实际速率完全可以超过并行总线。统考考过这个结论：PCI-Express 就是高速串行总线，PCIe×16 是 16 条串行通路的组合，不是 16 位并行传输。同理，USB 的 D+、D− 是一对差分信号，两者共同表示一个串行数据状态，不是「同时传输 **2** 位数据」。

按功能层次分：片内总线、系统总线、通信总线。

- 片内总线：芯片内部各部件之间的连线，如 CPU 内部寄存器之间、寄存器与 ALU 之间、控制器与执行部件之间的连线。
- 系统总线：计算机系统内各功能部件（CPU、主存、I/O 接口）之间相互连接的总线，是通常默认所指的「总线」。按传送信息内容又细分为三类：
  - 数据总线（**data bus**）：传送数据信息。它是双向总线。这里的数据是广义的，包括 CPU 与主存之间交换的数据或指令、CPU 从 I/O 接口读入的状态字与中断类型号、CPU 写入 I/O 接口的命令字，以及地址/数据线复用时的地址信息。数据总线的条数称为数据总线宽度，它是衡量系统性能的重要指标。注意 **MDR** 的位数与数据总线宽度相等，但与存储字长不一定相等。
  - 地址总线（**address bus**）：给出数据总线上源数据或目的数据在主存单元或 I/O 端口中的地址，是单向总线（由主设备指向从设备）。地址线的位数决定 CPU 可直接寻址的空间大小，n 根地址线最大可寻址 2^n 个单元；地址线位数 = **MAR** 位数。
  - 控制总线（**control bus**）：传送控制命令和时序信号，用于总线控制权协商、设备控制命令与应答等。对其中某一根信号线是单向的（要么是 **CPU** 送往部件，要么是部件送往 **CPU**），但对整个控制总线而言是双向的。常见控制信号包括存储器读/写、I/O 读/写、传输应答、总线请求/许可/忙、中断请求/响应、时钟与复位信号等。
- 通信总线（外部总线）：计算机系统之间、或计算机系统与其他系统（远程通信设备、测试设备）之间传送信息的总线，如 USB。相对而言，系统总线与 I/O 总线也可统称为内部总线。

数据线上「不可能」传什么

统考反复考一类题：问系统总线的数据线上不可能传输下列哪一项。判断依据是看信号的性质属于数据还是属于控制：

- 指令、操作数 → 数据线；
- I/O 接口的命令字、状态字 → 数据线（它们本质是写入/读出寄存器的二进制数据）；
- 中断类型号 → 数据线（中断响应时由中断控制器送上数据总线）；
- 握手（应答）信号、读/写控制信号、总线请求/许可 → 控制总线，不是数据线。

所以第 1 类题的标准答案是「握手信号」。第 2 类题问「I/O 总线的数据线上传输的信息包括」，答案是命令字、状态字、中断类型号三者都包括。

按传输定时方式分：同步总线与异步总线。同步总线用统一时钟定时，异步总线用握手信号定时。这一对概念放在「总线事务和定时」中展开。

单总线结构不等于只有一根线

「单总线结构」是指 CPU、主存、I/O 设备（经 I/O 接口）全部挂在一组系统总线上，而不是说物理上只有一根信号线——这一组总线里仍然包含数据线、地址线、控制线。单总线的优点是结构简单、易扩充；缺点是所有部件共享一组线，同一时刻只能有一个部件向总线发送信息，系统效率受限，多用于小型机或微型机。

（二）总线的组成及性能指标

总线在物理上由一组信号线加总线控制器（仲裁机构）组成；逻辑上则包括三条信息通路（数据、地址、控制）以及总线控制权协商机制。评价一条总线，看下面六个指标。

*1.*总线宽度

总线宽度指数据总线的根数，单位是位（bit），如 8 位、16 位、32 位、64 位。在并行总线中它直接决定一次能并发传送多少位数据，是带宽公式里的第一个因子。计算带宽时通常要先把它换算成字节：$W(\mathrm{B}) = \text{总线宽度}/8$。

*2.*总线时钟频率与总线工作频率

- 总线时钟频率 f_{clk} 是总线时钟周期的倒数。同步总线中传输双方持有完全同步的时钟信号，时钟频率越高传输越快。
- 单时钟传输次数 k 指一个总线时钟周期内传送数据的次数：普通总线取 1；**DDR**（Double Data Rate）在时钟的上升沿和下降沿各传送一次，取 2；QDR 取 4。
- 总线工作频率（也称有效数据传输率）为

$$f_{work} = f_{clk} \times k$$

时钟频率 $\neq$ 工作频率

这是统考反复设的坑：同步总线由时钟信号定时，但时钟频率不一定等于工作频率。双沿传输时工作频率是时钟频率的 2 倍；反过来，如果一个总线周期占多个时钟周期，工作频率又会低于时钟频率。「时钟频率」与「工作频率」必须分开算。

*3.*总线传输周期

总线传输周期（简称总线周期）是指在总线上完成一次总线事务所用的时间，由若干个总线时钟周期组成。完整的总线周期一般包括总线请求、总线仲裁、设备寻址、数据传输、总线释放几个阶段。通常一个总线周期只传送一个总线宽度的数据。

两个推论要记住：

- 总线时钟频率越高，总线周期越短；
- 采用地址/数据线复用会加长总线周期（地址和数据要分时传送），所以复用能省引脚，但不能提高速率，反而可能降低速率。

*4.*总线带宽

总线带宽（**bus bandwidth**）是总线上最大的数据传输速率（单位时间传送的字节数），通常不考虑总线传输周期中总线请求、仲裁、寻址等开销，只按真正传送数据的部分计算。单位是 MB/s 或

GB/s，其中 **1 MB** = 10^6 **B**，**1 GB** = 10^9 **B**（总线带宽沿用通信与存储领域的十进制换算，不用 1024）。

$$BW = \frac{W}{8} \times f_{clk} \times k = \frac{W}{8} \times f_{work}$$

其中 W 为总线宽度（bit），f_{clk} 为时钟频率，k 为单时钟传输次数，f_{work} 为工作频率。

这个公式还有几种等价写法，考试里哪一种方便就用哪一种：

- 按「每次传送的字节数 × 每秒传送次数」写：$BW = D \times f_{work}$，D 为一次传送的字节数。若题目给的是「一个总线周期传送 D 字节，一个总线周期占 m 个时钟周期」，则 $BW = \dfrac{D}{m \cdot T_{clk}} = \dfrac{D \cdot f_{clk}}{m}$。
- 按「总线周期」写：$BW = \dfrac{D}{T_{bus}}$，T_{bus} 为总线传输周期。这就是题目里常见的 $W/(T)$ 形式：$BW = W_{\text{字节}}/T_{\text{总线周期}}$。
- 按「每秒传输次数」写（串行高速总线，如 QPI、PCIe）：BW = GT/s × 每次有效字节数 × 方向数。

四种表述，同一个式子

1. $BW = \dfrac{W}{8} \times f_{clk} \times k$
2. $BW = \dfrac{W}{8} \times f_{work}$
3. $BW = \dfrac{D}{T_{bus}}$（D 为一个总线周期传送的字节数）
4. $BW = \dfrac{D \cdot f_{clk}}{m}$（$m$ 为一个总线周期占用的时钟周期数）

它们互相等价，区别只是题目给了哪几个量。做题第一步永远是：先确定「一次传送多少字节」，再确定「每秒传送多少次」，两者相乘。

5.总线带宽典型算例

算例 1（时钟周期数与总线周期不相等，2009 年统考）

系统总线在一个总线周期中并行传输 4 B，一个总线周期占 **2** 个时钟周期，总线时钟频率 10 MHz，求总线带宽。

一个时钟周期传送 $4\ \text{B}/2 = 2\ \text{B}$，故

$$BW = 2\ \text{B} \times 10\ \text{MHz} = 20\ \text{MB/s}$$

陷阱：直接算成 $4 \times 10 = 40$ MB/s 就忽略了一个总线周期占 2 个时钟周期。

算例 2（双沿传输 + 地址/数据线复用，2014 年统考）

同步总线采用数据线和地址线复用，地址/数据线 32 根，总线时钟频率 66 MHz，每个时钟周期传送两次数据（上升沿、下降沿各一次），求最大数据传输率。

32 根线在数据阶段一次传送 32 bit = 4 B，单时钟传输次数 $k = 2$：

$$BW = 4\ \text{B} \times 66\ \text{MHz} \times 2 = 528\ \text{MB/s}$$

注意：题目问的是峰值传输率，地址/数据线复用只影响事务的时序结构，不需要在结果里再除以 2。

算例 3（多通道存储器总线，2019 年统考）

采用 3 通道存储器总线，内存条为 DDR3-1333（存储器总线工作频率 1333 MHz，总线宽度 64 位），求总带宽。

DDR3-1333 中的 1333 已经是双倍传输后的有效工作频率，不能再乘 2。单通道一次传送 8 B：

$$BW_{单} = 8\ \text{B} \times 1333\ \text{MHz} \approx 10.66\ \text{GB/s}, \qquad BW_{总} = 3 \times 10.66 \approx 32\ \text{GB/s}$$

算例 4（串行全双工总线，2020 年统考）

QPI 是点对点全双工同步串行总线，每方向每次传输 20 位（16 位数据 + 4 位校验位），每个时钟周期传送 2 次，时钟频率 2.4 GHz。求总线带宽。

每秒传输次数 $= 2.4\ \text{GHz} \times 2 = 4.8\ \text{GT/s}$；每次有效数据 16 bit = 2 B；全双工两个方向同时传输：

$$BW = 4.8\ \text{GT/s} \times 2\ \text{B} \times 2 = 19.2\ \text{GB/s}$$

两个易错点：4 位校验位不计入有效数据带宽；全双工的「×2」是方向数，不能漏。

算例 5（非突发事务逐次计算，2023 年统考）

存储器总线宽度 64 位，总线时钟频率 1 GHz，传输一个数据或地址需 1 个时钟周期，不支持突发传送。主存每次准备一个 64 位数据需 6 ns，主存块大小 32 B。求读取一个主存块的时间。

时钟周期 $= 1$ ns。总线宽度 64 位 = 8 B，一个 32 B 主存块需传送 $32/8 = 4$ 次。不支持突发，则每次都要单独走一次完整事务：送地址 1 个周期（1 ns）+ 主存准备 6 ns + 传数据 1 个周期（1 ns）= 8 ns。

$$T = 4 \times 8\ \text{ns} = 32\ \text{ns}$$

总线带宽与「实际数据传输率」不是一回事

总线带宽是总线的固有性能指标，只反映「有数据传送时」的最快速率，不受传送方式影响，也不把地址阶段、仲裁阶段、存储器准备时间算进去。如果题目问的是「处理器一直发起某类事务时总线的实际数据传输率」，那就要用「一批数据量 ÷ 完成这批数据所需的全部时间（含地址、准备、仲裁开销）」来算，两者的答案会明显不同。真题曾在选项中同时给出这两个数来迷惑考生（例如 2024 年那道给足干扰条件的题，答案只需按有数据传送的时钟周期算）。

6.总线复用

总线信号线分为专用与复用两类。专用信号线只传一种信号（系统总线的数据线、地址线、控制线就是专用总线）；复用总线是让一组传输线具有多种用途，分时传送不同类型的信息。常见形式：

- 地址总线与数据总线复用：某时刻传地址，另一时刻传数据。PCI 总线是典型例子。
- 同一功能的线缆复用：DRAM 芯片的行、列地址线复用，在不同时刻分别送行地址和列地址，大幅减少芯片引脚。
- 串行总线可以看作最极端的总线复用：所有信息都通过一根（或一对差分）线传递。

复用的优点是提高总线利用率、减少引脚数目和布线空间、降低成本；缺点是控制逻辑复杂，且分时传送机制会引起总线性能下降（事务时间变长）。

复用能提高速率吗？不能

「采用信号线复用技术可减少信号线数量」正确；「采用地址/数据线复用可提高同步总线数据传输率」错误。能提高数据传输率的是：增加总线宽度、提高总线工作频率、支持突发传送这三项。突发传送之所以有效，是因为它只送一次首地址就把连续多个数据传完，摊薄了地址和仲裁开销，提高了有效（实际）数据传输率。

7.其他性能指标

- 总线负载能力：总线上可连接的设备数量，与总线的驱动能力和电气特性有关。
- 总线复用程度、是否支持突发、是否支持分离事务：都是影响有效带宽的结构性因素。

（三）总线事务和定时

*1.*总线事务与突发传送

总线事务（**bus transaction**）是指在总线上一对主从设备之间完成一次信息交换的过程，即从请求总线到完成总线使用的操作序列。发起事务并能获得总线控制权的设备叫主设备（**master**），被主设备寻址、被动响应的设备叫从设备（**slave**）。

一条总线上可以挂多对主从设备，但同一时刻只有一个活动主设备

主设备既可以发送数据给从设备，也可以接收从设备的数据，是主动的一方；从设备只能被动响应。如果主设备采用广播方式发送数据，则可以有多个从设备同时接收。总线事务的类型由操作性质决定，典型的有存储器读、存储器写、I/O 读、I/O 写、中断响应等。

一个完整的总线周期可分五个阶段：

1. 总线请求：CPU 或 DMA 等主设备需要使用总线时发出传输申请。
2. 总线仲裁：多个主设备同时请求时，由总线仲裁机构决定谁获得下一个传输周期的总线控制权。
3. 设备寻址：获得控制权的主设备把要访问的从设备地址及相关命令送到总线上，启动从设备。
4. 数据传输：主从设备之间进行数据交换，数据单向或双向传送。该阶段通常一次只传送一个字长的数据。
5. 总线释放：主设备不再占用总线，撤除相关信息，释放总线控制权。

「一次总线事务」是常考的定义题

题干若描述「主设备只需给出一个首地址，从设备就能从首地址开始的若干连续单元读出或写入多个数据」，答案是突发（猝发）传输。要能与另外几组概念区分开：并行/串行描述一次通过几根通道传送数据位；同步/异步描述定时方式；它们都不是「事务的组织方式」。

总线突发（猝发）传送方式（成组传送事务）：一次性传送地址上连续的一批数据。主设备只需给出这批数据的首地址，随后就可以在传输阶段连续传送多个地址连续单元的数据；在这批数据传输过程中总线一直被占用，直到整批数据传完才释放。

- 非突发方式：每个传送周期都要先送地址再送数据，主从设备之间通常每次只能传一个字长。
- 突发方式：每个总线周期仍只传一个计算机字长，但不释放总线，直到一组信息全部传完。

突发传送的代价是按地址递增规则访问，因此适用于连续成块的数据（存储器块读、Cache 行填充等）。它的收益是把「地址开销」摊薄到多个数据上，从而提高总线的有效数据传输率。

算例（突发传送的事务时间，2012 年统考）

同步总线时钟频率 100 MHz、宽度 32 位、地址/数据线复用、每传一个地址或数据占 1 个时钟周期，支持突发传送。求一次「主存写」总线事务传输 128 位数据所需的最少时间。

时钟周期 $= 1/100\ \text{MHz} = 10\ \text{ns}$。复用线传首地址需 1 个周期 = 10 ns；32 位宽度下传 128 位需 $128/32 = 4$ 个周期 = 40 ns。

$$T = 10\ \text{ns} + 40\ \text{ns} = 50\ \text{ns}$$

陷阱：漏掉地址周期会得到 40 ns；把复用误当成「要额外多除一次」会得到更离谱的数。题目说「至少」，意味着不计仲裁、存储器等待和额外应答时间。

*2.*总线定时：同步定时与异步定时

主设备获得总线使用权后，每一步操作何时开始、何时结束、信号保持多久，就是总线定时问题。常见有四种：同步方式、异步方式、半同步方式、分离事务方式，前两种是考纲重点。

同步定时方式。进行信息交换的两部件或设备通过统一的时钟进行同步，即都必须使用同一个时钟，在规定时钟节拍内完成规定的总线操作。挂在总线上的所有设备都从时钟线上获得定时信号。例如规定「存储器读」协议：主设备在第 1 个时钟周期发送地址和存储器读命令，从设备总是在第 5 个时钟周期把数据放到总线上作为响应，处理器也在第 5 个周期取数。写操作的传输周期长度同样是固定的。

- 优点：主从设备协调简单，传输协议简单，传输速率高。
- 缺点：总线定时以最慢设备所花时间为标准，因此适合存取时间相差不大的多个功能部件之间通信；存在时钟偏移问题，所以同步总线不能过长，否则会降低传输效率；同步定时中主从设备默认对方正常工作，可靠性不高。
- 适用：总线长度短、设备速度相近、可靠性要求高的场合，如直接与 CPU 相连的系统总线。

异步定时方式。没有统一的时钟限制，只依靠各部件或设备内部的定时操作，以信号握手的方式完成规定的总线操作：主设备发出请求（**Request**）信号，从设备收到后发出回答（**Acknowledge**）信号。由于不需要公共时钟规定请求与应答的时间，总线周期的长度不再相同，且有效规避了同步定时的时钟偏移问题，总线传输距离更长。

- 优点：总线周期灵活可变，主从设备不需强制同步，可以支持速度不同的两个设备挂在同一条总线上。
- 缺点：控制逻辑比同步方式复杂，速度比同步定时慢。

根据请求和回答信号的撤销是否互锁，异步定时分为三种：

方式	请求信号 **REQ** 何时撤销	回答信号 **ACK** 何时撤销	速度	可靠性
不互锁	发出后经过一段时间自动撤销，不等 ACK	收到 REQ 后经过一段时间自动撤销，不等 REQ 撤销	最快	最差
半互锁	必须等收到 **ACK** 之后才能撤销	自动撤销，不受 REQ 约束	中等	中等
全互锁	必须等收到 **ACK** 之后才能撤销	必须等 **REQ** 撤销之后才能撤销	最慢	最高

三种握手方式的记忆口诀

不互锁：谁也不等谁，发完就撤 → 快而不可靠。半互锁：主设备等从设备（请求等应答才撤），从设备不等主设备（应答自动撤）。全互锁：互相等（请求等应答才撤，应答等请求撤了才撤）→ 最慢最可靠。

结论排序：速度不互锁 > 半互锁 > 全互锁；可靠性全互锁 > 半互锁 > 不互锁。

半同步定时方式。保留同步定时的特点：所有地址、命令、数据信号的发出时间都严格参照系统时钟的某个前沿开始，接收方采用系统时钟后沿判断识别；同时又像异步定时那样允许不同速度的设备协同工作。为此增设一条 **Wait** 响应信号线：总线在每个时钟到来时采样 Wait 信号，若无效说明数据未准备好，下个时钟再采样，直到有效才去数据线上取数据。

半同步兼有同步与异步的优点：所有信号仍由公共时钟定时，又允许不同速率设备协同工作，还能有效解决异步传输中的噪声敏感问题。适用于系统工作速度不高、但由许多速度差异较大的设备组成的简单系统。

同步总线的时钟由谁提供

「同步通信方式中，同步时钟信号可由各设备提供」是错误说法。同步总线上的设备依据统一的公共时钟工作，该时钟通常由总线控制器或统一时钟源提供，不能由各设备提供彼此独立的时钟，否则就不叫同步总线了。

同步/异步常考结论汇总

- 异步通信中全互锁协议最慢（正确）。
- 异步通信中非互锁协议可靠性最差（正确）。
- 同步通信中时钟信号可由各设备提供（错误）。
- 半同步通信中握手信号的采样由同步时钟控制（正确）。
- 异步总线由握手信号定时，一次握手过程完成的是一个总线宽度的数据交换，不是一位数据（常说成「完成一位」是错的）。
- 同步总线的时钟频率不一定等于工作频率（正确）。

*3.*分离事务通信方式（拓展）

前面三种定时方式中，从主设备发出地址和读/写命令到数据传输结束，整个传输周期内总线使用权完全由主设备及被选中的从设备占据。但从设备准备数据的阶段，总线其实纯属空闲等待。分离事务通信方式把一个事务分成两个子过程：

1. 主设备获得总线后把事务类型（总线命令）、地址及其他信息（如主设备编号）送到总线上，从设备记录这些信息后，主设备立即释放总线，其他设备便可使用；
2. 从设备准备好数据后重新申请总线，获得使用权后把主设备编号和数据送上总线，主设备据此接收数据。

- 优点：在不传送数据的间隙释放总线，让其他申请者使用，提高总线利用率。
- 缺点：控制相当复杂、开销大，事务响应时间变长。

*4.*总线仲裁

当多个主设备同时请求总线时，必须由总线仲裁机构（总线控制器 / 仲裁器）决定谁获得下一个传输周期的使用权。仲裁方式分为集中式与分布式两大类。

集中式仲裁：仲裁逻辑集中在一处（一般是 CPU 或专门的仲裁器），各主设备通过专门的控制线与它相连。

1. 链式查询方式：所有设备共用一条总线请求（**BR**）线和一条总线忙（**BS**）线，另有一条总线允许（**BG**）线像链条一样从仲裁器依次穿过各设备。仲裁器收到请求后发出 BG 信号，BG 沿链依次传递，第一个收到 **BG** 且自己有请求的设备获得总线控制权，同时截断 BG 不再往下传。
   - 优点：只用三根控制线，结构简单，容易扩充（增减设备只需改链条连接）。
   - 缺点：优先级固定（离仲裁器越近越高），且对故障敏感——链条上某一级损坏，其后的设备都无法获得总线。
2. 计数器定时查询方式：撤销 BG 线，改用一组设备地址线。仲裁器收到请求后启动计数器开始计数，计数值通过地址线送到各设备，计数值与自己设备号相等且有请求的设备获得控制权并停止计数。
   - 优点：优先级可由计数器初值灵活改变（每次从 0 开始 → 固定优先级；每次从中止点继续 → 循环优先级，各设备机会均等）；对故障不如链式敏感。
   - 缺点：需要 $\lceil \log_2 n \rceil$ 根地址线，控制线比链式多。

3. 独立请求方式：每个设备都有一对独立的总线请求线 **BR** 和总线允许线 **BG**，直接连到仲裁器。
   - 优点：响应速度快（仲裁器可并行看到所有请求），优先级灵活（可由程序设定），对单个设备故障不敏感。
   - 缺点：控制线数量多（$2n$ 根），总线控制器复杂，扩展性受引脚限制。

分布式仲裁：不设中央仲裁器，每个主设备都带有仲裁逻辑和自己的仲裁号，各设备把仲裁号送到共享的仲裁线上，每个设备各自判断自己是否胜出（谁的仲裁号大谁赢），胜者获得总线。优点是可靠性高、无单点故障；缺点是逻辑分散、难以实现复杂的优先级策略。

三种集中式仲裁的对比考法

常考「只用三根控制线」→ 链式查询；「优先级固定、对故障敏感」→ 链式查询；「优先级可循环改变」→ 计数器定时查询；「响应最快、控制线最多」→ 独立请求。

三、I/O 接口（I/O 控制器）

（一）I/O 接口的功能和基本结构

外部设备与主机在工作速度、数据格式、时序控制上差异极大，且各自有独立的时钟。因此必须在主机与外设之间设置专门的逻辑部件来解决同步协调、速度匹配和数据格式转换问题，这个部件就是 **I/O 接口（I/O interface）**，也叫设备控制器、**I/O** 模块、适配器（**adapter**）。

I/O 接口不等于 I/O 设备

I/O 设备由执行 **I/O** 操作的机械部分（设备本体）和执行控制 **I/O** 的电子部件（接口/控制器）组成，通常把这两部分分开。所以：

- 磁盘驱动器是设备本体，不是 I/O 接口；磁盘控制器才是接口。
- 打印机适配器、网络控制器、可编程中断控制器都属于 I/O 接口。

1.I/O 接口的功能

1. 数据缓冲：用数据缓冲寄存器匹配 CPU 与外设之间的速度差异，起到「蓄水池」作用。
2. 错误检测（以及必要的纠错）：如奇偶校验、CRC 校验、传输超时检测，把出错信息记入状态寄存器供 CPU 查询。
3. 控制与定时：接收 CPU 的命令，产生设备所需的控制信号与时序，协调主机与外设的异步工作。
4. 数据格式转换：如串行 ↔ 并行转换（串行接口）、字节 ↔ 字装配（不同字长的设备）、数模/模数转换等。
5. 与主机和设备的通信：主机一侧按总线标准通信（地址译码、读写控制、中断请求），设备一侧按设备自身的电气与协议标准通信。
6. 此外还有设备寻址（让 CPU 能选中某个设备及其某个端口）与状态检测功能。

2.I/O 接口的基本结构

I/O 接口内部可分为两部分：连接总线的总线接口（必须按总线标准设计，属标准部分）和连接设备的内部接口（因设备而异，属非标准部分）。不论何种接口，一般都应包含下列部件：

1. 数据缓冲寄存器（**DBR**，数据端口）：输入时存放从设备读来的数据等待 CPU 取走，输出时暂存 CPU 送来的数据等待送往设备。
2. 设备状态寄存器（状态端口）：反馈设备状态，如设备忙、设备就绪、设备错误等。程序查询方

式中 CPU 正是反复读它来判断设备状态。

3. 设备命令寄存器（控制寄存器，控制端口）：接收 CPU 发来的设备控制命令，如设备复位、设备识别、读写控制、启动/停止。简单设备（如普通键盘、字符终端）可能根本没有命令寄存器。
4. 设备存储器（非必需）：供设备自身运算和处理使用，如显卡中的显存。
5. 地址译码器和 **I/O** 控制逻辑：接收地址线和控制线送来的信息，对控制寄存器中的命令字译码，把控制信号送到外设侧；同时把数据缓冲寄存器的数据送到外设或从外设接收数据；还要收集外设状态到状态寄存器。
6. 中断控制逻辑：产生中断请求、接收中断响应、提供中断类型号（向量中断方式下），并参与中断屏蔽判断。

状态寄存器与控制寄存器为何常合二为一

状态寄存器和控制寄存器在传送方向上是相反的（状态字由接口送往 CPU，是读；命令字由 CPU 写入接口，是写），而且 CPU 对它们的访问在时间上一般是错开的，所以很多接口把二者合并成同一个寄存器，通过读、写方向来区分。这就是真题里「状态端口和控制端口可以合用同一个寄存器」这一说法的来源。

*3.*接口三组总线各传什么

与 I/O 接口相连的数据线、地址线、控制线都是双向的（因为 DMA 方式下接口本身也能当主设备），但传送内容不同：

- 数据线：真正的 I/O 数据（经数据缓冲寄存器）、CPU 写入命令寄存器的控制字/命令字、CPU 从状态寄存器读出的状态字。
- 地址线：I/O 端口或设备寄存器的地址（CPU 在访问时给出，由接口内的地址译码器识别并选中寄存器）；DMA 方式下接口作为主设备发出的主存地址。
- 控制线：总线控制命令与时序信号（读/写、时钟）、总线控制权协商信号（DMA 请求）、应答信号、中断响应信号。

「命令字走数据线」与「控制信号走控制线」不是一回事

CPU 向接口控制寄存器写入命令字时，命令字是一组二进制数据值，它在数据总线上传送；而读/写控制信号、握手信号是控制信号，在控制总线上传送。前者是「要做什么」（What），后者是「现在执行读还是写、什么时候有效」（How/When）。2011 与 2012 年都考过这个区分。

（二）I/O 端口及其编址

把接口中 CPU 可访问的寄存器统称为 **I/O** 端口（**I/O port**）：数据缓冲寄存器简称数据端口，状态寄存器简称状态端口，控制寄存器简称控制端口。所有 I/O 端口编号组成的空间称为 **I/O** 地址空间。

访问这些端口的指令称为 **I/O** 指令（如 IA-32 的 `in`、`ins`、`out`、`outs`）。I/O 指令只能在操作系统内核的底层 I/O 软件中使用，因此 **I/O** 指令是特权指令。

I/O 指令传送的是什么

I/O 指令由 CPU 执行，典型功能是「通用寄存器 ↔ **I/O** 端口」之间的数据传送。设备与端口之间的交换由接口电路和设备控制逻辑完成；CPU 的通用寄存器不会绕过接口直接连外设。所以「I/O 指令实现的数据传送发生在通用寄存器和 **I/O** 端口之间」是正确的。

1.I/O 端口的两种编址方式

对比项	统一编址（存储器映射编址）	独立编址（**I/O** 映射方式）
概念	把 I/O 端口当成主存单元一样统一编址	I/O 端口地址与存储器地址无关，各自独立
访问指令	不需要专门的 **I/O** 指令，用普通访存指令（LOAD/STORE）即可读写端口	必须使用专门的输入输出指令（如 x86 的 IN/OUT）
优点	① 端口编址空间大；② 读写控制逻辑简单；③ I/O 访问与主存访问共用一组指令，保护机制可由分段/分页存储管理实现，无需专门保护机制	① 用专用 I/O 指令，程序清晰、便于编程；② **I/O** 端口地址位数少，地址译码速度快；③ **I/O** 端口地址不占用主存地址空间
缺点	① 端口占用了主存地址空间，主存地址空间变小；② 外设寻址时间长（地址位数多、译码慢）	① I/O 指令类型少，一般只能对端口传送，程序设计灵活性差；② 需要 **CPU** 提供存储器读/写与 **I/O** 读/写两组控制信号，增加控制逻辑复杂性
如何区分访问对象	通过地址码区分（按地址范围判断是主存还是端口）	通过不同指令区分（I/O 指令的地址码给出端口号，不同指令产生不同的总线控制信号）
典型处理器	MIPS、ARM 等 RISC 处理器	Intel x86 处理器

统一编址与独立编址的两个反向陷阱

1.「采用独立编址时，I/O 端口地址和主存地址可能相同」——正确。两者用不同地址空间，数值上重叠是允许的，靠指令（及相应的总线控制信号）来区分。
2.「采用统一编址时，CPU 不能用访存指令访问 I/O 端口」——错误。统一编址又称存储器映射 I/O，CPU 正是通过普通访存指令访问 I/O 端口。

统一编址的端口不能被 Cache 缓存

I/O 地址虽然映射到主存空间，但接口中的数据是动态变化的。若把端口内容放进 Cache，CPU 读到的就可能是过时副本，无法了解设备的实时状态变化。所以映射到主存空间的 I/O 区域必须标记为不可缓存。

四、I/O 方式

I/O 方式的发展主线是「尽量减少主机对 **I/O** 控制的干预」。从程序查询（CPU 全程参与）到程序中断（CPU 只在数据就绪时参与），再到 DMA（CPU 只做预处理和后处理），CPU 的参与度逐级降低，数据传输单位从「字节/字」变为「数据块」。

（一）程序查询方式

程序查询方式（程序直接控制方式）：I/O 完全依靠 CPU 执行程序实现。CPU 要与设备交换数据时，先设置接口命令寄存器启动设备；设备准备期间，CPU 反复读取接口中的状态寄存器查询设备是否就绪，据此决定是进行数据传送还是继续等待。反复查询的过程称为轮询（**polling**）。字符设备一次轮询传一个字节或字，块设备一次轮询可传一个数据块。

基本查询流程（以有命令寄存器的复杂设备为例）：

1. 读状态端口，判断设备是否就绪；未就绪则继续查询；
2. 就绪后，向控制端口写入命令与参数（写端口地址、传送长度、传送方向等），启动设备；

3. 再次不断查询状态端口，等待「命令执行完毕/数据就绪」；
4. 就绪后通过数据端口完成一次数据传送；
5. 若是成块传送，修改内存地址与计数器，回到第 3 步循环。

两种查询策略：

- 独占查询（忙等待，**busy-waiting**）：启动设备后就一直查询状态寄存器直到设备就绪，期间 CPU 不能执行其他任务。一次轮询只传一个数据单元。适用于单任务操作系统；对超高速设备也可以接受（设备响应快，等待时间极短）。
- 定时查询：CPU 启动设备的同时启动一个定时器，设备准备期间 CPU 不忙等而是继续工作，定时时间到产生定时中断，在中断服务程序中查询设备状态。缺点是定时时间难以确定：过短则数据未就绪白查，过长则数据已就绪却得不到及时处理。适用于外设不支持中断的多任务系统。

*1.*查询方式下的 *CPU* 时间开销

核心思想：开销 = 每秒查询次数 × 每次查询的 **CPU** 时间。

$$T_{\text{CPU 占空比}} = \frac{n_{\text{查询次数/秒}} \times t_{\text{每次查询}}}{1\ \text{s}}$$

其中每秒查询次数由设备的数据率和每次传送的单位决定：

$$n_{\text{查询次数/秒}} = \frac{\text{设备数据传输率}}{\text{每次传送的字节数}}$$

每次查询耗时 t 可以由「查询程序的时钟周期数 × 时钟周期」得到，也可以由「指令条数 × CPI × 时钟周期」得到。

算例 1（定时查询开销，**2011** 年统考）

处理器主频 50 MHz，采用定时查询方式控制设备 A。查询程序运行一次至少用 **500** 个时钟周期；设备 A 工作期间为不丢数据每秒至少查询 **200** 次。求 CPU 用于设备 A 的 I/O 时间占整个 CPU 时间的百分比至少是多少。

每秒用于查询的时钟周期数 $= 200 \times 500 = 100\,000 = 0.1$ M；CPU 每秒共 50 M 个时钟周期：

$$\frac{0.1\ \text{M}}{50\ \text{M}} = 0.2\%$$

也可以先算一次查询耗时 $500/50\ \text{MHz} = 10\ \mu\text{s}$，每秒查询 200 次共占 2 ms，$2\ \text{ms}/1\ \text{s} = 0.2\%$，结果一致。

算例 2（查询间隔与开销，**2018** 年统考第 **43** 题第 **1** 问）

计算机主频 500 MHz，CPI 为 4，设备 A 的数据传输率为 2 MB/s，接口有 32 位数据缓冲寄存器，每次输入/输出至少执行 10 条指令。

（**1**）最多间隔多长时间查询一次才不丢数据?

设备准备一个 32 位（4 B）数据的时间为

$$t = \frac{4\ \text{B}}{2\ \text{MB/s}} = 2\ \mu\text{s}$$

超过 2 μs 不查，缓冲区数据未取走而新数据又到来，就会覆盖丢失，所以最多每隔 **2** μ**s** 查询一次。

（**2**）**CPU** 用于设备 **A** 输入输出的时间占 **CPU** 总时间的百分比至少是多少?

每次输入/输出需要 $10 \times 4 = 40$ 个时钟周期；每秒查询次数 $= 1\ \text{s}/2\ \mu\text{s} = 5 \times 10^5$ 次：

$$\frac{40\times 5\times 10^5}{500\times 10^6}=\frac{2\times 10^7}{5\times 10^8}=4\%$$

算例 3（查询方式对高速设备不可行的量化，袁春风教材例题的口径）

查询程序每次操作至少 400 个时钟周期，处理器主频 500 MHz。求三种设备下查询开销占比。

- 鼠标：每秒至少查询 30 次 $\rightarrow \dfrac{30\times 400}{500\times 10^6}\approx 0.002\%$，可忽略。
- 软盘：16 位（2 B）为单位，50 kB/s $\rightarrow \dfrac{50\text{ kB/s}}{2\text{ B}}=25\text{ k 次/秒}\rightarrow \dfrac{25\times 10^3\times 400}{500\times 10^6}=2\%$，偏高但尚可忍受。
- 硬盘：16 B 为单位，4 MB/s $\rightarrow \dfrac{4\text{ MB/s}}{16\text{ B}}=250\text{ k 次/秒}\rightarrow \dfrac{250\times 10^3\times 400}{500\times 10^6}=20\%$，CPU 五分之一的时间用于查询，不可取。

结论：设备越快、每次传送单位越小，查询方式的开销越大，因此查询方式不适合高速成块传送。

「程序查询只适合慢速设备」不严谨

中断方式有额外开销（两次上下文切换 + 中断服务程序本身）。当设备准备数据的时间比这些开销还短时，用中断反而比忙等待轮询效率更低（中断成为瓶颈）。因此程序查询方式并不一定只适合慢速设备，一些高速设备（如 NVMe SSD 处理同步 I/O）也采用轮询方式。

（二）程序中断方式

*1.*中断的基本概念

中断（**interrupt**）是指 CPU 在执行程序过程中，出现某些来自 **CPU** 外部的、与当前指令执行无关的紧急事件，CPU 暂停现行程序，转去执行相应的处理程序，处理完后返回断点继续执行。异常（**exception**）则是指令执行过程中在处理器内部发生的、与当前指令同步相关的特殊事件。

中断与异常的判定标准

判断依据是「事件是否与当前指令同步相关」，而不是名称里有没有「存储」「I/O」字样：

- 与当前指令同步 → 异常（内部异常）：整数除以 0、访存缺页、存储保护错、非法指令。
- 与当前指令无关、来自 CPU 外部 → 中断（外部中断）：DMA 传送结束、打印机缺纸、I/O 完成、时钟中断、电源故障。

注意区分：外部中断在当前指令执行结束后检测；异常在当前指令执行过程中检测。

外部中断分为：

- 可屏蔽中断：通过 `INTR` 线请求，受中断允许位（IA-32 中是 `EFLAGS` 的 `IF`）控制，**CPU** 处于关中断状态时不会被响应；优先级最低。
- 不可屏蔽中断（**NMI**）：通过 `NMI` 线请求，用于电源掉电、硬件故障等紧急事件，不受中断允许位和中断屏蔽字控制，关中断状态下也能响应，优先级最高。NMI 的中断向量号通常固定，响应时不需要从外部读中断类型号。

中断源的记录与判优。每个中断源设一个中断请求标记触发器，状态为 1 表示有请求；这些触发器可组成中断请求标记寄存器，可集中在 CPU 中，也可分散在各中断源中。CPU 在每条指令执行的最后时刻自动检测中断请求引脚，发现请求后按优先级决定是否响应、响应哪个。

中断判优（中断响应优先级）依据「该中断源若得不到及时响应，致使机器工作出错的严重程度」确定：

1. 不可屏蔽中断 > 内部异常 > 可屏蔽中断；

2. 内部异常中，硬件故障 > 软件中断；
3. **DMA** 中断请求优先于 **I/O** 设备传送的中断请求；
4. I/O 传送类中断中，高速设备优先于低速设备，输入设备优先于输出设备，实时设备优先于普通设备。

判优可用硬件（链式排队器、中断控制器中的优先级编码器）或软件（按优先级从高到低逐级查询各中断源）实现。软件判优硬件简单、优先级可通过改变查询顺序灵活调整，但速度慢，可能无法实时响应。

中断排队不是「先来先服务」

「中断控制器按所接收中断请求的先后次序进行中断优先级排队」是错误的。中断排队依据预设的优先级（固定优先级或轮询优先级），不是按请求到达的时间先后。

*2.*中断响应过程

CPU 响应中断必须同时满足三个条件：

1. 至少有一个未被屏蔽的中断请求（中断请求触发器置 1，并已送排队器判优）；
2. **CPU** 允许中断且处于开中断状态（中断允许触发器 `EINT` = 1；异常和不可屏蔽中断不受此限制）；
3. 当前指令执行完毕（外部中断，异常不受此限制），且没有更紧急的任务。

三个条件里的常见错误说法

- 「一旦可屏蔽中断请求信号有效，CPU 将立即响应」——错误。请求有效 ≠ 立即响应，还要等当前指令结束、且处于开中断状态、且未被屏蔽。
- 「有中断请求时，CPU 立即暂停当前指令执行」——错误。CPU 不会打断正在执行的指令（这会破坏指令的原子性），只在指令边界响应。
- 「仅在用户态下 CPU 才能检测和响应中断」——错误。内核态下只要开中断且优先级条件满足，同样可以响应（这正是多重中断嵌套的基础）。

CPU 响应中断后进入中断响应周期。该周期内的操作由硬件直接实现，称为中断隐指令。中断隐指令不是指令系统中一条真正的指令，只是「硬件自动完成的一系列操作」的虚拟说法。它完成的操作是：

1. 关中断：置中断允许位为 0，屏蔽所有可屏蔽中断。目的是保证断点和现场在被完整保存之前不被新的中断打断。IA-32 中可通过特权指令 `sti`/`cli` 置位/清位 `IF`。
2. 保存断点：把指令无法直接读取的 `PC` 和 `PSW`（程序状态字寄存器）内容自动保存到栈或特定寄存器中。断点一般是下一条指令的地址；但对异常则不一定——「缺页故障」的断点是发生页故障的当前指令地址（处理完要重新执行该指令），「陷阱」的断点是陷阱指令后面一条指令的地址。断点与异常类型有关。
3. 形成中断服务程序入口地址并送 **PC**（引出中断服务程序 / 中断源识别）：若是特权级切换（用户态 → 内核态），硬件还会自动完成用户栈到内核栈的切换。

中断隐指令到底做了哪些事

统考原题（2012 年）：响应外部中断的过程中，中断隐指令完成的操作，除保护断点外，还包括——正确答案是 Ⅰ 关中断与 Ⅲ 形成中断服务程序入口地址并送 **PC**；Ⅱ「保存通用寄存器的内容」不属于中断隐指令，那是进入中断服务程序后由软件完成的第一件事。

口诀：中断隐指令 = 关中断 + 保护断点（**PC/PSW**）+ 找入口（中断源识别）。

中断源的识别方式有两种：

- 软件识别：CPU 中设原因寄存器（**Cause**）记录异常原因或中断类型标志，操作系统用一个统一的查询程序按优先级顺序查原因寄存器，先查到的先处理。
- 硬件识别（向量中断方式）：处理程序的首地址 PC 和初始 PSW 称为中断向量（**interrupt vector**），所有中断向量集中存放构成中断向量表。每个中断源有唯一中断类型号，中断向量位置与类型号相关（类型 0 存第 0 表项，类型 1 存第 1 表项……）。也由 **CPU** 外部的中断控制器根据各 I/O 设备的中断请求和中断屏蔽情况，结合中断响应优先级，识别当前请求的中断类型号，并通过数据总线送到 CPU。

CPU 获得中断类型号后，据此计算中断向量地址（**VA**），从向量表中取出中断服务程序入口地址送 PC，转去执行中断服务程序。

*3.*中断处理过程

中断服务程序（软件）的执行流程，完整版（支持多重中断）为：

1. 关中断（已由中断隐指令完成）；
2. 保存断点（已由中断隐指令完成，保存 PC 和 PSW）；
3. 保存现场和旧屏蔽字：现场信息指用户可见的工作寄存器（通用寄存器等）的内容，通过压栈指令保存，由软件实现；中断屏蔽字寄存器虽用户不可见，也由操作系统一并保存；
4. 设置新屏蔽字（由 I/O 指令访问中断控制器完成）；
5. 开中断：允许更高级中断请求得到响应，实现中断嵌套；
6. 执行中断服务例程：这是中断请求的真正目的，如程序 I/O 中的数据输入/输出；
7. 关中断：保证恢复现场和屏蔽字时不被中断；
8. 恢复现场和旧屏蔽字；
9. 开中断；
10. 中断返回：最后一条指令通常是中断返回指令 **IRET**，返回到原程序断点继续执行。

对于单级中断（不允许嵌套），去掉第 4、5 步和第 7、9 步中与开/关中断相关的部分，流程简化为：保护现场 → 中断事件处理 → 恢复现场 → 开中断 → 中断返回。

硬件做什么、软件做什么——一条界线

硬件（中断隐指令）：关中断、保存断点（PC、PSW）、中断源识别与引出服务程序、特权级与堆栈切换。
软件（中断服务程序）：保存通用寄存器等现场、设置新屏蔽字、执行服务操作、恢复现场与屏蔽字、中断返回。

记住这条界线，2018、2024 那类「谁负责保存什么」的题就不会错。：

- 2015 年统考：「处理外部中断时，应该由操作系统保存的是」→ 答案是通用寄存器的内容（PC 由硬件保存，TLB 和 Cache 内容不需要在单次中断中专门保存）。
- 2024 年统考：「保存断点和程序状态字在中断响应阶段完成」正确；「保存通用寄存器和设置新中断屏蔽字由软件实现」正确。

`IRET` 指令执行时完成的工作：① 从内核栈弹出 `EIP`、`CS`、`EFLAGS`，恢复断点和程序状态；② 检查当前处理程序的 CPL 是否等于 CS 中最低两位，若相等说明中断前后处于同一特权级，`IRET` 结束；否则继续下一步；③ 从内核栈弹出 `SS` 和 `ESP`，恢复中断前的栈，即由内核态改回用户态并完成堆栈切换。执行完 `IRET`，CPU 自然回到被中断的进程继续执行。

为什么断点要保存到栈里

为支持异常或中断的嵌套处理，大多数处理器把断点保存在栈中（如 80x86）。如果系统不支持多重中断嵌套，可以把断点保存在特定寄存器中而不必送栈（如 MIPS 用 EPC 寄存器专门存放断点），后者开销更小——因为访问栈就是访问存储器，比访问寄存器慢。

*4.*多重中断和中断屏蔽

- 单重中断：CPU 在执行中断服务程序过程中，又出现新的更高优先级的中断请求，但 CPU 不予响应，这就是单重中断。此时 CPU 在中断处理期间一直处于关中断状态。
- 多重中断（中断嵌套）：CPU 暂停现行的中断服务程序，转去处理新的中断请求。此时中断处理期间 CPU 并非始终关中断——在保护好现场并设置新屏蔽字后会重新开中断。

CPU 要具备多重中断功能，必须满足两个条件：

1. 在中断服务程序中提前设置开中断指令；
2. 优先级别高的中断源有权中断优先级别低的中断源。

中断处理优先级与中断屏蔽。这里要区分两个不同概念：

对比项	中断响应优先级（中断判优）	中断处理优先级（中断屏蔽）
由谁决定	由中断查询程序或硬件判优电路决定，反映多个中断同时请求时哪个先被响应	由各自的中断屏蔽字（屏蔽字寄存器内容）动态设定，反映本中断与其他所有中断之间的处理优先关系
可否修改	响应优先级往往是硬件线路已设置好，不便于改动	在多重中断系统中通常用中断屏蔽字动态分配，可在程序中执行 I/O 指令修改屏蔽字内容即可改变处理次序
生效时机	只在当前无正在执行的中断、需要从队列中判优选择时使用	任何时刻立即生效：一出现处理优先级更高的中断就立即响应并抢占
队列含义	队列中存放的是未被响应的中断，它们的处理优先级一定都比正在执行的中断低（否则早就抢占了）	当处理优先级高的中断都执行完毕，中断返回最初状态，再从队列中取响应优先级最高的给予响应

如果不使用中断屏蔽技术，处理优先级与响应优先级相同。

屏蔽字的设置规律：

1. 一般用 **1** 表示屏蔽，**0** 表示正常申请；
2. 每个中断源对应一个屏蔽字（在处理该中断源的中断服务程序时，屏蔽字寄存器中的内容就是该中断源对应的屏蔽字）；
3. 屏蔽字中 **1** 越多，优先级越高；每个屏蔽字中至少有一个 **1**（至少要能屏蔽自身的中断）。

算例（屏蔽字计算，2011 年统考）

某计算机有五级中断 $L_4 \sim L_0$，中断屏蔽字为 $M_4M_3M_2M_1M_0$，$M_i = 1$ 表示对 L_i 级中断进行屏蔽。中断响应优先级从高到低为 $L_0 \to L_1 \to L_2 \to L_3 \to L_4$；要求中断处理优先级从高到低为 $L_4 \to L_0 \to L_2 \to L_1 \to L_3$。求 L_1 的中断处理程序中设置的中断屏蔽字。

解题步骤：

1. 写出要求的处理优先级序列：$L_4 \to L_0 \to L_2 \to L_1 \to L_3$。
2. 对目标中断 L_1：处理优先级比它高的允许打断它 $\to$ 对应位为 **0**；它自己以及处理优先级比它低的要屏蔽 $\to$ 对应位为 **1**。

3. 比 L_1 高的有 L_4、L_0、$L_2 \to M_4=0$、$M_0=0$、$M_2=0$；本级 L_1 与更低的 $L_3 \to M_1=1$、$M_3=1$。
4. 按 $M_4M_3M_2M_1M_0$ 排列：$01010=$ **01010**。

关键：屏蔽字由「处理优先级」决定，不是由「响应优先级」决定。题目给响应优先级只是为了增加干扰或后续提问。

中断屏蔽字 ≠ 响应优先级

「中断屏蔽字用于确定中断响应的优先级」是错误说法。响应优先级由硬件排队或优先级判定电路决定；中断屏蔽字控制的是中断嵌套关系，即处理优先级。这是 2024 年统考的原题考点。

多重中断的其他常见判断

- 「在一条指令执行结束时响应中断」正确；
- 「中断处理期间 CPU 处于关中断状态」在多重中断语境下错误（服务程序中会重新开中断）；在单重中断语境下则正确；
- 「中断请求的产生与当前指令的执行无关」正确（异步性）；
- 「CPU 通过采样中断请求信号检测中断请求」正确；
- 「CPU 处于关中断状态时也能响应 NMI 请求」正确。

5.中断方式与查询方式的对比

对比项	程序查询方式	程序中断方式
CPU 参与方式	CPU 全程主动轮询，设备与 CPU 串行工作	CPU 启动设备后继续执行原程序，设备就绪时主动通知，CPU 与设备并行工作
谁发起数据传送	CPU 轮询发现就绪后主动传送	设备主动发出中断请求，CPU 被动响应
数据传送实现者	CPU 执行查询程序	CPU 执行中断服务程序（软件）
CPU 时间开销	与查询次数成正比，设备越快、传送单位越小开销越大	与中断次数 × 每次中断开销成正比；额外还有两次上下文切换开销
实时性	依赖查询频率，可能漏掉数据或浪费查询	由设备主动触发，实时性好，无需纠结轮询频率
适用场合	单任务系统、超高速设备、操作系统引导阶段	键盘、鼠标、针式打印机等低速字符设备

中断方式为什么不适合高速设备

设外设准备数据的时间为 t_{dev}，中断响应与处理时间为 t_{int}。为避免「前一数据未被取走、后一数据又到来造成覆盖」，必须满足

$$t_{dev} \geq t_{int}$$

即外设准备数据的时间应不小于中断响应和处理时间。「外设准备数据的时间应小于中断处理时间」是错误说法（2022 年统考考点）。设备越快，t_{dev} 越小，而 t_{int} 由几十到上千个时钟周期构成，很快就无法满足，因此高速设备应改用 DMA。

6.中断方式下的 *CPU* 开销与效率计算

$$T_{\text{CPU 占空比}}=\frac{n_{\text{中断次数/秒}} \times C_{\text{每次中断时钟周期数}}}{f_{\text{CPU}}}$$

其中中断次数由设备数据率和每次中断传送的数据量决定：

$$n_{\text{中断次数/秒}} = \frac{\text{设备数据传输率}}{\text{每次中断传送的字节数}}$$

每次中断的时钟周期数 C 通常由「中断响应周期数 + 中断服务程序指令数 × CPI + 其他开销」给出。

算例 1（中断开销，2019 年统考）

设备以中断方式与 CPU 交换数据，CPU 主频 1 GHz，接口中数据缓冲寄存器为 32 位，设备数据传输率 50 kB/s，每次中断开销（含中断响应和中断处理）为 1000 个时钟周期。求 CPU 用于该设备 I/O 的时间占整个 CPU 时间的百分比最多是多少。

每次中断最多传送 32 位 = 4 B：

$$n = \frac{50\ \text{kB/s}}{4\ \text{B}} = 12.5\ \text{k 次/秒}$$

每次中断耗时 $= 1000/1\ \text{GHz} = 1\ \mu\text{s}$，每秒中断开销时间 $= 12.5 \times 10^3 \times 1\ \mu\text{s} = 12.5\ \text{ms}$：

$$\frac{12.5\ \text{ms}}{1\ \text{s}} = 1.25\%$$

算例 2（中断 vs DMA 的开销对比，2010 年统考第 43 题）

CPU 主频 500 MHz，CPI = 5。外设数据传输率 0.5 MB/s，以 **32** 位为传输单位，中断服务程序 18 条指令，中断服务的其他开销相当于 2 条指令的执行时间。

（**1**）中断方式下 **CPU** 用于该外设 **I/O** 的时间占比？

每次中断的时钟周期数 $= 5 \times 18 + 5 \times 2 = 100$；每秒中断次数 $= 0.5\ \text{MB/s} \div 4\ \text{B} = 125\,000$：

$$\frac{100 \times 125\,000}{500 \times 10^6} = \frac{12.5 \times 10^6}{500 \times 10^6} = 2.5\%$$

（**2**）数据传输率提高到 **5 MB/s**，改用 **DMA**，每次 **DMA** 传送块大小 **5000 B**，**DMA** 预处理 + 后处理总开销 **500** 个时钟周期，**CPU** 占比？

每秒 DMA 次数 $= 5\ \text{MB/s} \div 5000\ \text{B} = 1000$：

$$\frac{1000 \times 500}{500 \times 10^6} = \frac{0.5 \times 10^6}{500 \times 10^6} = 0.1\%$$

同样吞吐量下，中断方式的开销是 DMA 的 **25** 倍，这就是高速设备必须用 DMA 的量化理由。

算例 3（中断允许的最大延迟，2014 年统考）

设备中断请求的响应和处理时间为 100 ns，每 400 ns 发出一次中断请求，中断响应所允许的最长延迟时间为 50 ns。求 CPU 用于该设备的 I/O 时间占整个 CPU 时间的百分比至少是多少。

$$\frac{100\ \text{ns}}{400\ \text{ns}} = 25\%$$

陷阱：那 50 ns 是「请求可以等待的上限」，等待期间 CPU 可以执行其他工作，不属于 CPU 用于该设备 I/O 的时间，不能加进分子。这是本题设置 B、C 两个干扰项的原因。

算例 4（异步串行中断，2012 年统考第 43 题）

CPU 主频 50 MHz，CPI = 4。设备 D 以异步串行方式向主机传送 7 位 ASCII 字符，规程中有 1 位奇校验位和 1 位停止位，从 D 接收启动命令到字符送入 I/O 端口需要 0.5 ms。I/O 端口每收到一个字符申请一次中断，中断响应需 10 个时钟周期，中断服务程序共 20 条指令。

（**1**）每传送一个字符在通信线上共需传输多少位？每秒最多可送多少个字符？

1（起始位）+7（数据位）+1（校验位）+1（停止位）= **10** 位。每字符耗时 0.5 ms，故每秒最多 $1000/0.5 = $ **2000** 个字符。

（**2**）读取 **1000** 个字符所需时间，以及 **CPU** 用于该任务的时间（时钟周期数）？

时钟周期 $= 1/50$ MHz $= 20$ ns。设备送字符到端口的耗时 $= 0.5$ ms$/20$ ns $= 2.5 \times 10^4$ 个时钟周期；一次中断响应 10 个周期；中断服务程序前 15 条指令（第 15 条启动 D 工作）$= 4 \times 15 = 60$ 个周期。

单字符传送时间约 $2.5 \times 10^4 + 10 + 60 = 25\,070$ 个时钟周期，1000 个字符约

$$1000 \times 25\,070 = 2.507 \times 10^7 \text{ 个时钟周期}$$

CPU 用于该任务的时间（每次响应 + 全部服务指令）$= 1000 \times (10 + 4 \times 20) = 1000 \times 90 = 9 \times 10^4$ 个时钟周期。

（**3**）中断响应阶段 **CPU** 进行了哪些操作？关中断、保护断点和程序状态、识别中断源。

*7.*中断方式与 *I/O* 端口交换的信息

中断 I/O 方式下，CPU 与打印控制接口的 I/O 端口之间可以交换：打印字符（写数据端口）、设备状态（读状态端口）、控制命令（写控制端口）。不可能交换的是主存地址——数据由 CPU 通过指令搬运，接口不直接访问主存，不关心数据来自内存何处；主存地址是 DMA 控制器必须掌握的信息。

（三）DMA 方式

1.DMA 的基本概念

直接内存访问（**Direct Memory Access**，**DMA**）：在总线上设置 **DMA** 控制器（**DMAC**），由 DMAC 临时接管总线，代替 CPU 控制外部设备与内存之间的批量数据交换。数据无须由 **CPU** 寄存器中转，完全消除了程序查询和程序中断方式中 CPU 进行实际数据传输的开销。

DMA 的工作轮廓是：

1. 传送前 CPU 访问 DMAC 接口，设置主存地址、数据块长度、传输方向等参数；
2. CPU 向设备的 I/O 接口发送命令和参数，启动设备；
3. 传输过程中 CPU 继续执行其他程序；设备需要传送时由 **DMAC** 向 **CPU** 申请总线控制权完成传输；
4. 一个数据块传送结束后，DMAC 通过中断方式请求 CPU 对缓冲区和 DMAC 进行后处理。

DMA 主要用于高速设备的块数据传输（磁盘、显卡、网卡、声卡），也可用于内存之间的大批量数据搬移。

DMA 的三句话概括

1. 请求的是总线控制权，不是 CPU 时间；
2. 实际数据传送由硬件完成，不由 CPU 执行程序完成，因此不涉及现场保护；
3. 传送单位是数据块，CPU 只在块传送开始（预处理）和结束（后处理）时参与。

2.DMA 控制器的组成

DMAC 连接在系统总线上，既可充当从设备（预处理阶段），也可充当主设备（数据传输阶段）。其基本组成：

部件	作用
主存地址寄存器 / 地址计数器 **AR**	存放交换数据的内存地址。预处理阶段由 CPU 设置；每次 DMA 传送后自动修改（递增或递减），指向下一个内存单元
字计数器 **WC**	记录数据块长度。预处理阶段设置；每传送 1 个字或字节自动减 **1**；减到 0 时数据传送完毕，发出 **DMA** 中断请求
数据缓冲寄存器 **DBR**	DMAC 作从设备时接收 CPU 传来的数据，作主设备时暂存传输数据。若不经该寄存器而让数据直接经数据总线从设备到内存，则性能更好
命令寄存器 / 控制寄存器	接收 CPU 的控制命令（启动 DMA、传输方向、工作方式等）
状态寄存器	向 CPU 反馈 DMAC 的状态（传输完成、发生错误及错误类型）
DMA 控制逻辑（含中断机构）	含控制与时序电路、状态标志；负责修改地址计数器和字计数器、指定传输方向、对 DMA 请求与 CPU 响应进行协调和同步；还支持中断机制，一组数据交换完毕时向 CPU 发 DMA 中断请求

握手信号：

- DMAC 需要总线控制权时，通过系统总线中的 **HOLD**（总线请求）信号向 CPU 发出请求；CPU 的总线控制器若允许，回送 **HLDA**（总线响应/授权）信号。
- 支持 DMA 的 I/O 设备通过 **DREQ**（**DMA** 请求）与 **DACK**（**DMA** 应答）两条专用控制线与 DMAC 相连。这两条线通常不属于系统总线，而是 DMAC 与 I/O 接口之间的私有通道。
- 一个 DMAC 支持多个通路，每个通路各有独立的主存地址寄存器和字计数器，可同时为多个设备提供 DMA 服务。

DMAC 的角色在换

- 预处理阶段：CPU 是主设备，DMAC 是从设备（CPU 像访问普通 I/O 端口一样用 `OUT` 等指令写 DMAC 的寄存器）。
- 数据传输阶段：DMAC 是主设备，CPU 变成旁观者。

~~DMAC~~ 连接的对象是「设备控制器」而不是「设备」

物理设备（硬盘、网卡）需要专门的设备控制器处理底层协议、时序与数据缓冲；DMAC 是通用的「搬运工」，无法直接理解成千上万种设备的具体细节。所以 **DMAC** 沟通的对象应该是设备控制器（**I/O** 接口），而不是设备本体。同理，「DMAC 与设备之间」也不是数据传输通路的端点。

3.DMA 传送过程

（**1**）预处理阶段

1. CPU 通过 **I/O** 指令把参数写入 DMAC：主存地址寄存器 **AR**（数据在主存的起始地址）、字计数器 **WC**（要传送的数据总量）、控制/命令寄存器（传输方向：设备 → 内存还是内存 → 设备）。
2. CPU 向 **I/O** 接口（设备控制器）发送启动命令，命令中包含设备地址、磁盘扇区号等信息，写入设备控制器自己的寄存器（如设备地址寄存器 DAR）。
3. CPU 向 DMAC 控制寄存器写入「启动 **DMA**」命令位，然后把发起 **I/O** 请求的进程挂起（阻塞），调度另一个就绪进程投入运行。

预处理结束时：DMAC 已掌握本次任务的所有细节，I/O 设备已开始准备数据，CPU 已完全脱身，**CPU** 与 **I/O** 操作实现并行。

（**2**）数据传送阶段（以周期挪用为例，本质是一个循环，每次循环传一个字）

1. 设备就绪，发出请求：I/O 设备准备好一个数据字放入内部缓冲寄存器后，通过 DREQ 向 DMAC 发出 DMA 请求。
2. **DMAC** 申请总线：DMAC 通过 HOLD（或 BREQ）向 CPU 发出总线请求。
3. **CPU** 授权总线：CPU 在当前机器周期（或总线事务/总线周期）结束后响应，断开自己与总线的连接（总线驱动器进入高阻态），发出 HLDA。
4. **DMAC** 主导数据传输（在一个存储周期内完成）：把 **AR** 中的地址送上地址总线；向控制总线发出相应控制信号（如存储器写 MEMW、I/O 读 IORD）；通过 DACK 回应发出请求的设备；I/O 接口把数据字放上数据总线；主存的存储控制器把数据写入指定单元。
5. 更新状态并归还总线：**AR** 自动加 **1**（或减 **1**），**WC** 自动减 **1**；DMAC 撤销 HOLD，立即把总线控制权归还 **CPU**。CPU 可继续执行若干周期，直到下一次 DREQ 到来——这就是「周期窃取」的精髓。

循环直到 **WC** 减为 **0**，DMAC 通过 INTR 向 CPU 发出中断请求，数据传输阶段结束。

（**3**）后处理阶段

1. DMAC 通过 INTR 发出中断请求（触发原因：正常完成——WC 递减至 0；或异常中止——内存校验错、设备无响应等）。
2. CPU 响应中断、保存现场、转入中断服务程序。
3. CPU 通过 I/O 指令读取 DMAC 的状态寄存器，检查「传输完成」标志、「发生错误」标志及错误类型。
4. 根据状态决策：传输成功则唤醒因 **I/O** 请求而阻塞的进程；传输出错则做错误处理（重传、报错、终止任务）。
5. 中断返回，CPU 继续原程序。

DMA 传送的三段与 CPU 的两次介入

预处理（CPU 设置参数 + 启动设备）→ 数据传送（DMAC 硬件全权搬运，CPU 不参与）→ 后处理（CPU 执行中断服务程序收尾）。

CPU 只在首（预处理）尾（后处理）介入；中间的海量搬运与 CPU 无关。所以不存在由 **CPU** 持续执行的「**DMA** 传送程序」——那是 2023 年统考的错误选项。

4.DMA 与 CPU 共享主存的三种方式

DMAC 和 CPU 都要访问主存，为避免访存冲突，常用三种方式：

方式	基本思想	工作过程	优点	缺点
停止 **CPU** 访存（成组传送）	DMA 优先，一旦开始不传完不罢休	DMAC 申请总线，CPU 在完成当前指令（或总线周期）后释放总线并暂停/等待；DMAC 在连续多个存储周期内把整块数据传完；传完归还总线，CPU 恢复	控制最简单（请求、建立、归还只做一次）；数据传输率最高，适合大块高速传输	传输期间 CPU 无法响应任何中断，实时性差；即使 CPU 当前不需要访存也被强制暂停，**CPU** 时间被浪费
周期挪用（周期窃取）	按需分配，DMA 只在需要时借用一个存储周期	设备准备好一个字，DMAC 就申请一次总线；CPU 在当前机器周期结束后释放；DMAC 占用一个存储周期传完这个字后立即归还	CPU 与 DMA 分时交替使用总线，CPU 不会被长时间阻塞，DMA 响应及时	每次搬一个字都要请求/仲裁/归还，有额外开销（整个挪用过程实际可能花 2 ~ 5 个存储周期）；若 DMA 与 CPU 同时访存，DMA 优先，CPU 被延缓

方式	基本思想	工作过程	优点	缺点
DMA 与 **CPU** 交替访存	把内存工作周期一分为二，各用一半	把每个内存工作周期固定划分为 T_1、T_2 两段，强制规定 **CPU** 只能在 T_1 访存，**DMAC** 只能在 T_2 访存	不需要总线请求（时间预先分配好），CPU 感觉不到 DMA 的存在，可连续工作	硬件代价高（需要复杂的时钟与内存控制逻辑）；资源浪费——某一方空闲时它的时间片就浪费了，实质上使内存总带宽减半；实际应用很少

周期挪用不是一个块传完才还总线

「周期挪用方式下，在整个数据块传送过程中 CPU 不可以访问主存储器」是错误说法。周期挪用只在需要传送数据时窃取相应的存储周期，相邻两次 DMA 传送之间 CPU 仍可正常访存。只有「停止 CPU 访存」（成组/连续传送）方式下，CPU 才会在整个数据块传送期间被禁止访问主存。这正是 2020 年统考 512 B / 32 位缓冲寄存器那道题的正确选项与错误选项的分界。

周期挪用与访存冲突

DMA 操作不会修改 **CPU** 寄存器，所以不存在现场保护问题。若周期挪用期间 CPU 不访问内存，则对 CPU 执行没有性能影响；若恰好也访问内存（形成访存冲突），则 **DMA** 优先访问内存——因为若 DMA 请求得不到及时响应，I/O 传输的数据可能丢失。周期挪用已成为 DMA 传输方式的主要方法。

算例（周期挪用的请求次数，**2012** 年统考第 **43** 题第 **2** 问）

CPU 主频 80 MHz，CPI = 4，平均每条指令访存 1.5 次，主存与 Cache 之间块大小 16 B，Cache 命中率 99%，存储器总线宽度 32 位。

（**1**）**MIPS** 数与每秒 **Cache** 缺失次数?

MIPS $= 80\ \mathrm{M}/4 = 20$。每秒 Cache 缺失次数 $= 20\ \mathrm{M} \times 1.5 \times (1 - 99\%) = 300\,000$ 次/秒。

（**2**）**Cache** 缺失时主存带宽需求?

$16\ \mathrm{B} \times 300\,000/\mathrm{s} = 4.8\ \mathrm{MB/s}$。

（**3**）缺页时磁盘接口每秒发出的 **DMA** 请求次数至少是多少?

缺页率 0.0005%，故每秒缺页异常 $= 300\,000 \times 0.0005\% = 1.5$ 次。页面大小 4 KB = 4096 B，磁盘 I/O 接口的数据缓冲寄存器 32 位 = 4 B，每传 4 B 发一次 DMA 请求：

$$1.5 \times \frac{4096}{4} = 1.5 \times 1024 = 1536\ \text{次/秒}$$

（**4**）**CPU** 与 **DMA** 同时要求使用存储器总线时谁优先？为什么?

DMA 优先级更高，因为若 DMA 请求得不到及时响应，I/O 传输的数据可能丢失。

算例（一个扇区读写中的总线请求次数，**2015** 年统考第 **43** 题第 **3** 问）

磁盘转速 7200 r/min，扇区 512 B，采用周期挪用 **DMA**，磁盘控制器数据缓冲区大小 64 位。

一个扇区读写中 DMA 控制器发出的总线请求次数 $= 512\ \mathrm{B}/64\ \mathrm{b} = 512/8 =$ **64** 次。若 CPU 检测到 DMA 请求时也需要访存，**DMAC** 仍能获得总线使用权，因为一旦磁盘开始读写就必须按时完成数据传送，否则会丢数据。

5.DMA 与中断方式的对比

对比项	程序中断方式	**DMA** 方式
谁请求	I/O 设备请求 CPU 的处理时间（请求 CPU 中断服务）	DMAC 请求总线使用权（总线控制权）

对比项	程序中断方式	**DMA** 方式
谁响应、何时响应	CPU 在一条指令执行结束后检测并响应	CPU 在一个总线事务（机器周期/总线周期）结束后响应并让出总线
传输单位	字节或字（一次中断传一个字）	数据块（一次 DMA 传一整块）
数据传送由谁完成	CPU 执行中断服务程序（软件）完成	**DMAC** 硬件直接控制总线完成，不经 CPU 寄存器
CPU 干预程度	每传一个字节/字都要 CPU 参与，需保护和恢复现场	CPU 只在预处理（设参数、启动）和后处理（结束中断处理）时介入，不改变 **CPU** 现场
优先级	较低	**DMA** 请求优先级高于 **CPU** 访存，也高于普通 **I/O** 传送中断请求
功能范围	不仅能传数据，还能处理各种随机事件（故障、状态变化），灵活性高	仅用于数据传输（也可用于内存间搬移）
适用场合	键盘、鼠标、打印机等低速字符设备	磁盘、SSD、网卡、显卡等高速块设备

DMA 与中断对比题的三个经典错误项

1.「中断 I/O 方式适用于所有外部设备」——错误。高速设备用中断会造成 CPU 开销过大。
2.「DMA 方式仅适用于快速外部设备」——片面。DMA 技术上也用于低速设备，只是不划算（初始化开销大）。
3.「DMA 方式下通过 **CPU** 执行 **DMA** 传送程序进行 I/O 操作」——错误。DMA 传送过程不存在 CPU 持续执行的传送程序。

（四）三种 **I/O** 方式的完整对比

对比项	程序查询方式	程序中断方式	**DMA** 方式
发起者	CPU 主动轮询	设备主动发中断请求	设备/DMAC 向 CPU 申请总线
CPU 与设备关系	串行（CPU 忙等）	并行（CPU 执行原程序）	并行（CPU 执行其他程序）
数据传送实现者	CPU 执行查询程序（软件）	CPU 执行中断服务程序（软件）	DMAC 硬件直接控制总线
传送单位	字节/字（或块，但 CPU 全程参与）	字节/字	数据块
CPU 介入时机	全程	每次数据就绪时	仅预处理与后处理
是否需要现场保护	不需要（无切换）	需要（保存/恢复现场）	不需要（不改变 CPU 现场）
请求的资源	—	CPU 处理时间	总线使用权
响应时机	立即（查询到即处理）	一条指令执行结束后	一个总线事务（机器周期）结束后
CPU 开销公式	$n_{\text{查询}} \times t_{\text{每次查询}}$	$n_{\text{中断}} \times C_{\text{每次中断}}$	$n_{\text{DMA 块}} \times C_{\text{预/后处理}}$
数据是否经 CPU 寄存器	经过	经过	不经过
典型设备	单任务系统、引导阶段、超高速同步设备	键盘、鼠标、针式打印机	磁盘、SSD、网卡、显卡

对比项	程序查询方式	程序中断方式	**DMA** 方式
优点	接口简单，不需额外硬件	消除轮询开销，实时性好，可处理随机事件	CPU 开销极小，吞吐率最高，适合大块高速传输
缺点	CPU 浪费严重，效率低	每次传送都要 CPU 参与，高速时开销过大	需要额外 DMAC 硬件，与 CPU 争用主存

本章小结表

考点	必背结论
总线两特点	共享、分时；同一时刻只有一个主设备能发送信息
系统总线三类	数据总线（双向，宽度 = MDR 位数）、地址总线（单向，位数 = MAR 位数，决定 2^n 寻址空间）、控制总线（单根单向、整体双向）
串行 vs 并行	并行适合近距离（系统总线），串行适合远距离（通信总线）；串行速率可以超过并行
总线带宽公式	$BW = \frac{W}{8} \times f_{clk} \times k = \frac{W}{8} \times f_{work} = \frac{D}{T_{bus}} = \frac{D \cdot f_{clk}}{m}$
提高带宽的手段	增加总线宽度、提高工作频率、支持突发传送（复用不能提速）
时钟频率 vs 工作频率	$f_{work} = f_{clk} \times$ 单时钟传输次数；DDR 取 2，普通取 1
总线复用	优点：省引脚、提高利用率、降成本；缺点：控制复杂、性能下降
突发传送	只送一次首地址，连续传多个数据，期间不释放总线；提高有效数据传输率
总线带宽 vs 实际传输率	带宽是固有指标，不受传送方式影响，不含地址/仲裁/准备开销
同步定时	统一时钟；协调简单、速率高；以最慢设备为标准、有时钟偏移、不能过长、可靠性不高
异步定时	握手信号，总线周期长度可变，可挂不同速度设备；逻辑复杂、速度较慢
三种互锁	速度：不互锁 > 半互锁 > 全互锁；可靠性：全互锁 > 半互锁 > 不互锁
半同步	公共时钟 + **Wait** 信号线；握手信号由时钟边沿采样
分离事务	请求与应答分两段，中间释放总线 → 提高总线利用率
集中式仲裁	链式查询（3 根线，优先级固定，对故障敏感）；计数器定时查询（优先级可循环）；独立请求（最快，线最多，优先级灵活）
I/O 接口非设备	磁盘驱动器是设备；控制器/适配器/网络控制器/中断控制器是接口
接口三类寄存器	数据缓冲寄存器（数据端口）、状态寄存器（状态端口）、控制寄存器（控制端口）；状态与控制方向相反，可合用同一寄存器
数据线上传什么	指令、操作数、命令字、状态字、中断类型号；握手信号走控制线
统一编址	端口与主存同一地址空间，用访存指令访问；占用主存空间、寻址慢；保护机制可复用分页/分段；端口不能进 **Cache**
独立编址	独立地址空间，用专用 **I/O** 指令（x86 IN/OUT）；不占主存空间、译码快；指令类型少、需两组控制信号
I/O 指令	传送发生在通用寄存器 ↔ **I/O** 端口之间；是特权指令
程序查询	CPU 与设备串行工作；开销 $= n_{\text{查询}} \times t_{\text{每次}}$；查询间隔必须 ≤ 设备准备一个数据的时间

考点	必背结论
定时查询	定时器到时查状态；定时过短白查、过长丢数据
中断响应三条件	有未被屏蔽的请求；CPU 开中断（异常和 NMI 不受限）；当前指令执行完毕
中断隐指令	关中断 + 保存断点（**PC/PSW**）+ 找服务程序入口；不清通用寄存器、不保存现场
断点差别	中断断点 = 下一条指令地址；缺页 fault 断点 = 当前指令地址；陷阱断点 = 陷阱指令后一条
服务程序流程	保存现场和旧屏蔽字 → 设置新屏蔽字 → 开中断 → 服务 → 关中断 → 恢复现场和屏蔽字 → 开中断 → 中断返回
单重 vs 多重	单重：处理期间一直关中断；多重：设新屏蔽字后重新开中断，允许嵌套
两种优先级	响应优先级由硬件/查询程序定，解决「同时请求先响应谁」；处理优先级由屏蔽字动态定，决定能否被抢占
屏蔽字规则	1 屏蔽、0 允许；**1** 越多优先级越高；每个屏蔽字至少一个 **1**；由处理优先级决定，不是响应优先级
中断开销	$T = \frac{n_{\text{中断}} \times C_{\text{每次}}}{f_{CPU}}$；须满足 $t_{dev} \geq t_{int}$ 才不丢数据
DMAC 组成	主存地址寄存器 AR、字计数器 WC、数据缓冲寄存器 DBR、（命令/状态寄存器）、DMA 控制逻辑（含中断机构）
DMA 握手线	HOLD（DMAC 请求）、HLDA（CPU 授权）；DREQ/DACK 是 DMAC 与 I/O 接口的专用线，不属系统总线
DMA 三阶段	预处理（CPU 设参数、启动设备）→ 数据传送（DMAC 主设备搬运）→ 后处理（结束中断，CPU 收尾）
AR/WC 变化	每传一次：AR ±1、WC −1；WC = 0 时发 DMA 中断请求
停止 CPU 访存	简单、速率最高；CPU 全程被暂停、无法响应中断
周期挪用	按需借一个存储周期，用完立即归还；CPU 可正常访存（除冲突时 DMA 优先）；挪用过程实际占 2 ~ 5 个存储周期
交替访存	时间片对半分配；无需总线请求；硬件复杂、带宽减半，实际少用
DMA vs 中断	请求总线权 vs 请求 CPU 时间；块 vs 字；总线事务后响应 vs 指令后响应；硬件 vs 软件搬运；不保存现场 vs 保存现场

易错清单

总线和 I/O 章 20 个高频陷阱

1. 并行总线不一定比串行总线快。串行总线线少、串扰小，可把频率做高，实际速率可超过并行；PCIe、USB 都是串行。
2. 地址总线是单向的，控制总线整体是双向的。DMA 控制器也能送主存地址，但地址总线的「单向」是指由主设备指向从设备。
3. **MDR** 位数 = 数据总线宽度 $\neq$ 存储字长；**MAR** 位数 = 地址线位数。
4. 时钟频率 $\neq$ 工作频率。DDR 双沿传输时 $f_{work} = 2f_{clk}$；一个总线周期占多个时钟周期时二者又不同。
5. **DDR3-1333** 的 **1333** 已是有效工作频率，不能再乘 2；QPI 的「每秒传输次数」才是 $f_{clk} \times 2$。
6. 总线带宽不考虑地址、仲裁、存储器准备时间。题目问「实际数据传输率」时要另算，别把两者混用。
7. 地址/数据线复用会加长总线周期，不提高速率；能提高速率的是加宽、提频、突发传送。
8. 突发传送的总线事务要算上首地址那一个周期。2012 年那题漏掉地址周期就会算成 40 ns 而不是 50 ns。
9. 同步定时以最慢设备为标准，所以只适合速度相近的设备；异步定时才能挂不同速度的设备。
10. 异步一次握手完成的是一个总线宽度的数据交换，不是一位。

11. 全互锁最慢但最可靠，不互锁最快但最不可靠；半互锁只锁主设备一侧。
12. 中断隐指令只做三件事：关中断、保存断点（PC/PSW）、引出中断服务程序。保存通用寄存器由软件做（2012、2018 年都考过）。
13. 状态端口与控制端口可以合用同一个寄存器，靠读/写方向区分。
14. 统一编址用访存指令访问端口，不是「不能用访存指令」；独立编址时端口地址与主存地址可以数值相同，靠指令区分。
15. 映射到主存空间的 **I/O** 端口不能被 **Cache** 缓存，否则读到的是过时副本，无法感知设备实时状态。
16. 中断响应优先级 $\neq$ 中断处理优先级。屏蔽字改变的是处理优先级（能否被嵌套抢占），不是响应优先级（同时请求时先响应谁）。2011、2024 年两次考这个区分。
17. 屏蔽字按「处理优先级」写：比本级处理优先级高的位写 0（允许打断），本级和更低的写 1。2011 年 L_1 的答案是 01010。
18. 中断服务程序内的顺序：保存现场 $\rightarrow$ 服务 $\rightarrow$ 恢复现场 $\rightarrow$ 开中断 $\rightarrow$ 中断返回（单级中断）；题干若限定「中断服务程序内」，就不要把硬件负责的关中断、保护断点写进去。
19. 中断方式要求 $t_{dev} \geq t_{int}$，即外设准备数据的时间不小于中断处理时间，否则数据被覆盖丢失。
20. 周期挪用不是整块独占总线。CPU 在相邻两次 DMA 传送之间仍可访存；只有「停止 CPU 访存」方式才在整个块传送期间禁止 CPU 访存。DMA 与 CPU 冲突时 **DMA** 优先，且 DMA 传送不需要保护现场（不修改 CPU 寄存器）。